

CS114: Complete Notes and Exam Pack

Quoc Kien

Detailed table of contents

How to Use This Complete Pack	8
Cách dùng tài liệu này	8
Phần 1: Học máy: Suy luận Bình phương Tối thiểu OLS	9
1. Mô hình Hồi quy Tuyến tính Đơn	9
Bản đồ ký hiệu và quy trình thay số	9
2. Hàm mất mát: Residual Sum of Squares	10
3. Suy luận hệ số chặn β_0	10
4. Suy luận hệ số góc β_1	11
5. Quy trình giải bài OLS trong 60 giây	12
6. Bảng Công thức Thi: Quy trình Thay số OLS	13
7. Minh họa bằng Python	13
8. Bộ bài tập mẫu có lời giải	15
Bài 1: Tính đường OLS từ 3 điểm	15
Bài 2: Dữ liệu 10 dòng và dự đoán	16
Bài 3: Dùng residual để kiểm tra đường OLS	17
Phần 2: Học máy: Hồi quy Tuyến tính từ Xác suất, MLE, và Ma trận	19
1. Vì sao OLS có thể xuất hiện từ xác suất?	19
Bản đồ ký hiệu ma trận	19
2. Maximum Likelihood Estimation dẫn tới OLS	20
3. Biểu diễn ma trận	21
4. Suy luận Normal Equations	22
5. Ý nghĩa hình học	23
6. Quy trình giải bài Matrix OLS	24
7. Bảng Công thức Thi: Matrix OLS	25
Dạng bài chứng minh: Gaussian MLE dẫn tới Normal Equations	26
8. Bộ bài tập mẫu có lời giải	27
Bài 1: Giải OLS bằng ma trận với hai điểm	27

Bài 2: Bộ dữ liệu 10 dòng, giải đầy đủ bằng matrix OLS	27
Phần 3: Học máy: Phân loại và Tối ưu Hồi quy Logistic	30
1. Bản chất của Bài toán Phân loại (Classification)	30
Bản đồ ký hiệu và luồng tính	30
Tại sao Tuyến tính (Linear Regression) thất bại với Phân loại?	30
2. Mô hình Logistic Regression & Hàm Sigmoid	31
Công thức Xác suất Hợp nhất	31
3. Chứng minh Hàm Mất mát Cross-Entropy bằng MLE	32
4. Chứng minh Tính Lồi (Convex Optimization) bằng Ma trận Hessian	32
Đạo hàm Bậc một (Gradient) via Chain Rule	33
Đạo hàm Bậc hai (Ma trận Hessian)	33
5. Các Phương pháp Tối ưu hóa Toán học	34
Thuật toán Gradient Descent (Tối ưu bậc 1)	34
Phương pháp Newton - Raphson (Tối ưu bậc 2)	34
6. Kiểm tra Thực thi: Sigmoid và Ranh giới Quyết định	36
7. Quy trình giải bài Logistic Regression	38
8. Bảng Công thức Thi: Quy trình Thay số cho Logistic Regression	38
Dạng bài: chứng minh MLE dẫn tới Binary Cross-Entropy	40
Bộ bài tập mẫu có lời giải	41
Bài 2: Bảng 12 dòng, tính xác suất, loss, accuracy	41
Bài 3: Một bước cập nhật gradient	43
Phần 4: Học máy: Họ Hàm Mũ và Mô hình Tuyến tính Tổng quát (GLM)	44
1. Họ Phân Phối Số Mũ (Exponential Family Distribution - EFD)	44
Bản đồ ký hiệu Exponential Family	44
Các tính chất toán học cốt lõi (Properties)	45
2. Chuẩn hóa các phân phối kinh điển về dạng EFD	45
2.1 Phân phối Bernoulli (Nền tảng của Logistic Regression)	45
2.2 Phân phối Gaussian (Nền tảng của Linear Regression)	46
3. Bản chất Tính Lồi của bài toán MLE thuộc họ EFD	47
4. Khung Mô Hình Tuyến Tính Tổng Quát (Generalized Linear Models - GLMs)	47
5. Bản đồ ánh xạ hệ thống GLM	48
6. Giai đoạn Dự đoán và Thuật toán Tối ưu hóa Tổng quát	48
Giai đoạn Dự đoán (Testing Phase)	48
Thuật toán Cập nhật Trọng số Tổng quát (Gradient Descent)	49
7. Thực hành Minh họa bằng Code Python	49
8. Quy trình nhận diện Exponential Family và GLM	52
9. Bảng Công thức Thi: Chọn GLM và Thay số	52
Dạng bài: chứng minh Bernoulli sinh ra sigmoid	53
Bộ bài tập mẫu có lời giải	54
Bài 2: Bảng 10 dòng, chọn inverse link phù hợp	55
Bài 3: Tính log-likelihood Bernoulli từ 10 quan sát	56

Phần 5: Học máy: Thuật toán Học Tạo sinh	58
1. Triết lý Mô hình Phân biệt (Discriminative) vs. Mô hình Sinh (Generative)	58
Bản đồ ký hiệu cho mô hình sinh	58
2. Phân tích Phân biệt Gaussian (Gaussian Discriminant Analysis - GDA)	59
2.1 Nhắc lại kiến thức: Phân phối Chuẩn Đa Biến	59
2.2 Các giả định nền tảng của GDA	59
3. Ước lượng Tham số GDA bằng Maximum Likelihood Estimation (MLE)	60
3.1 Ước lượng Tham số Tiên nghiệm ϕ	60
3.2 Ước lượng các Vector Kỳ vọng μ_0 và μ_1	61
3.3 Ước lượng Ma trận Hiệp phương sai Chung Σ	61
4. Chứng minh GDA tương đương toán học với Hàm Sigmoid	63
Biến đổi Đại số chi tiết	63
5. Thuật toán Phân loại Naive Bayes (Dữ liệu rời rạc)	64
5.1 Giả định Độc lập Điều kiện Naive Bayes	65
5.2 Thiết lập các Tham số và MLE	65
5.3 Kỹ thuật mịn hóa Laplace (Laplace Smoothing)	66
6. Thực hành Triển khai GDA bằng Python từ đầu (Scratch)	66
7. Quy trình giải bài GDA và Naive Bayes	69
8. Bảng Công thức Thi: Cách Thay số cho Bộ phân loại Sinh	70
Dạng bài: Naive Bayes từ bảng dữ liệu rời rạc	71
Bộ bài tập mẫu có lời giải	73
Bài 2: Naive Bayes với bảng đếm từ	73
Bài 3: Gaussian Discriminant Analysis từ 12 điểm một chiều	75
 Phần 6: Học máy: Cây Quyết định Nâng cao và Phương pháp Kết hợp	 77
1. Cơ chế Hoạt động của Cây Quyết định (Decision Tree)	77
Bản đồ ký hiệu khi tính split	77
2. Hàm Mục Tiêu và Các Độ Đo Ô Nhiễm (Impurity Metrics)	78
2.1 Đối với Bài toán Phân loại (Classification)	78
2.2 Đối với Bài toán Hồi quy (Regression Trees - CART)	79
3. Kiểm Soát Overfitting & Kỹ Thuật Cắt Tỉa Cây (Pruning)	79
4. Các Phương pháp Học Tập Hợp (Ensemble Methods)	80
4.1 Phương pháp Bagging (Bootstrap Aggregating)	80
4.2 Phương pháp Boosting & Bản chất Toán học của XGBoost	80
5. Quy trình chọn split trong cây quyết định	81
6. Bảng Công thức Thi: Máy tính Phân tách Cây Quyết định	82
Dạng bài: Information Gain từ bảng nhiều thuộc tính	84
Bộ bài tập mẫu có lời giải	85
Bài 2: Chọn split tốt nhất từ bộ dữ liệu 12 dòng	85
Bài 3: Lá hồi quy và MSE sau split	87
 Phần 7: Học máy: Mạng Nơ-ron và Lan truyền Ngược Vector hóa	 89
1. Bức tranh tinh thần: mạng nơ-ron đang lưu gì?	89

2. Một nơ-ron đơn: từ đầu vào đến dự đoán	90
3. Lan truyền xuôi: dữ liệu chảy từ trái sang phải	91
4. Lan truyền ngược: lỗi chảy từ phải sang trái	92
5. Vì sao sigmoid dễ làm mất gradient?	94
6. Dropout và Chuẩn hóa Mini-batch	96
Dropout	96
Chuẩn hóa mini-batch	96
7. Từ ký hiệu đến thao tác: đọc một bài backprop không bị lẫn	99
Dịch bảng trọng số thành phương trình	99
Công thức thao tác cho mạng 2-2-1, squared error	100
8. Quy trình debug một bài neural network	101
9. Bảng Công thức Thi: Lan truyền Xuôi, Lan truyền Ngược, Cập nhật	102
10. Bộ bài tập mẫu có lời giải	103
Bài 1: Xác định kích thước ma trận	103
Bài 2: Lan truyền xuôi qua mạng 2-2-1	104
Bài 3: Một bước lan truyền ngược ở tầng đầu ra	105
Bài 4: Backprop đầy đủ qua hidden layer với ký hiệu rõ ràng	106
Phần 8: Học máy: Tối ưu hóa, Thiên lệch-Phương sai, và Điều chuẩn	109
1. Tối ưu hóa bằng Gradient Descent	109
Bản đồ ký hiệu tối ưu hóa và regularization	109
2. Batch, Stochastic, và Mini-batch	110
3. Underfitting và Overfitting	112
4. Bias-Variance Decomposition	113
5. Regularization	114
6. Chọn siêu tham số bằng Cross-validation	114
7. Quy trình xử lý bài tối ưu hóa và regularization	116
8. Bảng Công thức Thi: Bias, Variance, Regularization	116
Dạng bài: một bước gradient descent có regularization	117
9. Bộ bài tập mẫu có lời giải	118
Bài 1: Tính objective Ridge	118
Bài 2: Chọn λ từ bảng cross-validation	118
Bài 3: Một bước Gradient Descent trên dữ liệu 12 dòng	119
Phần 9: Học máy: Học Không Giám sát và Phân cụm K-means	121
1. Tổng quan về Học Không Giám Sát (Unsupervised Learning)	121
Bản đồ ký hiệu K-means	121
2. Giải thuật Phân Cụm K-means (K-means Clustering)	122
2.1 Hàm Mục Tiêu Định Lượng (Distortion Function / Cost Function)	122
2.2 Quy Trình Vòng Lặp Hội Tụ (Coordinate Descent)	123
3. Cải Tiến Chọn Tâm Ban Đầu: K-means++	126
4. Thực Hành Trace Thuật Toán Bằng Tay (Bản Số Hóa)	127
4.1 Khởi tạo dữ liệu gốc (Input Data)	127

4.2 Vòng Lặp 1 (Iteration 1)	127
4.3 Vòng Lặp 2 (Iteration 2)	128
4.4 Kết luận hội tụ (Convergence)	129
5. Thực Hành Hiện Thực Hóa Thuật Toán Bằng Python	129
6. Quy trình trace K-means bằng tay	132
7. Bảng Công thức Thi: Các Điểm Cốt lõi của K-means	133
Dạng bài chứng minh: khoảng cách tới mean và khoảng cách từng cặp	133
Bộ bài tập mẫu có lời giải	135
Bài 2: K-means trên bộ dữ liệu 10 điểm hai chiều	135
Bài 3: Chọn K bằng elbow từ bảng distortion	137
Part 10: Exam Papers and Answer Keys	138
Exam Appendix 1: CS114 - Exam_Recovered	139
ĐỀ THI MACHINE LEARNING	139
Câu 1. Maximum Likelihood Estimation cho Logistic Regression	139
Câu 2. (Chưa khôi phục được nội dung)	140
Câu 3. Backpropagation cho mạng neural network đơn giản	140
Câu 4. Naive Bayes Classification	141
Câu 5. Decision Tree với Information Gain dựa trên Entropy	141
Câu 6. Quan hệ giữa khoảng cách đến mean và khoảng cách giữa các cặp điểm trong cụm	142
Exam Appendix 2: CS114 - Exam_Answer	144
ĐÁP ÁN ĐỀ THI MACHINE LEARNING	144
Câu 1. Maximum Likelihood Estimation cho Logistic Regression	144
Câu 2. (Chưa khôi phục được nội dung)	146
Câu 3. Backpropagation cho Mạng Neural Network Đơn Giản	146
Câu 4. Naive Bayes Classification	149
Câu 5. Decision Tree với Information Gain dựa trên Entropy	151
Câu 6. Chứng minh quan hệ khoảng cách trong cụm	154
Exam Appendix 3: CS114 - Exam_01	156
ĐỀ THI MACHINE LEARNING — ĐỀ 01	156
Câu 1. OLS Linear Regression bằng Ma trận	156
Câu 2. Logistic Regression — Thay số Tính Xác suất và Loss	157
Câu 3. Họ Phân Phối Số Mũ (Exponential Family) và GLM	157
Câu 4. Neural Network Forward Pass — Mạng 3-2-1	158
Câu 5. Decision Tree — Gini Index	158
Câu 6. K-means Clustering — Trace Thuật Toán	159
Câu 7. Bias-Variance Decomposition	160
Câu 8. Tính phi tuyến trong Mạng Neural (Nâng cao)	160

Exam Appendix 4: CS114 - Exam_01_Answer	161
ĐÁP ÁN ĐỀ THI MACHINE LEARNING — ĐỀ 01	161
Câu 1. OLS Linear Regression bằng Ma trận	161
Câu 2. Logistic Regression — Thay số	163
Câu 3. Exponential Family và GLM	164
Câu 4. Neural Network Forward Pass — Mạng 3-2-1	165
Câu 5. Decision Tree — Gini Index	166
Câu 6. K-means Clustering — Trace	168
Câu 7. Bias-Variance Decomposition	169
Câu 8. Tính phí tuyển trong Mạng Neural (Nâng cao)	170
Exam Appendix 5: CS114 - Exam_02	171
ĐỀ THI MACHINE LEARNING — ĐỀ 02	171
Câu 1. MLE cho Linear Regression — Chứng minh OLS	171
Câu 2. Gaussian Discriminant Analysis (GDA)	171
Câu 3. Logistic Regression — Gradient Descent Một Bước	172
Câu 4. Decision Tree — Entropy và Information Gain	173
Câu 5. Backpropagation — Mạng 2-2-1 với Squared Error	174
Câu 6. K-means — Chọn K và Distortion	175
Câu 7. Thử thách: Hội tụ của K-means và Giới hạn của Đảm bảo Hội tụ	175
Exam Appendix 6: CS114 - Exam_02_Answer	176
ĐÁP ÁN ĐỀ THI MACHINE LEARNING — ĐỀ 02	176
Câu 1. MLE cho Linear Regression — Chứng minh OLS	176
Câu 2. Gaussian Discriminant Analysis	177
Câu 3. Logistic Regression — Gradient Descent	179
Câu 4. Decision Tree — Entropy	181
Câu 5. Backpropagation — Mạng 2-2-1	182
Câu 6. K-means	183
Câu 7. Thử thách: Hội tụ của K-means và Giới hạn của Đảm bảo Hội tụ	185
Exam Appendix 7: CS114 - Exam_03	189
ĐỀ THI MACHINE LEARNING — ĐỀ 03	189
Câu 1. Hồi quy Tuyến tính Đơn (OLS từ đầu)	189
Câu 2. Chứng minh Hàm Sigmoid từ Exponential Family	190
Câu 3. Naive Bayes với Laplace Smoothing	190
Câu 4. Regularization và Ridge Regression	191
Câu 5. Neural Network — Xác định Kích thước Ma trận	191
Câu 6. K-means — Chứng minh Tâm Cụm Là Trung Bình	192
Câu 7. Thử thách: Newton's Method cho Logistic Regression	192

Exam Appendix 8: CS114 - Exam_03_Answer	194
ĐÁP ÁN ĐỀ THI MACHINE LEARNING — ĐỀ 03	194
Câu 1. Hồi quy Tuyến tính Đơn	194
Câu 2. Exponential Family	195
Câu 3. Naive Bayes	196
Câu 4. Ridge Regression	197
Câu 5. Neural Network — Kích thước	198
Câu 6. K-means — Chứng minh Tâm Cụm	200
Câu 7. Thử thách: Newton's Method cho Logistic Regression	201
Exam Appendix 9: CS114 - Exam_04	204
ĐỀ THI MACHINE LEARNING — ĐỀ 04	204
Câu 1. Logistic Regression — Chứng minh Tính Lỗi	204
Câu 2. OLS bằng Ma trận — Bộ dữ liệu 2 biến	204
Câu 3. GDA — Chứng minh Ranh giới Tuyến tính	205
Câu 4. Decision Tree — Regression Tree với MSE	206
Câu 5. Backpropagation qua Mạng 2-3-1	206
Câu 6. Gradient Descent với Regularization	207
Câu 7. Thử thách: Batch Normalization Forward và Backward	208
Exam Appendix 10: CS114 - Exam_04_Answer	209
ĐÁP ÁN ĐỀ THI MACHINE LEARNING — ĐỀ 04	209
Câu 1. Logistic Regression — Chứng minh Tính Lỗi	209
Câu 2. OLS 2 biến	210
Câu 3. GDA — Ranh giới Tuyến tính	212
Câu 4. Regression Tree	213
Câu 5. Backpropagation 2-3-1	215
Câu 6. Gradient Descent với Ridge	216
Câu 7. Thử thách: Batch Normalization Forward và Backward	217
Exam Appendix 11: CS114 - Exam_05	221
ĐỀ THI MACHINE LEARNING — ĐỀ 05	221
Câu 1. Cross-Entropy Loss và Logistic Regression Hoàn chỉnh	221
Câu 2. Naive Bayes — So sánh Có và Không có Laplace Smoothing	222
Câu 3. Decision Tree — So sánh Entropy và Gini	222
Câu 4. Neural Network — Dropout và Batch Normalization	223
Câu 5. Dropout trong Neural Network	223
Câu 6. Chứng minh MLE của Exponential Family là Lỗi	224
Câu 7. Thử thách: Vanishing Gradient bằng Quy tắc Chuỗi	224
Exam Appendix 12: CS114 - Exam_05_Answer	225
ĐÁP ÁN ĐỀ THI MACHINE LEARNING — ĐỀ 05	225
Câu 1. Logistic Regression Hoàn chỉnh	225

Câu 2. Naive Bayes — Laplace Smoothing	227
Câu 3. Decision Tree — Entropy vs Gini	228
Câu 4. Dropout và Batch Normalization	229
Câu 5. Dropout trong Neural Network	230
Câu 6. MLE Exponential Family	230
Câu 7. Thử thách: Vanishing Gradient bằng Quy tắc Chuỗi	232

Cheers **236**

How to Use This Complete Pack

This document joins the full CS114 notes with every exam and answer key in the `exam/` directory.

- Part 1 contains the complete notebook-based knowledgebase.
- Part 2 contains the recovered sample exam, five practice exams, and all answer keys.
- Use the PDF table of contents or bookmarks for fast traversal.

Cách dùng tài liệu này

Tài liệu này ghép toàn bộ 9 notebook CS114 thành một bản PDF lớn. Mỗi notebook bắt đầu ở một trang mới và có heading cấp 1 riêng, nên có thể đi nhanh bằng mục lục PDF hoặc thanh bookmarks của trình đọc PDF.

Các phần bài tập vẫn được viết bằng lời giải tay: thay số, tính toán, kết luận. Các đoạn Python chỉ phục vụ minh họa, kiểm tra đạo hàm, hoặc tạo hình trực quan.

Phần 1: Học máy: Suy luận Bình phương Tối thiểu OLS

Nguồn: `quarto/LR_OLS_Derivation.qmd`

1. Mô hình Hồi quy Tuyến tính Đơn

Bản đồ ký hiệu và quy trình thay số

Ký hiệu	Nghĩa	Khi làm bài dùng thế nào?
x_i	Giá trị đầu vào của mẫu thứ i	Cột dữ liệu độc lập.
y_i	Giá trị mục tiêu thật của mẫu thứ i	Cột cần dự đoán.
$\hat{y}_i$	Giá trị mô hình dự đoán	$\hat{y}_i = \beta_0 + \beta_1 x_i$.
$\bar{x}, \bar{y}$	Trung bình của x và y	Dùng để tính hệ số góc và hệ số chặn.
β_1	Hệ số góc	Đo y đổi bao nhiêu khi x tăng 1 đơn vị.
β_0	Hệ số chặn	Giá trị dự đoán khi $x = 0$.
e_i	Residual/sai số	$e_i = y_i - \hat{y}_i$.

Quy trình tính tay: tính $\bar{x}, \bar{y}$ trước, sau đó tính $\sum(x_i - \bar{x})(y_i - \bar{y})$ và $\sum(x_i - \bar{x})^2$, cuối cùng suy ra β_1, β_0 , rồi kiểm tra bằng residual.

Trong hồi quy tuyến tính đơn, ta mô hình hóa quan hệ giữa biến phụ thuộc y và biến độc lập x bằng một đường thẳng:

$$\hat{y}_i = \beta_0 + \beta_1 x_i$$

Trong đó:

- β_0 là **hệ số chặn**: giá trị dự đoán của y khi $x = 0$.
- β_1 là **hệ số góc**: khi x tăng 1 đơn vị, $\hat{y}$ thay đổi trung bình β_1 đơn vị.
- $\hat{y}_i$ là giá trị mô hình dự đoán cho quan sát thứ i .

Mục tiêu của OLS là chọn β_0, β_1 sao cho sai lệch giữa dữ liệu thật y_i và dự đoán $\hat{y}_i$ là nhỏ nhất.

2. Hàm mất mát: Residual Sum of Squares

Sai số của điểm thứ i là:

$$e_i = y_i - \hat{y}_i = y_i - \beta_0 - \beta_1 x_i$$

OLS dùng tổng bình phương sai số:

$$RSS = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

Ta thường đặt thêm hệ số $\frac{1}{2}$ để khi đạo hàm, số 2 triệt tiêu:

$$f(\beta_0, \beta_1) = \frac{1}{2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

Thực tế: bình phương làm sai số âm/dương đều thành dương, đồng thời phạt mạnh các điểm lệch xa.

3. Suy luận hệ số chặn β_0

Lấy đạo hàm riêng theo β_0 :

$$\frac{\partial f}{\partial \beta_0} = \sum_{i=1}^n -(y_i - \beta_0 - \beta_1 x_i) = 0$$

Nhân hai vế với -1 :

$$\sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) = 0$$

Tách tổng:

$$\sum_{i=1}^n y_i - n\beta_0 - \beta_1 \sum_{i=1}^n x_i = 0$$

Chuyển vế:

$$n\beta_0 = \sum_{i=1}^n y_i - \beta_1 \sum_{i=1}^n x_i$$

Chia cho n :

$$\beta_0 = \frac{1}{n} \sum_{i=1}^n y_i - \beta_1 \frac{1}{n} \sum_{i=1}^n x_i$$

Vì $\bar{x} = \frac{1}{n} \sum x_i$ và $\bar{y} = \frac{1}{n} \sum y_i$, ta có:

$$\boxed{\beta_0 = \bar{y} - \beta_1 \bar{x}}$$

Điều này cũng cho thấy đường OLS luôn đi qua điểm trung bình $(\bar{x}, \bar{y})$.

4. Suy luận hệ số góc β_1

Lấy đạo hàm riêng theo β_1 :

$$\frac{\partial f}{\partial \beta_1} = \sum_{i=1}^n -(y_i - \beta_0 - \beta_1 x_i)x_i = 0$$

Nhân với -1 :

$$\sum_{i=1}^n (x_i y_i - \beta_0 x_i - \beta_1 x_i^2) = 0$$

Thay $\beta_0 = \bar{y} - \beta_1 \bar{x}$ và dùng $\sum x_i = n\bar{x}$:

$$\sum x_i y_i - (\bar{y} - \beta_1 \bar{x})n\bar{x} - \beta_1 \sum x_i^2 = 0$$

Khai triển:

$$\sum x_i y_i - n\bar{x}\bar{y} + n\beta_1 \bar{x}^2 - \beta_1 \sum x_i^2 = 0$$

Gom các hạng tử chứa β_1 :

$$\sum x_i y_i - n\bar{x}\bar{y} = \beta_1 \left(\sum x_i^2 - n\bar{x}^2 \right)$$

Suy ra:

$$\beta_1 = \frac{\sum_{i=1}^n x_i y_i - n\bar{x}\bar{y}}{\sum_{i=1}^n x_i^2 - n\bar{x}^2}$$

Dạng dễ nhớ hơn:

$$\beta_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Tỉ số đo mức x và y cùng biến thiên; mẫu số đo độ phân tán của x .

5. Quy trình giải bài OLS trong 60 giây

Khi gặp một bài OLS tuyến tính đơn, hãy đi theo thứ tự này:

1. Lập bảng $x_i, y_i, x_i - \bar{x}, y_i - \bar{y}$.
2. Tính hai tổng quan trọng:

$$S_{xy} = \sum_i (x_i - \bar{x})(y_i - \bar{y}), \quad S_{xx} = \sum_i (x_i - \bar{x})^2$$

3. Tính:

$$\beta_1 = \frac{S_{xy}}{S_{xx}}, \quad \beta_0 = \bar{y} - \beta_1 \bar{x}$$

4. Viết mô hình cuối:

$$\hat{y} = \beta_0 + \beta_1 x$$

5. Nếu đề hỏi kiểm tra chất lượng, tính residual:

$$e_i = y_i - \hat{y}_i$$

Bẫy thường gặp: không được dùng công thức $\beta_1 = \frac{\sum x_i y_i}{\sum x_i^2}$ trừ khi dữ liệu đã được centered hoặc mô hình không có intercept. Với OLS thường có intercept, phải dùng độ lệch quanh trung bình.

6. Bảng Công thức Thi: Quy trình Thay số OLS

Bước	Công thức	Ý nghĩa
1. Tính trung bình	$\bar{x} = \frac{1}{n} \sum x_i, \bar{y} = \frac{1}{n} \sum y_i$	Xác định tâm dữ liệu.
2. Tính S_{xy}	$S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y})$	Đo x, y biến thiên cùng nhau.
3. Tính S_{xx}	$S_{xx} = \sum (x_i - \bar{x})^2$	Đo độ phân tán của x .
4. Tính slope	$\beta_1 = \frac{S_{xy}}{S_{xx}}$	Độ dốc đường hồi quy.
5. Tính intercept	$\beta_0 = \bar{y} - \beta_1 \bar{x}$	Dịch đường thẳng cho đi qua điểm trung bình.
6. Dự đoán	$\hat{y} = \beta_0 + \beta_1 x$	Thay x mới vào mô hình.
7. Tính residual	$e_i = y_i - \hat{y}_i$	Sai số dọc của từng điểm.

Mẹo nhớ nhanh.

- $\beta_1 > 0$: x tăng thì y có xu hướng tăng.
- $\beta_1 < 0$: x tăng thì y có xu hướng giảm.
- Nếu $S_{xx} = 0$, mọi x_i giống nhau nên không xác định được slope.
- Đường OLS luôn đi qua $(\bar{x}, \bar{y})$.

7. Minh họa bằng Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
X = 2 * np.random.rand(50, 1)
```

```

y = 4 + 3 * X + np.random.randn(50, 1)

x_mean = np.mean(X)
y_mean = np.mean(y)
beta_1 = np.sum((X - x_mean) * (y - y_mean)) / np.sum((X - x_mean) ** 2)
beta_0 = y_mean - beta_1 * x_mean

X_new = np.array([[0], [2]])
y_predict = beta_0 + beta_1 * X_new

print("beta_0:", round(float(beta_0), 3))
print("beta_1:", round(float(beta_1), 3))

plt.figure(figsize=(8, 5))
plt.scatter(X, y, color="#4C78A8", label="Dữ liệu")
plt.plot(X_new, y_predict, color="#F58518", linewidth=2, label=f"OLS: y =  
↪ {beta_0:.2f} + {beta_1:.2f}x")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Minh họa hồi quy tuyến tính OLS")
plt.legend()
plt.grid(alpha=0.25)
plt.show()

```

```

beta_0: 4.097
beta_1: 2.888

```

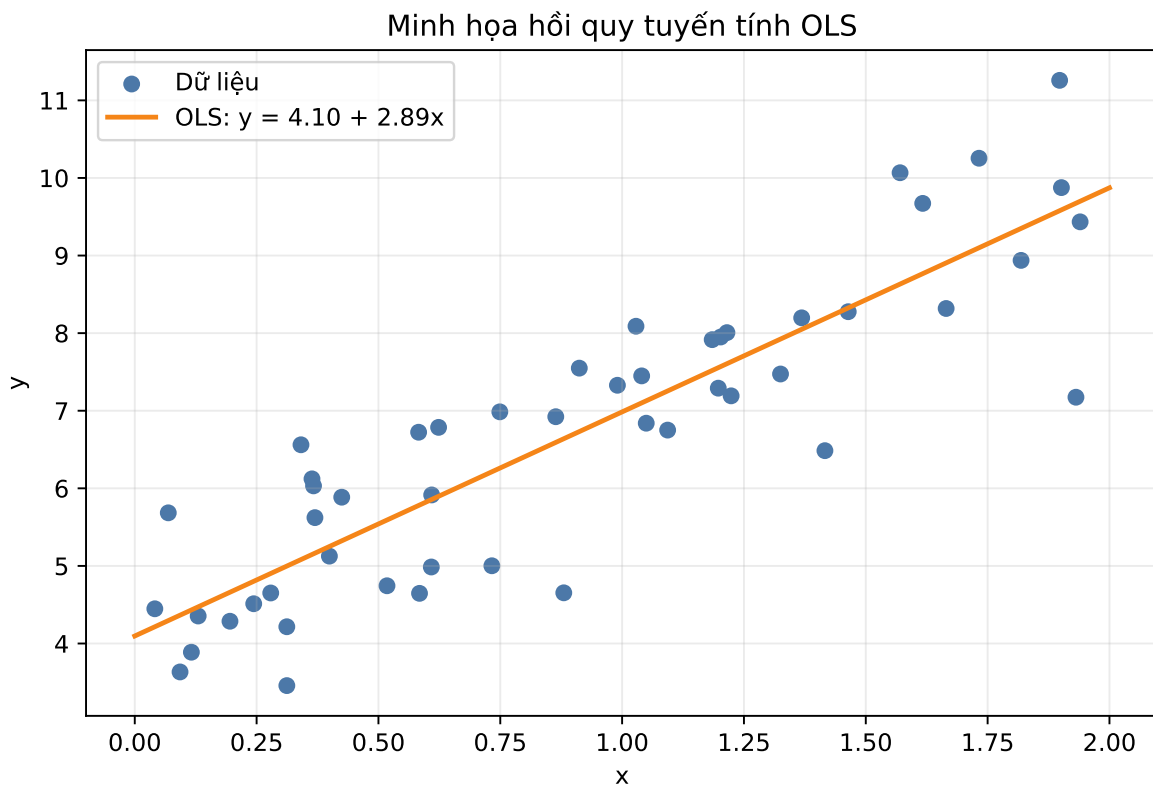


Figure 1: Đường OLS khớp dữ liệu và các residual theo phương dọc.

8. Bộ bài tập mẫu có lời giải

Bài 1: Tính đường OLS từ 3 điểm

Đề bài. Cho ba điểm $(1, 2)$, $(2, 3)$, $(3, 5)$. Tìm đường OLS $\hat{y} = \beta_0 + \beta_1 x$.

Lời giải.

$$\bar{x} = \frac{1 + 2 + 3}{3} = 2, \quad \bar{y} = \frac{2 + 3 + 5}{3} = \frac{10}{3}$$

$$S_{xy} = (1 - 2)(2 - \frac{10}{3}) + (2 - 2)(3 - \frac{10}{3}) + (3 - 2)(5 - \frac{10}{3}) = 3$$

$$S_{xx} = (1 - 2)^2 + (2 - 2)^2 + (3 - 2)^2 = 2$$

$$\beta_1 = \frac{3}{2} = 1.5, \quad \beta_0 = \frac{10}{3} - 1.5(2) = \frac{1}{3}$$

Kết luận.

$$\hat{y} = \frac{1}{3} + 1.5x$$

Bài 2: Dữ liệu 10 dòng và dự đoán

Đề bài. Cho bảng dữ liệu:

x	1	2	3	4	5	6	7	8	9	10
y	3	5	6	8	11	12	14	17	18	20

Hãy fit đường OLS và dự đoán y khi $x = 12$.

Tính trung bình:

$$\bar{x} = \frac{1 + 2 + \dots + 10}{10} = 5.5$$

$$\bar{y} = \frac{3 + 5 + 6 + 8 + 11 + 12 + 14 + 17 + 18 + 20}{10} = 11.4$$

Tính hai tổng chính:

$$S_{xy} = \sum_{i=1}^{10} (x_i - \bar{x})(y_i - \bar{y}) = 159.0$$

$$S_{xx} = \sum_{i=1}^{10} (x_i - \bar{x})^2 = 82.5$$

Hệ số góc:

$$\beta_1 = \frac{S_{xy}}{S_{xx}} = \frac{159.0}{82.5} \approx 1.927$$

Hệ số chặn:

$$\beta_0 = \bar{y} - \beta_1 \bar{x} = 11.4 - 1.927(5.5) \approx 0.800$$

Đường OLS:

$$\hat{y} = 0.800 + 1.927x$$

Dự đoán tại $x = 12$:

$$\hat{y} = 0.800 + 1.927(12) \approx 23.927$$

Kết luận. Khi $x = 12$, mô hình dự đoán $y \approx \boxed{23.93}$.

Bài 3: Dùng residual để kiểm tra đường OLS

Đề bài. Với bộ dữ liệu ở Bài 2, sau khi fit OLS, hãy kiểm tra hai điều kiện:

$$\sum_{i=1}^n e_i = 0, \quad \sum_{i=1}^n x_i e_i = 0$$

Trong đó $e_i = y_i - \hat{y}_i$. Giải thích vì sao hai điều kiện này quan trọng.

Lời giải. Điều kiện thứ nhất nói residual không còn lệch trung bình lên hoặc xuống. Điều kiện thứ hai nói residual không còn tương quan tuyến tính với x . Nếu một trong hai điều kiện còn lớn, ta vẫn có thể chỉnh intercept hoặc slope để giảm RSS.

Từ $\hat{y} = 0.800 + 1.927x$, residual xấp xỉ:

x	1	2	3	4	5	6	7	8	9	10
$e = y - \hat{y}$	0.273	0.345	-0.582	-0.509	0.564	-0.364	-0.291	0.782	-0.145	-0.073

Tổng residual:

$$\sum e_i \approx 0.273 + 0.345 - 0.582 - 0.509 + 0.564 - 0.364 - 0.291 + 0.782 - 0.145 - 0.073 \approx 0$$

Tổng $x_i e_i$:

$$\sum x_i e_i \approx 0$$

RSS:

$$\sum e_i^2 \approx 1.964$$

Kết luận. Hai tổng gần 0 xác nhận nghiệm OLS đã dừng đúng tại điểm cực tiểu. Đây là cách kiểm tra nhanh khi đề bài yêu cầu chứng minh đường fit đã tối ưu.

Phần 2: Học máy: Hồi quy Tuyến tính từ Xác suất, MLE, và Ma trận

Nguồn: `quarto/LR_MLE_Matrix_Derivation.qmd`

1. Vì sao OLS có thể xuất hiện từ xác suất?

Bản đồ ký hiệu ma trận

Ký hiệu	Nghĩa	Kích thước thường gặp
$\mathbf{X}$	Ma trận thiết kế, thường có cột 1 cho bias	$n \times p$
$\mathbf{y}$	Vector mục tiêu thật	$n \times 1$
β	Vector hệ số cần học	$p \times 1$
$\hat{\mathbf{y}}$	Vector dự đoán	$\mathbf{X}\beta$
$\mathbf{r}$	Vector residual	$\mathbf{y} - \mathbf{X}\beta$
σ^2	Phương sai nhiễu Gaussian	Một số dương

Khi gặp bài MLE dẫn tới OLS, hãy đọc theo chuỗi: giả định Gaussian cho y_i , viết likelihood, lấy log, bỏ các hằng số không phụ thuộc β , rồi nhận ra cực đại log-likelihood tương đương cực tiểu $\|\mathbf{y} - \mathbf{X}\beta\|_2^2$.

Trong mô hình hồi quy tuyến tính, ta không giả sử dữ liệu nằm hoàn hảo trên một đường hoặc một mặt phẳng. Ta giả sử mỗi nhãn thật được tạo bởi phần tín hiệu tuyến tính cộng với nhiễu:

$$y_i = \beta^T \mathbf{x}_i + \epsilon_i$$

Trong đó:

- $\mathbf{x}_i$ là vector đặc trưng của mẫu thứ i . Nếu mô hình có intercept, phần tử đầu tiên của $\mathbf{x}_i$ thường là 1.
- β là vector hệ số cần học.
- ϵ_i là phần nhiễu mà mô hình không giải thích được.

Giả định quan trọng nhất:

$$\epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

Nghĩa là sai số có trung bình bằng 0, độc lập giữa các mẫu, và có cùng phương sai σ^2 . Khi đó:

$$y_i \mid \mathbf{x}_i; \beta \sim \mathcal{N}(\beta^T \mathbf{x}_i, \sigma^2)$$

Mật độ xác suất của một nhãn y_i là:

$$p(y_i \mid \mathbf{x}_i; \beta) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y_i - \beta^T \mathbf{x}_i)^2}{2\sigma^2}\right)$$

Trực giác: nếu mô hình dự đoán gần y_i , sai số nhỏ, mũ âm nhỏ, xác suất cao. Nếu mô hình dự đoán rất xa, sai số bình phương lớn, xác suất thấp.

2. Maximum Likelihood Estimation dẫn tới OLS

Vì các mẫu độc lập, xác suất quan sát toàn bộ dữ liệu là tích của xác suất từng mẫu:

$$L(\beta) = \prod_{i=1}^n p(y_i \mid \mathbf{x}_i; \beta)$$

Thay công thức Gaussian vào:

$$L(\beta) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y_i - \beta^T \mathbf{x}_i)^2}{2\sigma^2}\right)$$

Tích nhiều số nhỏ dễ khó nhìn, nên ta lấy log. Vì log là hàm tăng đơn điệu, cực đại của L cũng là cực đại của $\ell = \log L$:

$$\ell(\beta) = \sum_{i=1}^n \left[\log\left(\frac{1}{\sqrt{2\pi}\sigma}\right) - \frac{(y_i - \beta^T \mathbf{x}_i)^2}{2\sigma^2} \right]$$

Gộp hằng số:

$$\ell(\beta) = n \log\left(\frac{1}{\sqrt{2\pi}\sigma}\right) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \beta^T \mathbf{x}_i)^2$$

Khi tối ưu theo β :

- Hạng $n \log \left(\frac{1}{\sqrt{2\pi}\sigma} \right)$ là hằng số.
- $\frac{1}{2\sigma^2}$ là hệ số dương.
- Tối đa hóa số âm của tổng bình phương sai số tương đương tối thiểu hóa tổng bình phương sai số.

Vì vậy:

$$\max_{\beta} \ell(\beta) \iff \min_{\beta} \frac{1}{2} \sum_{i=1}^n (y_i - \beta^T \mathbf{x}_i)^2$$

Kết luận lớn: OLS không chỉ là một mẹo hình học. Nó là nghiệm MLE nếu nhiễu của hồi quy tuyến tính là Gaussian i.i.d.

3. Biểu diễn ma trận

Với n mẫu và p đặc trưng, gom dữ liệu thành:

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \vdots \\ \mathbf{x}_n^T \end{pmatrix} \in \mathbb{R}^{n \times p}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} \in \mathbb{R}^n, \quad \beta = \begin{pmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_p \end{pmatrix} \in \mathbb{R}^p$$

Vector dự đoán toàn bộ dữ liệu là:

$$\hat{\mathbf{y}} = \mathbf{X}\beta$$

Vector sai số:

$$\mathbf{r} = \mathbf{y} - \mathbf{X}\beta$$

Loss OLS dạng ma trận:

$$L(\beta) = \frac{1}{2} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 = \frac{1}{2} (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta)$$

Nếu bạn thấy biểu thức này trong đề thi, hãy đọc nó như sau: lấy từng residual, bình phương, cộng lại, rồi nhân $\frac{1}{2}$.

4. Suy luận Normal Equations

Khai triển loss:

$$\begin{aligned}L(\beta) &= \frac{1}{2}(\mathbf{y}^T - \beta^T \mathbf{X}^T)(\mathbf{y} - \mathbf{X}\beta) \\&= \frac{1}{2}(\mathbf{y}^T \mathbf{y} - \mathbf{y}^T \mathbf{X}\beta - \beta^T \mathbf{X}^T \mathbf{y} + \beta^T \mathbf{X}^T \mathbf{X}\beta)\end{aligned}$$

Hai hạng giữa là scalar và là chuyển vị của nhau, nên bằng nhau:

$$\mathbf{y}^T \mathbf{X}\beta = \beta^T \mathbf{X}^T \mathbf{y}$$

Do đó:

$$L(\beta) = \frac{1}{2}(\mathbf{y}^T \mathbf{y} - 2\beta^T \mathbf{X}^T \mathbf{y} + \beta^T \mathbf{X}^T \mathbf{X}\beta)$$

Lấy gradient theo β :

$$\nabla_{\beta} L = -\mathbf{X}^T \mathbf{y} + \mathbf{X}^T \mathbf{X}\beta$$

Tại điểm cực tiểu, gradient bằng vector 0:

$$-\mathbf{X}^T \mathbf{y} + \mathbf{X}^T \mathbf{X}\beta = \mathbf{0}$$

Suy ra hệ phương trình chuẩn:

$$\boxed{\mathbf{X}^T \mathbf{X}\beta = \mathbf{X}^T \mathbf{y}}$$

Nếu $\mathbf{X}^T \mathbf{X}$ khả nghịch:

$$\boxed{\beta = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}}$$

Trong code, nên dùng `np.linalg.solve(X.T @ X, X.T @ y)` thay vì tự tính nghịch đảo, vì solve thường ổn định số học hơn.

5. Ý nghĩa hình học

Normal equations có thể viết lại:

$$\mathbf{X}^T(\mathbf{y} - \mathbf{X}\beta) = 0$$

Tức là:

- $\hat{\mathbf{y}} = \mathbf{X}\beta$ nằm trong không gian cột của $\mathbf{X}$.
- Residual $\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}}$ vuông góc với từng cột của $\mathbf{X}$.
- OLS chọn hình chiếu vuông góc của $\mathbf{y}$ lên không gian mà mô hình có thể biểu diễn.

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([
    [1.0, 0.0],
    [1.0, 1.0],
    [1.0, 2.0],
    [1.0, 3.0],
])
y = np.array([1.0, 2.2, 1.9, 3.2])
beta = np.linalg.solve(X.T @ X, X.T @ y)
y_hat = X @ beta
residual = y - y_hat

print("beta:", np.round(beta, 3))
print("X.T @ residual:", np.round(X.T @ residual, 10))

plt.figure(figsize=(7, 4))
plt.scatter(X[:, 1], y, label="y quan sát", color="#4C78A8", s=70)
plt.plot(X[:, 1], y_hat, label="đường fit OLS", color="#F58518", linewidth=2)
for x_i, y_i, y_fit in zip(X[:, 1], y, y_hat):
    plt.plot([x_i, x_i], [y_fit, y_i], color="#AOAECO", linestyle="--")
plt.title("Normal equations như phép chiếu vuông góc")
plt.xlabel("đặc trưng x")
plt.ylabel("mục tiêu y")
plt.legend()
plt.grid(alpha=0.25)
plt.show()
```

```
beta: [1.13 0.63]
X.T @ residual: [-0. -0.]
```

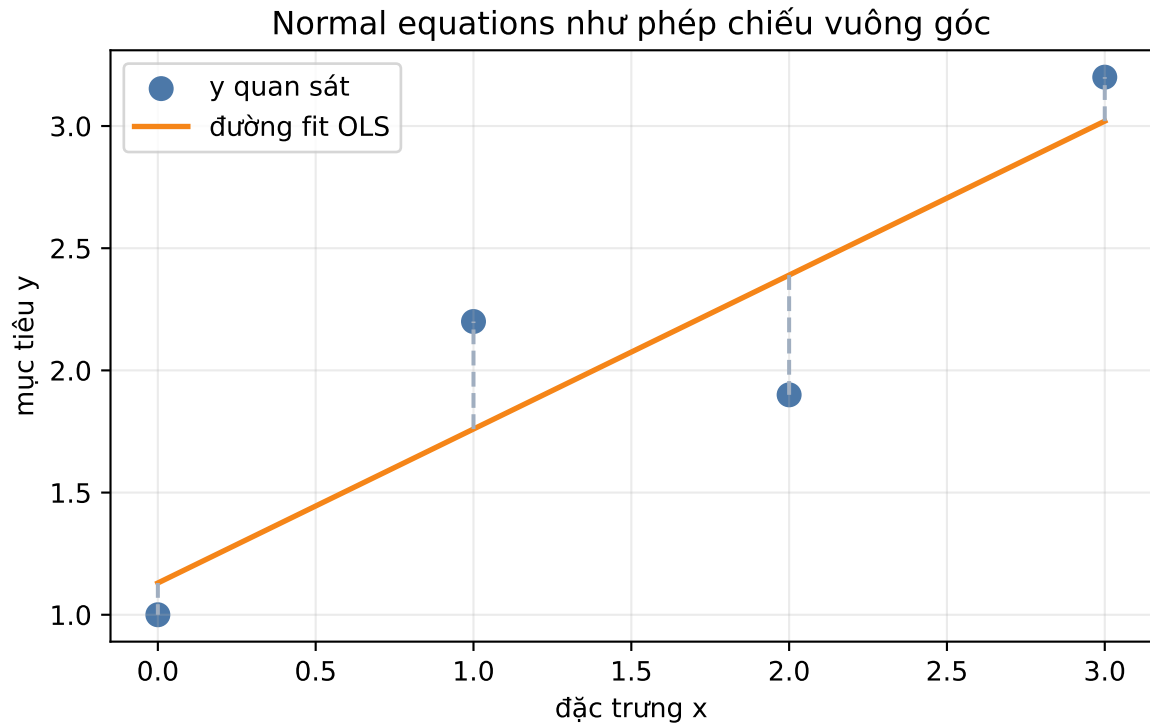


Figure 2: OLS chiếu y lên không gian dự đoán; residual vuông góc với các cột của X.

6. Quy trình giải bài Matrix OLS

Khi đề cho nhiều biến hoặc yêu cầu dùng ma trận, hãy làm theo checklist:

1. Tạo $\mathbf{X}$, nhớ thêm cột 1 nếu mô hình có intercept.
2. Kiểm tra kích thước:

$$\mathbf{X} : n \times p, \quad \beta : p \times 1, \quad \mathbf{y} : n \times 1$$

3. Tính ba khối:

$$\mathbf{X}^T \mathbf{X}, \quad \mathbf{X}^T \mathbf{y}, \quad (\mathbf{X}^T \mathbf{X})^{-1}$$

4. Tính nghiệm:

$$\hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

5. Kiểm tra residual bằng trực giao:

$$\mathbf{X}^T (\mathbf{y} - \mathbf{X} \hat{\beta}) = \mathbf{0}$$

Bẫy thường gặp: nếu $\mathbf{X}^T \mathbf{X}$ không khả nghịch, không dùng được công thức inverse trực tiếp. Khi đó cần pseudo-inverse, regularization, hoặc loại bỏ đặc trưng phụ thuộc tuyến tính.

7. Bảng Công thức Thi: Matrix OLS

Nhiệm vụ	Công thức	Cách dùng khi làm bài
Mô hình xác suất	$y_i \mid \mathbf{x}_i \sim \mathcal{N}(\beta^T \mathbf{x}_i, \sigma^2)$	Nhận ra OLS từ giả định nhiều Gaussian.
Likelihood	$L = \prod_i p(y_i \mid \mathbf{x}_i; \beta)$	Mẫu độc lập nên xác suất nhân lại.
Log-likelihood	$\ell = \log L$	Biến tích thành tổng, dễ đạo hàm.
Loss ma trận	$L = \frac{1}{2} \ \mathbf{y} - \mathbf{X}\beta\ _2^2$	Dạng ngắn của RSS.
Gradient	$\nabla L = -\mathbf{X}^T \mathbf{y} + \mathbf{X}^T \mathbf{X} \beta$	Đặt bằng 0 để tìm nghiệm đóng.
Normal equations	$\mathbf{X}^T \mathbf{X} \beta = \mathbf{X}^T \mathbf{y}$	Hệ tuyến tính cần giải.
Nghiệm đóng	$\beta = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$	Dùng khi ma trận khả nghịch.
Pseudoinverse	$\beta = \mathbf{X}^+ \mathbf{y}$	Dùng khi $\mathbf{X}^T \mathbf{X}$ suy biến hoặc gần suy biến.

Quy trình thay số:

1. Thêm cột 1 nếu cần intercept.
2. Tính $\mathbf{G} = \mathbf{X}^T \mathbf{X}$.
3. Tính $\mathbf{h} = \mathbf{X}^T \mathbf{y}$.
4. Giải $\mathbf{G}\beta = \mathbf{h}$.
5. Dự đoán bằng $\hat{\mathbf{y}} = \mathbf{X}\beta$.
6. Kiểm tra $\mathbf{X}^T (\mathbf{y} - \hat{\mathbf{y}}) \approx \mathbf{0}$.

Dạng bài chứng minh: Gaussian MLE dẫn tới Normal Equations

Khi đề yêu cầu chứng minh OLS từ giả định Gaussian, hãy viết theo chuỗi sau:

Giả định:

$$y_i = \beta^T \mathbf{x}_i + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

Suy ra:

$$p(y_i | \mathbf{x}_i; \beta) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y_i - \beta^T \mathbf{x}_i)^2}{2\sigma^2}\right)$$

Log-likelihood toàn bộ dữ liệu:

$$\ell(\beta) = C - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \beta^T \mathbf{x}_i)^2$$

Trong đó C không phụ thuộc β . Vì vậy:

$$\arg \max_{\beta} \ell(\beta) = \arg \min_{\beta} \sum_{i=1}^n (y_i - \beta^T \mathbf{x}_i)^2$$

Dạng ma trận:

$$L(\beta) = \frac{1}{2} \|\mathbf{y} - \mathbf{X}\beta\|_2^2$$

Gradient:

$$\nabla L = -\mathbf{X}^T(\mathbf{y} - \mathbf{X}\beta)$$

Đặt bằng 0:

$$\mathbf{X}^T \mathbf{X} \beta = \mathbf{X}^T \mathbf{y}$$

Đây là Normal Equations.

8. Bộ bài tập mẫu có lời giải

Bài 1: Giải OLS bằng ma trận với hai điểm

Đề bài. Fit đường thẳng có intercept qua hai điểm $(x, y) = (0, 1)$ và $(1, 3)$ bằng công thức ma trận.

Lời giải.

Thiết kế ma trận:

$$\mathbf{X} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 1 \\ 3 \end{pmatrix}$$

Tính:

$$\mathbf{X}^T \mathbf{X} = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}, \quad \mathbf{X}^T \mathbf{y} = \begin{pmatrix} 4 \\ 3 \end{pmatrix}$$

Normal equations:

$$\begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix} = \begin{pmatrix} 4 \\ 3 \end{pmatrix}$$

Từ phương trình thứ hai: $\beta_0 + \beta_1 = 3$. Từ phương trình thứ nhất: $2\beta_0 + \beta_1 = 4$. Trừ hai phương trình:

$$\beta_0 = 1, \quad \beta_1 = 2$$

Vậy:

$$\boxed{\hat{y} = 1 + 2x}$$

Bài 2: Bộ dữ liệu 10 dòng, giải đầy đủ bằng matrix OLS

Đề bài. Cho dữ liệu học tập sau. Mô hình có dạng $\hat{y} = \beta_0 + \beta_1 h + \beta_2 q$, trong đó h là số giờ học, q là số quiz đã làm. Hãy tính hệ số OLS và dự đoán điểm cho sinh viên học $h = 6$ giờ, làm $q = 5$ quiz.

dòng	h	q	y
1	1	1	52
2	2	1	57
3	2	2	60
4	3	2	65
5	4	3	71
6	5	3	76
7	5	4	79
8	6	4	84
9	7	5	90
10	8	5	95

Lời giải bằng công thức. Tạo:

$$\mathbf{X} = \begin{pmatrix} 1 & h_1 & q_1 \\ 1 & h_2 & q_2 \\ \vdots & \vdots & \vdots \\ 1 & h_{10} & q_{10} \end{pmatrix}, \quad \beta = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Từ bảng dữ liệu, ta có:

$$\mathbf{X}^T \mathbf{X} = \begin{pmatrix} 10 & 43 & 30 \\ 43 & 233 & 159 \\ 30 & 159 & 110 \end{pmatrix}$$

và

$$\mathbf{X}^T \mathbf{y} = \begin{pmatrix} 729 \\ 3434 \\ 2377 \end{pmatrix}$$

Do đó normal equations là:

$$\begin{pmatrix} 10 & 43 & 30 \\ 43 & 233 & 159 \\ 30 & 159 & 110 \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{pmatrix} = \begin{pmatrix} 729 \\ 3434 \\ 2377 \end{pmatrix}$$

Giải hệ tuyến tính này cho:

$$\beta \approx \begin{pmatrix} 45.323 \\ 4.613 \\ 2.581 \end{pmatrix}$$

Vậy mô hình là:

$$\hat{y} = 45.323 + 4.613h + 2.581q$$

Dự đoán cho $h = 6, q = 5$:

$$\hat{y} = 45.323 + 4.613(6) + 2.581(5)$$

$$\hat{y} = 45.323 + 27.678 + 12.905 = 85.906$$

Cách đọc kết quả. Hệ số $\beta_1 \approx 4.613$ cho biết khi giữ số quiz không đổi, học thêm một giờ làm điểm dự đoán tăng trung bình khoảng 4.613. Hệ số $\beta_2 \approx 2.581$ cho biết khi giữ số giờ học không đổi, làm thêm một quiz làm điểm dự đoán tăng trung bình khoảng 2.581.

Kết luận. Dự đoán điểm cho sinh viên học 6 giờ và làm 5 quiz là khoảng 85.91.

Phần 3: Học máy: Phân loại và Tối ưu Hồi quy Logistic

Nguồn: `quarto/Classification_Logistic_Regression.qmd`

1. Bản chất của Bài toán Phân loại (Classification)

Bản đồ ký hiệu và luồng tính

Ký hiệu	Nghĩa	Cách dùng khi thay số
$\mathbf{x}^{(i)}$	Vector đặc trưng của mẫu thứ i	Nhân với vector hệ số.
β	Vector trọng số, gồm cả bias nếu đã thêm cột 1	Thứ được học/cập nhật.
$z^{(i)}$	Điểm tuyến tính trước sigmoid	$z^{(i)} = \beta^T \mathbf{x}^{(i)}$.
p_i hoặc $\hat{y}^{(i)}$	Xác suất dự đoán lớp 1	$p_i = \sigma(z^{(i)})$.
$y^{(i)}$	Nhãn thật, 0 hoặc 1	So với p_i để tính loss/gradient.
J	Cross-entropy loss	Hàm cần tối thiểu hóa.
∇J	Gradient	Dùng trong cập nhật $\beta \leftarrow \beta - \alpha \nabla J$.

Luồng làm bài: tính z , đưa qua sigmoid để có p , áp ngưỡng để dự đoán lớp, tính loss, rồi nếu cần cập nhật thì dùng sai số $p - y$ nhân với đặc trưng.

Trong bài toán phân loại nhị phân (Binary Classification), tập dữ liệu huấn luyện bao gồm n mẫu có dạng $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, trong đó nhãn mục tiêu bị giới hạn trong hai giá trị rời rạc:

$$y^{(i)} \in \{0, 1\}$$

- $y^{(i)} = 1$: Lớp dương tính (Positive class).
- $y^{(i)} = 0$: Lớp âm tính (Negative class).

Tại sao Tuyến tính (Linear Regression) thất bại với Phân loại?

Nếu cố chấp sử dụng mô hình tuyến tính $f_{\beta}(\mathbf{x}) = \beta^T \mathbf{x}$ cho tác vụ phân loại, đường ranh giới quyết định sẽ bị ảnh hưởng nghiêm trọng bởi các điểm dữ liệu dị biệt (Outliers). Khi xuất hiện một điểm dữ liệu nằm rất xa nhưng mang nhãn cực đoan, việc tối thiểu hóa bình phương

khoảng cách (MSE) sẽ ép đường thẳng tuyến tính phải xoay một góc rất lớn để bù trừ sai số. Hệ quả là đường ranh giới bị dịch chuyển một cách vô lý, trực tiếp gây ra phân loại sai cho các điểm dữ liệu bình thường ở gần.

Do đó, ta cần một hàm kích hoạt có khả năng ép toàn bộ không gian số thực về khoảng xác suất $(0, 1)$, đó chính là **Hàm số Sigmoid**.

2. Mô hình Logistic Regression & Hàm Sigmoid

Logistic Regression mô hình hóa **xác suất có điều kiện** để một mẫu dữ liệu được phân loại vào lớp 1:

$$P(y^{(i)} = 1 \mid \mathbf{x}^{(i)}) = f_{\beta}(\mathbf{x}^{(i)}) = \sigma(\beta^T \mathbf{x}^{(i)})$$

Hàm số kích hoạt Sigmoid $\sigma(z)$ được định nghĩa toán học là:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

Biểu thức tính xác suất cho từng trường hợp nhãn cụ thể của mẫu dữ liệu: * $P(y^{(i)} = 1 \mid \mathbf{x}^{(i)}; \beta) = f_{\beta}(\mathbf{x}^{(i)})$ * $P(y^{(i)} = 0 \mid \mathbf{x}^{(i)}; \beta) = 1 - f_{\beta}(\mathbf{x}^{(i)})$

Công thức Xác suất Hợp nhất

Để thuận tiện cho việc thiết lập hàm toán học tối ưu, hai biểu thức trên được gộp lại thành một phương trình duy nhất dưới dạng tích lũy thừa nhờ tính chất nhị phân của nhãn $y^{(i)} \in \{0, 1\}$:

$$P(y^{(i)} \mid \mathbf{x}^{(i)}; \beta) = [f_{\beta}(\mathbf{x}^{(i)})]^{y^{(i)}} [1 - f_{\beta}(\mathbf{x}^{(i)})]^{1-y^{(i)}}$$

3. Chứng minh Hàm Mất mát Cross-Entropy bằng MLE

Với giả định các mẫu dữ liệu trong tập huấn luyện độc lập và phân phối cùng quy luật (i.i.d.), hàm lực lượng liên hợp xác suất (Likelihood Function) được thiết lập bằng tích xác suất của tất cả các mẫu:

$$L(\beta) = \prod_{i=1}^n P(y^{(i)} | \mathbf{x}^{(i)}; \beta) = \prod_{i=1}^n [f_{\beta}(\mathbf{x}^{(i)})]^{y^{(i)}} [1 - f_{\beta}(\mathbf{x}^{(i)})]^{1-y^{(i)}}$$

Theo nguyên lý ước lượng hợp lý cực đại (Maximum Likelihood Estimation - MLE), ta cần tìm bộ tham số β sao cho giá trị $L(\beta)$ đạt cực đại. Vì hàm logarit là hàm đồng biến đơn điệu, bài toán tương đương với việc tối đa hóa hàm Log-Likelihood:

$$\max_{\beta} L(\beta) \iff \max_{\beta} \log L(\beta) \iff \min_{\beta} -\log L(\beta)$$

Tiến hành lấy logarit tự nhiên và đổi dấu để chuyển sang bài toán tối thiểu hóa hàm mất mát:

$$J(\beta) = -\log L(\beta) = -\sum_{i=1}^n \log \left([f_{\beta}(\mathbf{x}^{(i)})]^{y^{(i)}} [1 - f_{\beta}(\mathbf{x}^{(i)})]^{1-y^{(i)}} \right)$$

Sử dụng tính chất của logarit để đưa số mũ xuống thành hệ số, ta thu được hàm mất mát **Binary Cross-Entropy Loss**:

$$J(\beta) = -\sum_{i=1}^n [y^{(i)} \log f_{\beta}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log(1 - f_{\beta}(\mathbf{x}^{(i)}))]$$

4. Chứng minh Tính Lồi (Convex Optimization) bằng Ma trận Hessian

Để các thuật toán tối ưu hóa dựa trên đạo hàm hoạt động tin cậy và không bị kẹt ở các điểm cực tiểu cục bộ, hàm mất mát $J(\beta)$ phải là một hàm lồi. Điều này đồng nghĩa với việc ma trận đạo hàm bậc hai (Hessian Matrix) phải luôn bán xác định dương ($\nabla^2 J(\beta) \succeq 0$) với mọi giá trị của β .

Đạo hàm Bậc một (Gradient) via Chain Rule

Đặt biến trung gian tuyến tính $z^{(i)} = \beta^T \mathbf{x}^{(i)}$ và giá trị dự đoán $\hat{y}^{(i)} = \sigma(z^{(i)}) = f_\beta(\mathbf{x}^{(i)})$. Áp dụng quy tắc chuỗi (Chain Rule) để tính đạo hàm riêng cho từng mẫu dữ liệu:

$$\frac{\partial J^{(i)}(\beta)}{\partial \beta} = \frac{\partial J^{(i)}}{\partial \hat{y}^{(i)}} \cdot \frac{\partial \hat{y}^{(i)}}{\partial z^{(i)}} \cdot \frac{\partial z^{(i)}}{\partial \beta}$$

Tính toán chi tiết từng thành phần vi phân:

1. **Thành phần Loss:** $\frac{\partial J^{(i)}}{\partial \hat{y}^{(i)}} = \frac{-y^{(i)}}{\hat{y}^{(i)}} + \frac{1-y^{(i)}}{1-\hat{y}^{(i)}} = \frac{\hat{y}^{(i)} - y^{(i)}}{\hat{y}^{(i)}(1-\hat{y}^{(i)})}$
2. **Thành phần Sigmoid:** $\frac{\partial \hat{y}^{(i)}}{\partial z^{(i)}} = \sigma(z^{(i)})(1 - \sigma(z^{(i)})) = \hat{y}^{(i)}(1 - \hat{y}^{(i)})$
3. **Thành phần Tuyến tính:** $\frac{\partial z^{(i)}}{\partial \beta} = \mathbf{x}^{(i)}$

Nhân kết hợp cả ba thành phần lại với nhau để triệt tiêu đại lượng mẫu số:

$$\frac{\partial J^{(i)}(\beta)}{\partial \beta} = \left[\frac{\hat{y}^{(i)} - y^{(i)}}{\hat{y}^{(i)}(1 - \hat{y}^{(i)})} \right] \cdot [\hat{y}^{(i)}(1 - \hat{y}^{(i)})] \cdot \mathbf{x}^{(i)} = (\hat{y}^{(i)} - y^{(i)})\mathbf{x}^{(i)}$$

Đạo hàm Bậc hai (Ma trận Hessian)

Tiếp tục lấy đạo hàm bậc hai bằng cách biệt hóa toán tử Gradient thu được theo vector tham số β^T :

$$H = \nabla^2 J(\beta) = \frac{\partial^2 J(\beta)}{\partial \beta \partial \beta^T} = \sum_{i=1}^n \frac{\partial}{\partial \beta} [(\hat{y}^{(i)} - y^{(i)})\mathbf{x}^{(i)}]$$

Vì nhãn dữ liệu thực tế $y^{(i)}$ là một hằng số cố định và ta đã biết $\frac{\partial \hat{y}^{(i)}}{\partial \beta} = \hat{y}^{(i)}(1 - \hat{y}^{(i)})\mathbf{x}^{(i)}$, biểu thức ma trận Hessian tổng thể rút gọn thành:

$$H = \sum_{i=1}^n \hat{y}^{(i)}(1 - \hat{y}^{(i)})\mathbf{x}^{(i)}(\mathbf{x}^{(i)})^T$$

Biện luận tính lồi: * Vì đầu ra hàm xác suất Sigmoid luôn nằm trong biên hạn hẹp $\hat{y}^{(i)} \in (0, 1)$, hệ số trọng số luôn dương: $\hat{y}^{(i)}(1 - \hat{y}^{(i)}) > 0$. * Bản chất phép nhân ngoài của một vector với chính nó dưới dạng $\mathbf{xx}^T$ luôn tạo dựng nên một ma trận bán xác định dương.

Do tổng các ma trận bán xác định dương là một ma trận bán xác định dương, ta kết luận $H \succeq 0 \forall \beta$. Hàm mục tiêu của Logistic Regression **hoàn toàn là hàm lồi (Convex Function)**.

5. Các Phương pháp Tối ưu hóa Toán học

Thuật toán Gradient Descent (Tối ưu bậc 1)

Thực hiện cập nhật véc-tơ trọng số lặp bằng cách đi ngược hướng với vector độ dốc Gradient hệ số học α :

$$\beta^{(t+1)} = \beta^{(t)} - \alpha \nabla J(\beta^{(t)}) = \beta^{(t)} + \alpha \sum_{i=1}^n (y^{(i)} - f_{\beta}(\mathbf{x}^{(i)})) \mathbf{x}^{(i)}$$

Công thức cập nhật chi tiết cho từng thành phần chỉ số tham số β_j trong không gian đa chiều:

$$\beta_j^{(t+1)} = \beta_j^{(t)} + \alpha \sum_{i=1}^n (y^{(i)} - f_{\beta}(\mathbf{x}^{(i)})) x_j^{(i)}$$

Phương pháp Newton - Raphson (Tối ưu bậc 2)

Để tìm điểm cực trị của hàm lỗi bằng cách giải hệ phương trình đạo hàm bằng không $\nabla J(\beta) = 0$, phương pháp Newton sử dụng khai triển Taylor bậc hai nhằm mượn thêm thông tin độ cong từ ma trận Hessian.

Công thức cập nhật đa biến sử dụng ma trận nghịch đảo Hessian tại mỗi bước lặp:

$$\beta^{(t+1)} = \beta^{(t)} - H^{-1} \nabla J(\beta^{(t)})$$

Dạng ma trận dùng trong bài thi Newton.

Với ma trận thiết kế $\mathbf{X} \in \mathbb{R}^{n \times p}$, vector xác suất $\mathbf{p} = \sigma(\mathbf{X}\beta)$, và vector nhãn $\mathbf{y}$:

$$\nabla J(\beta) = \mathbf{X}^T (\mathbf{p} - \mathbf{y})$$

Đặt ma trận đường chéo:

$$\mathbf{R} = \text{diag}(p_1(1-p_1), p_2(1-p_2), \dots, p_n(1-p_n))$$

thì Hessian là:

$$\nabla^2 J(\beta) = \mathbf{X}^T \mathbf{R} \mathbf{X}$$

và bước Newton có dạng:

$$\beta_{new} = \beta - (\mathbf{X}^T \mathbf{R} \mathbf{X})^{-1} \mathbf{X}^T (\mathbf{p} - \mathbf{y})$$

Để kiểm tra tính lồi, lấy bất kỳ vector $\mathbf{v}$:

$$\mathbf{v}^T \nabla^2 J \mathbf{v} = \mathbf{v}^T \mathbf{X}^T \mathbf{R} \mathbf{X} \mathbf{v} = (\mathbf{X} \mathbf{v})^T \mathbf{R} (\mathbf{X} \mathbf{v}) \geq 0$$

vì mọi phần tử đường chéo $p_i(1 - p_i)$ đều không âm. Đây là lý do Logistic Regression với cross-entropy có nghiệm tối ưu toàn cục, còn Newton's Method có thể tận dụng độ cong để đi rất nhanh gần nghiệm.

So sánh đặc tính vận hành:

Thuật toán Tối ưu	Cơ chế Đạo hàm	Ưu điểm Vận hành	Nhược điểm Khuyết góc
Gradient Descent	Chỉ sử dụng bậc một (∇J)	Chi phí tính toán trên mỗi bước lặp rất thấp ($\mathcal{O}(p)$). Phù hợp với không gian dữ liệu lớn chứa hàng triệu thuộc tính p .	Tốc độ hội tụ chậm (Linear Convergence). Tốn nhiều thời gian thử nghiệm để tìm ra hệ số học α phù hợp.
Newton's Method	Kết hợp bậc một và ma trận bậc hai ($H^{-1} \nabla J$)	Tốc độ hội tụ cực kỳ nhanh nhờ đặc tính hội tụ bậc hai (Quadratic Convergence). Số vòng lặp yêu cầu rất ít và không cần tùy chỉnh hyperparameter hệ số học α .	Chi phí tính toán ma trận nghịch đảo Hessian rất lớn ($\mathcal{O}(p^3)$) tại mỗi bước lặp. Hoàn toàn bất khả thi nếu số lượng thuộc tính p lớn.

6. Kiểm tra Thực thi: Sigmoid và Ranh giới Quyết định

Các công thức phía trên sẽ dễ nhớ hơn nếu gắn với hai trục giác hình học: sigmoid biến một điểm số thực thành xác suất, và ngưỡng $P(y = 1 \mid \mathbf{x}) = 0.5$ tạo ra ranh giới quyết định tuyến tính.

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

np.random.seed(7)
n = 120
X0 = np.random.normal(loc=[-1.2, -0.8], scale=[0.55, 0.55], size=(n // 2, 2))
X1 = np.random.normal(loc=[1.2, 1.0], scale=[0.65, 0.65], size=(n // 2, 2))
X = np.vstack([X0, X1])
y = np.hstack([np.zeros(n // 2), np.ones(n // 2)])
X_design = np.c_[np.ones(n), X]

beta = np.zeros(3)
lr = 0.4
for _ in range(800):
    probs = sigmoid(X_design @ beta)
    grad = X_design.T @ (y - probs) / n
    beta += lr * grad

pred = (sigmoid(X_design @ beta) >= 0.5).astype(int)
accuracy = (pred == y).mean()
print("Beta học được:", np.round(beta, 3))
print("Độ chính xác huấn luyện:", round(float(accuracy), 3))

z = np.linspace(-8, 8, 300)
xx, yy = np.meshgrid(np.linspace(-3, 3, 160), np.linspace(-3, 3, 160))
grid_design = np.c_[np.ones(xx.size), xx.ravel(), yy.ravel()]
surface = sigmoid(grid_design @ beta).reshape(xx.shape)

fig, ax = plt.subplots(1, 2, figsize=(9, 3.8))
ax[0].plot(z, sigmoid(z), color="#4C78A8")
ax[0].axhline(0.5, color="black", linestyle="--", linewidth=1)
ax[0].axvline(0.0, color="black", linestyle="--", linewidth=1)
ax[0].set_title("Sigmoid biến điểm số thành xác suất")
ax[0].set_xlabel("điểm tuyến tính z")
```

```

ax[0].set_ylabel(" $\sigma(z)$ ")
ax[0].grid(alpha=0.25)

contour = ax[1].contourf(xx, yy, surface, levels=20, cmap="RdBu_r",
    ↪ alpha=0.75)
ax[1].scatter(X0[:, 0], X0[:, 1], label="class 0", edgecolor="black",
    ↪ alpha=0.85)
ax[1].scatter(X1[:, 0], X1[:, 1], label="class 1", edgecolor="black",
    ↪ alpha=0.85)
ax[1].contour(xx, yy, surface, levels=[0.5], colors="black", linewidths=2)
ax[1].set_title("Ranh giới xác suất học được")
ax[1].set_xlabel(" $x_1$ ")
ax[1].set_ylabel(" $x_2$ ")
ax[1].legend()
fig.colorbar(contour, ax=ax[1], label=" $P(y=1 \mid x)$ ")
plt.tight_layout()
plt.show()

```

Beta học được: [-0.101 4.325 3.452]

Độ chính xác huấn luyện: 1.0

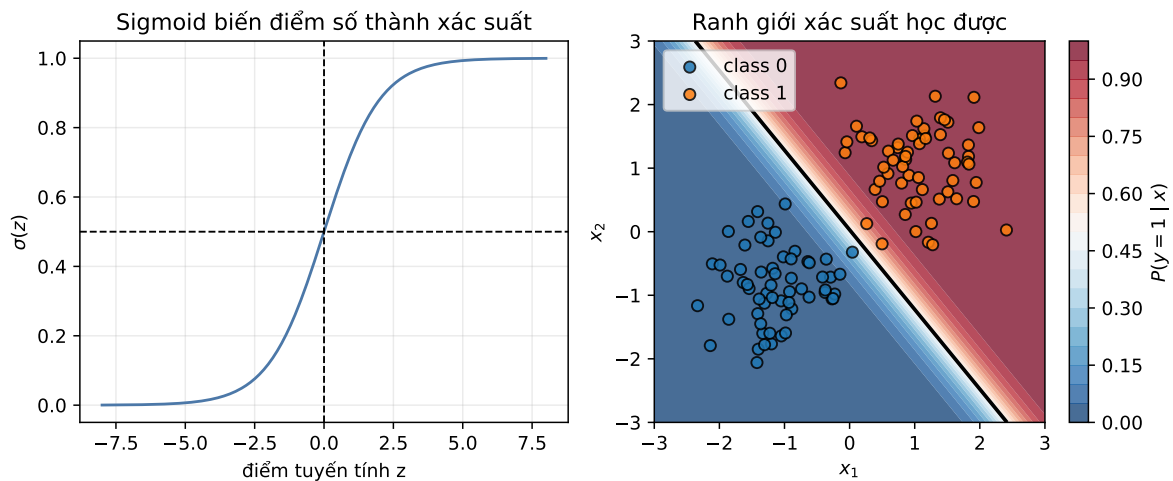


Figure 3: Logistic Regression ánh xạ điểm tuyến tính qua sigmoid và học ranh giới xác suất tuyến tính.

7. Quy trình giải bài Logistic Regression

Một bài Logistic Regression thường chỉ xoay quanh bốn thao tác:

1. Tính điểm tuyến tính

$$z_i = \beta^T \mathbf{x}_i$$

2. Đổi thành xác suất

$$p_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}$$

3. Tính loss

$$J_i = -y_i \log p_i - (1 - y_i) \log(1 - p_i)$$

4. Tính gradient hoặc cập nhật

$$\nabla J = \frac{1}{n} \mathbf{X}^T (\mathbf{p} - \mathbf{y}), \quad \beta_{new} = \beta - \alpha \nabla J$$

Bẫy thường gặp: nếu đang tối thiểu hóa cross-entropy thì dùng $(p - y)$; nếu đang cực đại hóa log-likelihood thì dấu đổi thành $(y - p)$. Hai cách viết đúng nhưng cập nhật phải nhất quán với mục tiêu.

8. Bảng Công thức Thi: Quy trình Thay số cho Logistic Regression

Dùng phần này khi đề bài nhắc đến “phân loại nhị phân”, “xác suất thuộc lớp 1”, “log odds”, hoặc “cross-entropy”.

Nhiệm vụ	Công thức	Cách dùng
Điểm tuyến tính	$z^{(i)} = \beta^T \mathbf{x}^{(i)}$	Nén toàn bộ đặc trưng thành một số thực.
Xác suất	$\hat{y}^{(i)} = \sigma(z^{(i)}) = \frac{1}{1 + e^{-z^{(i)}}}$	Biến điểm số bất kỳ thành xác suất trong $(0, 1)$.
Luật quyết định	$\hat{y}^{(i)} \geq 0.5 \Rightarrow \text{lớp 1}$	Tương đương $z^{(i)} \geq 0$ nếu ngưỡng là 0.5.

Nhiệm vụ	Công thức	Cách dùng
Loss một mẫu	$J^{(i)} = -y^{(i)} \log \hat{y}^{(i)} - (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$	Phạt rất nặng khi mô hình tự tin nhưng sai.
Gradient	$\nabla J = \sum_i (\hat{y}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}$	Sai số dự đoán nhân với vector đặc trưng.
Hessian	$H = \sum_i \hat{y}^{(i)} (1 - \hat{y}^{(i)}) \mathbf{x}^{(i)} (\mathbf{x}^{(i)})^T$	Bán xác định dương, nên hàm mục tiêu lồi.
Cập nhật Gradient Descent	$\beta \leftarrow \beta - \alpha \nabla J$	Đi ngược chiều gradient của cross-entropy.

Bẫy thường gặp khi thi.

- Logistic Regression là mô hình tuyến tính theo không gian đặc trưng, nhưng xác suất đầu ra phi tuyến vì có sigmoid.
- Không mặc định dùng MSE để suy luận Logistic Regression; cross-entropy xuất phát từ Bernoulli MLE.
- Dấu của $z = \beta^T \mathbf{x}$ quyết định lớp khi ngưỡng là 0.5.
- Độ lớn $|z|$ thể hiện mức tự tin: dương lớn nghĩa là gần lớp 1, âm lớn nghĩa là gần lớp 0.

```
import numpy as np

beta = np.array([-0.4, 1.2, -0.8])
X_demo = np.array([
    [1.0, 0.0, 0.0],
    [1.0, 2.0, 1.0],
    [1.0, -1.0, 2.0],
])
y_demo = np.array([0, 1, 0])
scores = X_demo @ beta
probs = 1 / (1 + np.exp(-scores))
losses = -y_demo * np.log(probs) - (1 - y_demo) * np.log(1 - probs)

print("Điểm z:", np.round(scores, 3))
print("Xác suất:", np.round(probs, 3))
print("Lớp dự đoán:", (probs >= 0.5).astype(int))
print("Cross-entropy từng mẫu:", np.round(losses, 3))
print("Loss trung bình:", round(float(losses.mean()), 3))
```

```
Điểm z: [-0.4  1.2 -3.2]
Xác suất: [0.401 0.769 0.039]
Lớp dự đoán: [0 1 0]
```

Cross-entropy từng mẫu: [0.513 0.263 0.04]
Loss trung bình: 0.272

Dạng bài: chứng minh MLE dẫn tới Binary Cross-Entropy

Khi đề yêu cầu “viết likelihood, log-likelihood, rồi chứng minh tương đương với cross-entropy”, hãy làm theo khung này.

Với $p_i = P(y_i = 1 \mid x_i; \beta) = \sigma(\beta^T \mathbf{x}_i)$:

$$P(y_i \mid x_i; \beta) = p_i^{y_i} (1 - p_i)^{1-y_i}$$

Likelihood toàn bộ dữ liệu:

$$L(\beta) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i}$$

Log-likelihood:

$$\ell(\beta) = \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Negative log-likelihood:

$$-\ell(\beta) = -\sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Đây chính là binary cross-entropy nếu bỏ hoặc chia thêm hệ số $\frac{1}{n}$. Vì nhân/chia bởi hằng số dương không đổi điểm tối ưu:

$$\arg \max \ell(\beta) = \arg \min [-\ell(\beta)]$$

Gradient của log-likelihood dùng cho gradient ascent:

$$\nabla \ell(\beta) = \sum_{i=1}^n (y_i - p_i) \mathbf{x}_i$$

Gradient của cross-entropy dùng cho gradient descent:

$$\nabla J(\beta) = \frac{1}{n} \sum_{i=1}^n (p_i - y_i) \mathbf{x}_i$$

Hai công thức khác nhau vì một bên tối đa hóa, một bên tối thiểu hóa.

Bộ bài tập mẫu có lời giải

Đề bài. Cho mô hình Logistic Regression có $\beta = (-0.4, 1.2, -0.8)$, trong đó phần tử đầu tiên là bias. Với mẫu $\mathbf{x} = (1, 2, 1)$ và nhãn thật $y = 1$, hãy tính xác suất dự đoán, lớp dự đoán, và loss cross-entropy.

Lời giải từng bước.

1. Tính điểm tuyến tính:

$$z = -0.4 + 1.2(2) - 0.8(1) = 1.2$$

2. Đưa qua sigmoid:

$$\hat{y} = \sigma(1.2) = \frac{1}{1 + e^{-1.2}} \approx 0.768$$

3. Vì $0.768 \geq 0.5$, mô hình dự đoán **lớp 1**.

4. Vì $y = 1$, loss là:

$$J = -\log(0.768) \approx 0.264$$

Kết luận. Mô hình dự đoán đúng lớp 1 với xác suất khoảng 76.8%, và loss thấp vì dự đoán khá tự tin.

Bài 2: Bảng 12 dòng, tính xác suất, loss, accuracy

Đề bài. Cho mô hình $\beta = (-1.0, 0.9, 0.7)$, trong đó phần tử đầu là bias. Với 12 mẫu sau, hãy tính z , xác suất, lớp dự đoán, cross-entropy trung bình, và accuracy.

dòng	x_1	x_2	y
1	0.0	0.0	0
2	0.5	0.2	0
3	1.0	0.4	0
4	1.0	1.0	1
5	1.5	0.8	1
6	2.0	0.5	1
7	-0.5	1.0	0
8	0.2	1.8	1
9	2.5	1.0	1
10	-1.0	0.5	0
11	1.2	-0.2	0
12	0.8	1.5	1

Lời giải. Mỗi dòng làm cùng một quy trình:

$$z = -1.0 + 0.9x_1 + 0.7x_2, \quad \hat{p} = \frac{1}{1 + e^{-z}}$$

Ví dụ dòng 4:

$$z = -1.0 + 0.9(1.0) + 0.7(1.0) = 0.60$$

$$\hat{p} = \sigma(0.60) = \frac{1}{1 + e^{-0.60}} \approx 0.646$$

Vì $\hat{p} \geq 0.5$, dự đoán lớp 1. Vì $y = 1$, loss dòng 4 là:

$$-\log(0.646) \approx 0.437$$

Làm tương tự cho 12 dòng:

dòng	z	$\hat{p}$	lớp dự đoán	loss
1	-1.000	0.269	0	0.313
2	-0.410	0.399	0	0.509
3	0.180	0.545	1	0.787
4	0.600	0.646	1	0.437
5	0.910	0.713	1	0.338
6	1.150	0.760	1	0.275
7	-0.750	0.321	0	0.387
8	0.440	0.608	1	0.497
9	1.950	0.875	1	0.133
10	-1.550	0.175	0	0.192
11	-0.060	0.485	0	0.664
12	0.770	0.684	1	0.380

Tổng loss xấp xỉ:

$$0.313 + 0.509 + \dots + 0.380 = 4.914$$

Cross-entropy trung bình:

$$\bar{J} = \frac{4.914}{12} \approx 0.409$$

Mô hình sai duy nhất ở dòng 3, nên accuracy:

$$\text{Accuracy} = \frac{11}{12} \approx 0.917$$

Kết luận. Bài kiểu này không cần học lại mô hình. Chỉ cần thay số vào z , qua sigmoid, đặt ngưỡng 0.5, rồi tính loss từng dòng.

Bài 3: Một bước cập nhật gradient

Đề bài. Với cùng 12 dòng ở Bài 2, từ $\beta = (-1.0, 0.9, 0.7)$, hãy tính một bước gradient descent với learning rate $\alpha = 0.1$.

Lời giải. Gradient của cross-entropy là:

$$\nabla J = \frac{1}{n} \mathbf{X}^T (\hat{\mathbf{p}} - \mathbf{y})$$

Từ bảng ở Bài 2, lấy từng sai số xác suất $\hat{p}_i - y_i$, rồi nhân với từng cột của $\mathbf{X}$. Sau khi cộng và chia cho $n = 12$, ta được:

$$\nabla J \approx \begin{pmatrix} 0.0399 \\ -0.0765 \\ -0.1166 \end{pmatrix}$$

Cập nhật:

$$\beta_{new} = \beta - 0.1 \nabla J$$

$$\beta_{new} \approx \begin{pmatrix} -1.0 \\ 0.9 \\ 0.7 \end{pmatrix} - 0.1 \begin{pmatrix} 0.0399 \\ -0.0765 \\ -0.1166 \end{pmatrix} = \begin{pmatrix} -1.0040 \\ 0.9076 \\ 0.7117 \end{pmatrix}$$

Kết luận. Nếu $\hat{p} - y$ dương, mô hình đang dự đoán xác suất lớp 1 quá cao cho dòng đó; gradient sẽ đẩy hệ số theo hướng giảm xác suất đó.

Phần 4: Học máy: Họ Hàm Mũ và Mô hình Tuyến tính Tổng quát (GLM)

Nguồn: `quarto/Exponential_Family_GLMs.qmd`

1. Họ Phân Phối Số Mũ (Exponential Family Distribution - EFD)

Bản đồ ký hiệu Exponential Family

Ký hiệu	Nghĩa	Câu hỏi cần tự hỏi
y	Biến ngẫu nhiên/nhãn được mô hình hóa	y là nhị phân, liên tục, hay đếm?
η	Natural parameter	Thường liên hệ với $\theta^T x$.
$T(y)$	Sufficient statistic	Dữ liệu đi vào likelihood qua thống kê nào?
$a(\eta)$	Log-partition function	Đảm bảo phân phối chuẩn hóa về tổng/xác suất 1.
$b(y)$	Base measure	Phần phụ thuộc vào y nhưng không phụ thuộc η .
$\mu = \mathbb{E}[T(y)]$	Kỳ vọng của sufficient statistic	Bằng $a'(\eta)$ trong dạng chuẩn.

Mẹo nhận diện: đưa phân phối về dạng $b(y) \exp(\eta T(y) - a(\eta))$, sau đó đọc ra η , $T(y)$, $a(\eta)$, $b(y)$. Với GLM, phần học máy nằm ở cách nối η với đặc trưng $\mathbf{x}$.

Một phân phối xác suất đơn biến thuộc **Họ phân phối số mũ** nếu hàm mật độ xác suất (hoặc hàm khối xác suất) của nó có thể biến đổi về dạng chuẩn tắc sau:

$$P(y | \eta) = b(y) \exp(\eta^T T(y) - a(\eta))$$

Trong đó: * y : Biến ngẫu nhiên của dữ liệu quan sát. * η : Tham số tự nhiên (Natural parameter / Canonical parameter). * $T(y)$: Thống kê đủ (Sufficient statistic) — thông thường trong các bài toán cơ bản $T(y) = y$. * $b(y)$: Thước đo cơ sở (Base measure). * $a(\eta)$: Hàm chuẩn hóa log (Log-partition function), đóng vai trò như một hằng số chuẩn hóa để đảm bảo tổng tích phân xác suất bằng 1.

Các tính chất toán học cốt lõi (Properties)

Hàm log-partition $a(\eta)$ mang thông tin cực kỳ quan trọng về các mô-men của phân phối: 1.

Kỳ vọng (Mean): Đạo hàm bậc nhất của $a(\eta)$ chính là kỳ vọng của thống kê đủ:

$$\mathbb{E}[y \mid \eta] = \frac{\partial a(\eta)}{\partial \eta}$$

2. **Phương sai (Variance):** Đạo hàm bậc hai của $a(\eta)$ chính là phương sai của thống kê đủ:

$$\text{Var}(y \mid \eta) = \frac{\partial^2 a(\eta)}{\partial \eta^2}$$

2. Chuẩn hóa các phân phối kinh điển về dạng EFD

2.1 Phân phối Bernoulli (Nền tảng của Logistic Regression)

Phân phối Bernoulli mô hình hóa biến ngẫu nhiên nhị phân $y \in \{0, 1\}$ với xác suất thành công $P(y = 1) = \phi$:

$$P(y \mid \phi) = \phi^y (1 - \phi)^{1-y}$$

Sử dụng phép biến đổi mũ và logarit ($\exp(\log(\cdot))$) để đưa về dạng chuẩn tắc:

$$P(y \mid \phi) = \exp(\log(\phi^y (1 - \phi)^{1-y}))$$

$$P(y \mid \phi) = \exp(y \log \phi + (1 - y) \log(1 - \phi))$$

$$P(y \mid \phi) = \exp(y \log \phi + \log(1 - \phi) - y \log(1 - \phi))$$

$$P(y \mid \phi) = \exp\left(y \log\left(\frac{\phi}{1 - \phi}\right) + \log(1 - \phi)\right)$$

Đối chiếu với phương trình tổng quát của Họ phân phối số mũ, ta rút ra các thành phần chuẩn tắc: * **Base measure:** $b(y) = y$ * **Sufficient statistic:** $T(y) = y$ * **Tham số tự nhiên η :**

$$\eta = \log\left(\frac{\phi}{1 - \phi}\right)$$

Nếu ta giải phương trình này để tìm ϕ theo η :

$$e^\eta = \frac{\phi}{1-\phi} \implies e^{-\eta} = \frac{1-\phi}{\phi} = \frac{1}{\phi} - 1$$

$$\phi = \frac{1}{1+e^{-\eta}} = \sigma(\eta)$$

(Đây là minh chứng tối thượng vì sao hàm số Sigmoid lại xuất hiện một cách tự nhiên trong mô hình Logistic Regression). * **Log-partition function** $a(\eta)$:

$$a(\eta) = -\log(1-\phi) = -\log\left(1 - \frac{1}{1+e^{-\eta}}\right) = \log(1+e^\eta)$$

2.2 Phân phối Gaussian (Nền tảng của Linear Regression)

Xét phân phối chuẩn Gaussian với phương sai cố định $\sigma^2 = 1$ để đơn giản hóa biểu thức đại số:

$$P(y | \mu) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}(y - \mu)^2\right)$$

Khai triển hằng đẳng thức đáng nhớ bên trong hàm mũ:

$$P(y | \mu) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}(y^2 - 2y\mu + \mu^2)\right)$$

$$P(y | \mu) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}y^2\right) \exp\left(y\mu - \frac{1}{2}\mu^2\right)$$

Đối chiếu với cấu trúc dạng chuẩn tắc EFD, ta phân rã được các thành phần: * **Base measure:** $b(y) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}y^2\right)$ * **Sufficient statistic:** $T(y) = y$ * **Tham số tự nhiên η :** $\eta = \mu$ * **Log-partition function** $a(\eta)$: $a(\eta) = \frac{1}{2}\mu^2 = \frac{1}{2}\eta^2$

3. Bản chất Tính Lỗi của bài toán MLE thuộc họ EFD

Xét tập dữ liệu gồm n mẫu độc lập i.i.d. Hàm Log-Likelihood theo tham số tự nhiên η là:

$$\ell(\eta) = \sum_{i=1}^n \log P(y^{(i)} | \eta) = \sum_{i=1}^n [\log b(y^{(i)}) + \eta^T T(y^{(i)}) - a(\eta)]$$

Để tìm điểm tối ưu, ta lấy đạo hàm bậc một (Gradient) theo η :

$$\nabla_{\eta} \ell(\eta) = \sum_{i=1}^n \left[T(y^{(i)}) - \frac{\partial a(\eta)}{\partial \eta} \right] = \sum_{i=1}^n [T(y^{(i)}) - \mathbb{E}[y | \eta]]$$

Tiếp tục lấy đạo hàm bậc hai để thu được ma trận Hessian:

$$H = \nabla_{\eta}^2 \ell(\eta) = - \sum_{i=1}^n \frac{\partial^2 a(\eta)}{\partial \eta \partial \eta^T} = - \sum_{i=1}^n \text{Var}(y | \eta)$$

Vì giá trị phương sai luôn là một số không âm (và ma trận hiệp phương sai luôn bán xác định dương), ta có:

$$\text{Var}(y | \eta) \succeq 0 \implies H \preceq 0$$

Do ma trận Hessian của hàm Log-Likelihood luôn bán xác định âm, hàm số này **chắc chắn là hàm lõm (Concave)**. Hệ quả là, hàm số đối ngẫu **Negative Log-Likelihood sẽ luôn là hàm lồi (Convex)**. Điều này đảm bảo mọi bài toán tối ưu hóa MLE thuộc Họ phân phối số mũ đều có một điểm cực trị tối ưu toàn cục duy nhất.

4. Khung Mô Hình Tuyến Tính Tổng Quát (Generalized Linear Models - GLMs)

Mô hình tuyến tính tổng quát (GLMs) là phương pháp chuẩn hóa toán học cho phép chúng ta mở rộng ranh giới dự đoán tuyến tính sang các dạng phân phối nhãn nhị phân khác nhau. Một mô hình được cấu thành từ 3 giả định nền tảng (Assumptions) sau:

1. **Phân phối mục tiêu:** Biến mục tiêu y cho trước điều kiện biến đầu vào $\mathbf{x}$ phải tuân theo một phân phối xác suất thuộc Họ phân phối số mũ (EFD) với tham số tự nhiên η :

$$y | \mathbf{x}; \beta \sim \text{Exponential Family}(\eta)$$

2. **Thành phần Tuyến tính:** Tham số tự nhiên η được giả định biến thiên tuyến tính theo các đặc trưng đầu vào thông qua tổ hợp trọng số β :

$$\eta = \beta^T \mathbf{x}$$

3. **Hàm liên kết (Link Function):** Kỳ vọng của mô hình $\mu = \mathbb{E}[y | \mathbf{x}]$ liên kết với thành phần tuyến tính thông qua một hàm số g được gọi là hàm liên kết:

$$\eta = g(\mu) = \beta^T \mathbf{x} \implies \mu = g^{-1}(\beta^T \mathbf{x})$$

Nếu ta chọn $\eta = g(\mu)$ sao cho hàm liên kết khớp trực tiếp với định nghĩa tham số tự nhiên, hàm đó được gọi là **Canonical Link Function**.

5. Bản đồ ánh xạ hệ thống GLM

Bảng dưới đây tổng hợp cách khung lý thuyết GLM kiến tạo nên các thuật toán Machine Learning quen thuộc từ các giả định phân phối khác nhau:

Phân phối toán học	Tham số tự nhiên η	Canonical Link Function $g(\mu)$	Hàm dự đoán Inverse Link $g^{-1}(\beta^T \mathbf{x})$	Thuật toán ML cấu thành
Gaussian ($\sigma^2 = 1$)	$\eta = \mu$	Identity link: $\eta = \mu$	$\hat{y} = \beta^T \mathbf{x}$	Linear Regression
Bernoulli	$\eta = \log\left(\frac{\phi}{1-\phi}\right)$	Logit link: $\eta = \log\left(\frac{\mu}{1-\mu}\right)$	$\hat{y} = \frac{1}{1+e^{-\beta^T \mathbf{x}}} = \sigma(\beta^T \mathbf{x})$	Logistic Regression

6. Giai đoạn Dự đoán và Thuật toán Tối ưu hóa Tổng quát

Giai đoạn Dự đoán (Testing Phase)

Tại giai đoạn kiểm thử với một mẫu dữ liệu mới $\mathbf{x}$, mục tiêu của chúng ta là đưa ra kỳ vọng có điều kiện của nhãn y :

$$f_{\beta}(\mathbf{x}) = \mathbb{E}[y | \mathbf{x}; \beta] = \left. \frac{\partial a(\eta)}{\partial \eta} \right|_{\eta=\beta^T \mathbf{x}} = g^{-1}(\beta^T \mathbf{x})$$

Thuật toán Cập nhật Trọng số Tổng quát (Gradient Descent)

Một điều kỳ diệu mang tính hệ thống của GLM đó là: Nếu chúng ta sử dụng *Canonical Link Function*, thì đạo hàm của hàm mất mát Negative Log-Likelihood với mọi phân phối thuộc họ EFD đều cho ra cùng một dạng toán học duy nhất!

Công thức cập nhật vector trọng số tổng quát qua Gradient Descent là:

$$\beta^{(t+1)} = \beta^{(t)} + \alpha \sum_{i=1}^n (y^{(i)} - f_{\beta}(\mathbf{x}^{(i)})) \mathbf{x}^{(i)}$$

Viết chi tiết cho từng thành phần chỉ số thuộc tính β_j :

$$\beta_j^{(t+1)} = \beta_j^{(t)} + \alpha \sum_{i=1}^n (y^{(i)} - g^{-1}(\beta^T \mathbf{x}^{(i)})) x_j^{(i)}$$

Nhận xét tối thượng: Cho dù bạn đang làm Linear Regression với phân phối chuẩn, Logistic Regression với phân phối nhị phân, hay thậm chí Poisson Regression với dữ liệu đếm, quy tắc cập nhật trọng số luôn luôn không đổi. Toàn bộ sự khác biệt về mặt bản chất mô hình đã được gói gọn hoàn hảo bên trong hàm ngược liên kết g^{-1} .

7. Thực hành Minh họa bằng Code Python

Dưới đây là đoạn mã Python giúp bạn mô phỏng lại cách một mô hình GLM (như Logistic Regression) tự động kích hoạt hàm liên kết dựa trên phân phối Bernoulli để tối ưu hóa bộ trọng số:

```
import numpy as np
import matplotlib.pyplot as plt

class GLMBernoulliLogistic:
    def __init__(self, learning_rate=0.1, iters=1000):
        self.lr = learning_rate
        self.iters = iters
        self.weights = None

    def _inverse_link_function(self, z):
        # Hàm ngược liên kết ( $g^{-1}$ ) cho phân phối Bernoulli chính là hàm
        ↪ Sigmoid
```

```

        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)

        # Vòng lặp tối ưu hóa Gradient Descent tổng quát của hệ thống GLM
        for _ in range(self.iters):
            # 1. Thành phần tuyến tính:  $\eta = \beta^T \cdot x$ 
            eta = np.dot(X, self.weights)

            # 2. Kỳ vọng mô hình qua hàm ngược liên kết:  $\mu = g^{-1}(\eta)$ 
            y_pred = self._inverse_link_function(eta)

            # 3. Cập nhật trọng số theo quy tắc tổng quát của GLM
            gradient = np.dot(X.T, (y - y_pred)) / n_samples
            self.weights += self.lr * gradient

    def predict_prob(self, X):
        eta = np.dot(X, self.weights)
        return self._inverse_link_function(eta)

# Giả lập dữ liệu nhị phân để chạy thử nghiệm mô hình
np.random.seed(42)
X_dummy = np.random.randn(100, 2)
# Tạo nhãn tuyến tính giả lập bị ép qua sigmoid
true_eta = 1.5 * X_dummy[:, 0] - 2.0 * X_dummy[:, 1]
y_dummy = (1 / (1 + np.exp(-true_eta)) > 0.5).astype(int)

# Khởi tạo và huấn luyện mô hình GLM chuẩn tắc
model = GLMBernoulliLogistic(learning_rate=0.5, iters=500)
model.fit(X_dummy, y_dummy)
print("Bộ trọng số tối ưu tìm được qua GLM:", model.weights)

xx, yy = np.meshgrid(np.linspace(-3, 3, 160), np.linspace(-3, 3, 160))
grid = np.c_[xx.ravel(), yy.ravel()]
probs = model.predict_prob(grid).reshape(xx.shape)

plt.figure(figsize=(7, 5))
contour = plt.contourf(xx, yy, probs, levels=20, cmap="RdBu_r", alpha=0.75)
plt.colorbar(contour, label="$P(y=1 \mid x)$")
plt.scatter(X_dummy[y_dummy == 0, 0], X_dummy[y_dummy == 0, 1], label="class
↪ 0", edgecolor="black", alpha=0.85)

```

```
plt.scatter(X_dummy[y_dummy == 1, 0], X_dummy[y_dummy == 1, 1], label="class
↪ 1", edgecolor="black", alpha=0.85)
plt.contour(xx, yy, probs, levels=[0.5], colors="black", linewidths=2)
plt.title("Bernoulli GLM probability surface")
plt.xlabel("$x_1$")
plt.ylabel("$x_2$")
plt.legend()
plt.grid(alpha=0.2)
plt.show()
```

Bộ trọng số tối ưu tìm được qua GLM: [4.74278036 -5.98881054]

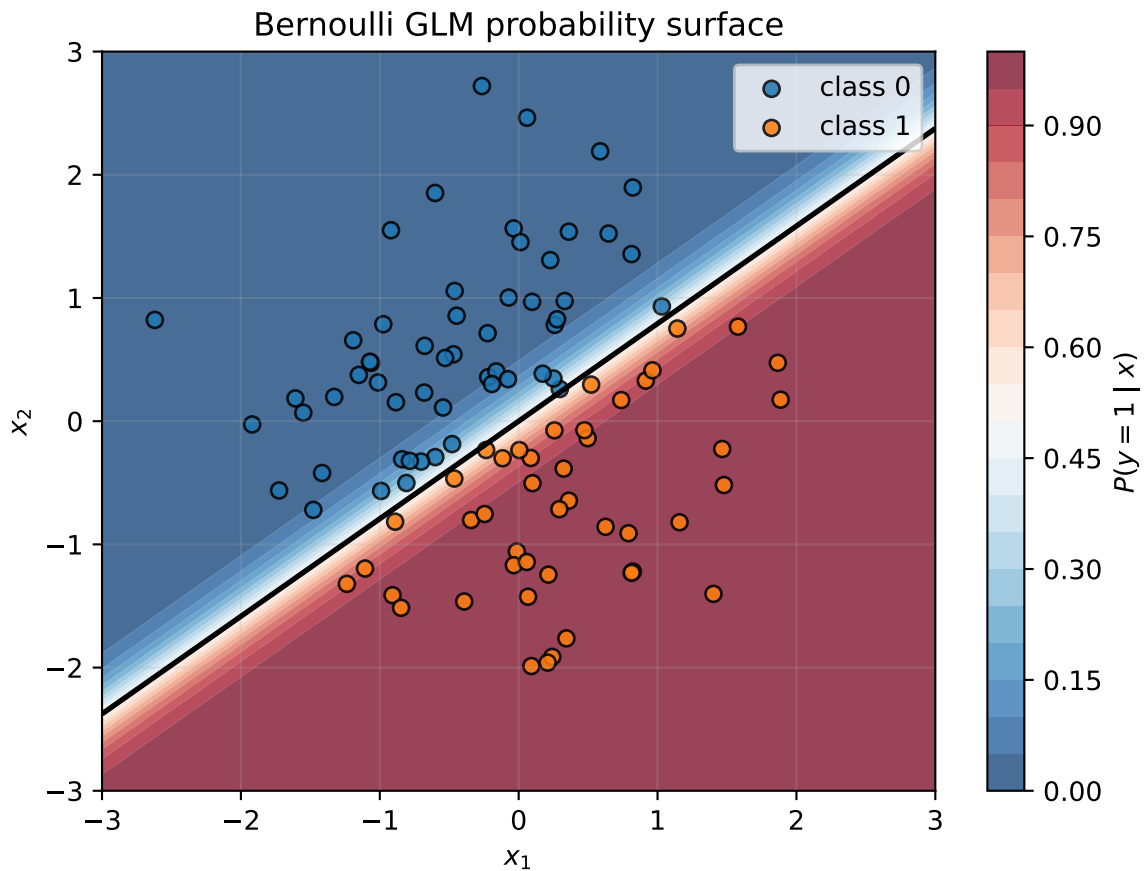


Figure 4: A Bernoulli GLM learns a logistic probability surface through the inverse link function.

8. Quy trình nhận diện Exponential Family và GLM

Khi đề yêu cầu chứng minh một phân phối thuộc Exponential Family:

1. Viết lại mật độ/xác suất dưới dạng log:

$$\log P(y | \eta) = \log b(y) + \eta^T T(y) - a(\eta)$$

2. Nhóm các phần theo ba loại:

- Phần nhân với tham số là $\eta^T T(y)$.
- Phần chỉ phụ thuộc tham số và dùng để chuẩn hóa là $a(\eta)$.
- Phần chỉ phụ thuộc dữ liệu là $b(y)$.

3. Nếu đề hỏi GLM, xác định:

$$\eta = \theta^T x, \quad \mu = \mathbb{E}[Y | X = x], \quad g(\mu) = \eta$$

4. Nếu đề hỏi tối ưu, dùng tính chất:

$$a'(\eta) = \mathbb{E}[T(Y)], \quad a''(\eta) = \text{Var}(T(Y)) \geq 0$$

Bẫy thường gặp: đừng nhầm η với μ . η là tham số tự nhiên trong không gian tuyến tính; μ là kỳ vọng ở không gian dữ liệu.

9. Bảng Công thức Thi: Chọn GLM và Thay số

Bắt đầu mọi bài GLM bằng cách xác định kiểu của biến mục tiêu.

Kiểu biến mục tiêu	Phân phối	Tham số tự nhiên / Link	Inverse Link / Dự đoán
Giá trị thực $y \in \mathbb{R}$	Gaussian	$\eta = \mu = \beta^T \mathbf{x}$	$\hat{y} = \beta^T \mathbf{x}$
Nhị phân $y \in \{0, 1\}$	Bernoulli	$\eta = \log \frac{\phi}{1-\phi}$	$\phi = \sigma(\beta^T \mathbf{x})$
Dữ liệu đếm $y \in \{0, 1, 2, \dots\}$	Poisson	$\eta = \log \lambda$	$\lambda = e^{\beta^T \mathbf{x}}$

Mẫu giải GLM tổng quát với canonical link.

1. Tính điểm tuyến tính: $\eta^{(i)} = \beta^T \mathbf{x}^{(i)}$.
2. Đổi điểm tuyến tính thành kỳ vọng: $\mu^{(i)} = g^{-1}(\eta^{(i)})$.
3. Tính sai số: $y^{(i)} - \mu^{(i)}$.
4. Cập nhật trọng số:

$$\beta \leftarrow \beta + \alpha \sum_i (y^{(i)} - \mu^{(i)}) \mathbf{x}^{(i)}$$

Mẹo nhớ: phân phối quyết định inverse link; inverse link quyết định cách dự đoán.

```
import numpy as np

eta = np.array([-2.0, 0.0, 2.0])
gaussian_mu = eta
bernoulli_mu = 1 / (1 + np.exp(-eta))
poisson_mu = np.exp(eta)

print("eta:", eta)
print("Inverse link Gaussian:", np.round(gaussian_mu, 3))
print("Inverse link Bernoulli:", np.round(bernoulli_mu, 3))
print("Inverse link Poisson:", np.round(poisson_mu, 3))
```

```
eta: [-2.  0.  2.]
Inverse link Gaussian: [-2.  0.  2.]
Inverse link Bernoulli: [0.119 0.5  0.881]
Inverse link Poisson: [0.135 1.  7.389]
```

Dạng bài: chứng minh Bernoulli sinh ra sigmoid

Với Bernoulli:

$$P(y; \phi) = \phi^y (1 - \phi)^{1-y}, \quad y \in \{0, 1\}$$

Lấy log:

$$\log P(y; \phi) = y \log \phi + (1 - y) \log(1 - \phi)$$

Biến đổi:

$$\log P(y; \phi) = y \log \frac{\phi}{1-\phi} + \log(1-\phi)$$

So với dạng Exponential Family:

$$P(y | \eta) = b(y) \exp(\eta T(y) - a(\eta))$$

ta đọc được:

$$\eta = \log \frac{\phi}{1-\phi}, \quad T(y) = y, \quad a(\eta) = \log(1 + e^\eta)$$

Từ $\eta = \log \frac{\phi}{1-\phi}$:

$$e^\eta = \frac{\phi}{1-\phi}$$

$$\phi = \frac{e^\eta}{1 + e^\eta} = \frac{1}{1 + e^{-\eta}} = \sigma(\eta)$$

Vì GLM đặt $\eta = \theta^T x$, ta có:

$$P(y = 1 | x) = \sigma(\theta^T x)$$

Đây chính là Logistic Regression nhìn từ Exponential Family.

Bộ bài tập mẫu có lời giải

Đề bài. Cho $\beta = (0.5, 1.0, -0.25)$ và $\mathbf{x} = (1, 2, 4)$, trong đó phần tử đầu là bias. Tính dự đoán nếu ta dùng Gaussian GLM, Bernoulli GLM và Poisson GLM.

Lời giải từng bước.

1. Tính điểm tuyến tính:

$$\eta = \beta^T \mathbf{x} = 0.5 + 1.0(2) - 0.25(4) = 1.5$$

2. Nếu là Gaussian:

$$\hat{y} = \eta = 1.5$$

3. Nếu là Bernoulli:

$$P(y = 1 | \mathbf{x}) = \sigma(1.5) = \frac{1}{1 + e^{-1.5}} \approx 0.818$$

4. Nếu là Poisson:

$$\lambda = e^{1.5} \approx 4.482$$

Kết luận. Cùng một điểm tuyến tính $\eta = 1.5$, nhưng mỗi GLM biến nó thành dự đoán khác nhau tùy theo phân phối mục tiêu.

Bài 2: Bảng 10 dòng, chọn inverse link phù hợp

Đề bài. Cho 10 giá trị η từ mô hình tuyến tính. Hãy tính dự đoán cho ba trường hợp: Gaussian, Bernoulli, và Poisson. Sau đó giải thích khi nào dùng từng loại.

dòng	η
1	-2.0
2	-1.2
3	-0.5
4	0.0
5	0.3
6	0.8
7	1.1
8	1.6
9	2.0
10	2.5

Lời giải. Ba inverse link cần dùng:

$$\mu_{\text{Gaussian}} = \eta, \quad \mu_{\text{Bernoulli}} = \sigma(\eta) = \frac{1}{1 + e^{-\eta}}, \quad \mu_{\text{Poisson}} = e^{\eta}$$

Ví dụ với $\eta = 0.8$:

$$\mu_{\text{Gaussian}} = 0.8$$

$$\mu_{\text{Bernoulli}} = \frac{1}{1 + e^{-0.8}} \approx 0.690$$

$$\mu_{\text{Poisson}} = e^{0.8} \approx 2.226$$

Làm tương tự cho toàn bộ bảng:

dòng	η	Gaussian μ	Bernoulli $P(y = 1)$	Poisson rate
1	-2.0	-2.0	0.119	0.135
2	-1.2	-1.2	0.231	0.301
3	-0.5	-0.5	0.378	0.607
4	0.0	0.0	0.500	1.000
5	0.3	0.3	0.574	1.350
6	0.8	0.8	0.690	2.226
7	1.1	1.1	0.750	3.004
8	1.6	1.6	0.832	4.953
9	2.0	2.0	0.881	7.389
10	2.5	2.5	0.924	12.182

Kết luận.

- Gaussian dùng khi y là số thực liên tục.
- Bernoulli dùng khi $y \in \{0, 1\}$.
- Poisson dùng khi y là số đếm không âm.

Bài 3: Tính log-likelihood Bernoulli từ 10 quan sát

Đề bài. Với xác suất dự đoán của Bernoulli GLM ở Bài 2 và nhãn thật dưới đây, hãy tính log-likelihood trung bình.

dòng	1	2	3	4	5	6	7	8	9	10
y	0	0	0	0	1	1	1	1	1	1

Lời giải. Bernoulli log-likelihood từng dòng:

$$\ell_i = y_i \log(p_i) + (1 - y_i) \log(1 - p_i)$$

Với $y = 0$, log-likelihood là $\log(1 - p)$. Với $y = 1$, log-likelihood là $\log(p)$.

dòng	p	y	log-likelihood
1	0.119	0	$\log(0.881) = -0.127$
2	0.231	0	$\log(0.769) = -0.263$
3	0.378	0	$\log(0.622) = -0.474$
4	0.500	0	$\log(0.500) = -0.693$
5	0.574	1	$\log(0.574) = -0.554$

dòng	p	y	log-likelihood
6	0.690	1	$\log(0.690) = -0.371$
7	0.750	1	$\log(0.750) = -0.287$
8	0.832	1	$\log(0.832) = -0.184$
9	0.881	1	$\log(0.881) = -0.127$
10	0.924	1	$\log(0.924) = -0.079$

Tổng log-likelihood:

$$-0.127 - 0.263 - 0.474 - 0.693 - 0.554 - 0.371 - 0.287 - 0.184 - 0.127 - 0.079 = -3.160$$

Log-likelihood trung bình:

$$\bar{\ell} = \frac{-3.160}{10} = -0.316$$

Negative log-likelihood trung bình:

$$-\bar{\ell} = 0.316$$

Kết luận. Logistic Regression chính là Bernoulli GLM với logit link. Khi tối ưu cross-entropy, ta đang tối thiểu hóa negative log-likelihood.

Phần 5: Học máy: Thuật toán Học Tạo sinh

Nguồn: *quarto/GLA.qmd*

1. Triết lý Mô hình Phân biệt (Discriminative) vs. Mô hình Sinh (Generative)

Bản đồ ký hiệu cho mô hình sinh

Ký hiệu	Nghĩa	Cách dùng khi giải bài
ϕ	Prior của lớp 1, $P(y = 1)$	Đếm số mẫu lớp 1 chia tổng số mẫu.
μ_0, μ_1	Mean của từng lớp trong GDA	Lấy trung bình các vector thuộc mỗi lớp.
Σ	Hiệp phương sai chung trong GDA	Gộp sai lệch quanh mean của đúng lớp.
$P(x y)$	Likelihood sinh dữ liệu trong từng lớp	Gaussian với GDA, tích xác suất rời rạc với Naive Bayes.
$P(y x)$	Posterior để phân loại	Tỉ lệ với $P(x y)P(y)$.
α	Hệ số Laplace smoothing	Cộng vào bộ đếm để tránh xác suất bằng 0.

Quy trình chung: đếm prior, ước lượng phân phối của x trong từng lớp, nhân likelihood với prior, rồi chuẩn hóa hoặc so sánh hai điểm số chưa chuẩn hóa.

Khi giải quyết một bài toán phân loại, hai hướng tiếp cận cốt lõi bao gồm:

- **Mô hình Phân biệt (Discriminative Models):** Học trực tiếp xác suất điều kiện $P(y | \mathbf{x})$ từ dữ liệu (ví dụ: Logistic Regression). Mô hình này cố gắng tìm ra một đường ranh giới tối ưu để phân tách các lớp dữ liệu mà không cần quan tâm đến việc dữ liệu đó được sinh ra như thế nào.
- **Mô hình Sinh (Generative Models):** Thay vì học trực tiếp ranh giới, mô hình sinh đi tìm hiểu cách từng lớp dữ liệu được tạo ra một cách độc lập bằng cách mô hình hóa xác suất đồng thời (Joint Probability):

$$P(\mathbf{x}, y) = P(\mathbf{x} | y)P(y)$$

Sau khi đã học được phân phối $P(\mathbf{x} | y)$ cho từng lớp và xác suất tiên nghiệm (Prior) $P(y)$, mô hình áp dụng **Định lý Bayes** để tính xác suất hậu nghiệm (Posterior) tại giai đoạn dự đoán:

$$P(y = 1 | \mathbf{x}) = \frac{P(\mathbf{x} | y = 1)P(y = 1)}{P(\mathbf{x})}$$

Trong đó, mẫu số được khai triển bằng công thức xác suất đầy đủ:

$$P(\mathbf{x}) = P(\mathbf{x} | y = 1)P(y = 1) + P(\mathbf{x} | y = 0)P(y = 0)$$

Tùy thuộc vào bản chất của các thuộc tính đầu vào $\mathbf{x}$, ta có hai thuật toán sinh kinh điển: 1. **Thuộc tính liên tục** ($\mathbf{x} \in \mathbb{R}^p$): Gaussian Discriminant Analysis (GDA). 2. **Thuộc tính rời rạc**: Naive Bayes.

2. Phân tích Phân biệt Gaussian (Gaussian Discriminant Analysis - GDA)

GDA giả định rằng dữ liệu đầu vào ứng với mỗi nhãn cụ thể tuân theo một **Phân phối Chuẩn Đa Biến** (Multivariate Gaussian Distribution).

2.1 Nhắc lại kiến thức: Phân phối Chuẩn Đa Biến

Một vector ngẫu nhiên $\mathbf{x} \in \mathbb{R}^p$ tuân theo phân phối chuẩn đa biến với vector kỳ vọng $\mu \in \mathbb{R}^p$ và ma trận hiệp phương sai $\Sigma \in \mathbb{R}^{p \times p}$ (ký hiệu $\mathbf{x} \sim \mathcal{N}(\mu, \Sigma)$) có hàm mật độ xác suất là:

$$P(\mathbf{x}; \mu, \Sigma) = \frac{1}{(2\pi)^{p/2} |\Sigma|^{1/2}} \times \exp \left(-\frac{1}{2} (\mathbf{x} - \mu)^T \Sigma^{-1} (\mathbf{x} - \mu) \right).$$

Trong đó $|\Sigma|$ là định thức của ma trận Σ .

2.2 Các giả định nền tảng của GDA

Xét bài toán phân loại nhị phân ($y \in \{0, 1\}$). GDA áp đặt các giả định phân phối sau: * Nhãn mục tiêu y tuân theo phân phối Bernoulli với tham số ϕ :

$$y \sim \text{Bernoulli}(\phi) \implies P(y) = \phi^y (1 - \phi)^{1-y}$$

* Dữ liệu đầu vào điều kiện theo lớp $y = 0$ tuân theo phân phối chuẩn:

$$\mathbf{x} | y = 0 \sim \mathcal{N}(\mu_0, \Sigma)$$

* Dữ liệu đầu vào điều kiện theo lớp $y = 1$ tuân theo phân phối chuẩn:

$$\mathbf{x} \mid y = 1 \sim \mathcal{N}(\mu_1, \Sigma)$$

Lưu ý quan trọng: GDA giả định các lớp có vector kỳ vọng riêng biệt ($\mu_0 \neq \mu_1$) nhưng **chia sẻ chung một ma trận hiệp phương sai** Σ . Điều này giúp ranh giới quyết định toán học sau này là một đường thẳng (tuyến tính).

3. Ước lượng Tham số GDA bằng Maximum Likelihood Estimation (MLE)

Để tìm các tham số $\{\phi, \mu_0, \mu_1, \Sigma\}$, ta thiết lập hàm Log-Likelihood đồng thời trên toàn bộ n mẫu dữ liệu huấn luyện:

$$\begin{aligned} \ell(\phi, \mu_0, \mu_1, \Sigma) &= \sum_{i=1}^n \log P(\mathbf{x}^{(i)}, y^{(i)}) \\ &= \sum_{i=1}^n [\log P(\mathbf{x}^{(i)} \mid y^{(i)}) + \log P(y^{(i)})]. \end{aligned}$$

Khai triển chi tiết hàm mục tiêu tối ưu hóa:

$$\begin{aligned} \ell &= \sum_{i=1}^n \left[\log \left(\frac{1}{(2\pi)^{p/2} |\Sigma|^{1/2}} \right) \right. \\ &\quad \left. - \frac{1}{2} (\mathbf{x}^{(i)} - \mu_{y^{(i)}})^T \Sigma^{-1} (\mathbf{x}^{(i)} - \mu_{y^{(i)}}) \right. \\ &\quad \left. + y^{(i)} \log \phi + (1 - y^{(i)}) \log(1 - \phi) \right]. \end{aligned}$$

3.1 Ước lượng Tham số Tiên nghiệm ϕ

Lấy đạo hàm riêng của ℓ theo biến vô hướng ϕ và đặt bằng 0:

$$\begin{aligned} \frac{\partial \ell}{\partial \phi} &= \sum_{i=1}^n \left[\frac{y^{(i)}}{\phi} - \frac{1 - y^{(i)}}{1 - \phi} \right] \\ &= 0. \end{aligned}$$

$$\begin{aligned}
\sum_{i=1}^n \frac{y^{(i)} - \phi}{\phi(1 - \phi)} &= 0 \\
\Rightarrow \sum_{i=1}^n y^{(i)} - n\phi &= 0 \\
\Rightarrow \phi &= \frac{1}{n} \sum_{i=1}^n y^{(i)}.
\end{aligned}$$

3.2 Ước lượng các Vector Kỳ vọng μ_0 và μ_1

Để tìm μ_0 , ta chỉ quan tâm đến các số hạng chứa μ_0 (các mẫu có $y^{(i)} = 0$). Áp dụng quy tắc đạo hàm ma trận $\frac{\partial}{\partial \mathbf{z}}(\mathbf{a} - \mathbf{z})^T \mathbf{A}(\mathbf{a} - \mathbf{z}) = -2\mathbf{A}(\mathbf{a} - \mathbf{z})$:

$$\begin{aligned}
\nabla_{\mu_0} \ell &= \sum_{i:y^{(i)}=0} \Sigma^{-1}(\mathbf{x}^{(i)} - \mu_0) \\
&= \mathbf{0}.
\end{aligned}$$

$$\begin{aligned}
\Sigma^{-1} \sum_{i:y^{(i)}=0} (\mathbf{x}^{(i)} - \mu_0) &= \mathbf{0} \\
\Rightarrow \mu_0 &= \frac{\sum_{i=1}^n (1 - y^{(i)}) \mathbf{x}^{(i)}}{\sum_{i=1}^n (1 - y^{(i)})}.
\end{aligned}$$

Tương tự cho lớp 1, ta có công thức nghiệm đóng:

$$\mu_1 = \frac{\sum_{i=1}^n y^{(i)} \mathbf{x}^{(i)}}{\sum_{i=1}^n y^{(i)}}$$

3.3 Ước lượng Ma trận Hiệp phương sai Chung Σ

Bằng phép toán tối ưu hóa ma trận nâng cao, ta thu được ma trận hiệp phương sai thực nghiệm mẫu:

$$\begin{aligned}
\Sigma &= \frac{1}{n} \sum_{i=1}^n (\mathbf{x}^{(i)} - \mu_{y^{(i)}}) \\
&\quad \times (\mathbf{x}^{(i)} - \mu_{y^{(i)}})^T.
\end{aligned}$$

Với bài GDA một chiều ($p = 1$), ma trận hiệp phương sai chung trở thành **variance chung**:

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu_{y_i})^2$$

Điểm dễ sai nhất là ký hiệu μ_{y_i} . Nó không phải một mean cố định cho mọi dòng. Mỗi dòng dùng mean của đúng lớp chứa dòng đó:

Nếu dòng có nhãn	Số hạng phải cộng vào variance
$y_i = 0$	$(x_i - \mu_0)^2$
$y_i = 1$	$(x_i - \mu_1)^2$

Quy trình thay số bằng tay.

1. Tính μ_0 từ tất cả điểm thuộc lớp 0.
2. Tính μ_1 từ tất cả điểm thuộc lớp 1.
3. Lập hai tổng sai lệch bình phương:

$$SS_0 = \sum_{i:y_i=0} (x_i - \mu_0)^2, \quad SS_1 = \sum_{i:y_i=1} (x_i - \mu_1)^2$$

4. Gộp lại bằng mẫu số là tổng số điểm n , không phải $n - 1$:

$$\sigma^2 = \frac{SS_0 + SS_1}{n}$$

Ví dụ nhanh: nếu lớp 0 có $x = \{1.0, 1.5, 2.0, 2.5\}$ thì $\mu_0 = 1.75$ và:

$$SS_0 = (1.0 - 1.75)^2 + (1.5 - 1.75)^2 + (2.0 - 1.75)^2 + (2.5 - 1.75)^2 = 1.25$$

Nếu lớp 1 có $x = \{4.8, 5.2, 5.6, 6.0\}$ thì $\mu_1 = 5.40$ và:

$$SS_1 = (4.8 - 5.4)^2 + (5.2 - 5.4)^2 + (5.6 - 5.4)^2 + (6.0 - 5.4)^2 = 0.80$$

Với $n = 8$:

$$\sigma^2 = \frac{1.25 + 0.80}{8} = 0.25625$$

Đây là variance dùng tiếp trong mật độ Gaussian:

$$p(x | y = c) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu_c)^2}{2\sigma^2}\right)$$

4. Chứng minh GDA tương đương toán học với Hàm Sigmoid

Một điểm bộc phá lý thuyết đỉnh cao là: Nếu dữ liệu thỏa mãn các giả định của GDA, thì xác suất hậu nghiệm $P(y = 1 \mid \mathbf{x})$ được suy diễn ra sẽ có dạng chính xác là một **Hàm số Sigmoid**.

Biến đổi Đại số chi tiết

Áp dụng Định lý Bayes:

$$\begin{aligned} P(y = 1 \mid \mathbf{x}) &= \frac{P(\mathbf{x} \mid y = 1)P(y = 1)}{P(\mathbf{x} \mid y = 1)P(y = 1) + P(\mathbf{x} \mid y = 0)P(y = 0)} \\ &= \frac{1}{1 + \frac{P(\mathbf{x} \mid y = 0)P(y = 0)}{P(\mathbf{x} \mid y = 1)P(y = 1)}}. \end{aligned}$$

Ta viết lại biểu thức dưới dạng hàm mũ:

$$P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + \exp\left(-\log\left(\frac{P(\mathbf{x} \mid y = 1)P(y = 1)}{P(\mathbf{x} \mid y = 0)P(y = 0)}\right)\right)}.$$

Đặt giá trị bên trong hàm mũ là số hạng z :

$$\begin{aligned} z &= \log\left(\frac{P(\mathbf{x} \mid y = 1)}{P(\mathbf{x} \mid y = 0)}\right) \\ &\quad + \log\left(\frac{P(y = 1)}{P(y = 0)}\right). \end{aligned}$$

Thay hàm mật độ xác suất chuẩn đa biến vào thành phần thứ nhất của z :

$$\log\left(\frac{P(\mathbf{x} \mid y = 1)}{P(\mathbf{x} \mid y = 0)}\right) = \log\left(\frac{\frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}Q_1\right)}{\frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}Q_0\right)}\right),$$

trong đó:

$$\begin{aligned} Q_1 &= (\mathbf{x} - \mu_1)^T \Sigma^{-1} (\mathbf{x} - \mu_1), \\ Q_0 &= (\mathbf{x} - \mu_0)^T \Sigma^{-1} (\mathbf{x} - \mu_0). \end{aligned}$$

Triệt tiêu hằng số chuẩn hóa mặt trước và lấy logarit tự nhiên để triệt tiêu hàm số exp:

$$-\frac{1}{2}(\mathbf{x} - \mu_1)^T \Sigma^{-1} (\mathbf{x} - \mu_1) + \frac{1}{2}(\mathbf{x} - \mu_0)^T \Sigma^{-1} (\mathbf{x} - \mu_0).$$

Khai triển các đa thức ma trận (lưu ý rằng $\mathbf{x}^T \Sigma^{-1} \mu_1 = \mu_1^T \Sigma^{-1} \mathbf{x}$ vì đây là đại lượng vô hướng):

$$-\frac{1}{2} [\mathbf{x}^T \Sigma^{-1} \mathbf{x} - 2\mu_1^T \Sigma^{-1} \mathbf{x} + \mu_1^T \Sigma^{-1} \mu_1] + \frac{1}{2} [\mathbf{x}^T \Sigma^{-1} \mathbf{x} - 2\mu_0^T \Sigma^{-1} \mathbf{x} + \mu_0^T \Sigma^{-1} \mu_0].$$

Ta thấy số hạng bậc hai $\mathbf{x}^T \Sigma^{-1} \mathbf{x}$ bị triệt tiêu hoàn toàn (nhờ giả định chung ma trận Σ). Gom các số hạng còn lại theo biến $\mathbf{x}$:

$$(\mu_1^T \Sigma^{-1} - \mu_0^T \Sigma^{-1}) \mathbf{x} - \frac{1}{2} \mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2} \mu_0^T \Sigma^{-1} \mu_0.$$

Kết hợp với thành phần tiên nghiệm $\log\left(\frac{\phi}{1-\phi}\right)$, toàn bộ biểu thức z có cấu trúc tuyến tính hoàn hảo:

$$z = \theta^T \mathbf{x} + \theta_0$$

Trong đó: $\theta = \Sigma^{-1}(\mu_1 - \mu_0)$ $\theta_0 = -\frac{1}{2}\mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2}\mu_0^T \Sigma^{-1} \mu_0 + \log \frac{\phi}{1-\phi}$

Hệ quả là:

$$P(y = 1 | \mathbf{x}) = \frac{1}{1 + e^{-(\theta^T \mathbf{x} + \theta_0)}} = \sigma(\theta^T \mathbf{x} + \theta_0).$$

5. Thuật toán Phân loại Naive Bayes (Dữ liệu rời rạc)

Khi các đặc trưng đầu vào $\mathbf{x}$ là các biến rời rạc định danh (ví dụ: bài toán lọc thư rác từ các từ vựng xuất hiện), giả định phân phối liên tục Gauss không còn phù hợp. Ta chuyển sang sử dụng **Naive Bayes**.

5.1 Giả định Độc lập Điều kiện Naive Bayes

Nếu không có giả định đơn giản hóa, để mô hình hóa một vector thuộc tính nhị phân $\mathbf{x} \in \{0, 1\}^p$, ta cần một bảng xác suất khổng lồ chứa $2^p - 1$ tham số, điều này gây bùng nổ dữ liệu toán học.

Giả định “Ngây thơ” (Naive) đặt ra là: **Các đặc trưng đầu vào x_j độc lập với nhau khi biết trước nhãn lớp y .**

$$P(x_1, x_2, \dots, x_p | y) = \prod_{j=1}^p P(x_j | y)$$

5.2 Thiết lập các Tham số và MLE

Xét bài toán phân loại văn bản (Text Classification) với từ điển kích thước p . Đặt: $\phi_{j|y=1} = P(x_j = 1 | y = 1)$ $\phi_{j|y=0} = P(x_j = 1 | y = 0)$ $\phi_y = P(y = 1)$

Hàm Log-Likelihood đồng thời trên toàn bộ tập dữ liệu huấn luyện:

$$\ell(\phi_y, \phi_{j|y=1}, \phi_{j|y=0}) = \sum_{i=1}^n \log \left(\prod_{j=1}^p P(x_j^{(i)} | y^{(i)}) \cdot P(y^{(i)}) \right).$$

Tối ưu hóa hàm mục tiêu bằng phương pháp MLE, ta thu được các tỷ lệ đếm thực nghiệm trực quan:

$$\phi_{j|y=1} = \frac{\sum_{i=1}^n \mathbf{1}\{x_j^{(i)} = 1 \wedge y^{(i)} = 1\}}{\sum_{i=1}^n \mathbf{1}\{y^{(i)} = 1\}}$$

$$\phi_{j|y=0} = \frac{\sum_{i=1}^n \mathbf{1}\{x_j^{(i)} = 1 \wedge y^{(i)} = 0\}}{\sum_{i=1}^n \mathbf{1}\{y^{(i)} = 0\}}$$

$$\phi_y = \frac{\sum_{i=1}^n \mathbf{1}\{y^{(i)} = 1\}}{n}$$

5.3 Kỹ thuật mịn hóa Laplace (Laplace Smoothing)

Nếu một từ vựng mới chưa từng xuất hiện trong lớp 1 ở tập huấn luyện, giá trị đếm bằng 0 dẫn đến $\phi_{j|y=1} = 0$. Khi gặp văn bản mới chứa từ này, phép nhân tích chuỗi sẽ triệt tiêu toàn bộ xác suất hậu nghiệm xuống bằng 0 một cách sai lầm ($P(y = 1 | \mathbf{x}) = 0$).

Để khắc phục, kỹ thuật **Laplace Smoothing** cộng thêm 1 vào tử số và cộng thêm kích thước miền giá trị vào mẫu số:

$$\phi_{j|y=1} = \frac{\sum_{i=1}^n \mathbf{1}\{x_j^{(i)} = 1 \wedge y^{(i)} = 1\} + 1}{\sum_{i=1}^n \mathbf{1}\{y^{(i)} = 1\} + 2}$$

6. Thực hành Triển khai GDA bằng Python từ đầu (Scratch)

Đoạn mã Python dưới đây hiện thực hóa toàn bộ các bước tính toán MLE của thuật toán GDA và dự đoán xác suất hậu nghiệm:

```
import numpy as np
import matplotlib.pyplot as plt

class GaussianDiscriminantAnalysis:
    def __init__(self):
        self.phi = None
        self.mu0 = None
        self.mu1 = None
        self.sigma = None

    def fit(self, X, y):
        n_samples, n_features = X.shape

        # 1. Ước lượng tham số Bernoulli phi
        self.phi = np.mean(y)

        # 2. Phân tách các mẫu dữ liệu theo từng lớp
        X0 = X[y == 0]
        X1 = X[y == 1]

        # 3. Tính toán các vector kỳ vọng mu0 và mu1
        self.mu0 = np.mean(X0, axis=0)
        self.mu1 = np.mean(X1, axis=0)
```

```

# 4. Tính toán ma trận hiệp phương sai chung mẫu shared sigma
dev0 = X0 - self.mu0
dev1 = X1 - self.mu1

# Cộng gộp tổng bình phương độ lệch trên toàn bộ dữ liệu
shared_covariance = (np.dot(dev0.T, dev0) + np.dot(dev1.T, dev1)) /
↪ n_samples
self.sigma = shared_covariance

def _multivariate_gaussian(self, X, mu, sigma):
    # Tính toán hàm mật độ xác suất chuẩn đa biến Gauss
    p = len(mu)
    det_sigma = np.linalg.det(sigma)
    inv_sigma = np.linalg.inv(sigma)

    # Tính hằng số chuẩn hóa mật trước
    norm_const = 1.0 / ((2 * np.pi) ** (p / 2) * np.sqrt(det_sigma))

    # Tính khoảng cách Mahalanobis trong hàm mũ
    dev = X - mu
    exponent = -0.5 * np.sum(np.dot(dev, inv_sigma) * dev, axis=1)
    return norm_const * np.exp(exponent)

def predict_proba(self, X):
    # Giai đoạn dự đoán áp dụng Định lý Bayes
    px_y0 = self._multivariate_gaussian(X, self.mu0, self.sigma)
    px_y1 = self._multivariate_gaussian(X, self.mu1, self.sigma)

    # Tính xác suất đồng thời joint probability
    joint_y0 = px_y0 * (1 - self.phi)
    joint_y1 = px_y1 * self.phi

    # Chuẩn hóa posterior
    posterior_y1 = joint_y1 / (joint_y0 + joint_y1)
    return posterior_y1

# Kiểm thử thuật toán với dữ liệu giả lập mẫu
np.random.seed(42)
X_data = np.vstack([np.random.multivariate_normal([1.0, 1.0], [[1, 0], [0,
↪ 1]], 50),
                    np.random.multivariate_normal([3.0, 4.0], [[1, 0], [0,
↪ 1]], 50)])

```

```

y_data = np.hstack([np.zeros(50), np.ones(50)])

gda = GaussianDiscriminantAnalysis()
gda.fit(X_data, y_data)
print("Vector mu0 thực nghiệm tối ưu của GDA:", gda.mu0)
print("Vector mu1 thực nghiệm tối ưu của GDA:", gda.mu1)
print("Estimated shared covariance:")
print(np.round(gda.sigma, 3))

xx, yy = np.meshgrid(np.linspace(-2, 6, 160), np.linspace(-2, 7, 160))
grid = np.c_[xx.ravel(), yy.ravel()]
posterior = gda.predict_proba(grid).reshape(xx.shape)

plt.figure(figsize=(7, 5))
contour = plt.contourf(xx, yy, posterior, levels=20, cmap="RdBu_r",
    ↪ alpha=0.75)
plt.colorbar(contour, label="$P(y=1 \mid x)$")
plt.scatter(X_data[y_data == 0, 0], X_data[y_data == 0, 1], label="class 0",
    ↪ edgecolor="black", alpha=0.85)
plt.scatter(X_data[y_data == 1, 0], X_data[y_data == 1, 1], label="class 1",
    ↪ edgecolor="black", alpha=0.85)
plt.contour(xx, yy, posterior, levels=[0.5], colors="black", linewidths=2)
plt.title("GDA posterior surface and decision boundary")
plt.xlabel("$x_1$")
plt.ylabel("$x_2$")
plt.legend()
plt.grid(alpha=0.2)
plt.show()

```

```

Vector mu0 thực nghiệm tối ưu của GDA: [0.86432437 0.9279826 ]
Vector mu1 thực nghiệm tối ưu của GDA: [2.90454712 4.14006205]
Estimated shared covariance:
[[0.726 0.025]
 [0.025 0.976]]

```

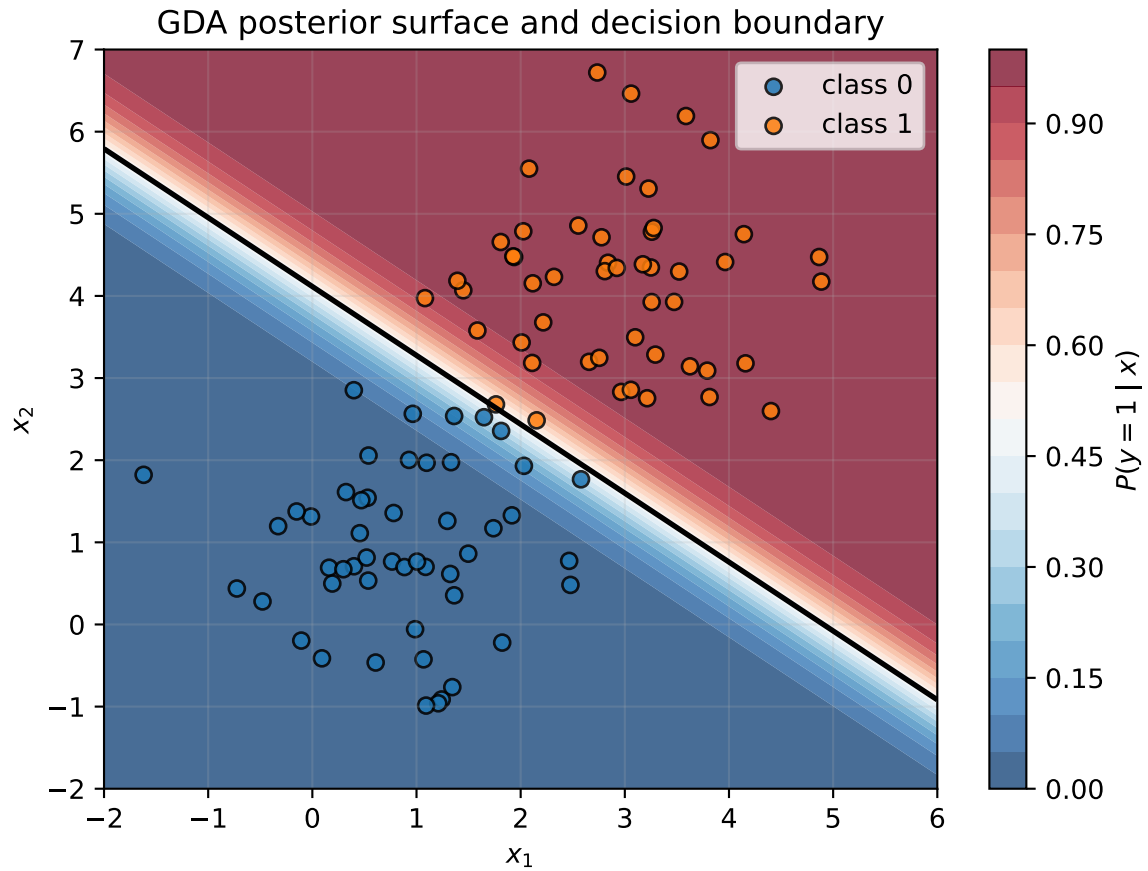


Figure 5: GDA posterior probabilities learned from two Gaussian-like classes.

7. Quy trình giải bài GDA và Naive Bayes

Với mô hình sinh, luôn tách bài thành hai phần: học phân phối và so sánh posterior.

GDA:

1. Tính prior $\phi = P(y = 1)$.
2. Tính mean từng lớp μ_0, μ_1 .
3. Tính covariance chung Σ nếu đề yêu cầu.
4. Với điểm mới, so sánh:

$$P(\mathbf{x} \mid y = 1)P(y = 1) \quad \text{và} \quad P(\mathbf{x} \mid y = 0)P(y = 0)$$

Naive Bayes:

1. Đếm prior của từng lớp.
2. Đếm likelihood từng thuộc tính theo từng lớp.
3. Nếu có xác suất 0, dùng Laplace smoothing:

$$P(x_j = v \mid y = c) = \frac{\text{count}(x_j = v, y = c) + \alpha}{\text{count}(y = c) + \alpha|\mathcal{V}_j|}$$

4. Nhân các xác suất:

$$\text{score}(c) = P(y = c) \prod_j P(x_j \mid y = c)$$

Bẫy thường gặp: không cần tính mẫu số $P(x)$ nếu chỉ so sánh lớp, vì nó giống nhau cho mọi lớp.

8. Bảng Công thức Thi: Cách Thay số cho Bộ phân loại Sinh

Mô hình sinh luôn đi theo mẫu Bayes sau:

$$P(y \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid y)P(y)}{\sum_c P(\mathbf{x} \mid y = c)P(y = c)}$$

Mô hình	Khi nào dùng	Tham số cần ước lượng	Luật dự đoán
GDA	Đặc trưng liên tục, mỗi lớp gần giống Gaussian	$\phi, \mu_0, \mu_1, \Sigma$ chung	Chọn lớp có $P(\mathbf{x} \mid y)P(y)$ lớn hơn.
Naive Bayes	Đặc trưng nhị phân/đếm/văn bản	$P(y)$ và từng $P(x_j \mid y)$	Nhân likelihood của các đặc trưng với prior.
Logistic Regression	Muốn học trực tiếp ranh giới quyết định	θ	Tính $\sigma(\theta^T \mathbf{x})$.

Công thức tham số GDA.

$$\phi = \frac{1}{n} \sum_i y^{(i)}$$

$$\mu_0 = \frac{\sum_i (1 - y^{(i)}) \mathbf{x}^{(i)}}{\sum_i (1 - y^{(i)})},$$

$$\mu_1 = \frac{\sum_i y^{(i)} \mathbf{x}^{(i)}}{\sum_i y^{(i)}}.$$

$$\Sigma = \frac{1}{n} \sum_i (\mathbf{x}^{(i)} - \mu_{y^{(i)}})(\mathbf{x}^{(i)} - \mu_{y^{(i)}})^T$$

Mẹo nhớ Naive Bayes: prior cho biết lớp đó phổ biến đến đâu; likelihood cho biết đặc trưng quan sát được phù hợp với lớp đó đến đâu.

```
px_y0 = 0.08
px_y1 = 0.24
prior_y0 = 0.60
prior_y1 = 0.40

joint0 = px_y0 * prior_y0
joint1 = px_y1 * prior_y1
posterior_y1 = joint1 / (joint0 + joint1)

print("Điểm joint cho y=0:", round(joint0, 3))
print("Điểm joint cho y=1:", round(joint1, 3))
print("P(y=1 | x):", round(posterior_y1, 3))
print("Lớp dự đoán:", int(posterior_y1 >= 0.5))
```

```
Điểm joint cho y=0: 0.048
Điểm joint cho y=1: 0.096
P(y=1 | x): 0.667
Lớp dự đoán: 1
```

Dạng bài: Naive Bayes từ bảng dữ liệu rời rạc

Với dữ liệu email có ba thuộc tính nhị phân như **promo**, **urgent**, **team**, cách làm luôn giống nhau.

Giả sử cần phân loại email mới:

promo	urgent	team
Yes	No	Yes

Bước 1: tính prior:

$$P(\text{Spam}) = \frac{\#\text{Spam}}{n},$$

$$P(\text{Not Spam}) = \frac{\#\text{Not Spam}}{n}.$$

Bước 2: tính likelihood theo từng lớp:

$$P(\text{promo}=\text{Yes} \mid \text{Spam}),$$

$$P(\text{urgent}=\text{No} \mid \text{Spam}),$$

$$P(\text{team}=\text{Yes} \mid \text{Spam}).$$

và tương tự cho lớp Not Spam.

Bước 3: nhân điểm số chưa chuẩn hóa:

$$\begin{aligned} \text{score}(\text{Spam}) &= P(\text{Spam}) \\ &\times P(\text{promo}=\text{Yes} \mid \text{Spam}) \\ &\times P(\text{urgent}=\text{No} \mid \text{Spam}) \\ &\times P(\text{team}=\text{Yes} \mid \text{Spam}). \end{aligned}$$

$$\begin{aligned} \text{score}(\text{Not Spam}) &= P(\text{Not Spam}) \\ &\times P(\text{promo}=\text{Yes} \mid \text{Not Spam}) \\ &\times P(\text{urgent}=\text{No} \mid \text{Not Spam}) \\ &\times P(\text{team}=\text{Yes} \mid \text{Not Spam}). \end{aligned}$$

Bước 4: chọn lớp có score lớn hơn. Nếu muốn xác suất chuẩn hóa:

$$P(\text{Spam} \mid x) = \frac{\text{score}(\text{Spam})}{\text{score}(\text{Spam}) + \text{score}(\text{Not Spam})}$$

Nếu một likelihood bằng 0, dùng Laplace smoothing:

$$P(x_j = v \mid y = c) = \frac{\text{count}(x_j = v, y = c) + 1}{\text{count}(y = c) + |\mathcal{V}_j|}$$

Bộ bài tập mẫu có lời giải

Đề bài. Một bộ phân loại sinh cho biết:

- $P(\mathbf{x} \mid y = 0) = 0.08, P(y = 0) = 0.60$
- $P(\mathbf{x} \mid y = 1) = 0.24, P(y = 1) = 0.40$

Hãy tính $P(y = 1 \mid \mathbf{x})$ và lớp dự đoán.

Lời giải từng bước.

1. Tính joint score cho lớp 0:

$$P(\mathbf{x} \mid y = 0)P(y = 0) = 0.08(0.60) = 0.048$$

2. Tính joint score cho lớp 1:

$$P(\mathbf{x} \mid y = 1)P(y = 1) = 0.24(0.40) = 0.096$$

3. Chuẩn hóa bằng tổng hai joint score:

$$P(y = 1 \mid \mathbf{x}) = \frac{0.096}{0.048 + 0.096} = 0.667$$

4. Vì $0.667 > 0.5$, dự đoán **lớp 1**.

Kết luận. Dù prior của lớp 1 thấp hơn, likelihood của $\mathbf{x}$ dưới lớp 1 cao hơn đủ mạnh để posterior nghiêng về lớp 1.

Bài 2: Naive Bayes với bảng đếm từ

Đề bài. Một bộ phân loại spam dùng 10 email huấn luyện. Bảng dưới là số email trong từng lớp có chứa mỗi từ. Dùng Laplace smoothing $\alpha = 1$ để phân loại email mới chứa các từ *coupon*, *discount*, *report*.

lớp	số email	coupon	discount	report	team
spam	4	3	4	0	1
normal	6	1	0	5	4

Giả sử các từ là Bernoulli feature: từ xuất hiện hoặc không xuất hiện.

Lời giải. Prior:

$$P(\text{spam}) = \frac{4}{10} = 0.4, \quad P(\text{normal}) = \frac{6}{10} = 0.6$$

Với Laplace smoothing:

$$P(w = 1 \mid c) = \frac{\#(w = 1, c) + \alpha}{N_c + 2\alpha}$$

Với lớp spam:

$$P(\textit{coupon} = 1 \mid \textit{spam}) = \frac{3 + 1}{4 + 2} = \frac{4}{6} = 0.667$$

$$P(\textit{discount} = 1 \mid \textit{spam}) = \frac{4 + 1}{4 + 2} = \frac{5}{6} = 0.833$$

$$P(\textit{report} = 1 \mid \textit{spam}) = \frac{0 + 1}{4 + 2} = \frac{1}{6} = 0.167$$

Joint score spam:

$$0.4(0.667)(0.833)(0.167) \approx 0.03704$$

Với lớp normal:

$$P(\textit{coupon} = 1 \mid \textit{normal}) = \frac{1 + 1}{6 + 2} = \frac{2}{8} = 0.250$$

$$P(\textit{discount} = 1 \mid \textit{normal}) = \frac{0 + 1}{6 + 2} = \frac{1}{8} = 0.125$$

$$P(\textit{report} = 1 \mid \textit{normal}) = \frac{5 + 1}{6 + 2} = \frac{6}{8} = 0.750$$

Joint score normal:

$$0.6(0.250)(0.125)(0.750) = 0.01406$$

Chuẩn hóa posterior:

$$P(\textit{spam} \mid \textit{words}) = \frac{0.03704}{0.03704 + 0.01406} \approx 0.725$$

Vì $0.725 > 0.5$, dự đoán **spam**.

Kết luận. Naive Bayes nhân nhiều xác suất điều kiện lại. Khi làm bài tay, nên tính joint score từng lớp trước, rồi chuẩn hóa ở bước cuối.

Bài 3: Gaussian Discriminant Analysis từ 12 điểm một chiều

Đề bài. Cho dữ liệu một chiều. Ước lượng mean, variance chung, prior của mỗi lớp theo GDA, rồi phân loại điểm mới $x = 2.2$.

dòng	x	y
1	0.8	0
2	1.0	0
3	1.1	0
4	1.3	0
5	1.5	0
6	1.7	0
7	2.4	1
8	2.6	1
9	2.8	1
10	3.0	1
11	3.1	1
12	3.3	1

Lời giải.

Prior hai lớp bằng nhau:

$$P(y = 0) = P(y = 1) = \frac{6}{12} = 0.5$$

Mean lớp 0:

$$\mu_0 = \frac{0.8 + 1.0 + 1.1 + 1.3 + 1.5 + 1.7}{6} = \frac{7.4}{6} \approx 1.233$$

Mean lớp 1:

$$\mu_1 = \frac{2.4 + 2.6 + 2.8 + 3.0 + 3.1 + 3.3}{6} = \frac{17.2}{6} \approx 2.867$$

Variance chung của GDA:

$$\sigma^2 = \frac{1}{12} \sum_{i=1}^{12} (x_i - \mu_{y_i})^2 \approx 0.0922$$

Với $x = 2.2$, mật độ Gaussian theo từng lớp:

$$p(x = 2.2 \mid y = 0) = \frac{1}{\sqrt{2\pi(0.0922)}} \exp\left(-\frac{(2.2 - 1.233)^2}{2(0.0922)}\right) \approx 0.00828$$

$$p(x = 2.2 \mid y = 1) = \frac{1}{\sqrt{2\pi(0.0922)}} \exp\left(-\frac{(2.2 - 2.867)^2}{2(0.0922)}\right) \approx 0.1180$$

Nhân prior:

$$score_0 = 0.00828(0.5) = 0.00414,$$

$$score_1 = 0.1180(0.5) = 0.05902.$$

Posterior lớp 1:

$$P(y = 1 \mid x = 2.2) = \frac{0.05902}{0.00414 + 0.05902} \approx 0.934$$

Vì $0.934 > 0.5$, dự đoán **lớp 1**.

Kết luận. GDA học phân phối của từng lớp trước, rồi dùng Bayes để đổi likelihood thành posterior. Với dữ liệu một chiều, điểm nằm gần mean lớp nào hơn thường nghiêng về lớp đó, nhưng prior và variance vẫn ảnh hưởng.

Phần 6: Học máy: Cây Quyết định Nâng cao và Phương pháp Kết hợp

Nguồn: `quarto/Decision_Tree.qmd`

1. Cơ chế Hoạt động của Cây Quyết định (Decision Tree)

Bản đồ ký hiệu khi tính split

Ký hiệu	Nghĩa	Cách dùng
S	Tập dữ liệu tại node hiện tại	Tính impurity cha.
A	Thuộc tính/điều kiện đang xét để chia	Thử từng thuộc tính hoặc từng ngưỡng.
S_v	Nhánh con khi thuộc tính nhận giá trị v	Tính impurity con.
p_c	Tỉ lệ mẫu thuộc lớp c	Dùng trong entropy/Gini.
$H(S)$	Entropy	Đo độ lẫn lộn bằng log.
$Gini(S)$	Gini impurity	Đo độ lẫn lộn không dùng log.
$IG(S, A)$	Information Gain	Impurity cha trừ impurity con có trọng số.
SSE	Tổng bình phương sai số trong regression tree	Chọn split làm SSE nhỏ nhất.

Luồng làm bài: tính impurity của node cha, chia dữ liệu theo từng ứng viên split, tính impurity/SSE của từng nhánh, lấy trung bình có trọng số, rồi chọn split làm giảm lỗi nhiều nhất.

Cây quyết định là thuật toán học máy phi tuyến tính hoạt động theo cơ chế chia để trị (Top-down, Greedy, Recursively partitioning). Thuật toán liên tục phân tách không gian thuộc tính thành các vùng hình chữ nhật loại trừ lẫn nhau (Mutually exclusive regions) sao cho dữ liệu ở mỗi vùng con đạt độ đồng nhất (Purity) cao nhất.

Tại một nút (node) bất kỳ, không gian dữ liệu được phân hoạch dựa trên một thuộc tính j và một ngưỡng giá trị t :

$$S(j, t) = \{\{x \in \mathcal{D} \mid x_j \leq t\}, \{x \in \mathcal{D} \mid x_j > t\}\} = \{R_1, R_2\}$$

The important visual property is axis alignment: each split cuts one feature at one threshold, so deeper trees build more refined rectangular regions.

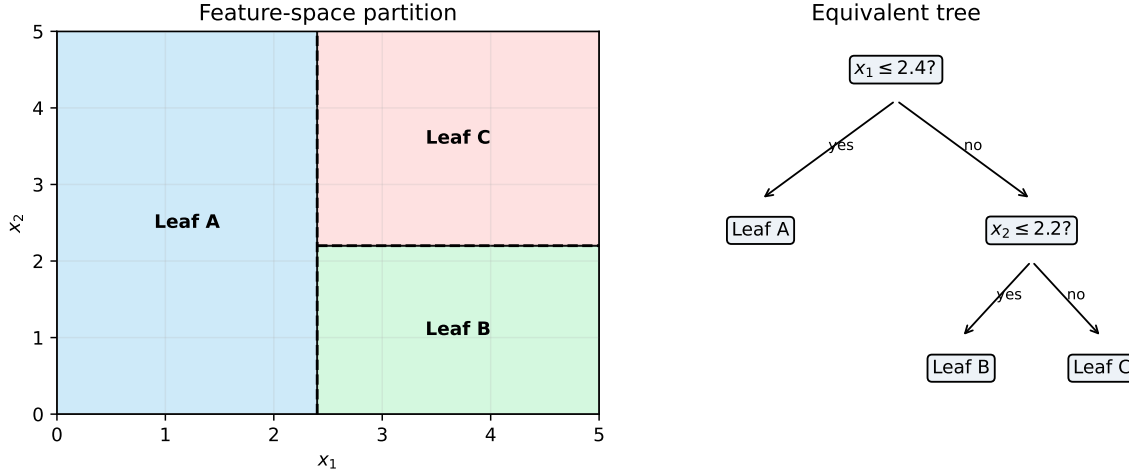


Figure 6: A shallow decision tree and the axis-aligned regions it creates.

2. Hàm Mục Tiêu và Các Độ Đo Ô Nhiễm (Impurity Metrics)

Để đánh giá chất lượng của một phép phân tách tại nút R , ta định nghĩa hàm chi phí thông qua độ ô nhiễm $L(R)$.

2.1 Đối với Bài toán Phân loại (Classification)

Gọi p_c là tỷ lệ số mẫu thuộc lớp $c \in \{1, 2, \dots, C\}$ bên trong vùng R :

$$p_c = \frac{1}{|R|} \sum_{i \in R} \mathbf{1}\{y^{(i)} = c\}$$

- **Độ bất định Entropy (Entropy):** Đo lường mức độ hỗn loạn của thông tin. Hàm số đạt cực đại bằng $\log_2 C$ khi dữ liệu phân phối đều và đạt cực tiểu bằng 0 khi toàn bộ mẫu thuộc về một lớp duy nhất:

$$H(R) = - \sum_{c=1}^C p_c \log_2 p_c$$

- **Chỉ số Gini (Gini Index):** Đo lường xác suất phân loại sai một mẫu dữ liệu nếu ta chọn nhãn ngẫu nhiên dựa trên phân phối của vùng đó:

$$G(R) = 1 - \sum_{c=1}^C p_c^2$$

Khi phân hoạch một nút cha R thành hai nút con R_1 và R_2 , thuật toán tham lam sẽ quét qua toàn bộ các thuộc tính j và các ngưỡng t để chọn ra phép phân tách tối đa hóa **Information Gain**:

$$\max_{j,t} \text{Gain}(R, j, t) = H(R) - \left(\frac{|R_1|}{|R|} H(R_1) + \frac{|R_2|}{|R|} H(R_2) \right)$$

2.2 Đối với Bài toán Hồi quy (Regression Trees - CART)

Đối với bài toán dự đoán giá trị liên tục, ta không sử dụng Entropy hay Gini mà tối ưu hóa bằng **Sai số bình phương trung bình (MSE)**.

- **Hàm mục tiêu ô nhiễm tại nút R :**

$$L(R) = \frac{1}{|R|} \sum_{i \in R} (y^{(i)} - \bar{y}_R)^2$$

Trong đó $\bar{y}_R$ là giá trị trung bình cộng của các nhãn mục tiêu nằm trong vùng R :

$$\bar{y}_R = \frac{1}{|R|} \sum_{i \in R} y^{(i)}$$

- **Giá trị dự đoán:** Khi một mẫu dữ liệu mới rơi vào nút lá R , giá trị dự đoán đầu ra của mô hình chính là hằng số $\bar{y}_R$.
- **Tiêu chí phân hoạch:** Thuật toán tìm kiếm thuộc tính j và ngưỡng t sao cho tổng sai số bình phương sau phân tách là nhỏ nhất:

$$\min_{j,t} \left[\sum_{i \in R_1} (y^{(i)} - \bar{y}_{R_1})^2 + \sum_{i \in R_2} (y^{(i)} - \bar{y}_{R_2})^2 \right]$$

3. Kiểm Soát Overfitting & Kỹ Thuật Cắt Tỉa Cây (Pruning)

Nếu cây quyết định được phát triển tự do, nó sẽ mọc cho đến khi tất cả các nút lá đều đồng nhất tuyệt đối ($L(R) = 0$). Hiện tượng này gây ra hiện tượng Overfitting nghiêm trọng (High Variance). Để kiểm soát, ta sử dụng thuật toán **Cost-Complexity Pruning** (Cắt tỉa chi phí - phức tạp).

Ta thiết lập hàm mục tiêu tối ưu hóa có chứa thành phần phạt dựa trên kích thước cấu trúc cây:

$$C_\alpha(T) = \sum_{m=1}^{|T|} |R_m| L(R_m) + \alpha |T|$$

Trong đó: * $|T|$ là số lượng nút lá của cây T . * $L(R_m)$ là độ ô nhiễm tại nút lá thứ m . * $\alpha \geq 0$ là siêu tham số điều khiển sự đánh đổi (trade-off) giữa độ chính xác và độ phức tạp cấu trúc. α càng lớn, thuật toán càng ép cây phải bị cắt tỉa gọn hơn.

4. Các Phương Phương Học Tập Hợp (Ensemble Methods)

4.1 Phương pháp Bagging (Bootstrap Aggregating)

- **Nguyên lý:** Xây dựng song song nhiều cây quyết định hoàn toàn độc lập. Mỗi cây được huấn luyện trên một tập dữ liệu con được tạo ra bằng phương pháp lấy mẫu lặp lại ngẫu nhiên (Bootstrap sampling) từ tập dữ liệu huấn luyện gốc.
- **Đại diện tiêu biểu - Random Forest:** Để giảm thiểu độ tương quan giữa các cây, Random Forest bổ sung một cơ chế ngẫu nhiên hóa: Tại mỗi nút phân tách, thuật toán chỉ cho phép chọn thuộc tính tối ưu từ một tập con gồm k thuộc tính được chọn ngẫu nhiên ($k < p$, thông thường $k = \sqrt{p}$).

4.2 Phương pháp Boosting & Bản chất Toán học của XGBoost

Trái ngược với Bagging, Boosting xây dựng chuỗi các cây một cách tuần tự (Sequential), cây sau tập trung sửa sai cho các cây phía trước.

XGBoost (Extreme Gradient Boosting) nâng cấp thuật toán bằng cách áp dụng **Khai triển Taylor bậc hai** để xấp xỉ hàm mất mát tổng thể tại bước lặp thứ t , giúp tìm ra cấu trúc cây tối ưu một cách cực kỳ chính xác.

Hàm mất mát tại bước lặp thứ t khi thêm cây $f_t(\mathbf{x})$ vào mô hình:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y^{(i)}, \hat{y}^{(t-1)(i)} + f_t(\mathbf{x}^{(i)})) + \Omega(f_t)$$

Áp dụng khai triển Taylor bậc hai xấp xỉ hàm $l(y, \hat{y} + \Delta\hat{y}) \approx l(y, \hat{y}) + g\Delta\hat{y} + \frac{1}{2}h(\Delta\hat{y})^2$:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^n \left[l(y^{(i)}, \hat{y}^{(t-1)(i)}) + g_i f_t(\mathbf{x}^{(i)}) + \frac{1}{2} h_i f_t^2(\mathbf{x}^{(i)}) \right] + \Omega(f_t)$$

Trong đó g_i và h_i lần lượt là đạo hàm bậc một (Gradient) và đạo hàm bậc hai (Hessian) của hàm mất mát cũ tính tại giá trị dự đoán trước đó:

$$g_i = \frac{\partial l(y^{(i)}, \hat{y}^{(t-1)(i)})}{\partial \hat{y}^{(t-1)(i)}} \quad \text{và} \quad h_i = \frac{\partial^2 l(y^{(i)}, \hat{y}^{(t-1)(i)})}{\partial (\hat{y}^{(t-1)(i)})^2}$$

Bỏ qua các hằng số không ảnh hưởng đến quá trình tối ưu hóa, hàm mục tiêu rút gọn của XGBoost tại bước t là:

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^n \left[g_i f_t(\mathbf{x}^{(i)}) + \frac{1}{2} h_i f_t^2(\mathbf{x}^{(i)}) \right] + \gamma |T| + \frac{1}{2} \lambda \sum_{j=1}^{|T|} w_j^2$$

Thông tin đạo hàm bậc hai h_i đóng vai trò như một hệ số điều hướng độ cong của không gian lỗi, giúp XGBoost hội tụ nhanh vượt trội so với Gradient Boosting truyền thống chỉ dùng đạo hàm bậc một.

5. Quy trình chọn split trong cây quyết định

Khi đề cho bảng dữ liệu và yêu cầu chọn node gốc:

1. Tính impurity của node cha.
2. Với từng thuộc tính hoặc ngưỡng split, chia dữ liệu thành các nhánh.
3. Tính impurity của từng nhánh.
4. Tính impurity sau split:

$$Impurity_{after} = \sum_v \frac{|S_v|}{|S|} Impurity(S_v)$$

5. Tính mức giảm:

$$Gain = Impurity(S) - Impurity_{after}$$

6. Chọn split có gain lớn nhất hoặc SSE nhỏ nhất.

Bẫy thường gặp: entropy/Gini dùng cho classification; SSE/MSE dùng cho regression tree. Đừng trộn entropy với biến mục tiêu liên tục.

6. Bảng Công thức Thi: Máy tính Phân tách Cây Quyết định

Khi đề bài hỏi một phép tách có tốt hay không, hãy tính độ ô nhiễm trước và sau khi tách.

Đại lượng	Công thức	Giá trị tốt nhất
Xác suất lớp	$p_c = \frac{\# \text{mẫu thuộc lớp } c}{ R }$	Dùng bên trong
Entropy	$H(R) = -\sum_c p_c \log_2 p_c$	Entropy/Gini.
Gini	$G(R) = 1 - \sum_c p_c^2$	0 nghĩa là nút hoàn toàn thuần nhất.
Độ ô nhiễm con có trọng số	$\frac{ R_1 }{ R } I(R_1) + \frac{ R_2 }{ R } I(R_2)$	0 nghĩa là nút hoàn toàn thuần nhất.
Information Gain	$I(R) - \left[\frac{ R_1 }{ R } I(R_1) + \frac{ R_2 }{ R } I(R_2) \right]$	Càng thấp càng tốt.
Giá trị lá hồi quy	$\bar{y}_R = \frac{1}{ R } \sum_{i \in R} y_i$	Càng cao càng tốt.
Cost-complexity pruning	$C_\alpha(T) = \sum_m R_m L(R_m) + \alpha T $	Dự đoán trung bình mục tiêu trong nút lá. α càng lớn, cây càng bị cắt gọn.

```
import numpy as np
import matplotlib.pyplot as plt

def entropy(counts):
    counts = np.array(counts, dtype=float)
    probs = counts[counts > 0] / counts.sum()
    return -np.sum(probs * np.log2(probs))

def gini(counts):
    counts = np.array(counts, dtype=float)
    probs = counts / counts.sum()
    return 1 - np.sum(probs ** 2)

parent = [5, 5]
left = [4, 1]
right = [1, 4]
n_parent = sum(parent)
weighted_entropy = (sum(left) / n_parent) * entropy(left) + (sum(right) /
    ↪ n_parent) * entropy(right)
weighted_gini = (sum(left) / n_parent) * gini(left) + (sum(right) / n_parent)
    ↪ * gini(right)
```

```

print("Entropy nút cha:", round(entropy(parent), 3))
print("Entropy con có trọng số:", round(weighted_entropy, 3))
print("Information Gain:", round(entropy(parent) - weighted_entropy, 3))
print("Gini nút cha:", round(gini(parent), 3))
print("Gini con có trọng số:", round(weighted_gini, 3))

labels = ["nút cha", "nút con"]
entropy_vals = [entropy(parent), weighted_entropy]
gini_vals = [gini(parent), weighted_gini]
x = np.arange(len(labels))
width = 0.35
plt.figure(figsize=(6, 3.5))
plt.bar(x - width / 2, entropy_vals, width, label="entropy")
plt.bar(x + width / 2, gini_vals, width, label="gini")
plt.xticks(x, labels)
plt.ylabel("độ ô nhiễm")
plt.title("Phép tách tốt làm giảm độ ô nhiễm")
plt.legend()
plt.grid(axis="y", alpha=0.25)
plt.show()

```

```

Entropy nút cha: 1.0
Entropy con có trọng số: 0.722
Information Gain: 0.278
Gini nút cha: 0.5
Gini con có trọng số: 0.32

```

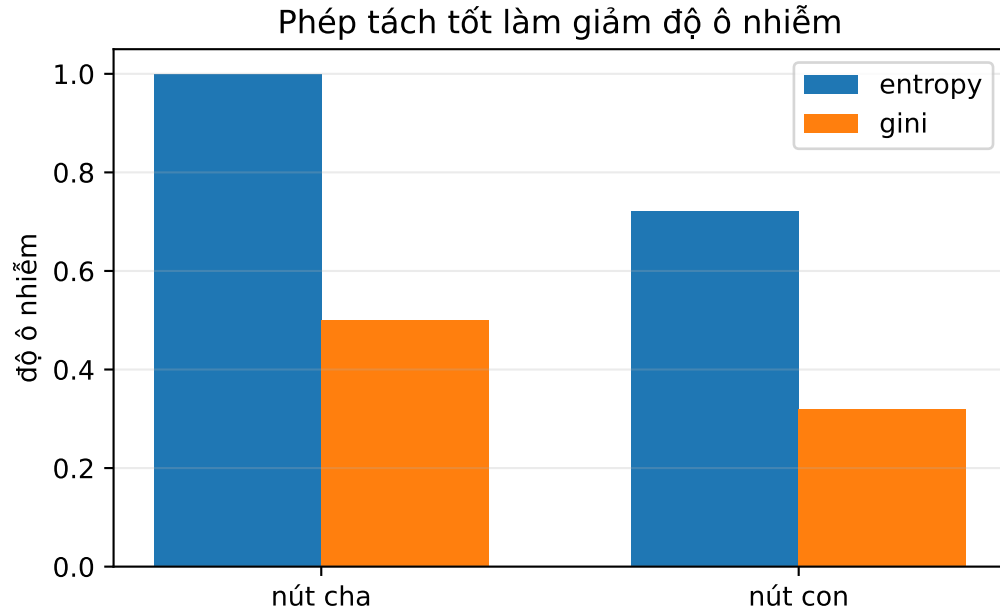


Figure 7: Information gain compares parent impurity with weighted child impurity.

Dạng bài: Information Gain từ bảng nhiều thuộc tính

Khi bảng có nhiều cột rời rạc như Schedule, Load, Budget, Deadline, hãy tính từng thuộc tính theo cùng một khung.

Với thuộc tính A có các giá trị $v_1, \dots, v_k$:

$$Gain(S, A) = H(S) - \sum_{r=1}^k \frac{|S_{v_r}|}{|S|} H(S_{v_r})$$

Mẫu bảng phụ nên lập:

Giá trị của A	Số mẫu	Số Yes	Số No	Entropy
v_1	$ S_{v_1} $	count	count	$H(S_{v_1})$
v_2	$ S_{v_2} $	count	count	$H(S_{v_2})$

Sau đó tính weighted entropy:

$$H_{after}(A) = \sum_v \frac{|S_v|}{|S|} H(S_v)$$

và:

$$Gain(S, A) = H(S) - H_{after}(A)$$

Cách trình bày rõ ràng: tính $H(S)$ một lần ở đầu, rồi mỗi thuộc tính chỉ cần một bảng phụ và một dòng gain. Đừng trộn các count của thuộc tính này sang thuộc tính khác.

Bộ bài tập mẫu có lời giải

Đề bài. Một nút cha có 10 mẫu: 5 mẫu lớp 0 và 5 mẫu lớp 1. Một phép tách tạo ra:

- Nút trái: 4 mẫu lớp 0, 1 mẫu lớp 1.
- Nút phải: 1 mẫu lớp 0, 4 mẫu lớp 1.

Hãy tính Information Gain theo Entropy.

Lời giải từng bước.

1. Entropy nút cha:

$$H(R) = -0.5 \log_2(0.5) - 0.5 \log_2(0.5) = 1$$

2. Entropy nút trái:

$$H(R_1) = -0.8 \log_2(0.8) - 0.2 \log_2(0.2) \approx 0.722$$

3. Entropy nút phải có cùng phân phối đảo ngược, nên:

$$H(R_2) \approx 0.722$$

4. Entropy sau tách:

$$\frac{5}{10}(0.722) + \frac{5}{10}(0.722) = 0.722$$

5. Information Gain:

$$IG = 1 - 0.722 = 0.278$$

Kết luận. Phép tách này hữu ích vì làm Entropy giảm từ 1 xuống 0.722.

Bài 2: Chọn split tốt nhất từ bộ dữ liệu 12 dòng

Đề bài. Cho bộ dữ liệu nhị phân sau. Hãy so sánh hai split: $x_1 \leq 2.5$ và $x_2 \leq 1.5$ bằng Information Gain theo Gini. Split nào tốt hơn?

dòng	x_1	x_2	y
1	1.0	1.0	0
2	1.2	1.4	0
3	1.5	2.0	0
4	2.0	1.1	0
5	2.4	1.8	1
6	2.8	1.2	1
7	3.0	2.0	1
8	3.2	2.4	1
9	3.8	1.0	1
10	4.0	2.8	1
11	4.3	1.7	1
12	4.8	2.2	1

Lời giải. Gini của một nút:

$$G(R) = 1 - p_0^2 - p_1^2$$

Information Gain theo Gini:

$$IG = G(R) - \frac{|R_L|}{|R|}G(R_L) - \frac{|R_R|}{|R|}G(R_R)$$

Nút cha có 4 mẫu lớp 0 và 8 mẫu lớp 1:

$$G(R) = 1 - \left(\frac{4}{12}\right)^2 - \left(\frac{8}{12}\right)^2 = 1 - \frac{16}{144} - \frac{64}{144} = 0.444$$

Với split $x_1 \leq 2.5$:

- Nút trái có 5 mẫu: 4 lớp 0, 1 lớp 1.
- Nút phải có 7 mẫu: 0 lớp 0, 7 lớp 1.

$$G(R_L) = 1 - \left(\frac{4}{5}\right)^2 - \left(\frac{1}{5}\right)^2 = 0.320$$

$$G(R_R) = 1 - 0^2 - 1^2 = 0$$

Gini sau split:

$$\frac{5}{12}(0.320) + \frac{7}{12}(0) = 0.133$$

Gain:

$$IG_{x_1} = 0.444 - 0.133 = 0.311$$

Với split $x_2 \leq 1.5$:

- Nút trái có 5 mẫu: 2 lớp 0, 3 lớp 1.
- Nút phải có 7 mẫu: 2 lớp 0, 5 lớp 1.

$$G(R_L) = 1 - \left(\frac{2}{5}\right)^2 - \left(\frac{3}{5}\right)^2 = 0.480$$

$$G(R_R) = 1 - \left(\frac{2}{7}\right)^2 - \left(\frac{5}{7}\right)^2 \approx 0.245$$

Gini sau split:

$$\frac{5}{12}(0.480) + \frac{7}{12}(0.245) \approx 0.343$$

Gain:

$$IG_{x_2} = 0.444 - 0.343 = 0.102$$

Kết luận. Chọn split có gain lớn hơn. Trong bài thi, nếu hai split cùng hợp lý trực giác, cứ để Gini/Entropy quyết định.

Bài 3: Lá hồi quy và MSE sau split

Đề bài. Với dữ liệu hồi quy 10 dòng, xét split $x \leq 5$. Mỗi lá dự đoán bằng trung bình y trong lá. Tính MSE sau split.

x	1	2	3	4	5	6	7	8	9	10
y	3	4	5	6	8	13	14	16	18	20

Lời giải.

Lá trái chứa $y = \{3, 4, 5, 6, 8\}$:

$$\hat{y}_L = \frac{3 + 4 + 5 + 6 + 8}{5} = \frac{26}{5} = 5.2$$

Sai số bình phương trong lá trái:

$$(3 - 5.2)^2 + (4 - 5.2)^2 + (5 - 5.2)^2 + (6 - 5.2)^2 + (8 - 5.2)^2 = 14.8$$

Lá phải chứa $y = \{13, 14, 16, 18, 20\}$:

$$\hat{y}_R = \frac{13 + 14 + 16 + 18 + 20}{5} = \frac{81}{5} = 16.2$$

Sai số bình phương trong lá phải:

$$(13 - 16.2)^2 + (14 - 16.2)^2 + (16 - 16.2)^2 + (18 - 16.2)^2 + (20 - 16.2)^2 = 32.8$$

Tổng SSE sau split:

$$SSE = 14.8 + 32.8 = 47.6$$

MSE sau split:

$$MSE = \frac{47.6}{10} = 4.76$$

Kết luận. Với cây hồi quy, lá không bỏ phiếu lớp mà lấy trung bình mục tiêu. Split tốt là split làm tổng sai số trong lá nhỏ đi.

Phần 7: Học máy: Mạng Nơ-ron và Lan truyền Ngược Vector hóa

Nguồn: `quarto/Neural_Network.qmd`

1. Bức tranh tinh thần: mạng nơ-ron đang lưu gì?

Một mạng nơ-ron không phải là một công thức bí ẩn. Nó là nhiều tầng biến đổi dữ liệu liên tiếp. Ở mỗi tầng, có hai pha:

1. **Pha tuyến tính:** gom thông tin từ tầng trước bằng trọng số và độ lệch.
2. **Pha phi tuyến:** đưa kết quả qua hàm kích hoạt để tạo giá trị kích hoạt cho tầng sau.

Với tầng l :

$$\mathbf{Z}^{[l]} = \mathbf{W}^{[l]} \mathbf{A}^{[l-1]} + \mathbf{b}^{[l]}$$

$$\mathbf{A}^{[l]} = g^{[l]}(\mathbf{Z}^{[l]})$$

Ý nghĩa của từng ký hiệu:

Ký hiệu	Nằm ở đâu trong mạng?	Nói bằng lời
$\mathbf{A}^{[0]}$	Tầng đầu vào	Dữ liệu gốc, thường chính là $\mathbf{X}^T$.
$\mathbf{W}^{[l]}$	Các cạnh nối từ tầng $l - 1$ sang tầng l	Trọng số cho biết giá trị kích hoạt tầng trước ảnh hưởng mạnh yếu thế nào đến tầng sau.
$\mathbf{b}^{[l]}$	Mỗi nơ-ron ở tầng l có một độ lệch	Dịch điểm tuyến tính lên hoặc xuống trước khi kích hoạt.
$\mathbf{Z}^{[l]}$	Bên trong tầng l , trước hàm kích hoạt	Điểm tuyến tính trước kích hoạt.
$\mathbf{A}^{[l]}$	Đầu ra của tầng l	Giá trị kích hoạt sau khi áp dụng hàm g . Đây là thứ được gửi sang tầng kế tiếp.
$d\mathbf{Z}^{[l]}$	Khi lan truyền ngược qua tầng l	Tín hiệu lỗi tại điểm trước kích hoạt.
$d\mathbf{W}^{[l]}, d\mathbf{b}^{[l]}$	Cùng chỗ với $\mathbf{W}^{[l]}, \mathbf{b}^{[l]}$	Hướng cần chỉnh trọng số và độ lệch để hàm mất mát giảm.

Nếu chỉ nhớ một câu: lan truyền xuôi tạo ra Z rồi A ; lan truyền ngược tạo ra dZ rồi dW, db ; bước cập nhật dùng dW, db để sửa W, b .

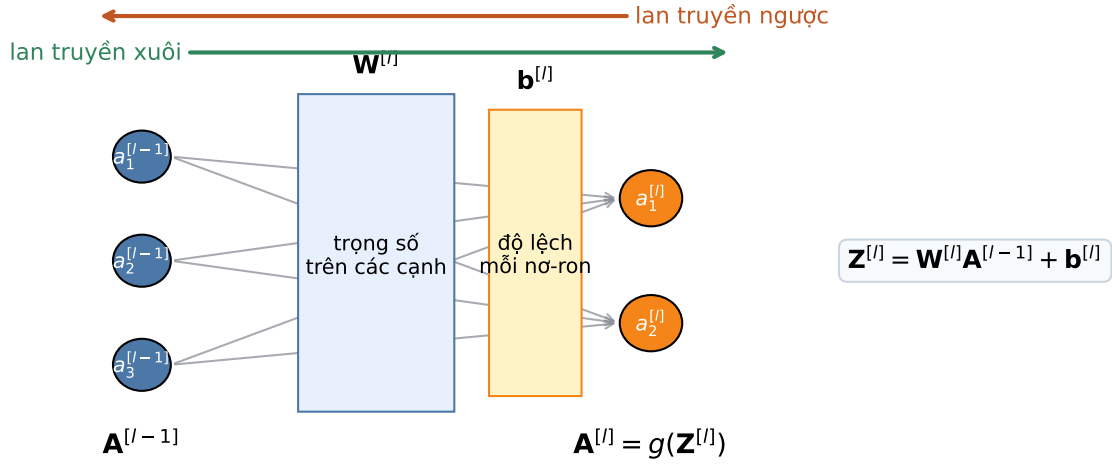


Figure 8: Vị trí của A , W , b , Z trong một tầng kết nối đầy đủ.

2. Một nơ-ron đơn: từ đầu vào đến dự đoán

Một nơ-ron nhận vector đầu vào $\mathbf{x}$, nhân với trọng số $\mathbf{w}$, cộng độ lệch b , rồi đưa qua hàm kích hoạt:

$$z = \mathbf{w}^T \mathbf{x} + b$$

$$a = g(z)$$

Nếu g là sigmoid, a có thể đọc như xác suất. Nếu g là ReLU, a là tín hiệu dương được truyền tiếp.

Ví dụ:

$$\mathbf{x} = (2, 3), \quad \mathbf{w} = (0.5, -0.2), \quad b = 0.1$$

$$z = 0.5(2) - 0.2(3) + 0.1 = 0.5$$

Nếu dùng sigmoid:

$$a = \sigma(0.5) = \frac{1}{1 + e^{-0.5}} \approx 0.622$$

Điều quan trọng: z là điểm thô trước kích hoạt, còn a là tín hiệu đã xử lý để đưa sang bước sau.

3. Lan truyền xuôi: dữ liệu chảy từ trái sang phải

Với mini-batch có m mẫu, ta không xử lý từng mẫu riêng lẻ mà xếp chúng thành ma trận.

Nếu tầng $l-1$ có n_{l-1} nơ-ron và tầng l có n_l nơ-ron:

Đại lượng	Kích thước	Vai trò
$\mathbf{A}^{[l-1]}$	$n_{l-1} \times m$	Giá trị kích hoạt của tầng trước cho toàn bộ mini-batch.
$\mathbf{W}^{[l]}$	$n_l \times n_{l-1}$	Mỗi hàng là trọng số đi vào một nơ-ron của tầng l .
$\mathbf{b}^{[l]}$	$n_l \times 1$	Độ lệch của từng nơ-ron, được cộng cho mọi cột trong mini-batch.
$\mathbf{Z}^{[l]}$	$n_l \times m$	Điểm tuyến tính của tầng l .
$\mathbf{A}^{[l]}$	$n_l \times m$	Đầu ra của tầng l sau hàm kích hoạt.

Tưởng tượng mỗi cột là một mẫu. Mỗi hàng là một nơ-ron. Vì vậy $\mathbf{A}^{[l]}$ có hàng là nơ-ron tầng l và cột là mẫu trong mini-batch.

Lan truyền xuôi: tính dự đoán và mất mát

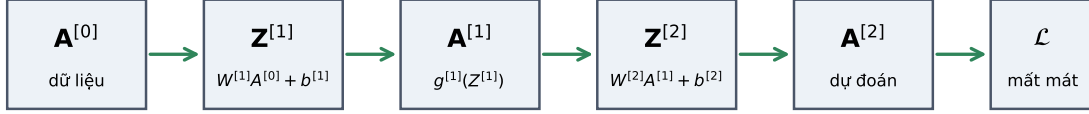


Figure 9: Lan truyền xuôi tạo Z rồi A ở từng tầng.

4. Lan truyền ngược: lỗi chảy từ phải sang trái

Lan truyền xuôi trả lời câu hỏi: **mô hình dự đoán gì?**

Lan truyền ngược trả lời câu hỏi: **mỗi tham số phải bị sửa bao nhiêu để hàm mất mát giảm?**

Ký hiệu quan trọng nhất là $\mathbf{dZ}^{[l]}$:

$$\mathbf{dZ}^{[l]} = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}^{[l]}}$$

Đọc bằng lời: nếu điểm trước kích hoạt $\mathbf{Z}^{[l]}$ thay đổi một chút, hàm mất mát thay đổi bao nhiêu?

Sau khi có $\mathbf{dZ}^{[l]}$, ta tính được:

$$\mathbf{dW}^{[l]} = \frac{1}{m} \mathbf{dZ}^{[l]} (\mathbf{A}^{[l-1]})^T$$

$$\mathbf{db}^{[l]} = \frac{1}{m} \sum_{i=1}^m \mathbf{dZ}^{(i)[l]}$$

Ý nghĩa trực giác:

- $\mathbf{dZ}^{[l]}$ là mức lỗi của từng nơ-ron trong từng mẫu.
- $\mathbf{A}^{[l-1]}$ là tín hiệu đã đi vào tầng đó.

- dW bằng **lỗi nhân tín hiệu đầu vào**. Nếu đầu vào lớn và lỗi lớn, cạnh đó cần được sửa nhiều.
- db là lỗi trung bình của từng nơ-ron.

Với tầng ẩn:

$$d\mathbf{Z}^{[l]} = ((\mathbf{W}^{[l+1]})^T d\mathbf{Z}^{[l+1]}) \odot g'(\mathbf{Z}^{[l]})$$

Phần $(\mathbf{W}^{[l+1]})^T d\mathbf{Z}^{[l+1]}$ kéo lỗi từ tầng sau về tầng trước. Phần $\odot g'(\mathbf{Z}^{[l]})$ kiểm tra hàm kích hoạt ở tầng này có cho gradient đi qua hay không.

Hadamard product $\odot$ là gì?

Ký hiệu $\odot$ nghĩa là nhân từng phần tử tương ứng, không phải nhân ma trận. Hai ma trận/vector phải có cùng kích thước:

$$\begin{pmatrix} 2 & -1 \\ 3 & 4 \end{pmatrix} \odot \begin{pmatrix} 5 & 6 \\ 0 & -2 \end{pmatrix} = \begin{pmatrix} 2 \cdot 5 & (-1) \cdot 6 \\ 3 \cdot 0 & 4 \cdot (-2) \end{pmatrix} = \begin{pmatrix} 10 & -6 \\ 0 & -8 \end{pmatrix}$$

Trong backprop, Hadamard product xuất hiện vì mỗi neuron ở cùng một tầng có “cổng đạo hàm activation” riêng. Ví dụ:

$$d\mathbf{Z}^{[l]} = \underbrace{(\mathbf{W}^{[l+1]})^T d\mathbf{Z}^{[l+1]}}_{\text{lỗi được kéo từ tầng sau}} \odot \underbrace{g'(\mathbf{Z}^{[l]})}_{\text{đạo hàm activation của từng neuron}}$$

Nếu neuron nào có đạo hàm activation gần 0, phần tử tương ứng trong $d\mathbf{Z}^{[l]}$ cũng bị làm nhỏ lại. Đây là lý do sigmoid bão hòa gây vanishing gradient.

Sau khi có gradient, cập nhật tham số:

$$\mathbf{W}^{[l]} \leftarrow \mathbf{W}^{[l]} - \alpha d\mathbf{W}^{[l]}$$

$$\mathbf{b}^{[l]} \leftarrow \mathbf{b}^{[l]} - \alpha d\mathbf{b}^{[l]}$$

Trong đó α là learning rate.

Lan truyền ngược: hàm mất mát gửi tín hiệu sửa tham số

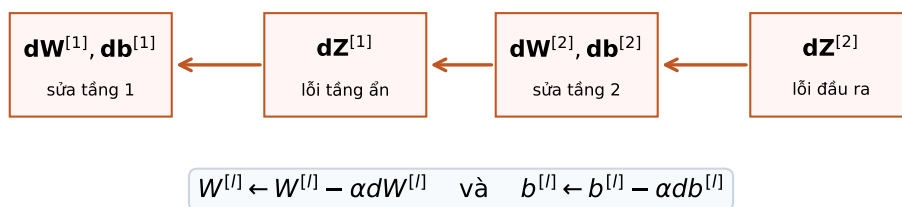


Figure 10: Lan truyền ngược đưa lỗi về từng tầng rồi tạo dW , db để cập nhật tham số.

5. Vì sao sigmoid dễ làm mất gradient?

Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Đạo hàm:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Khi z rất lớn, $\sigma(z) \approx 1$, nên $\sigma'(z) \approx 0$. Khi z rất âm, $\sigma(z) \approx 0$, nên $\sigma'(z) \approx 0$. Nếu nhiều tầng đều nhân với đạo hàm gần 0, tín hiệu đạo hàm về các tầng đầu gần như biến mất. Đây là hiện tượng **gradient biến mất**.

Cận trên exam-style cho vanishing gradient.

Vì $\sigma(z) \in (0, 1)$, đặt $s = \sigma(z)$ thì:

$$\sigma'(z) = s(1 - s)$$

Hàm $s(1 - s)$ đạt cực đại tại $s = \frac{1}{2}$, nên:

$$0 < \sigma'(z) \leq \frac{1}{4}$$

Với một mạng sâu dạng vô hướng:

$$a^{[0]} = x, \quad z^{[\ell]} = w_\ell a^{[\ell-1]}, \quad a^{[\ell]} = \sigma(z^{[\ell]})$$

gradient về trọng số tầng đầu có dạng tích theo quy tắc chuỗi:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a^{[L]}} \left[\prod_{\ell=2}^L \sigma'(z^{[\ell]}) w_\ell \right] \sigma'(z^{[1]}) a^{[0]}$$

Nếu $|w_\ell| \leq 1$, phần tín hiệu đi từ tầng cuối về tầng đầu bị chặn bởi:

$$\prod_{\ell=2}^L |\sigma'(z^{[\ell]}) w_\ell| \leq \left(\frac{1}{4}\right)^{L-1}$$

Ví dụ với $L = 8$:

$$\left(\frac{1}{4}\right)^7 = \frac{1}{16384} \approx 0.000061$$

Nghĩa là chỉ riêng việc nhân chuỗi các đạo hàm sigmoid đã có thể làm tín hiệu nhỏ đi hơn 16 nghìn lần. Nếu nhiều neuron rơi vào vùng bão hòa, đạo hàm còn nhỏ hơn $\frac{1}{4}$, nên gradient biến mất nhanh hơn nữa.

ReLU:

$$g(z) = \max(0, z)$$

Đạo hàm của ReLU bằng 1 khi $z > 0$, nên ở miền dương tín hiệu đạo hàm đi qua dễ hơn.

Ba cách giảm vanishing gradient thường gặp:

- **ReLU:** tránh nhân gradient với một số luôn nhỏ hơn hoặc bằng 0.25 như sigmoid.
- **Batch Normalization:** giữ z ít rơi vào vùng bão hòa, nên đạo hàm của sigmoid/tanh ít bị kéo về 0 hơn.
- **Residual connection:** thêm đường tắt dạng $\mathbf{A}^{[l+1]} = F(\mathbf{A}^{[l]}) + \mathbf{A}^{[l]}$, giúp gradient có một nhánh identity để chảy ngược về tầng trước.

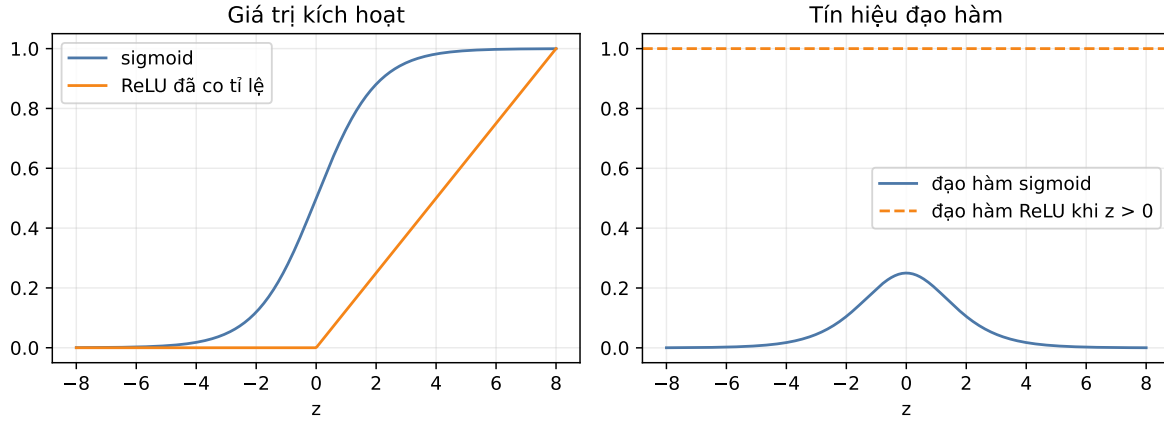


Figure 11: Sigmoid dễ bão hòa ở hai đầu, còn ReLU giữ gradient tốt ở miền dương.

6. Dropout và Chuẩn hóa Mini-batch

Dropout

Dropout tắt ngẫu nhiên một phần giá trị kích hoạt trong lúc huấn luyện:

$$\mathbf{A}_{\text{drop}}^{[l]} = \mathbf{A}^{[l]} \odot \mathbf{M}^{[l]}$$

Trong đó $\mathbf{M}^{[l]}$ là mặt nạ 0 hoặc 1. Nếu một nơ-ron bị tắt, mạng không được dựa quá nhiều vào nó. Nhờ vậy mô hình bớt overfit.

Khi kiểm thử, ta không tắt nơ-ron nữa. Với inverted dropout, lúc huấn luyện thường chia giá trị kích hoạt còn sống cho $1 - p$ để giữ kỳ vọng ổn định.

Chuẩn hóa mini-batch

Chuẩn hóa mini-batch chuẩn hóa $\mathbf{Z}^{[l]}$ trong một mini-batch:

$$\hat{z}^{(i)} = \frac{z^{(i)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

Sau đó học thêm scale và shift:

$$\tilde{z}^{(i)} = \gamma \hat{z}^{(i)} + \beta$$

Tác dụng chính:

- Giúp phân phối đầu vào của mỗi tầng ổn định hơn.
- Cho phép dùng learning rate lớn hơn.
- Thường làm quá trình train nhanh và mượt hơn.

Backward pass của Batch Normalization.

Trong bài tính tay, nếu một neuron có mini-batch $z_1, \dots, z_m$ và:

$$\tilde{z}_i = \gamma \hat{z}_i + \beta, \quad \hat{z}_i = \frac{z_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

thì gradient từ tầng sau thường được ký hiệu:

$$g_i = \frac{\partial L}{\partial \tilde{z}_i}$$

Hai gradient dễ nhất là:

$$\frac{\partial L}{\partial \gamma} = \sum_{i=1}^m g_i \hat{z}_i, \quad \frac{\partial L}{\partial \beta} = \sum_{i=1}^m g_i$$

Gradient quay về từng z_i có dạng gọn:

$$\frac{\partial L}{\partial z_i} = \frac{\gamma}{m\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \left[mg_i - \sum_{j=1}^m g_j - \hat{z}_i \sum_{j=1}^m g_j \hat{z}_j \right]$$

Cách nhớ: có ba phần điều chỉnh. Phần mg_i là gradient trực tiếp của phần tử i ; phần $-\sum_j g_j$ xuất hiện vì mean phụ thuộc vào cả batch; phần $-\hat{z}_i \sum_j g_j \hat{z}_j$ xuất hiện vì variance cũng phụ thuộc vào cả batch.

Ví dụ mini-batch nhanh. Với $z = (1, 3, 5, 7)$, ta có $\mu_{\mathcal{B}} = 4$, $\sigma_{\mathcal{B}}^2 = 5$, và:

$$\hat{z} = \left(-\frac{3}{\sqrt{5}}, -\frac{1}{\sqrt{5}}, \frac{1}{\sqrt{5}}, \frac{3}{\sqrt{5}} \right)$$

Nếu $\gamma = 2$, $\beta = -1$, thì:

$$\tilde{z} = 2\hat{z} - 1$$

Nếu $g = (1, -1, 2, -2)$:

$$\frac{\partial L}{\partial \beta} = 0, \quad \frac{\partial L}{\partial \gamma} = \frac{-6}{\sqrt{5}} \approx -2.683$$

Để tính tiếp gradient quay về từng z_i , dùng:

$$\sum_i g_i = 1 - 1 + 2 - 2 = 0, \quad \sum_i g_i \hat{z}_i = -\frac{6}{\sqrt{5}}$$

Vì $m = 4$, $\gamma = 2$, $\sqrt{\sigma_{\mathcal{B}}^2} = \sqrt{5}$:

$$\frac{\partial L}{\partial z_i} = \frac{2}{4\sqrt{5}} \left[4g_i - \hat{z}_i \left(-\frac{6}{\sqrt{5}} \right) \right]$$

Thay từng phần tử:

i	g_i	$\hat{z}_i$	$\frac{\partial L}{\partial z_i}$
1	1	$-\frac{3}{\sqrt{5}}$	$\frac{0.4}{2\sqrt{5}} \approx 0.089$
2	-1	$-\frac{1}{\sqrt{5}}$	$\frac{-5.2}{2\sqrt{5}} \approx -1.163$
3	2	$\frac{1}{\sqrt{5}}$	$\frac{9.2}{2\sqrt{5}} \approx 2.057$
4	-2	$\frac{3}{\sqrt{5}}$	$\frac{-4.4}{2\sqrt{5}} \approx -0.984$

Kiểm tra nhanh:

$$\sum_i \frac{\partial L}{\partial z_i} \approx 0.089 - 1.163 + 2.057 - 0.984 \approx 0$$

Tổng gradient về z gần bằng 0 vì BatchNorm đã trừ mean của batch; nếu mọi z_i cùng tăng một lượng như nhau, giá trị chuẩn hóa gần như không đổi.

Checklist BatchNorm khi làm tay.

1. Tính $\mu_{\mathcal{B}}$.
2. Tính $\sigma_{\mathcal{B}}^2$ bằng mẫu số m .
3. Tính từng $\hat{z}_i$.
4. Tính $\tilde{z}_i = \gamma \hat{z}_i + \beta$.
5. Với backward, tính trước $\sum_i g_i$ và $\sum_i g_i \hat{z}_i$.
6. Thay vào công thức $\partial L / \partial z_i$.

Đây là đúng kiểu thay số cần cho câu BatchNorm trong đề thi.

7. Từ ký hiệu đến thao tác: đọc một bài backprop không bị lẫn

Khi làm tay một mạng nhỏ, điều dễ gây nhầm nhất là cùng một neuron có nhiều ký hiệu. Bảng sau cố định nghĩa của từng loại ký hiệu:

Ký hiệu	Nghĩa	Có phải tham số học được không?	Dùng ở đâu?
x_1, x_2	Đầu vào của mẫu dữ liệu	Không	Dùng để tính tầng ẩn.
$w_{x_1 \rightarrow h_1}$	Trọng số nối từ x_1 sang neuron ẩn h_1	Có	Cập nhật bằng gradient descent.
b_{h_1}	Bias của neuron h_1	Có	Cộng vào trước activation.
z_{h_1}	Tổng tuyến tính đi vào neuron h_1	Không	$z_{h_1} = w_{x_1 \rightarrow h_1}x_1 + w_{x_2 \rightarrow h_1}x_2 + b_{h_1}$.
h_1 hoặc a_{h_1}	Activation sau sigmoid/ReLU của neuron h_1	Không	$h_1 = \sigma(z_{h_1})$.
z_o	Tổng tuyến tính đi vào output neuron	Không	$z_o = w_{h_1 \rightarrow o}h_1 + w_{h_2 \rightarrow o}h_2 + b_o$.
$\hat{y}$	Dự đoán cuối cùng	Không	$\hat{y} = \sigma(z_o)$.
δ_o	Lỗi tại output trước activation	Không	$\delta_o = \frac{\partial L}{\partial z_o}$.
δ_{h_1}	Lỗi tại hidden neuron trước activation	Không	$\delta_{h_1} = \frac{\partial L}{\partial z_{h_1}}$.
$\frac{\partial L}{\partial w}$ hoặc dW	Gradient của loss theo trọng số	Không	Dùng để cập nhật $w \leftarrow w - \eta \frac{\partial L}{\partial w}$.

Quy tắc một dòng: z là trước activation, $a/h/\hat{y}$ là sau activation, δ là lỗi quay ngược về trước activation, còn dW/db là thứ dùng để sửa tham số.

Dịch bảng trọng số thành phương trình

Khi đề cho bảng kết nối thay vì ma trận, hãy biến từng dòng thành một cạnh trong mạng. Ví dụ một mạng 2 input, 2 hidden, 1 output có bảng:

Kết nối	Trọng số
$x_1 \rightarrow h_1$	$w_{x_1 \rightarrow h_1}$
$x_2 \rightarrow h_1$	$w_{x_2 \rightarrow h_1}$
$x_1 \rightarrow h_2$	$w_{x_1 \rightarrow h_2}$

Kết nối	Trọng số
$x_2 \rightarrow h_2$	$w_{x_2 \rightarrow h_2}$
$h_1 \rightarrow o$	$w_{h_1 \rightarrow o}$
$h_2 \rightarrow o$	$w_{h_2 \rightarrow o}$

thì phương trình forward bắt buộc là:

$$z_{h_1} = w_{x_1 \rightarrow h_1} x_1 + w_{x_2 \rightarrow h_1} x_2 + b_{h_1}, \quad h_1 = \sigma(z_{h_1})$$

$$z_{h_2} = w_{x_1 \rightarrow h_2} x_1 + w_{x_2 \rightarrow h_2} x_2 + b_{h_2}, \quad h_2 = \sigma(z_{h_2})$$

$$z_o = w_{h_1 \rightarrow o} h_1 + w_{h_2 \rightarrow o} h_2 + b_o, \quad \hat{y} = \sigma(z_o)$$

Sau đó backprop đi ngược đúng thứ tự các mũi tên:

$$o \rightarrow h_1, h_2 \rightarrow x_1, x_2$$

Nói cách khác:

1. Tính lỗi output δ_o .
2. Dùng δ_o để tính gradient của các cạnh đi vào output.
3. Truyền lỗi về từng hidden neuron để có $\delta_{h_1}, \delta_{h_2}$.
4. Dùng $\delta_{h_1}, \delta_{h_2}$ để tính gradient của các cạnh đi vào hidden.

Mẫu câu nên viết trong bài làm: “Tôi dùng z cho tổng tuyến tính trước sigmoid, dùng h cho activation sau sigmoid, và dùng δ cho đạo hàm của loss theo z tại neuron đó.” Câu này tự bảo vệ bài làm khỏi lẫn ký hiệu.

Công thức thao tác cho mạng 2-2-1, squared error

Với hidden sigmoid và output sigmoid:

$$L = \frac{1}{2}(\hat{y} - y)^2$$

Forward:

$$z_{h_1} = w_{x_1 \rightarrow h_1} x_1 + w_{x_2 \rightarrow h_1} x_2 + b_{h_1}, \quad h_1 = \sigma(z_{h_1})$$

$$z_{h_2} = w_{x_1 \rightarrow h_2} x_1 + w_{x_2 \rightarrow h_2} x_2 + b_{h_2}, \quad h_2 = \sigma(z_{h_2})$$

$$z_o = w_{h_1 \rightarrow o} h_1 + w_{h_2 \rightarrow o} h_2 + b_o, \quad \hat{y} = \sigma(z_o)$$

Backward:

$$\delta_o = (\hat{y} - y) \hat{y} (1 - \hat{y})$$

$$\frac{\partial L}{\partial w_{h_1 \rightarrow o}} = \delta_o h_1, \quad \frac{\partial L}{\partial w_{h_2 \rightarrow o}} = \delta_o h_2, \quad \frac{\partial L}{\partial b_o} = \delta_o$$

$$\delta_{h_1} = \delta_o w_{h_1 \rightarrow o} h_1 (1 - h_1), \quad \delta_{h_2} = \delta_o w_{h_2 \rightarrow o} h_2 (1 - h_2)$$

$$\frac{\partial L}{\partial w_{x_1 \rightarrow h_1}} = \delta_{h_1} x_1, \quad \frac{\partial L}{\partial w_{x_2 \rightarrow h_1}} = \delta_{h_1} x_2$$

$$\frac{\partial L}{\partial w_{x_1 \rightarrow h_2}} = \delta_{h_2} x_1, \quad \frac{\partial L}{\partial w_{x_2 \rightarrow h_2}} = \delta_{h_2} x_2$$

Update:

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}, \quad b_{new} = b_{old} - \eta \frac{\partial L}{\partial b}$$

8. Quy trình debug một bài neural network

Trước khi tính dài, kiểm tra theo thứ tự:

1. Vẽ cấu trúc tầng: input, hidden, output.
2. Ghi rõ mỗi cạnh là trọng số nào và mỗi node có bias nào.
3. Tính toàn bộ z trước, rồi mới tính activation. Đừng trộn z_h với h .
4. Backprop luôn bắt đầu từ output:

$$\delta_o = \frac{\partial L}{\partial z_o}$$

5. Gradient của một trọng số luôn bằng:

lỗi tại neuron nhận $\times$ activation của neuron gửi

Ví dụ:

$$\frac{\partial L}{\partial w_{h_1 \rightarrow o}} = \delta_o h_1, \quad \frac{\partial L}{\partial w_{x_1 \rightarrow h_1}} = \delta_{h_1} x_1$$

Bẫy thường gặp: δ_h phải nhân thêm đạo hàm activation của hidden neuron. Nếu quên $h(1-h)$ với sigmoid, gradient hidden sẽ quá lớn và sai bản chất.

9. Bảng Công thức Thi: Lan truyền Xuôi, Lan truyền Ngược, Cập nhật

Giai đoạn	Công thức	Kích thước
Đầu vào	$\mathbf{A}^{[0]} = \mathbf{X}^T$	$n_0 \times m$
Tuyến tính xuôi	$\mathbf{Z}^{[l]} = \mathbf{W}^{[l]} \mathbf{A}^{[l-1]} + \mathbf{b}^{[l]}$	$n_l \times m$
Kích hoạt xuôi	$\mathbf{A}^{[l]} = g^{[l]}(\mathbf{Z}^{[l]})$	$n_l \times m$
Lỗi đầu ra, sigmoid + BCE	$\mathbf{dZ}^{[L]} = \mathbf{A}^{[L]} - \mathbf{Y}$	$n_L \times m$
Lỗi tầng ẩn	$\mathbf{dZ}^{[l]} = ((\mathbf{W}^{[l+1]})^T \mathbf{dZ}^{[l+1]}) \odot g'(\mathbf{Z}^{[l]})$	$n_l \times m$
Đạo hàm theo trọng số	$\mathbf{dW}^{[l]} = \frac{1}{m} \mathbf{dZ}^{[l]} (\mathbf{A}^{[l-1]})^T$	$n_l \times n_{l-1}$
Đạo hàm theo độ lệch	$\mathbf{db}^{[l]} = \frac{1}{m} \sum_i \mathbf{dZ}^{(i)[l]}$	$n_l \times 1$
Cập nhật trọng số	$\mathbf{W}^{[l]} \leftarrow \mathbf{W}^{[l]} - \alpha \mathbf{dW}^{[l]}$	giống $\mathbf{W}^{[l]}$
Cập nhật độ lệch	$\mathbf{b}^{[l]} \leftarrow \mathbf{b}^{[l]} - \alpha \mathbf{db}^{[l]}$	giống $\mathbf{b}^{[l]}$

Quy tắc kiểm tra nhanh:

- dW phải cùng kích thước với W .
- db phải cùng kích thước với b .
- dZ phải cùng kích thước với Z .
- Nếu phép nhân ma trận không khớp kích thước, hãy kiểm tra lại chiều mini-batch: thường mỗi cột là một mẫu.

dW từ backprop: $\begin{bmatrix} -0.045927 & 0.072171 \end{bmatrix}$

dW từ sai phân số: $\begin{bmatrix} -0.045927 & 0.072171 \end{bmatrix}$

Sai lệch tuyệt đối: $\begin{bmatrix} 0. & 0. \end{bmatrix}$

10. Bộ bài tập mẫu có lời giải

Bài 1: Xác định kích thước ma trận

Đề bài. Một mini-batch có $m = 5$ mẫu, tầng trước có $n_{l-1} = 3$ nơ-ron, tầng hiện tại có $n_l = 4$ nơ-ron. Hãy xác định kích thước của $\mathbf{A}^{[l-1]}$, $\mathbf{W}^{[l]}$, $\mathbf{b}^{[l]}$, $\mathbf{Z}^{[l]}$, $\mathbf{dW}^{[l]}$.

Lời giải.

Vì mỗi cột là một mẫu:

$$\mathbf{A}^{[l-1]} \in \mathbb{R}^{3 \times 5}$$

Vì tầng hiện tại có 4 nơ-ron, mỗi nơ-ron nhận 3 giá trị kích hoạt từ tầng trước:

$$\mathbf{W}^{[l]} \in \mathbb{R}^{4 \times 3}$$

Độ lệch có một số cho mỗi nơ-ron:

$$\mathbf{b}^{[l]} \in \mathbb{R}^{4 \times 1}$$

Tuyến tính xuôi:

$$\mathbf{Z}^{[l]} = \mathbf{W}^{[l]} \mathbf{A}^{[l-1]} + \mathbf{b}^{[l]}$$

Kích thước:

$$(4 \times 3)(3 \times 5) + (4 \times 1) = 4 \times 5$$

Nên:

$$\mathbf{Z}^{[l]} \in \mathbb{R}^{4 \times 5}$$

Đạo hàm theo trọng số luôn cùng kích thước với trọng số:

$$\mathbf{dW}^{[l]} \in \mathbb{R}^{4 \times 3}$$

Kết luận. Nếu dW không cùng kích thước với W , phép nhân trong backprop đã sai.

Bài 2: Lan truyền xuôi qua mạng 2-2-1

Đề bài. Cho mạng có 2 đầu vào, 2 nơ-ron ẩn dùng ReLU, và 1 đầu ra dùng sigmoid:

$$\mathbf{x} = \begin{pmatrix} 1.0 \\ 2.0 \end{pmatrix}, \quad \mathbf{W}^{[1]} = \begin{pmatrix} 0.5 & -0.4 \\ 0.3 & 0.8 \end{pmatrix}, \quad \mathbf{b}^{[1]} = \begin{pmatrix} 0.1 \\ -0.2 \end{pmatrix}$$

$$\mathbf{W}^{[2]} = \begin{pmatrix} 1.2 & -0.7 \end{pmatrix}, \quad b^{[2]} = 0.05$$

Tính $\mathbf{Z}^{[1]}$, $\mathbf{A}^{[1]}$, $Z^{[2]}$, và $A^{[2]}$.

Lời giải.

Tầng ẩn:

$$\mathbf{Z}^{[1]} = \begin{pmatrix} 0.5 & -0.4 \\ 0.3 & 0.8 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 0.1 \\ -0.2 \end{pmatrix}$$

Nơ-ron ẩn thứ nhất:

$$z_1^{[1]} = 0.5(1) - 0.4(2) + 0.1 = -0.2$$

Nơ-ron ẩn thứ hai:

$$z_2^{[1]} = 0.3(1) + 0.8(2) - 0.2 = 1.7$$

Do đó:

$$\mathbf{Z}^{[1]} = \begin{pmatrix} -0.2 \\ 1.7 \end{pmatrix}$$

Qua ReLU:

$$\mathbf{A}^{[1]} = \begin{pmatrix} \max(0, -0.2) \\ \max(0, 1.7) \end{pmatrix} = \begin{pmatrix} 0 \\ 1.7 \end{pmatrix}$$

Tầng đầu ra:

$$Z^{[2]} = \begin{pmatrix} 1.2 & -0.7 \end{pmatrix} \begin{pmatrix} 0 \\ 1.7 \end{pmatrix} + 0.05$$

$$Z^{[2]} = 1.2(0) - 0.7(1.7) + 0.05 = -1.14$$

Qua sigmoid:

$$A^{[2]} = \sigma(-1.14) = \frac{1}{1 + e^{1.14}} \approx 0.242$$

Kết luận. Mạng dự đoán xác suất khoảng 0.242 cho lớp 1.

Bài 3: Một bước lan truyền ngược ở tầng đầu ra

Đề bài. Với đầu ra của Bài 2 và nhãn thật $y = 1$, dùng sigmoid + binary cross-entropy. Tính $dZ^{[2]}$, $dW^{[2]}$, $db^{[2]}$.

Lời giải.

Với sigmoid + binary cross-entropy:

$$dZ^{[2]} = A^{[2]} - y$$

Thay số:

$$dZ^{[2]} = 0.242 - 1 = -0.758$$

Vì chỉ có một mẫu:

$$dW^{[2]} = dZ^{[2]}(A^{[1]})^T$$

$$dW^{[2]} = -0.758 \begin{pmatrix} 0 & 1.7 \end{pmatrix} = \begin{pmatrix} 0 & -1.289 \end{pmatrix}$$

Đạo hàm theo độ lệch:

$$db^{[2]} = dZ^{[2]} = -0.758$$

Nếu tốc độ học $\alpha = 0.1$, cập nhật tầng đầu ra:

$$W_1^{[2]} \leftarrow 1.2 - 0.1(0) = 1.2$$

$$W_2^{[2]} \leftarrow -0.7 - 0.1(-1.289) = -0.571$$

$$b^{[2]} \leftarrow 0.05 - 0.1(-0.758) = 0.126$$

Kết luận. Vì mô hình dự đoán quá thấp so với nhãn 1, đạo hàm làm độ lệch tăng đầu ra tăng và làm trọng số nối từ giá trị kích hoạt 1.7 bớt âm để lần sau xác suất lớp 1 cao hơn.

Bài 4: Backprop đầy đủ qua hidden layer với ký hiệu rõ ràng

Đề bài. Cho mạng 2-2-1, hidden sigmoid, output sigmoid, squared error:

$$L = \frac{1}{2}(\hat{y} - y)^2$$

Một mẫu dữ liệu:

$$x_1 = 2, \quad x_2 = -1, \quad y = 0$$

Tham số ban đầu:

Kết nối hoặc bias	Giá trị
$w_{x_1 \rightarrow h_1}$	0.25
$w_{x_2 \rightarrow h_1}$	-0.15
$w_{x_1 \rightarrow h_2}$	-0.35
$w_{x_2 \rightarrow h_2}$	0.45
$w_{h_1 \rightarrow o}$	0.55
$w_{h_2 \rightarrow o}$	-0.25
b_{h_1}, b_{h_2}, b_o	-0.05, 0.20, -0.10

Tính forward, loss, toàn bộ gradient, rồi cập nhật với $\eta = 0.1$.

Lời giải.

Forward tầng ẩn:

$$z_{h_1} = 0.25(2) + (-0.15)(-1) - 0.05 = 0.60, \quad h_1 = \sigma(0.60) \approx 0.6457$$

$$z_{h_2} = (-0.35)(2) + 0.45(-1) + 0.20 = -0.95, \quad h_2 = \sigma(-0.95) \approx 0.2789$$

Forward output:

$$z_o = 0.55(0.6457) + (-0.25)(0.2789) - 0.10 \approx 0.1854$$

$$\hat{y} = \sigma(0.1854) \approx 0.5462$$

Loss:

$$L = \frac{1}{2}(0.5462 - 0)^2 \approx 0.1492$$

Output error:

$$\delta_o = \frac{\partial L}{\partial z_o} = (\hat{y} - y)\hat{y}(1 - \hat{y}) = (0.5462 - 0)(0.5462)(0.4538) \approx 0.1354$$

Gradient tầng output:

$$\frac{\partial L}{\partial w_{h_1 \rightarrow o}} = \delta_o h_1 = (0.1354)(0.6457) \approx 0.0874$$

$$\frac{\partial L}{\partial w_{h_2 \rightarrow o}} = \delta_o h_2 = (0.1354)(0.2789) \approx 0.0378$$

$$\frac{\partial L}{\partial b_o} = \delta_o \approx 0.1354$$

Hidden error:

$$\delta_{h_1} = \delta_o w_{h_1 \rightarrow o} h_1 (1 - h_1) = (0.1354)(0.55)(0.6457)(0.3543) \approx 0.0170$$

$$\delta_{h_2} = \delta_o w_{h_2 \rightarrow o} h_2 (1 - h_2) = (0.1354)(-0.25)(0.2789)(0.7211) \approx -0.0068$$

Gradient tầng ẩn:

$$\frac{\partial L}{\partial w_{x_1 \rightarrow h_1}} = \delta_{h_1} x_1 = (0.0170)(2) \approx 0.0341, \quad \frac{\partial L}{\partial w_{x_2 \rightarrow h_1}} = \delta_{h_1} x_2 = (0.0170)(-1) \approx -0.0170$$

$$\frac{\partial L}{\partial w_{x_1 \rightarrow h_2}} = \delta_{h_2} x_1 = (-0.0068)(2) \approx -0.0136, \quad \frac{\partial L}{\partial w_{x_2 \rightarrow h_2}} = \delta_{h_2} x_2 = (-0.0068)(-1) \approx 0.0068$$

$$\frac{\partial L}{\partial b_{h_1}} = \delta_{h_1}, \quad \frac{\partial L}{\partial b_{h_2}} = \delta_{h_2}$$

Update với $\eta = 0.1$:

Tham số	Giá trị mới
$w_{x_1 \rightarrow h_1}$	$0.25 - 0.1(0.0341) = 0.2466$
$w_{x_2 \rightarrow h_1}$	$-0.15 - 0.1(-0.0170) = -0.1483$
$w_{x_1 \rightarrow h_2}$	$-0.35 - 0.1(-0.0136) = -0.3486$
$w_{x_2 \rightarrow h_2}$	$0.45 - 0.1(0.0068) = 0.4493$
$w_{h_1 \rightarrow o}$	$0.55 - 0.1(0.0874) = 0.5413$
$w_{h_2 \rightarrow o}$	$-0.25 - 0.1(0.0378) = -0.2538$
b_{h_1}	$-0.05 - 0.1(0.0170) = -0.0517$
b_{h_2}	$0.20 - 0.1(-0.0068) = 0.2007$
b_o	$-0.10 - 0.1(0.1354) = -0.1135$

Cách tự kiểm tra dấu. Vì $y = 0$ nhưng $\hat{y} = 0.5462$ còn cao, $\delta_o > 0$. Khi cập nhật $w \leftarrow w - \eta dW$, các gradient dương bị trừ đi, giúp lần dự đoán sau có xu hướng kéo $\hat{y}$ xuống thấp hơn.

Phần 8: Học máy: Tối ưu hóa, Thiên lệch-Phương sai, và Điều chuẩn

Nguồn: `quarto/Optimization_Bias_Variance-Regularization.qmd`

1. Tối ưu hóa bằng Gradient Descent

Bản đồ ký hiệu tối ưu hóa và regularization

Ký hiệu	Nghĩa	Cách dùng
$J(\theta)$	Hàm mục tiêu/loss cần tối thiểu hóa	Theo dõi xem mô hình đang tốt lên hay xấu đi.
θ hoặc β	Tham số mô hình	Thứ được cập nhật qua gradient descent.
α	Learning rate	Bước đi lớn hay nhỏ trên hướng gradient.
∇J	Gradient	Hướng tăng nhanh nhất của loss.
λ	Độ mạnh regularization	Lớn hơn nghĩa là phạt mô hình phức tạp mạnh hơn.
Bias	Sai lệch hệ thống của mô hình trung bình	Cao khi mô hình quá đơn giản.
Variance	Độ nhạy với tập huấn luyện	Cao khi mô hình quá phức tạp.
Train/validation/test	Ba vai trò dữ liệu khác nhau	Học tham số, chọn siêu tham số, đánh giá cuối.

Quy trình khi gặp bài tính: viết loss, tính gradient của phần dữ liệu, cộng gradient của penalty nếu có, cập nhật tham số, rồi giải thích kết quả theo bias/variance hoặc overfit/underfit.

Giả sử ta tối thiểu hóa loss hồi quy tuyến tính:

$$L(\beta) = \frac{1}{2n} \sum_{i=1}^n (y_i - \beta^T \mathbf{x}_i)^2$$

Gradient descent cập nhật tham số theo quy tắc:

$$\beta^{(t+1)} = \beta^{(t)} - \alpha \nabla L(\beta^{(t)})$$

Trong đó α là learning rate. Nếu α quá nhỏ, mô hình học chậm. Nếu α quá lớn, loss có thể dao động hoặc tăng.

Với từng hệ số β_j :

$$\frac{\partial L}{\partial \beta_j} = -\frac{1}{n} \sum_{i=1}^n (y_i - \beta^T \mathbf{x}_i) x_{i,j}$$

Suy ra cập nhật:

$$\beta_j \leftarrow \beta_j + \frac{\alpha}{n} \sum_{i=1}^n (y_i - \beta^T \mathbf{x}_i) x_{i,j}$$

Thực tế: nếu residual $y_i - \hat{y}_i$ dương, mô hình đang dự đoán thấp hơn thực tế. Ta tăng các hệ số tương ứng với đặc trưng dương để kéo dự đoán lên.

2. Batch, Stochastic, và Mini-batch

Biến thể	Dữ liệu dùng trong một lần cập nhật	Công thức	Khi nào nhớ tới
Batch Gradient Descent	Toàn bộ n mẫu	$\beta_j \leftarrow \beta_j + \frac{\alpha}{n} \sum_i (y_i - \hat{y}_i) x_{i,j}$	Mượt, ổn định, nhưng chậm với dữ liệu lớn.
Stochastic Gradient Descent	Một mẫu ngẫu nhiên	$\beta_j \leftarrow \beta_j + \alpha (y_i - \hat{y}_i) x_{i,j}$	Nhanh, nhiễu cao, loss dao động.
Mini-batch Gradient Descent	Một nhóm m mẫu	$\beta_j \leftarrow \beta_j + \frac{\alpha}{m} \sum_{i \in \mathcal{B}} (y_i - \hat{y}_i) x_{i,j}$	Cân bằng giữa tốc độ, vector hóa, và ổn định.

```
import numpy as np
import matplotlib.pyplot as plt

beta_grid = np.linspace(-1, 5, 300)
loss_grid = (beta_grid - 2.3) ** 2 + 0.4

beta = -0.5
alpha = 0.22
path = [beta]
for _ in range(12):
```

```

    grad = 2 * (beta - 2.3)
    beta = beta - alpha * grad
    path.append(beta)

path = np.array(path)
path_loss = (path - 2.3) ** 2 + 0.4

plt.figure(figsize=(7, 4))
plt.plot(beta_grid, loss_grid, color="#4C78A8", label="loss")
plt.scatter(path, path_loss, color="#F58518", zorder=3, label="các bước cập
↪ nhật")
for a, b, la, lb in zip(path[:-1], path[1:], path_loss[:-1], path_loss[1:]):
    plt.annotate("", xy=(b, lb), xytext=(a, la), arrowprops={"arrowstyle":
↪ "->", "color": "#F58518"})
plt.xlabel("tham số beta")
plt.ylabel("loss")
plt.title("Learning rate điều khiển độ dài bước đi")
plt.legend()
plt.grid(alpha=0.25)
plt.show()

```

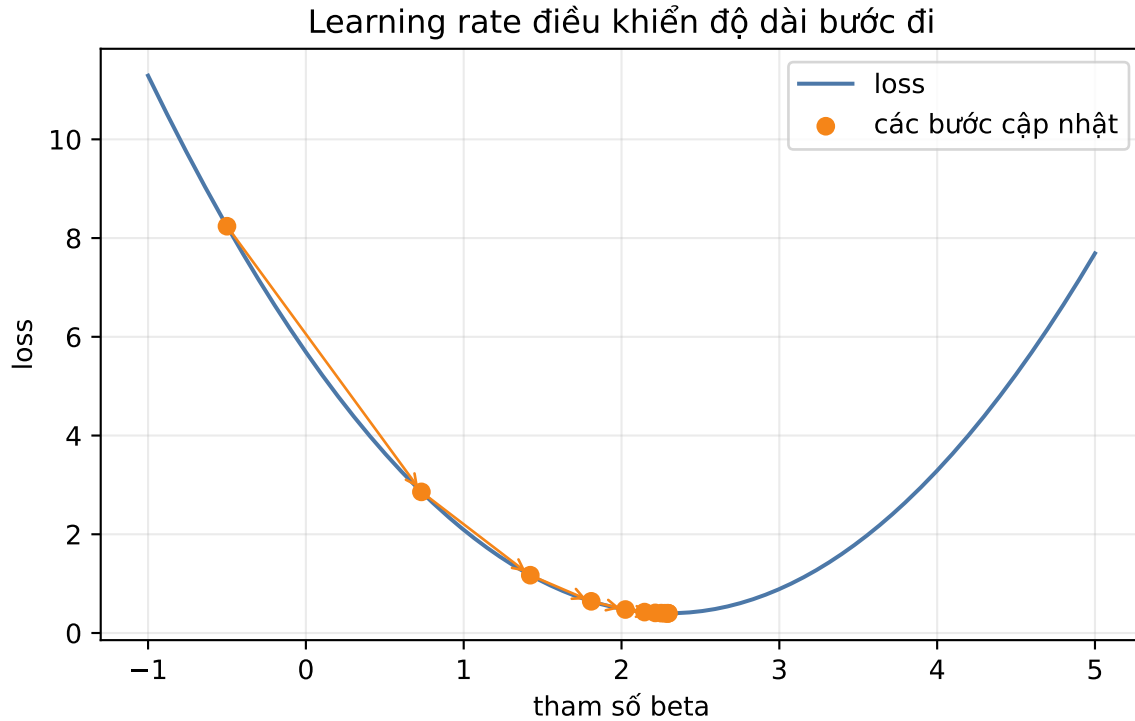


Figure 12: Gradient descent đi từng bước xuống đáy của hàm loss lồi.

3. Underfitting và Overfitting

Khi mô hình gặp dữ liệu mới, ta quan tâm test loss:

$$L_{\text{test}} = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} [(y - \hat{f}(\mathbf{x}))^2]$$

Hai kiểu lỗi chính:

- **Underfitting, high bias:** train loss cao, test loss cao. Mô hình quá đơn giản, không học được quy luật thật.
- **Overfitting, high variance:** train loss thấp, test loss cao. Mô hình quá linh hoạt, học cả nhiễu của tập train.

Một câu nhớ nhanh: bias là lỗi do giả định quá cứng; variance là lỗi do mô hình thay đổi quá mạnh khi tập train thay đổi.

4. Bias-Variance Decomposition

Giả sử:

$$y = f(\mathbf{x}) + \epsilon, \quad \mathbb{E}[\epsilon] = 0, \quad \text{Var}(\epsilon) = \sigma^2$$

Mô hình học từ một tập dữ liệu ngẫu nhiên $\mathcal{D}$, nên dự đoán $\hat{f}(\mathbf{x}; \mathcal{D})$ cũng là biến ngẫu nhiên. Tại một điểm $\mathbf{x}$ cố định:

$$\mathbb{E}[(y - \hat{f})^2] = \text{Bias}(\hat{f})^2 + \text{Var}(\hat{f}) + \sigma^2$$

Suy luận từng bước:

$$\mathbb{E}[(y - \hat{f})^2] = \mathbb{E}[(f + \epsilon - \hat{f})^2]$$

Khai triển bình phương:

$$\mathbb{E}[(f - \hat{f})^2 + 2\epsilon(f - \hat{f}) + \epsilon^2]$$

Vì $\mathbb{E}[\epsilon] = 0$, hạng chéo bằng 0. Vì $\mathbb{E}[\epsilon^2] = \sigma^2$, còn:

$$\mathbb{E}_{\mathcal{D}}[(f - \hat{f})^2] + \sigma^2$$

Thêm và bớt dự đoán trung bình $\mathbb{E}_{\mathcal{D}}[\hat{f}]$:

$$f - \hat{f} = (f - \mathbb{E}_{\mathcal{D}}[\hat{f}]) + (\mathbb{E}_{\mathcal{D}}[\hat{f}] - \hat{f})$$

Hạng thứ nhất tạo **bias bình phương**. Hạng thứ hai tạo **variance**. Hạng chéo bằng 0 vì độ lệch quanh trung bình có kỳ vọng bằng 0.

5. Regularization

Regularization thêm hình phạt vào loss để mô hình bớt đuối theo nhiễu:

$$L_{\text{reg}}(\beta) = L(\beta) + \lambda R(\beta)$$

Trong đó $\lambda \geq 0$ điều khiển độ mạnh của phạt.

Phương pháp	Penalty	Tác dụng
Ridge, L_2	$\lambda \sum_j \beta_j^2$	Co nhỏ hệ số một cách mượt, giảm variance.
Lasso, L_1	$\lambda \sum_j \beta_j $	Có thể đẩy một số hệ số về đúng 0, chọn đặc trưng.
Elastic Net	$\lambda_1 \ \beta\ _1 + \lambda_2 \ \beta\ _2^2$	Kết hợp sparse và ổn định.

Nếu λ quá nhỏ, mô hình có thể overfit. Nếu λ quá lớn, mô hình có thể underfit.

6. Chọn siêu tham số bằng Cross-validation

Quy trình K -fold cross-validation:

1. Chia tập train thành K fold.
2. Với mỗi ứng viên λ , train trên $K - 1$ fold và validate trên fold còn lại.
3. Lặp để mỗi fold được làm validation một lần.
4. Lấy trung bình validation loss.
5. Chọn λ có validation loss trung bình nhỏ nhất.
6. Train lại trên toàn bộ tập train bằng λ đã chọn.

Không dùng test set để chọn λ , vì test set phải là phần đánh giá cuối cùng.

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(3)
x = np.linspace(-3, 3, 35)
true_y = np.sin(x)
y = true_y + np.random.normal(0, 0.22, size=x.shape)
x_grid = np.linspace(-3, 3, 300)
```

```

true_grid = np.sin(x_grid)

degrees = [1, 5, 18]
titles = ["High bias\nunderfit", "Cân bằng", "High variance\noverfit"]
fig, axes = plt.subplots(1, 3, figsize=(10, 3.4), sharey=True)

for ax, degree, title in zip(axes, degrees, titles):
    coef = np.polyfit(x, y, degree)
    pred = np.polyval(coef, x_grid)
    train_mse = np.mean((np.polyval(coef, x) - y) ** 2)
    test_mse = np.mean((pred - true_grid) ** 2)
    ax.scatter(x, y, color="#4C78A8", s=35, label="mẫu train")
    ax.plot(x_grid, true_grid, color="black", linestyle="--", label="hàm
    ⇨ thật")
    ax.plot(x_grid, pred, color="#F58518", linewidth=2, label="mô hình")
    ax.set_title(f"{title}\nbậc={degree}")
    ax.text(-2.9, 1.15, f"train MSE={train_mse:.2f}\ntest
    ⇨ MSE={test_mse:.2f}", fontsize=8)
    ax.set_ylim(-1.5, 1.5)
    ax.grid(alpha=0.25)
axes[0].legend(fontsize=8, loc="lower left")
plt.tight_layout()
plt.show()

```

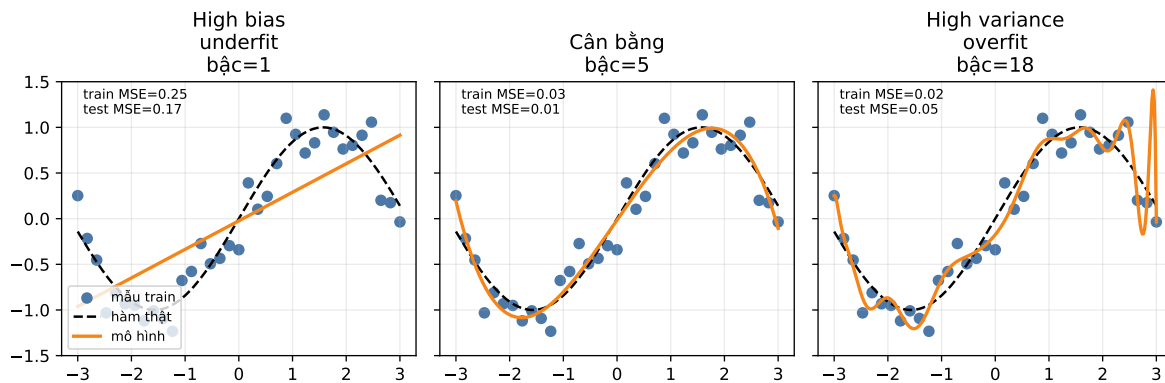


Figure 13: Underfitting, fit hợp lý, và overfitting trên cùng một bộ dữ liệu nhiễu.

7. Quy trình xử lý bài tối ưu hóa và regularization

Khi đề yêu cầu một bước cập nhật:

1. Viết dự đoán hiện tại tại $\hat{y}$.
2. Viết residual hoặc sai số dự đoán.
3. Tính gradient phần loss dữ liệu.
4. Tính gradient penalty:

$$\nabla_{\beta} \lambda \|\beta\|_2^2 = 2\lambda\beta$$

5. Cộng gradient:

$$\nabla J_{total} = \nabla J_{data} + \nabla J_{penalty}$$

6. Cập nhật:

$$\beta_{new} = \beta - \alpha \nabla J_{total}$$

Bẫy thường gặp: regularization thường không phạt bias/intercept trong nhiều triển khai. Nếu đề không nói rõ, hãy nêu giả định trước khi tính.

8. Bảng Công thức Thi: Bias, Variance, Regularization

Dấu hiệu	Chẩn đoán	Cách xử lý
Train loss cao, test loss cao	High bias	Thêm đặc trưng, tăng bậc mô hình, giảm regularization.
Train loss thấp, test loss cao	High variance	Tăng regularization, đơn giản hóa mô hình, thêm dữ liệu.
Train loss thấp, test loss thấp	Fit tốt	Giữ mô hình, kiểm tra thêm bằng dữ liệu mới.
Validation loss nhỏ nhất tại λ trung bình	Trade-off tốt	Chọn λ đó rồi train lại trên full train.
Loss dao động mạnh khi train	Learning rate lớn hoặc SGD nhiều	Giảm α , dùng mini-batch, chuẩn hóa dữ liệu.

Gradient cần nhớ:

$$\nabla L = -\frac{1}{n} \mathbf{X}^T (\mathbf{y} - \mathbf{X}\beta)$$

Ridge gradient add-on:

$$\nabla L_{\text{ridge}} = \nabla L + 2\lambda\beta$$

Bias-variance:

$$\text{MSE} = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

Dạng bài: một bước gradient descent có regularization

Giải sử loss Ridge:

$$J(\beta) = \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_2^2$$

Gradient tổng:

$$\nabla J = -\frac{1}{n} \mathbf{X}^T (\mathbf{y} - \mathbf{X}\beta) + 2\lambda\beta$$

Nếu không phạt bias, tách $\beta = (\beta_0, \beta_1, \dots, \beta_p)$ và chỉ phạt từ β_1 trở đi:

$$\nabla_{\beta_0} J = \nabla_{\beta_0} J_{\text{data}}$$

$$\nabla_{\beta_j} J = \nabla_{\beta_j} J_{\text{data}} + 2\lambda\beta_j, \quad j \geq 1$$

Sau đó cập nhật:

$$\beta_{\text{new}} = \beta - \alpha \nabla J$$

Cách kiểm tra trực giác: nếu λ lớn, các hệ số không phải bias bị kéo về 0 mạnh hơn. Nếu bắt đầu tại $\beta = 0$, gradient của penalty bằng 0 ở bước đầu tiên; tác dụng regularization xuất hiện rõ hơn ở các bước sau.

9. Bộ bài tập mẫu có lời giải

Bài 1: Tính objective Ridge

Đề bài. Một mô hình có train loss 0.20 và trọng số $\beta = (2, -1, 0.5)$. Tính ridge objective với $\lambda = 0.1$.

Lời giải.

Tính penalty:

$$\|\beta\|_2^2 = 2^2 + (-1)^2 + 0.5^2 = 4 + 1 + 0.25 = 5.25$$

Nhân với λ :

$$\lambda\|\beta\|_2^2 = 0.1(5.25) = 0.525$$

Cộng với loss gốc:

$$L_{\text{reg}} = 0.20 + 0.525 = 0.725$$

Kết luận. Ridge objective bằng 0.725.

Bài 2: Chọn λ từ bảng cross-validation

Đề bài. Sau 5-fold cross-validation, ta có validation MSE:

λ	fold 1	fold 2	fold 3	fold 4	fold 5
0.00	0.92	1.10	0.98	1.05	1.20
0.01	0.70	0.78	0.74	0.76	0.81
0.10	0.54	0.57	0.55	0.58	0.56
1.00	0.61	0.65	0.64	0.62	0.67
10.00	1.20	1.15	1.25	1.18	1.22

Chọn λ tốt nhất.

Lời giải. Tính trung bình từng dòng:

$$\lambda = 0 : \frac{0.92 + 1.10 + 0.98 + 1.05 + 1.20}{5} = 1.050$$

$$\lambda = 0.01 : \frac{0.70 + 0.78 + 0.74 + 0.76 + 0.81}{5} = 0.758$$

$$\lambda = 0.10 : \frac{0.54 + 0.57 + 0.55 + 0.58 + 0.56}{5} = 0.560$$

$$\lambda = 1.00 : \frac{0.61 + 0.65 + 0.64 + 0.62 + 0.67}{5} = 0.638$$

$$\lambda = 10.00 : \frac{1.20 + 1.15 + 1.25 + 1.18 + 1.22}{5} = 1.200$$

Bảng tóm tắt:

λ	validation MSE trung bình
0.00	1.050
0.01	0.758
0.10	0.560
1.00	0.638
10.00	1.200

Kết luận. Chọn $\lambda = 0.10$ vì validation MSE trung bình nhỏ nhất. $\lambda = 0$ có dấu hiệu overfit, còn $\lambda = 10$ quá mạnh nên dễ underfit.

Bài 3: Một bước Gradient Descent trên dữ liệu 12 dòng

Đề bài. Dữ liệu có một đặc trưng x và intercept. Bắt đầu với $\beta_0 = 0, \beta_1 = 0$, learning rate $\alpha = 0.01$. Tính một bước batch gradient descent cho loss MSE.

dòng	x	y
1	1	4
2	2	5
3	3	7
4	4	8
5	5	11
6	6	13
7	7	15
8	8	16
9	9	19

dòng	x	y
10	10	21
11	11	22
12	12	25

Lời giải. Với $\beta_0 = \beta_1 = 0$, mọi dự đoán ban đầu bằng 0, nên residual $r_i = y_i$.

Gradient theo dạng cập nhật cộng:

$$\beta_j \leftarrow \beta_j + \frac{\alpha}{n} \sum_{i=1}^n r_i x_{i,j}$$

Với intercept, $x_{i,0} = 1$.

Tính tổng residual:

$$\sum r_i = 4 + 5 + 7 + 8 + 11 + 13 + 15 + 16 + 19 + 21 + 22 + 25 = 166$$

Tính tổng residual nhân đặc trưng:

$$\sum r_i x_i = 4(1) + 5(2) + 7(3) + 8(4) + 11(5) + 13(6) + 15(7) + 16(8) + 19(9) + 21(10) + 22(11) + 25(12) = 1356$$

Cập nhật intercept:

$$\beta_0^{new} = 0 + \frac{0.01}{12}(166) \approx 0.1383$$

Cập nhật slope:

$$\beta_1^{new} = 0 + \frac{0.01}{12}(1356) = 1.1300$$

Kết luận. Một bước cập nhật đã kéo đường dự đoán từ đường phẳng $\hat{y} = 0$ về phía dữ liệu. Trong bài thi, chỉ cần nhớ: residual nhân đặc trưng rồi lấy trung bình.

Phần 9: Học máy: Học Không Giám sát và Phân cụm K-means

Nguồn: `quarto/Unsupervised_Learning_K_Means.qmd`

1. Tổng quan về Học Không Giám Sát (Unsupervised Learning)

Bản đồ ký hiệu K-means

Ký hiệu	Nghĩa	Vai trò trong thuật toán
$\mathbf{x}^{(i)}$	Điểm dữ liệu thứ i	Được gán vào một cụm.
K	Số cụm cần tìm	Chọn trước hoặc chọn bằng elbow/silhouette.
c_i	Nhãn cụm của điểm i	Kết quả của bước gán cụm.
μ_k	Tâm cụm thứ k	Trung bình các điểm đang thuộc cụm k .
J	Distortion/SSE	Tổng khoảng cách bình phương tới tâm cụm.
Iteration	Một vòng gán cụm rồi cập nhật tâm	Lặp đến khi nhãn/tâm không đổi.

Quy trình tính tay: lập bảng khoảng cách từ từng điểm tới từng tâm, chọn khoảng cách nhỏ nhất để gán cụm, tính lại tâm bằng trung bình, rồi tính distortion để kiểm tra thuật toán có giảm lỗi hay không.

Trong khi Học có giám sát (Supervised Learning) làm việc với cặp dữ liệu huấn luyện đầy đủ nhãn $\{(\mathbf{x}^{(i)}, y^{(i)})\}$, **Học không giám sát** chỉ nhận đầu vào là một tập hợp các vector đặc trưng không có nhãn mục tiêu kèm theo:

$$\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^n \quad \text{với } \mathbf{x}^{(i)} \in \mathbb{R}^p$$

Mục tiêu của hệ thống là tự tìm ra các mô thức cấu trúc ẩn (Hidden structures) hoặc quy luật phân bố của dữ liệu. Các nhánh bài toán chính bao gồm: * **Phân cụm (Clustering)**: Gom nhóm các điểm dữ liệu tương đồng vào cùng một cụm. * **Giảm chiều dữ liệu (Dimensionality Reduction)**: Nén không gian thuộc tính từ bậc cao về bậc thấp mà vẫn giữ tối đa lượng thông tin (ví dụ: PCA). * **Phát hiện dị biệt (Anomaly Detection)**: Tìm kiếm các điểm dữ liệu bất thường lệch sâu khỏi phân phối chuẩn.

2. Giải thuật Phân Cụm K-means (K-means Clustering)

K-means là thuật toán phân cụm dựa trên phân hoạch không gian (Partitioning method). Thuật toán cố gắng chia n mẫu dữ liệu thành K cụm khác nhau ($C_1, C_2, \dots, C_K$) cố định sao cho tổng bình phương khoảng cách từ các điểm đến tâm cụm (Centroid) tương ứng là nhỏ nhất.

2.1 Hàm Mục Tiêu Định Lượng (Distortion Function / Cost Function)

Về mặt toán học, K-means tối thiểu hóa hàm chi phí J , còn được gọi là độ méo dạng (Distortion Function):

$$J(c^{(1)}, \dots, c^{(n)}, \mu_1, \dots, \mu_K) = \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}^{(i)} - \mu_{c^{(i)}}\|^2$$

Trong đó: * $c^{(i)} \in \{1, 2, \dots, K\}$ là chỉ số cụm được gán cho mẫu dữ liệu $\mathbf{x}^{(i)}$. * $\mu_k \in \mathbb{R}^p$ là vector tọa độ tâm của cụm thứ k . * $\|\cdot\|$ đại diện cho khoảng cách hình học L2 (Euclidean distance).

Diễn giải cho người mới học. Với mỗi điểm dữ liệu, K-means chỉ hỏi một câu: “Tâm cụm nào gần điểm này nhất?” Khoảng cách bình phương là mức phạt cho phép gán đó. Hàm mục tiêu là mức phạt trung bình trên toàn bộ dữ liệu, nên phân cụm tốt nghĩa là các điểm nằm gần tâm cụm được gán của chúng.

Với một cụm cố định C_k , bước cập nhật tâm cụm giải bài toán nhỏ hơn:

$$\min_{\mu_k} \sum_{i \in C_k} \|\mathbf{x}^{(i)} - \mu_k\|^2$$

Lấy đạo hàm theo μ_k :

$$\sum_{i \in C_k} 2(\mu_k - \mathbf{x}^{(i)}) = 0$$

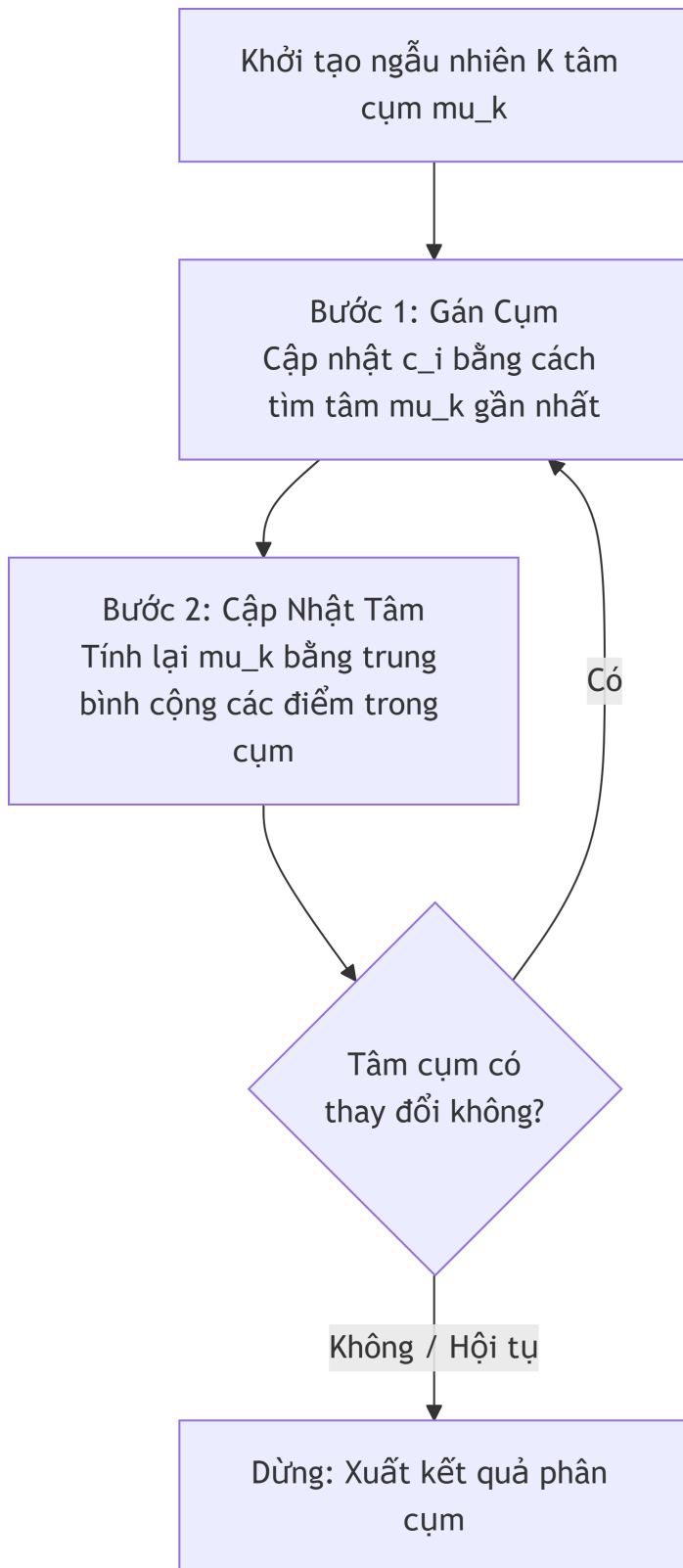
Cô lập tâm cụm:

$$|C_k| \mu_k = \sum_{i \in C_k} \mathbf{x}^{(i)} \implies \mu_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}^{(i)}$$

Vì vậy, tâm cụm tốt nhất cho một nhóm điểm cố định chính là trung bình cộng của các điểm trong nhóm đó.

2.2 Quy Trình Vòng Lặp Hội Tụ (Coordinate Descent)

Hàm mục tiêu J phụ thuộc vào hai nhóm biến độc lập: chỉ số gán cụm $c^{(i)}$ và tọa độ tâm μ_k . Để tối ưu, K -means thực hiện kỹ thuật tối ưu hóa luân phiên theo từng nhóm biến (Coordinate Descent):



1. **Bước gán cụm (Cluster Assignment Step):** Giữ cố định toàn bộ các tâm cụm μ_k , tối ưu hóa hàm chi phí bằng cách tìm chỉ số cụm $c^{(i)}$ gần nhất cho từng mẫu dữ liệu:

$$c^{(i)} \leftarrow \arg \min_k \|\mathbf{x}^{(i)} - \mu_k\|^2$$

2. **Bước cập nhật tâm cụm (Centroid Update Step):** Giữ cố định toàn bộ các chỉ số gán cụm $c^{(i)}$, tối ưu hóa tọa độ các tâm cụm μ_k bằng cách lấy đạo hàm riêng của J theo từng μ_k và đặt bằng 0. Kết quả nghiệm đóng chính là tọa độ trung bình cộng:

$$\mu_k \leftarrow \frac{\sum_{i=1}^n \mathbf{1}\{c^{(i)} = k\} \mathbf{x}^{(i)}}{\sum_{i=1}^n \mathbf{1}\{c^{(i)} = k\}} = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}^{(i)}$$

Chứng minh tính hội tụ: Vì tại mỗi phân đoạn của Bước 1 và Bước 2, giá trị của hàm chi phí J luôn luôn giảm hoặc giữ nguyên ($J^{(t+1)} \leq J^{(t)}$), đồng thời hàm J luôn bị chặn dưới bởi số 0, nên giải thuật K -means **chắc chắn luôn luôn hội tụ** về một điểm cực trị sau một số hữu hạn vòng lặp. Tuy nhiên, vì đây là hàm không lồi hoàn toàn, thuật toán có thể bị kẹt ở điểm cực tiểu cục bộ (Local Minimum) phụ thuộc vào cách khởi tạo tâm cụm ban đầu.

Bản chứng minh đầy đủ hơn để dùng khi thi.

K-means tối ưu hàm:

$$J(\mathbf{c}, \mu) = \sum_{i=1}^m \left\| \mathbf{x}^{(i)} - \mu_{c_i} \right\|_2^2$$

trong đó c_i là nhãn cụm của điểm thứ i .

Bước 1: Gán cụm khi tâm cố định. Với các tâm cụm không đổi, mỗi điểm được gán về tâm gần nhất:

$$c_i^{new} = \arg \min_k \left\| \mathbf{x}^{(i)} - \mu_k \right\|_2^2$$

nên:

$$\left\| \mathbf{x}^{(i)} - \mu_{c_i^{new}} \right\|_2^2 \leq \left\| \mathbf{x}^{(i)} - \mu_{c_i^{old}} \right\|_2^2$$

cho từng điểm. Cộng trên toàn bộ dữ liệu suy ra J không tăng.

Bước 2: Cập nhật tâm khi nhãn cố định. Với một cụm C_k đã cố định, ta cần giải:

$$\min_{\mu_k} \sum_{\mathbf{x}^{(i)} \in C_k} \|\mathbf{x}^{(i)} - \mu_k\|_2^2$$

Lấy gradient:

$$-2 \sum_{\mathbf{x}^{(i)} \in C_k} \mathbf{x}^{(i)} + 2|C_k|\mu_k = 0$$

suy ra:

$$\mu_k = \frac{1}{|C_k|} \sum_{\mathbf{x}^{(i)} \in C_k} \mathbf{x}^{(i)}$$

Tức là tâm mới chính là trung bình cộng, nghiệm tối ưu của cụm đó. Vì vậy bước cập nhật tâm cũng không làm tăng J .

Bước 3: Vì sao hữu hạn? Có tối đa K^m cách gán m điểm vào K cụm. Mỗi vòng lặp làm J giảm hoặc giữ nguyên, và khi phép gán không đổi thì các tâm cũng không đổi. Vì số trạng thái gán cụm là hữu hạn, thuật toán phải dừng sau hữu hạn bước.

Điều không được hiểu sai: Hội tụ không có nghĩa là đạt cực tiểu toàn cục. K-means là coordinate descent trên một bài toán có biến rời rạc c_i và tâm liên tục μ_k , nên hàm mục tiêu không lồi toàn cục. Một nghiệm có thể ổn định với hai bước Lloyd nhưng vẫn kém hơn một cách chia khác. Trong thực tế nên dùng K-means++, chạy nhiều lần với nhiều khởi tạo, rồi chọn nghiệm có distortion nhỏ nhất.

3. Cải Tiến Chọn Tâm Ban Đầu: K-means++

Để giảm thiểu rủi ro bị kẹt ở cực tiểu cục bộ do khởi tạo các tâm ban đầu quá gần nhau, thuật toán **K-means++** bổ sung quy tắc chọn tâm thông minh dựa trên xác suất khoảng cách:

1. Chọn ngẫu nhiên tâm cụm đầu tiên μ_1 từ tập dữ liệu.
2. Với mỗi mẫu dữ liệu $\mathbf{x}^{(i)}$, tính toán khoảng cách ngắn nhất từ nó đến các tâm cụm đã được chọn trước đó, ký hiệu là $D(\mathbf{x}^{(i)})$.
3. Chọn tâm cụm tiếp theo μ_k từ các điểm dữ liệu dựa trên phân phối xác suất tỷ lệ thuận với bình phương khoảng cách:

$$P(\mathbf{x}^{(i)}) = \frac{D(\mathbf{x}^{(i)})^2}{\sum_{j=1}^n D(\mathbf{x}^{(j)})^2}$$

(Điểm nào nằm càng xa các tâm cụm đã có thì càng có xác suất cao được chọn làm tâm cụm mới). 4. Lặp lại bước 2 và 3 cho đến khi đủ K tâm cụm, sau đó tiến hành chạy vòng lặp K -means chuẩn.

4. Thực Hành Trace Thuật Toán Bằng Tay (Bản Số Hóa)

Dưới đây là phần số hóa chi tiết bài toán chạy mô phỏng từng bước (Trace) từ ghi chép thực tế của bạn với 5 mẫu dữ liệu trong không gian 2D ($p = 2$), phân thành $K = 2$ cụm.

4.1 Khởi tạo dữ liệu gốc (Input Data)

- $\mathbf{x}^{(1)} = (1, 1)$
- $\mathbf{x}^{(2)} = (2, 1)$
- $\mathbf{x}^{(3)} = (4, 1)$
- $\mathbf{x}^{(4)} = (7, 1)$
- $\mathbf{x}^{(5)} = (9, 1)$

Chọn 2 tâm cụm khởi tạo ban đầu ngẫu nhiên:

- $\mu_1^{(0)} = (2, 1)$
- $\mu_2^{(0)} = (4, 1)$

4.2 Vòng Lặp 1 (Iteration 1)

Bước 1: Gán cụm dựa trên khoảng cách Euclidean gần nhất

Tính toán khoảng cách từ các điểm dữ liệu đến hai tâm $\mu_1^{(0)}$ và $\mu_2^{(0)}$:

- Mẫu $\mathbf{x}^{(1)} = (1, 1)$: Gần $\mu_1^{(0)}$ hơn $\Rightarrow c^{(1)} = 1$.
- Mẫu $\mathbf{x}^{(2)} = (2, 1)$: Trùng với $\mu_1^{(0)} \Rightarrow c^{(2)} = 1$.
- Mẫu $\mathbf{x}^{(3)} = (4, 1)$: Trùng với $\mu_2^{(0)} \Rightarrow c^{(3)} = 2$.
- Mẫu $\mathbf{x}^{(4)} = (7, 1)$: Gần $\mu_2^{(0)}$ hơn $\Rightarrow c^{(4)} = 2$.

- Mẫu $\mathbf{x}^{(5)} = (9, 1)$: Gần $\mu_2^{(0)}$ hơn $\implies c^{(5)} = 2$.

Kết thúc bước gán cụm, ta có thành phần thành viên:

- Cụm 1: $C_1 = \{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}\}$
- Cụm 2: $C_2 = \{\mathbf{x}^{(3)}, \mathbf{x}^{(4)}, \mathbf{x}^{(5)}\}$

Bước 2: Cập nhật lại tọa độ các tâm cụm

Tính toán trung bình cộng tọa độ các điểm thành viên trong mỗi cụm:

- Tâm cụm 1 mới:

$$\mu_1^{(1)} = \frac{(1, 1) + (2, 1)}{2} = (1.5, 1)$$

- Tâm cụm 2 mới:

$$\mu_2^{(1)} = \frac{(4, 1) + (7, 1) + (9, 1)}{3} = \left(\frac{20}{3}, 1\right) \approx (6.67, 1)$$

4.3 Vòng Lặp 2 (Iteration 2)

Bước 1: Gán lại cụm dựa trên các tâm mới $\mu_1^{(1)}$ và $\mu_2^{(1)}$

- Mẫu $\mathbf{x}^{(1)} = (1, 1)$: Khoảng cách đến $\mu_1^{(1)}$ là 0.5, đến $\mu_2^{(1)}$ là 5.17 $\implies c^{(1)} = 1$.
- Mẫu $\mathbf{x}^{(2)} = (2, 1)$: Khoảng cách đến $\mu_1^{(1)}$ là 0.5, đến $\mu_2^{(1)}$ là 4.67 $\implies c^{(2)} = 1$.
- Mẫu $\mathbf{x}^{(3)} = (4, 1)$: Khoảng cách đến $\mu_1^{(1)}$ là 2.5, đến $\mu_2^{(1)}$ là 2.67 $\implies$ Vẫn gần $\mu_1^{(1)}$ hơn!
Điểm $\mathbf{x}^{(3)}$ bị nhảy cụm từ cụm 2 sang cụm 1 $\implies c^{(3)} = 1$.
- Mẫu $\mathbf{x}^{(4)} = (7, 1)$: Gần $\mu_2^{(1)}$ hơn $\implies c^{(4)} = 2$.
- Mẫu $\mathbf{x}^{(5)} = (9, 1)$: Gần $\mu_2^{(1)}$ hơn $\implies c^{(5)} = 2$.

Thành phần thành viên mới sau khi phân định lại:

- Cụm 1: $C_1 = \{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \mathbf{x}^{(3)}\}$
- Cụm 2: $C_2 = \{\mathbf{x}^{(4)}, \mathbf{x}^{(5)}\}$

Bước 2: Cập nhật lại tọa độ các tâm cụm một lần nữa

- Tâm cụm 1 mới:

$$\mu_1^{(2)} = \frac{(1, 1) + (2, 1) + (4, 1)}{3} = \left(\frac{7}{3}, 1\right) \approx (2.33, 1)$$

- Tâm cụm 2 mới:

$$\mu_2^{(2)} = \frac{(7, 1) + (9, 1)}{2} = (8, 1)$$

4.4 Kết luận hội tụ (Convergence)

Nếu ta tiếp tục chạy sang Vòng lặp 3, việc gán cụm cho các mẫu dựa trên các tâm cụm mới $(2.33, 1)$ và $(8, 1)$ sẽ giữ nguyên kết quả thành viên không đổi so với Vòng lặp 2. Thuật toán chính thức **đạt trạng thái ổn định**: phép gán cụm không đổi và các tâm cụm không còn di chuyển. Với K -means, đây là nghiệm hội tụ cục bộ phụ thuộc vào khởi tạo, không phải bảo đảm nghiệm tối ưu toàn cục cho mọi bộ dữ liệu.

5. Thực Hành Hiện Thực Hóa Thuật Toán Bằng Python

Đoạn mã Python dưới đây lập trình lại toàn bộ quy trình Coordinate Descent của thuật toán K -means chuẩn để bạn chạy kiểm thử:

```
import numpy as np

class KMeansClustering:
    def __init__(self, k=2, max_iters=100):
        self.k = k
        self.max_iters = max_iters
        self.centroids = None

    def fit(self, X):
        # Khởi tạo ngẫu nhiên K tâm cụm từ các điểm dữ liệu thực tế
        n_samples, n_features = X.shape
        random_indices = np.random.choice(n_samples, self.k, replace=False)
```

```

        self.centroids = X[random_indices]

    for _ in range(self.max_iters):
        # Bước 1: Gán cụm (Cluster Assignment)
        # Tính khoảng cách từ tất cả các điểm đến toàn bộ K tâm cụm
        distances = np.linalg.norm(X[:, np.newaxis] - self.centroids,
↪ axis=2)
        labels = np.argmin(distances, axis=1)

        # Lưu lại tâm cụm cũ để kiểm tra điều kiện dừng hội tụ
        old_centroids = self.centroids.copy()

        # Bước 2: Cập nhật tâm cụm (Centroid Update)
        for cluster_idx in range(self.k):
            cluster_points = X[labels == cluster_idx]
            if len(cluster_points) > 0:
                self.centroids[cluster_idx] = np.mean(cluster_points,
↪ axis=0)

        # Nếu tọa độ tâm cụm không đổi, giải thuật đã hội tụ về một
        ↪ nghiệm ổn định
        if np.allclose(old_centroids, self.centroids):
            break

    return labels

# Nạp đúng dữ liệu mẫu từ bài tập trace bằng tay trong ghi chép
X_trace = np.array([
    [1.0, 1.0],
    [2.0, 1.0],
    [4.0, 1.0],
    [7.0, 1.0],
    [9.0, 1.0]
])

# Chạy giải thuật thực nghiệm
np.random.seed(0)
kmeans = KMeansClustering(k=2, max_iters=10)
final_labels = kmeans.fit(X_trace)
print("Tọa độ các tâm cụm hội tụ cuối cùng:\n", kmeans.centroids)
print("Nhãn cụm hội tụ được gán tương ứng cho từng mẫu:", final_labels)

```

Tọa độ các tâm cụm hội tụ cuối cùng:

```
[[8.          1.          ]  
 [2.33333333 1.          ]]
```

Nhãn cụm hội tụ được gán tương ứng cho từng mẫu: [1 1 1 0 0]

```
import numpy as np  
import matplotlib.pyplot as plt  
  
X_trace = np.array([  
    [1.0, 1.0],  
    [2.0, 1.0],  
    [4.0, 1.0],  
    [7.0, 1.0],  
    [9.0, 1.0]  
)  
  
centroids = np.array([[2.0, 1.0], [4.0, 1.0]])  
history = []  
  
for iteration in range(4):  
    distances = np.linalg.norm(X_trace[:, None, :] - centroids[None, :, :],  
    ↪ axis=2)  
    labels = np.argmin(distances, axis=1)  
    distortion = np.mean(np.min(distances, axis=1) ** 2)  
    history.append((iteration, centroids.copy(), labels.copy(), distortion))  
  
    new_centroids = centroids.copy()  
    for k in range(2):  
        if np.any(labels == k):  
            new_centroids[k] = X_trace[labels == k].mean(axis=0)  
    if np.allclose(new_centroids, centroids):  
        break  
    centroids = new_centroids  
  
print("Distortion theo từng vòng lặp:", [round(item[3], 3) for item in  
    ↪ history])  
  
fig, axes = plt.subplots(1, len(history), figsize=(4 * len(history), 3.2),  
    ↪ sharey=True)  
if len(history) == 1:  
    axes = [axes]  
  
colors = np.array(["#4C78A8", "#F58518"])
```

```

for ax, (iteration, centroids_i, labels_i, distortion_i) in zip(axes,
    ↪ history):
    ax.scatter(X_trace[:, 0], X_trace[:, 1], c=colors[labels_i], s=80,
    ↪ edgecolor="black", label="data points")
    ax.scatter(centroids_i[:, 0], centroids_i[:, 1], c=colors, marker="X",
    ↪ s=220, edgecolor="black", label="centroids")
    for point, label in zip(X_trace, labels_i):
        center = centroids_i[label]
        ax.plot([point[0], center[0]], [point[1], center[1]],
    ↪ color=colors[label], alpha=0.55)
    ax.set_title(f"iter {iteration}\nJ={distortion_i:.3f}")
    ax.set_xlim(0, 10)
    ax.set_ylim(0.4, 1.8)
    ax.set_xlabel("$x_1$")
    ax.grid(alpha=0.25)
axes[0].set_ylabel("$x_2$")
plt.tight_layout()
plt.show()

```

Distortion theo từng vòng lặp: [np.float64(7.0), np.float64(2.461), np.float64(1.333)]

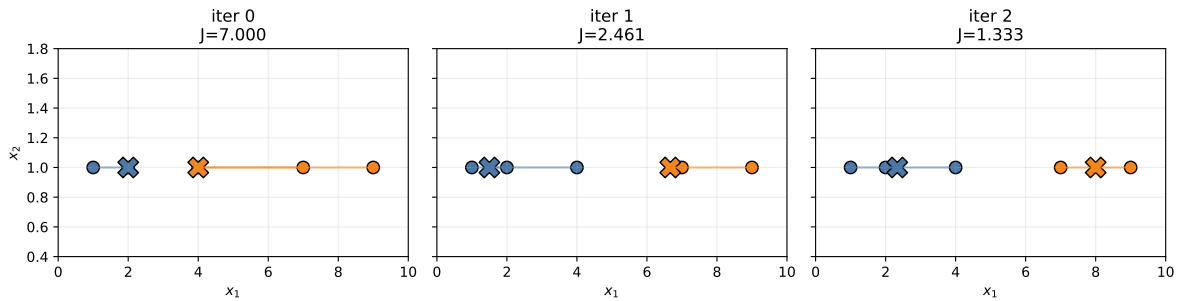


Figure 14: Trace trực quan của K-means: phép gán ổn định dần và distortion giảm đơn điệu.

6. Quy trình trace K-means bằng tay

Khi đề yêu cầu chạy K-means vài vòng:

1. Ghi tâm hiện tại $\mu_1, \dots, \mu_K$.
2. Lập bảng khoảng cách bình phương từ từng điểm tới từng tâm.

3. Gán điểm vào tâm gần nhất.
4. Tính tâm mới bằng trung bình từng cụm.
5. Tính distortion:

$$J = \sum_i \|\mathbf{x}^{(i)} - \mu_{c_i}\|_2^2$$

6. Lặp lại đến khi nhãn hoặc tâm không đổi.

Bẫy thường gặp: distortion nên được tính bằng tâm sau khi cập nhật nếu đề hỏi cuối vòng lặp hoàn chỉnh. Nếu đề hỏi sau bước gán, hãy nói rõ đang dùng tâm cũ hay tâm mới.

7. Bảng Công thức Thi: Các Điểm Cốt lõi của K-means

Khái niệm	Điều cần nhớ
Đầu vào	Chỉ có đặc trưng $\mathbf{x}^{(i)}$; không có nhãn $y^{(i)}$.
Mục tiêu	Tối thiểu hóa khoảng cách bình phương trung bình từ mỗi điểm đến tâm cụm được gán.
Bước gán cụm	Chọn tâm gần nhất: $c^{(i)} \leftarrow \arg \min_k \ \mathbf{x}^{(i)} - \mu_k\ ^2$.
Bước cập nhật tâm	Thay mỗi tâm cụm bằng trung bình các điểm được gán vào cụm đó.
Hội tụ	Chi phí không tăng, nhưng nghiệm cuối có thể là cục bộ, không chắc tối ưu toàn cục.
Điểm yếu chính	Nhạy với khởi tạo và giá trị K .
Cách cải thiện	Dùng K-means++ và chạy nhiều lần với các khởi tạo khác nhau.

Mẹo nhớ một dòng: bước gán chọn tâm gần nhất; bước cập nhật kéo tâm về trung bình các điểm được gán.

Dạng bài chứng minh: khoảng cách tới mean và khoảng cách từng cặp

Một đẳng thức hay gặp trong phân cụm là:

$$\sum_{i=1}^n \|x_i - \mu\|^2 = \frac{1}{2n} \sum_{i=1}^n \sum_{j=1}^n \|x_i - x_j\|^2$$

với:

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

Cách chứng minh gọn:

$$\sum_{i=1}^n \|x_i - \mu\|^2 = \sum_i \|x_i\|^2 - 2\mu^T \sum_i x_i + n\|\mu\|^2$$

Vì $\sum_i x_i = n\mu$:

$$\sum_{i=1}^n \|x_i - \mu\|^2 = \sum_i \|x_i\|^2 - n\|\mu\|^2$$

Mặt khác:

$$\sum_i \sum_j \|x_i - x_j\|^2 = \sum_i \sum_j (\|x_i\|^2 - 2x_i^T x_j + \|x_j\|^2)$$

Hai tổng đầu và cuối đều bằng $n \sum_i \|x_i\|^2$, còn tổng giữa:

$$2 \sum_i \sum_j x_i^T x_j = 2 \left(\sum_i x_i \right)^T \left(\sum_j x_j \right) = 2n^2 \|\mu\|^2$$

Do đó:

$$\sum_i \sum_j \|x_i - x_j\|^2 = 2n \sum_i \|x_i\|^2 - 2n^2 \|\mu\|^2$$

Chia hai vế cho $2n$:

$$\frac{1}{2n} \sum_i \sum_j \|x_i - x_j\|^2 = \sum_i \|x_i\|^2 - n\|\mu\|^2 = \sum_i \|x_i - \mu\|^2$$

Bộ bài tập mẫu có lời giải

Đề bài. Cho các điểm một chiều $x = \{1, 2, 4, 7, 9\}$ và $K = 2$. Khởi tạo $\mu_1 = 2, \mu_2 = 4$. Hãy thực hiện một vòng K-means đầu tiên.

Lời giải từng bước.

1. Gán cụm theo tâm gần nhất:

- $x = 1$: gần $\mu_1 = 2$ hơn, nên vào cụm 1.
- $x = 2$: trùng $\mu_1 = 2$, nên vào cụm 1.
- $x = 4$: trùng $\mu_2 = 4$, nên vào cụm 2.
- $x = 7$: gần $\mu_2 = 4$ hơn, nên vào cụm 2.
- $x = 9$: gần $\mu_2 = 4$ hơn, nên vào cụm 2.

2. Ta có:

$$C_1 = \{1, 2\}, \quad C_2 = \{4, 7, 9\}$$

3. Cập nhật tâm cụm:

$$\mu_1 = \frac{1+2}{2} = 1.5$$
$$\mu_2 = \frac{4+7+9}{3} = \frac{20}{3} \approx 6.67$$

Kết luận. Sau vòng đầu tiên, hai tâm mới là $\mu_1 = 1.5$ và $\mu_2 \approx 6.67$.

Bài 2: K-means trên bộ dữ liệu 10 điểm hai chiều

Đề bài. Cho 10 điểm:

dòng	x_1	x_2
1	1.0	1.0
2	1.2	0.8
3	0.8	1.3
4	1.5	1.1
5	2.0	1.0
6	7.0	7.0
7	7.5	6.8
8	6.8	7.4
9	8.0	7.2
10	7.2	7.8

Chạy K-means với $K = 2$, khởi tạo $\mu_1 = (1, 1)$ và $\mu_2 = (7, 7)$. Hãy theo dõi nhãn cụm, tâm cụm, và distortion đến khi hội tụ.

Lời giải.

Vòng 0 dùng tâm ban đầu:

$$\mu_1^{(0)} = (1, 1), \quad \mu_2^{(0)} = (7, 7)$$

Ví dụ điểm (2.0, 1.0):

$$d^2((2, 1), \mu_1) = (2 - 1)^2 + (1 - 1)^2 = 1$$

$$d^2((2, 1), \mu_2) = (2 - 7)^2 + (1 - 7)^2 = 61$$

Nên điểm này thuộc cụm 1. Các điểm 1 đến 5 gần μ_1 , các điểm 6 đến 10 gần μ_2 :

$$C_1 = \{(1.0, 1.0), (1.2, 0.8), (0.8, 1.3), (1.5, 1.1), (2.0, 1.0)\}$$

$$C_2 = \{(7.0, 7.0), (7.5, 6.8), (6.8, 7.4), (8.0, 7.2), (7.2, 7.8)\}$$

Distortion vòng 0 là trung bình khoảng cách bình phương đến tâm được gán:

$$J^{(0)} \approx 0.368$$

Cập nhật tâm cụm:

$$\mu_1^{(1)} = \left(\frac{1.0 + 1.2 + 0.8 + 1.5 + 2.0}{5}, \frac{1.0 + 0.8 + 1.3 + 1.1 + 1.0}{5} \right) = (1.30, 1.04)$$

$$\mu_2^{(1)} = \left(\frac{7.0 + 7.5 + 6.8 + 8.0 + 7.2}{5}, \frac{7.0 + 6.8 + 7.4 + 7.2 + 7.8}{5} \right) = (7.30, 7.24)$$

Vòng 1, khi dùng hai tâm mới, nhãn cụm không đổi:

$$C_1 = \text{điểm 1 đến 5}, \quad C_2 = \text{điểm 6 đến 10}$$

Distortion giảm:

$$J^{(1)} \approx 0.248$$

Vì cập nhật lại trung bình vẫn cho $(1.30, 1.04)$ và $(7.30, 7.24)$, thuật toán hội tụ.

Kết luận. Vì dữ liệu tách thành hai nhóm rất rõ, K-means hội tụ nhanh. Với dữ liệu thật, nên chạy nhiều khởi tạo vì kết quả phụ thuộc tâm ban đầu.

Bài 3: Chọn K bằng elbow từ bảng distortion

Đề bài. Giả sử chạy K-means với nhiều K thu được distortion:

K	1	2	3	4	5	6
distortion	38.5	8.2	5.7	4.9	4.5	4.2

Chọn K hợp lý.

Lời giải. Khi tăng K , distortion luôn giảm hoặc giữ nguyên. Ta tìm điểm mà mức giảm bắt đầu chậm lại:

- Từ $K = 1$ sang $K = 2$: giảm 30.3, rất lớn.
- Từ $K = 2$ sang $K = 3$: giảm 2.5.
- Các bước sau giảm rất ít.

Kết luận. Elbow nằm tại $K = 2$. Đây là lựa chọn hợp lý vì thêm cụm sau 2 không cải thiện distortion nhiều.

Part 10: Exam Papers and Answer Keys

The following appendix joins every exam Markdown file currently present in `exam/`.

Exam Appendix 1: CS114 - Exam_Recovered

Source: `exam/sample_exam/CS114 - Exam_Recovered.md`

ĐỀ THI MACHINE LEARNING

BẢN KHÔI PHỤC THEO MÔ TẢ

- **Thời gian làm bài:** 90 phút
 - **Hình thức:** Tự luận + tính toán
 - **Ghi chú:** Sinh viên được sử dụng máy tính cầm tay. Trình bày rõ công thức, các bước biến đổi và kết luận.
-

Câu 1. Maximum Likelihood Estimation cho Logistic Regression

Cho tập dữ liệu huấn luyện: $D = \{(x_i, y_i)\}_{i=1}^n$, với $y_i \in \{0, 1\}$. Mô hình Logistic Regression được định nghĩa bởi:

$$P(y_i = 1|x_i; w, b) = \sigma(w^T x_i + b)$$

Trong đó:

- $\sigma(z) = \frac{1}{1+e^{-z}}$
- $p_i = P(y_i = 1|x_i; w, b)$

Hãy:

1. Viết hàm likelihood của toàn bộ tập dữ liệu.
 2. Viết hàm log-likelihood.
 3. Chứng minh rằng việc cực đại hóa log-likelihood tương đương với việc cực tiểu hóa binary cross-entropy loss.
 4. Tính gradient của log-likelihood theo w và b .
 5. Nêu vì sao Logistic Regression thường không có nghiệm đóng cho MLE và cần dùng các phương pháp tối ưu lặp như Gradient Ascent, Gradient Descent hoặc Newton-Raphson.
-

Câu 2. (Chưa khôi phục được nội dung)

Phần này hiện chưa có đủ mô tả để khôi phục chính xác.

Câu 3. Backpropagation cho mạng neural network đơn giản

Cho một mạng neural network gồm:

- 2 input: x_1, x_2
- 1 hidden layer gồm 2 neuron (h_1, h_2), dùng sigmoid activation
- 1 output neuron (o), dùng sigmoid activation
- Loss function (với một mẫu dữ liệu): Squared error $L = \frac{1}{2}(\hat{y} - y)^2$

Giả sử có một mẫu dữ liệu với: $x_1 = 1, x_2 = 2, y = 1$.

Các trọng số ban đầu:

Kết nối	Trọng số
$x_1 \rightarrow h_1$	0.1
$x_2 \rightarrow h_1$	0.2
$x_1 \rightarrow h_2$	0.3
$x_2 \rightarrow h_2$	0.4
$h_1 \rightarrow o$	0.5
$h_2 \rightarrow o$	0.6

Bias ban đầu:

$$b_{h1} = 0.1, \quad b_{h2} = 0.1, \quad b_o = 0.1$$

Hãy thực hiện:

1. Forward propagation để tính h_1, h_2 và $\hat{y}$.
2. Tính loss L .
3. Tính đạo hàm $\frac{\partial L}{\partial w}$ đối với từng trọng số.
4. Nếu learning rate $\eta = 0.1$, hãy cập nhật các trọng số sau một bước Gradient Descent.

Biết rằng: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

Câu 4. Naive Bayes Classification

Cho bảng dữ liệu huấn luyện sau về phân loại email là Spam hoặc Not Spam:

STT	Có từ “free”	Có từ “win”	Có từ “meeting”	Nhãn
1	Yes	Yes	No	Spam
2	Yes	No	No	Spam
3	No	Yes	No	Spam
4	Yes	Yes	Yes	Spam
5	No	No	Yes	Not Spam
6	No	No	Yes	Not Spam
7	Yes	No	Yes	Not Spam
8	No	Yes	Yes	Not Spam
9	No	No	No	Not Spam
10	Yes	Yes	No	Spam

Sử dụng thuật toán Naive Bayes với giả định các thuộc tính độc lập có điều kiện theo nhãn.
Cho email mới cần dự đoán:

Có từ “free”	Có từ “win”	Có từ “meeting”
Yes	Yes	No

Hãy:

1. Tính prior probability $P(\text{Spam})$ và $P(\text{Not Spam})$.
2. Tính các conditional probabilities cần thiết.
3. Tính $P(\text{Spam}|x)$ và $P(\text{Not Spam}|x)$ theo Naive Bayes.
4. Kết luận email mới thuộc lớp nào.
5. Nếu có xác suất bằng 0, hãy áp dụng Laplace smoothing với $\alpha = 1$.

Câu 5. Decision Tree với Information Gain dựa trên Entropy

Cho bảng dữ liệu thời tiết dùng để quyết định có chơi tennis hay không:

STT	Outlook	Temperature	Humidity	Wind	Play
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No

STT	Outlook	Temperature	Humidity	Wind	Play
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Biết công thức tính:

- **Entropy:**

$$Entropy(S) = - \sum_c p_c \log_2(p_c)$$

- **Information Gain** của thuộc tính A :

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} \cdot Entropy(S_v)$$

Hãy:

1. Tính entropy ban đầu của tập dữ liệu S .
2. Tính Information Gain cho các thuộc tính: Outlook, Temperature, Humidity, Wind.
3. Chọn thuộc tính tốt nhất để làm node gốc của cây quyết định.
4. Vẽ cây quyết định sau bước chia đầu tiên.
5. Giải thích vì sao thuộc tính có Information Gain lớn nhất được chọn.

Câu 6. Quan hệ giữa khoảng cách đến mean và khoảng cách giữa các cặp điểm trong cụm

Cho một cụm dữ liệu $C = \{x_1, x_2, \dots, x_n\}$ với mỗi $x_i \in \mathbb{R}^d$. Gọi mean của cụm là:

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

Hãy chứng minh đẳng thức sau:

$$\sum_{i=1}^n \|x_i - \mu\|^2 = \frac{1}{2n} \sum_{i=1}^n \sum_{j=1}^n \|x_i - x_j\|^2$$

Từ đó suy ra:

$$\frac{1}{n} \sum_{i=1}^n \|x_i - \mu\|^2 = \frac{1}{2n^2} \sum_{i=1}^n \sum_{j=1}^n \|x_i - x_j\|^2$$

Nói cách khác: Trung bình khoảng cách bình phương từ các điểm trong cụm đến mean có quan hệ trực tiếp với trung bình khoảng cách bình phương giữa tất cả các cặp điểm trong cụm.

Lưu ý: Trong công thức chuẩn của clustering/K-means, đại lượng đúng là khoảng cách bình phương Euclidean, không phải khoảng cách Euclidean thông thường.

HẾT

Exam Appendix 2: CS114 - Exam_Answer

Source: `exam/sample_exam/CS114 - Exam_Answer.md`

ĐÁP ÁN ĐỀ THI MACHINE LEARNING

BẢN ĐÁP ÁN CHI TIẾT

Câu 1. Maximum Likelihood Estimation cho Logistic Regression

1.1 Hàm Likelihood

Với giả định các mẫu độc lập (i.i.d.), sử dụng công thức xác suất hợp nhất cho phân phối Bernoulli:

$$P(y_i | x_i; w, b) = p_i^{y_i} (1 - p_i)^{1-y_i}$$

Hàm likelihood của toàn bộ tập dữ liệu:

$$L(w, b) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i}$$

trong đó $p_i = \sigma(w^T x_i + b) = \frac{1}{1 + e^{-(w^T x_i + b)}}$.

1.2 Hàm Log-Likelihood

Lấy logarit tự nhiên:

$$\ell(w, b) = \log L(w, b) = \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

1.3 Chứng minh tương đương với Binary Cross-Entropy Loss

Binary cross-entropy loss được định nghĩa:

$$J(w, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Ta nhận thấy:

$$J(w, b) = -\frac{1}{n} \ell(w, b)$$

Do đó:

$$\max_{w, b} \ell(w, b) \iff \min_{w, b} (-\ell(w, b)) \iff \min_{w, b} J(w, b)$$

Kết luận: Cực đại hóa log-likelihood tương đương với cực tiểu hóa binary cross-entropy loss (khác nhau hệ số dương $\frac{1}{n}$ và đổi dấu, không ảnh hưởng đến điểm tối ưu).

1.4 Tính Gradient

Tính chất quan trọng của sigmoid: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

Đặt $z_i = w^T x_i + b$, áp dụng chain rule:

$$\frac{\partial \ell}{\partial w} = \sum_{i=1}^n \frac{\partial \ell}{\partial p_i} \cdot \frac{\partial p_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial w}$$

Tính từng thành phần:

- $\frac{\partial \ell}{\partial p_i} = \frac{y_i}{p_i} - \frac{1-y_i}{1-p_i} = \frac{y_i - p_i}{p_i(1-p_i)}$
- $\frac{\partial p_i}{\partial z_i} = p_i(1 - p_i)$
- $\frac{\partial z_i}{\partial w} = x_i$

Nhân lại và triệt tiêu mẫu số:

$$\frac{\partial \ell}{\partial w} = \sum_{i=1}^n (y_i - p_i) x_i$$

Tương tự cho bias:

$$\frac{\partial \ell}{\partial b} = \sum_{i=1}^n (y_i - p_i)$$

1.5 Vì sao không có nghiệm đóng?

Đặt gradient bằng 0:

$$\sum_{i=1}^n (y_i - \sigma(w^T x_i + b)) x_i = 0$$

Do hàm sigmoid $\sigma(\cdot)$ là hàm phi tuyến, phương trình trên không thể giải đại số để rút w và b ra dạng tường minh. Khác với Linear Regression (nơi mô hình tuyến tính cho phương trình bậc nhất), Logistic Regression tạo ra hệ phương trình siêu việt (transcendental equations) không có nghiệm đóng.

Do đó, ta phải sử dụng các phương pháp tối ưu lặp: - **Gradient Ascent/Descent**: Cập nhật $w \leftarrow w + \alpha \sum_i (y_i - p_i) x_i$, hội tụ tuyến tính. - **Newton-Raphson**: Sử dụng Hessian $H = -\sum_i p_i (1 - p_i) x_i x_i^T$, cập nhật $w \leftarrow w - H^{-1} \nabla \ell$, hội tụ bậc hai nhưng tốn $O(p^3)$ mỗi bước.

Câu 2. (Chưa khôi phục được nội dung)

Bỏ qua do đề gốc chưa khôi phục.

Câu 3. Backpropagation cho Mạng Neural Network Đơn Giản

3.1 Forward Propagation

Tầng ẩn — Neuron h_1 :

$$z_{h1} = w_{x1 \rightarrow h1} \cdot x_1 + w_{x2 \rightarrow h1} \cdot x_2 + b_{h1}$$

$$z_{h1} = 0.1(1) + 0.2(2) + 0.1 = 0.1 + 0.4 + 0.1 = 0.6$$

$$h_1 = \sigma(0.6) = \frac{1}{1 + e^{-0.6}} \approx 0.6457$$

Tầng ẩn — Neuron h_2 :

$$z_{h2} = 0.3(1) + 0.4(2) + 0.1 = 0.3 + 0.8 + 0.1 = 1.2$$

$$h_2 = \sigma(1.2) = \frac{1}{1 + e^{-1.2}} \approx 0.7685$$

Tầng đầu ra — Neuron o :

$$z_o = 0.5 \cdot h_1 + 0.6 \cdot h_2 + b_o = 0.5(0.6457) + 0.6(0.7685) + 0.1$$

$$z_o = 0.3229 + 0.4611 + 0.1 = 0.8840$$

$$\hat{y} = \sigma(0.8840) = \frac{1}{1 + e^{-0.8840}} \approx 0.7076$$

3.2 Tính Loss

$$L = \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(0.7076 - 1)^2 = \frac{1}{2}(-0.2924)^2 = \frac{1}{2}(0.08550) \approx 0.04275$$

3.3 Tính Gradient (Backpropagation)

Bước 1: Đạo hàm tại output

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = 0.7076 - 1 = -0.2924$$

$$\sigma'(z_o) = \hat{y}(1 - \hat{y}) = 0.7076 \times 0.2924 = 0.2069$$

$$\delta_o = \frac{\partial L}{\partial z_o} = (\hat{y} - y) \cdot \sigma'(z_o) = (-0.2924)(0.2069) = -0.06050$$

Bước 2: Gradient trọng số tầng output

$$\frac{\partial L}{\partial w_{h1 \rightarrow o}} = \delta_o \cdot h_1 = (-0.06050)(0.6457) = -0.03907$$

$$\frac{\partial L}{\partial w_{h2 \rightarrow o}} = \delta_o \cdot h_2 = (-0.06050)(0.7685) = -0.04650$$

$$\frac{\partial L}{\partial b_o} = \delta_o = -0.06050$$

Bước 3: Lan truyền ngược đến tầng ẩn

$$\delta_{h1} = \delta_o \cdot w_{h1 \rightarrow o} \cdot h_1(1 - h_1)$$

$$= (-0.06050)(0.5)(0.6457)(1 - 0.6457)$$

$$= (-0.06050)(0.5)(0.2288) = -0.006921$$

$$\delta_{h2} = \delta_o \cdot w_{h2 \rightarrow o} \cdot h_2(1 - h_2)$$

$$= (-0.06050)(0.6)(0.7685)(1 - 0.7685)$$

$$= (-0.06050)(0.6)(0.1780) = -0.006459$$

Bước 4: Gradient trọng số tầng ẩn

$$\frac{\partial L}{\partial w_{x1 \rightarrow h1}} = \delta_{h1} \cdot x_1 = (-0.006921)(1) = -0.006921$$

$$\frac{\partial L}{\partial w_{x2 \rightarrow h1}} = \delta_{h1} \cdot x_2 = (-0.006921)(2) = -0.013842$$

$$\frac{\partial L}{\partial w_{x1 \rightarrow h2}} = \delta_{h2} \cdot x_1 = (-0.006459)(1) = -0.006459$$

$$\frac{\partial L}{\partial w_{x2 \rightarrow h2}} = \delta_{h2} \cdot x_2 = (-0.006459)(2) = -0.012918$$

3.4 Cập nhật trọng số ($\eta = 0.1$)

Công thức: $w_{new} = w_{old} - \eta \cdot \frac{\partial L}{\partial w}$

Kết nối	w_{old}	Gradient	w_{new}
$x_1 \rightarrow h_1$	0.1	-0.006921	$0.1 - 0.1(-0.006921) = 0.10069$
$x_2 \rightarrow h_1$	0.2	-0.013842	$0.2 - 0.1(-0.013842) = 0.20138$
$x_1 \rightarrow h_2$	0.3	-0.006459	$0.3 - 0.1(-0.006459) = 0.30065$
$x_2 \rightarrow h_2$	0.4	-0.012918	$0.4 - 0.1(-0.012918) = 0.40129$
$h_1 \rightarrow o$	0.5	-0.03907	$0.5 - 0.1(-0.03907) = 0.50391$
$h_2 \rightarrow o$	0.6	-0.04650	$0.6 - 0.1(-0.04650) = 0.60465$

Bias cập nhật:

- $b_{h1} = 0.1 - 0.1(-0.006921) = 0.10069$
- $b_{h2} = 0.1 - 0.1(-0.006459) = 0.10065$
- $b_o = 0.1 - 0.1(-0.06050) = 0.10605$

Câu 4. Naive Bayes Classification

4.1 Prior Probability

Từ bảng dữ liệu: 5 mẫu Spam (dòng 1, 2, 3, 4, 10), 5 mẫu Not Spam (dòng 5, 6, 7, 8, 9).

$$P(\text{Spam}) = \frac{5}{10} = 0.5$$

$$P(\text{Not Spam}) = \frac{5}{10} = 0.5$$

4.2 Conditional Probabilities

Lớp Spam (5 mẫu):

Từ	Số mẫu Spam có “Yes”	$P(\text{Yes} \text{Spam})$
free	3 (dòng 1, 2, 4)	$\frac{3}{5} = 0.6$
win	3 (dòng 1, 3, 4)	$\frac{3}{5} = 0.6$
meeting	1 (dòng 4)	$\frac{1}{5} = 0.2$

Kiểm tra lại: Spam gồm dòng 1, 2, 3, 4, 10. - free=Yes trong Spam: dòng 1(Yes), 2(Yes), 3(No), 4(Yes), 10(Yes) → 4 mẫu - win=Yes trong Spam: dòng 1(Yes), 2(No), 3(Yes), 4(Yes), 10(Yes) → 4 mẫu - meeting=Yes trong Spam: dòng 1(No), 2(No), 3(No), 4(Yes), 10(No) → 1 mẫu

$$P(\text{free=Yes} \mid \text{Spam}) = \frac{4}{5} = 0.8$$

$$P(\text{win=Yes} \mid \text{Spam}) = \frac{4}{5} = 0.8$$

$$P(\text{meeting=No} \mid \text{Spam}) = 1 - P(\text{meeting=Yes} \mid \text{Spam}) = 1 - \frac{1}{5} = 0.8$$

Lớp Not Spam (5 mẫu: dòng 5, 6, 7, 8, 9):

- free=Yes: dòng 5(No), 6(No), 7(Yes), 8(No), 9(No) → 1 mẫu
- win=Yes: dòng 5(No), 6(No), 7(No), 8(Yes), 9(No) → 1 mẫu
- meeting=Yes: dòng 5(Yes), 6(Yes), 7(Yes), 8(Yes), 9(No) → 4 mẫu

$$P(\text{free=Yes} \mid \text{Not Spam}) = \frac{1}{5} = 0.2$$

$$P(\text{win=Yes} \mid \text{Not Spam}) = \frac{1}{5} = 0.2$$

$$P(\text{meeting=No} \mid \text{Not Spam}) = 1 - \frac{4}{5} = 0.2$$

4.3 Tính Posterior

Email mới: free=Yes, win=Yes, meeting=No.

Tính cho Spam:

$$P(\mathbf{x} \mid \text{Spam}) \cdot P(\text{Spam}) = (0.8)(0.8)(0.8)(0.5) = 0.256$$

Tính cho Not Spam:

$$P(\mathbf{x} \mid \text{Not Spam}) \cdot P(\text{Not Spam}) = (0.2)(0.2)(0.2)(0.5) = 0.004$$

Chuẩn hóa:

$$P(\text{Spam} \mid \mathbf{x}) = \frac{0.256}{0.256 + 0.004} = \frac{0.256}{0.260} \approx 0.985$$

$$P(\text{Not Spam} \mid \mathbf{x}) = \frac{0.004}{0.260} \approx 0.015$$

4.4 Kết luận

Vì $P(\text{Spam} \mid \mathbf{x}) \approx 0.985 > 0.5$, email mới được phân loại là **Spam**.

4.5 Laplace Smoothing

Trong bài này không có xác suất nào bằng 0, nên Laplace smoothing không thay đổi kết luận. Nếu áp dụng Laplace smoothing ($\alpha = 1$) cho feature nhị phân:

$$P(x_j = 1 \mid c) = \frac{\text{count}(x_j = 1, c) + 1}{N_c + 2}$$

Ví dụ: $P(\text{meeting=Yes} \mid \text{Spam}) = \frac{1+1}{5+2} = \frac{2}{7} \approx 0.286$ thay vì 0.2.

Câu 5. Decision Tree với Information Gain dựa trên Entropy

5.1 Entropy ban đầu

Trong 14 mẫu: 9 mẫu Yes, 5 mẫu No.

$$\begin{aligned} \text{Entropy}(S) &= -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} \\ &= -0.6429 \times (-0.6373) - 0.3571 \times (-1.4854) \\ &= 0.4098 + 0.5306 = 0.9403 \end{aligned}$$

5.2 Information Gain cho từng thuộc tính

Outlook

- **Sunny** (dòng 1,2,8,9,11): 2 Yes, 3 No $\rightarrow H = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.9710$
- **Overcast** (dòng 3,7,12,13): 4 Yes, 0 No $\rightarrow H = 0$
- **Rain** (dòng 4,5,6,10,14): 3 Yes, 2 No $\rightarrow H = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} = 0.9710$

$$\begin{aligned} Gain(S, Outlook) &= 0.9403 - \frac{5}{14}(0.9710) - \frac{4}{14}(0) - \frac{5}{14}(0.9710) \\ &= 0.9403 - 0.3468 - 0 - 0.3468 = 0.2467 \end{aligned}$$

Temperature

- **Hot** (dòng 1,2,3,13): 2 Yes, 2 No $\rightarrow H = 1.0$
- **Mild** (dòng 4,8,10,11,12,14): 4 Yes, 2 No $\rightarrow H = -\frac{4}{6} \log_2 \frac{4}{6} - \frac{2}{6} \log_2 \frac{2}{6} = 0.9183$
- **Cool** (dòng 5,6,7,9): 3 Yes, 1 No $\rightarrow H = -\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} = 0.8113$

$$\begin{aligned} Gain(S, Temperature) &= 0.9403 - \frac{4}{14}(1.0) - \frac{6}{14}(0.9183) - \frac{4}{14}(0.8113) \\ &= 0.9403 - 0.2857 - 0.3936 - 0.2318 = 0.0292 \end{aligned}$$

Humidity

- **High** (dòng 1,2,3,4,8,12,14): 3 Yes, 4 No $\rightarrow H = -\frac{3}{7} \log_2 \frac{3}{7} - \frac{4}{7} \log_2 \frac{4}{7} = 0.9852$
- **Normal** (dòng 5,6,7,9,10,11,13): 6 Yes, 1 No $\rightarrow H = -\frac{6}{7} \log_2 \frac{6}{7} - \frac{1}{7} \log_2 \frac{1}{7} = 0.5917$

$$\begin{aligned} Gain(S, Humidity) &= 0.9403 - \frac{7}{14}(0.9852) - \frac{7}{14}(0.5917) \\ &= 0.9403 - 0.4926 - 0.2959 = 0.1518 \end{aligned}$$

Wind

- **Weak** (dòng 1,3,4,5,8,9,10,13): 6 Yes, 2 No $\rightarrow H = -\frac{6}{8} \log_2 \frac{6}{8} - \frac{2}{8} \log_2 \frac{2}{8} = 0.8113$
- **Strong** (dòng 2,6,7,11,12,14): 3 Yes, 3 No $\rightarrow H = 1.0$

$$Gain(S, Wind) = 0.9403 - \frac{8}{14}(0.8113) - \frac{6}{14}(1.0)$$

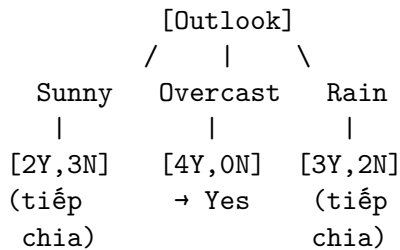
$$= 0.9403 - 0.4636 - 0.4286 = 0.0481$$

5.3 Chọn thuộc tính tốt nhất

Thuộc tính	Information Gain
Outlook	0.2467
Humidity	0.1518
Wind	0.0481
Temperature	0.0292

Kết luận: Chọn **Outlook** làm node gốc vì có Information Gain lớn nhất.

5.4 Cây quyết định sau bước chia đầu tiên



- Nhánh **Overcast** đã thuần nhất (4 Yes, 0 No) $\rightarrow$ lá = **Yes**
- Nhánh **Sunny** và **Rain** cần tiếp tục chia.

5.5 Giải thích

Thuộc tính có Information Gain lớn nhất được chọn vì nó **giảm entropy nhiều nhất** sau phép chia, nghĩa là nó phân tách dữ liệu thành các nhóm con đồng nhất nhất. Đây là chiến lược tham lam (greedy): tại mỗi bước, ta chọn phép chia tốt nhất cục bộ để nhanh nhất đạt được các nút lá thuần nhất.

Câu 6. Chứng minh quan hệ khoảng cách trong cụm

Chứng minh đẳng thức chính

Cần chứng minh:

$$\sum_{i=1}^n |x_i - \mu|^2 = \frac{1}{2n} \sum_{i=1}^n \sum_{j=1}^n |x_i - x_j|^2$$

Bước 1: Khai triển về phải

$$|x_i - x_j|^2 = |x_i|^2 - 2x_i^T x_j + |x_j|^2$$

Tính tổng kép:

$$\sum_{i=1}^n \sum_{j=1}^n |x_i - x_j|^2 = \sum_{i=1}^n \sum_{j=1}^n (|x_i|^2 - 2x_i^T x_j + |x_j|^2)$$

Tách tổng:

$$= \sum_{i=1}^n \sum_{j=1}^n |x_i|^2 - 2 \sum_{i=1}^n \sum_{j=1}^n x_i^T x_j + \sum_{i=1}^n \sum_{j=1}^n |x_j|^2$$

Mỗi hạng thứ nhất và hạng thứ ba đều bằng $n \sum_{i=1}^n |x_i|^2$:

$$= 2n \sum_{i=1}^n |x_i|^2 - 2 \left(\sum_{i=1}^n x_i \right)^T \left(\sum_{j=1}^n x_j \right)$$

Vì $\sum_{i=1}^n x_i = n\mu$:

$$= 2n \sum_{i=1}^n |x_i|^2 - 2(n\mu)^T (n\mu)$$

$$= 2n \sum_{i=1}^n |x_i|^2 - 2n^2 |\mu|^2$$

Chia cho $2n$:

$$\frac{1}{2n} \sum_{i=1}^n \sum_{j=1}^n |x_i - x_j|^2 = \sum_{i=1}^n |x_i|^2 - n|\mu|^2$$

Bước 2: Khai triển về trái

$$\begin{aligned}\sum_{i=1}^n |x_i - \mu|^2 &= \sum_{i=1}^n (|x_i|^2 - 2x_i^T \mu + |\mu|^2) \\ &= \sum_{i=1}^n |x_i|^2 - 2\mu^T \sum_{i=1}^n x_i + n|\mu|^2\end{aligned}$$

Thay $\sum_{i=1}^n x_i = n\mu$:

$$\begin{aligned}&= \sum_{i=1}^n |x_i|^2 - 2\mu^T (n\mu) + n|\mu|^2 \\ &= \sum_{i=1}^n |x_i|^2 - 2n|\mu|^2 + n|\mu|^2 \\ &= \sum_{i=1}^n |x_i|^2 - n|\mu|^2\end{aligned}$$

Bước 3: So sánh

Cả vế trái và vế phải đều bằng $\sum_{i=1}^n |x_i|^2 - n|\mu|^2$.

$$\therefore \sum_{i=1}^n |x_i - \mu|^2 = \frac{1}{2n} \sum_{i=1}^n \sum_{j=1}^n |x_i - x_j|^2 \quad \blacksquare$$

Suy ra hệ quả

Chia cả hai vế cho n :

$$\frac{1}{n} \sum_{i=1}^n |x_i - \mu|^2 = \frac{1}{2n^2} \sum_{i=1}^n \sum_{j=1}^n |x_i - x_j|^2 \quad \blacksquare$$

Ý nghĩa: Trung bình khoảng cách bình phương từ các điểm đến tâm cụm (vế trái — đại lượng K-means tối thiểu hóa) bằng đúng một nửa trung bình khoảng cách bình phương giữa tất cả các cặp điểm (vế phải). Điều này cho thấy việc tối thiểu hóa distortion trong K-means tương đương với việc tối thiểu hóa khoảng cách nội cụm trung bình giữa mọi cặp điểm.

HẾT ĐÁP ÁN

Exam Appendix 3: CS114 - Exam_01

Source: `exam/exam_01/CS114 - Exam_01.md`

ĐỀ THI MACHINE LEARNING — ĐỀ 01

- **Thời gian làm bài:** 90 phút
- **Hình thức:** Tự luận + tính toán
- **Ghi chú:** Sinh viên được sử dụng máy tính cầm tay. Trình bày rõ công thức, các bước biến đổi và kết luận.

Câu 1. OLS Linear Regression bằng Ma trận

Cho 5 điểm dữ liệu huấn luyện với mô hình $\hat{y} = \beta_0 + \beta_1 x$:

STT	x	y
1	0	1
2	1	3
3	2	4
4	3	6
5	4	8

Hãy:

1. Xây dựng ma trận thiết kế $\mathbf{X}$ (thêm cột 1 cho intercept) và vector $\mathbf{y}$.
2. Tính $\mathbf{X}^T \mathbf{X}$ và $\mathbf{X}^T \mathbf{y}$.
3. Giải hệ Normal Equations $\mathbf{X}^T \mathbf{X} \beta = \mathbf{X}^T \mathbf{y}$ để tìm β_0, β_1 .
4. Dự đoán $\hat{y}$ khi $x = 6$.
5. Kiểm tra điều kiện $\mathbf{X}^T (\mathbf{y} - \mathbf{X} \beta) = \mathbf{0}$.

Câu 2. Logistic Regression — Thay số Tính Xác suất và Loss

Cho mô hình Logistic Regression có $\beta = (0.5, -1.2, 0.8)$, phần tử đầu tiên là bias. Cho bảng 8 mẫu sau:

STT	x_1	x_2	y
1	1.0	0.5	1
2	-0.5	1.0	0
3	2.0	1.5	1
4	0.0	0.0	0
5	1.5	-1.0	0
6	-1.0	2.0	1
7	3.0	0.5	1
8	0.5	1.0	1

Hãy:

1. Tính điểm tuyến tính z , xác suất $\hat{p}$, lớp dự đoán cho từng mẫu.
2. Tính cross-entropy loss trung bình.
3. Tính accuracy.
4. Nêu nhận xét về mẫu nào mô hình dự đoán sai và giải thích.

Câu 3. Họ Phân Phối Số Mũ (Exponential Family) và GLM

Hãy:

1. Viết phân phối Bernoulli $P(y | \phi) = \phi^y(1 - \phi)^{1-y}$ dưới dạng chuẩn tắc Exponential Family. Xác định rõ η , $T(y)$, $a(\eta)$, $b(y)$.
2. Chứng minh rằng $\phi = \sigma(\eta) = \frac{1}{1+e^{-\eta}}$ (tức hàm Sigmoid xuất hiện tự nhiên từ Exponential Family).
3. Cho $\beta = (0.3, 1.5, -0.5)$ và $\mathbf{x} = (1, 2, 3)$. Tính dự đoán cho 3 trường hợp: Gaussian GLM, Bernoulli GLM, Poisson GLM.

Câu 4. Neural Network Forward Pass — Mạng 3-2-1

Cho mạng gồm:

- 3 input: $x_1 = 0.5$, $x_2 = -1.0$, $x_3 = 2.0$
- 1 hidden layer gồm 2 neuron (h_1, h_2), dùng ReLU activation
- 1 output neuron (o), dùng sigmoid activation

Trọng số:

$$\mathbf{W}^{[1]} = \begin{pmatrix} 0.3 & -0.2 & 0.5 \\ 0.1 & 0.4 & -0.3 \end{pmatrix}, \quad \mathbf{b}^{[1]} = \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix}$$

$$\mathbf{W}^{[2]} = (0.7 \quad -0.5), \quad b^{[2]} = 0.2$$

Hãy:

1. Tính $\mathbf{Z}^{[1]}$, $\mathbf{A}^{[1]}$ (qua ReLU).
2. Tính $Z^{[2]}$ và xác suất output $\hat{y} = \sigma(Z^{[2]})$.
3. Nếu nhãn thật $y = 1$, tính binary cross-entropy loss.
4. Tính $dZ^{[2]}$, $dW^{[2]}$, $db^{[2]}$ (backprop bước đầu tiên tại output).

Câu 5. Decision Tree — Gini Index

Cho bảng dữ liệu phân loại khách hàng mua sản phẩm:

STT	Tuổi	Thu nhập	Sinh viên	Mua
1	Trẻ	Cao	Không	Không
2	Trẻ	Cao	Không	Không
3	Trung niên	Cao	Không	Có
4	Già	Trung bình	Không	Có
5	Già	Thấp	Có	Có
6	Già	Thấp	Có	Không
7	Trung niên	Thấp	Có	Có
8	Trẻ	Trung bình	Không	Không
9	Trẻ	Thấp	Có	Có
10	Già	Trung bình	Có	Có

Hãy:

1. Tính Gini Index ban đầu của tập dữ liệu.
2. Tính Gini Gain cho thuộc tính **Tuổi** và **Sinh viên**.
3. Chọn thuộc tính nào tốt hơn để chia tại node gốc.
4. Vẽ cây sau bước chia đầu tiên.

Câu 6. K-means Clustering — Trace Thuật Toán

Cho 6 điểm dữ liệu trong không gian 2D:

STT	x_1	x_2
1	1	2
2	2	1
3	2	3
4	8	7
5	9	8
6	8	9

Cho $K = 2$, khởi tạo tâm cụm:

$$\mu_1^{(0)} = (1, 2), \quad \mu_2^{(0)} = (9, 8)$$

Hãy:

1. Thực hiện bước gán cụm (iteration 0) và tính distortion J .
2. Cập nhật tâm cụm.
3. Thực hiện bước gán cụm (iteration 1) và kiểm tra hội tụ.
4. Giải thích vì sao K-means luôn hội tụ nhưng có thể rơi vào cực tiểu cục bộ.

Câu 7. Bias-Variance Decomposition

Hãy:

1. Viết phương trình phân rã Test MSE thành Bias², Variance, và Noise.
2. Chứng minh từng bước phân rã, bắt đầu từ $\mathbb{E}[(y - \hat{f})^2]$ với $y = f(\mathbf{x}) + \epsilon$, $\mathbb{E}[\epsilon] = 0$, $\text{Var}(\epsilon) = \sigma^2$.
3. Cho bảng cross-validation sau, chọn λ tốt nhất:

λ	fold 1	fold 2	fold 3	fold 4
0.00	1.50	1.65	1.40	1.55
0.05	0.85	0.90	0.82	0.88
0.50	0.60	0.65	0.58	0.62
5.00	0.95	1.00	0.92	0.98

Câu 8. Tính phi tuyến trong Mạng Neural (Nâng cao)

Vì sao ta không thể dùng hàm kích hoạt tuyến tính (linear activation, $g(z) = z$) cho tất cả các tầng ẩn trong một mạng Neural Network có nhiều tầng? Chứng minh bằng toán học.

HẾT

Exam Appendix 4: CS114 - Exam_01_Answer

Source: `exam/exam_01/CS114 - Exam_01_Answer.md`

ĐÁP ÁN ĐỀ THI MACHINE LEARNING — ĐỀ 01

Câu 1. OLS Linear Regression bằng Ma trận

1.1 Ma trận thiết kế

$$\mathbf{X} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 1 \\ 3 \\ 4 \\ 6 \\ 8 \end{pmatrix}$$

1.2 Tính $\mathbf{X}^T \mathbf{X}$ và $\mathbf{X}^T \mathbf{y}$

$$\mathbf{X}^T \mathbf{X} = \begin{pmatrix} 5 & 10 \\ 10 & 30 \end{pmatrix}$$

Chi tiết: $\sum 1 = 5$, $\sum x_i = 0 + 1 + 2 + 3 + 4 = 10$, $\sum x_i^2 = 0 + 1 + 4 + 9 + 16 = 30$.

$$\mathbf{X}^T \mathbf{y} = \begin{pmatrix} 22 \\ 57 \end{pmatrix}$$

Chi tiết: $\sum y_i = 1 + 3 + 4 + 6 + 8 = 22$, $\sum x_i y_i = 0 + 3 + 8 + 18 + 32 = 61$.

Sửa: $\sum x_i y_i = 0(1) + 1(3) + 2(4) + 3(6) + 4(8) = 0 + 3 + 8 + 18 + 32 = 61$

$$\mathbf{X}^T \mathbf{y} = \begin{pmatrix} 22 \\ 61 \end{pmatrix}$$

1.3 Giải Normal Equations

$$\begin{pmatrix} 5 & 10 \\ 10 & 30 \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix} = \begin{pmatrix} 22 \\ 61 \end{pmatrix}$$

Từ PT1: $5\beta_0 + 10\beta_1 = 22 \rightarrow \beta_0 = \frac{22-10\beta_1}{5}$

Từ PT2: $10\beta_0 + 30\beta_1 = 61$

Thay PT1 vào PT2:

$$10 \cdot \frac{22 - 10\beta_1}{5} + 30\beta_1 = 61$$

$$2(22 - 10\beta_1) + 30\beta_1 = 61$$

$$44 - 20\beta_1 + 30\beta_1 = 61$$

$$10\beta_1 = 17 \implies \beta_1 = 1.7$$

$$\beta_0 = \frac{22 - 10(1.7)}{5} = \frac{22 - 17}{5} = 1.0$$

Mô hình: $\hat{y} = 1.0 + 1.7x$

1.4 Dự đoán

$$\hat{y}(6) = 1.0 + 1.7(6) = 1.0 + 10.2 = 11.2$$

x	y	$\hat{y}$	e
-----	-----	-----------	-----

1.5 Kiểm tra

Tính residual $e_i = y_i - \hat{y}_i$:

x	y	$\hat{y}$	e
0	1	1.0	0.0
1	3	2.7	0.3
2	4	4.4	-0.4
3	6	6.1	-0.1
4	8	7.8	0.2

$$\sum e_i = 0.0 + 0.3 - 0.4 - 0.1 + 0.2 = 0.0$$

$$\sum x_i e_i = 0(0) + 1(0.3) + 2(-0.4) + 3(-0.1) + 4(0.2) = 0 + 0.3 - 0.8 - 0.3 + 0.8 = 0.0$$

Câu 2. Logistic Regression — Thay số

$$\beta = (0.5, -1.2, 0.8), z = 0.5 - 1.2x_1 + 0.8x_2$$

STT	z	$\hat{p}$	Lớp dự đoán	y	Đúng?	Loss
1	$0.5 - 1.2 + 0.4 = -0.3$	0.426	0	1	Sai	$-\log(0.426) = 0.854$
2	$0.5 + 0.6 + 0.8 = 1.9$	0.870	1	0	Sai	$-\log(1 - 0.870) = 2.040$
3	$0.5 - 2.4 + 1.2 = -0.7$	0.332	0	1	Sai	$-\log(0.332) = 1.103$
4	$0.5 + 0 + 0 = 0.5$	0.622	1	0	Sai	$-\log(1 - 0.622) = 0.973$
5	$0.5 - 1.8 - 0.8 = -2.1$	0.109	0	0	Đúng	$-\log(1 - 0.109) = 0.115$

STT	z	$\hat{p}$	Lớp dự đoán	y	Đúng?	Loss
6	$0.5 + 1.2 + 1.6 = 3.3$	0.964	1	1	Đúng	$-\log(0.964) = 0.037$
7	$0.5 - 3.6 + 0.4 = -2.7$	0.063	0	1	Sai	$-\log(0.063) = 2.765$
8	$0.5 - 0.6 + 0.8 = 0.7$	0.668	1	1	Đúng	$-\log(0.668) = 0.404$

Cross-entropy trung bình:

$$\bar{J} = \frac{0.854 + 2.040 + 1.103 + 0.973 + 0.115 + 0.037 + 2.765 + 0.404}{8} = \frac{8.291}{8} \approx 1.036$$

Accuracy:

$$\text{Accuracy} = \frac{3}{8} = 0.375$$

Nhận xét: Mô hình dự đoán sai 5/8 mẫu, đặc biệt sai nghiêm trọng ở mẫu 2 (predict Spam nhưng thật Not Spam) và mẫu 7 (predict Not Spam nhưng thật Spam). Hệ số $\beta_1 = -1.2$ làm x_1 lớn kéo xác suất xuống, mâu thuẫn với dữ liệu ở mẫu 7 ($x_1 = 3, y = 1$). Mô hình cần được huấn luyện lại.

Câu 3. Exponential Family và GLM

3.1 Chuẩn hóa Bernoulli

$$\begin{aligned} P(y \mid \phi) &= \phi^y (1 - \phi)^{1-y} = \exp(y \log \phi + (1 - y) \log(1 - \phi)) \\ &= \exp\left(y \log \frac{\phi}{1 - \phi} + \log(1 - \phi)\right) \end{aligned}$$

Đổi chiều $P(y \mid \eta) = b(y) \exp(\eta T(y) - a(\eta))$:

- $\eta = \log \frac{\phi}{1 - \phi}$ (log-odds)
- $T(y) = y$
- $a(\eta) = -\log(1 - \phi) = \log(1 + e^\eta)$
- $b(y) = 1$

3.2 Chứng minh $\phi = \sigma(\eta)$

Từ $\eta = \log \frac{\phi}{1-\phi}$:

$$e^\eta = \frac{\phi}{1-\phi}$$

$$e^\eta(1-\phi) = \phi$$

$$e^\eta - e^\eta\phi = \phi$$

$$e^\eta = \phi(1 + e^\eta)$$

$$\phi = \frac{e^\eta}{1 + e^\eta} = \frac{1}{1 + e^{-\eta}} = \sigma(\eta) \quad \blacksquare$$

3.3 Dự đoán với 3 GLM

$$\eta = \beta^T \mathbf{x} = 0.3(1) + 1.5(2) + (-0.5)(3) = 0.3 + 3.0 - 1.5 = 1.8$$

- **Gaussian GLM:** $\hat{y} = \eta = 1.8$
- **Bernoulli GLM:** $P(y = 1) = \sigma(1.8) = \frac{1}{1+e^{-1.8}} \approx 0.858$
- **Poisson GLM:** $\hat{\lambda} = e^{1.8} \approx 6.050$

Câu 4. Neural Network Forward Pass — Mạng 3-2-1

4.1 Tầng ẩn

$$z_1^{[1]} = 0.3(0.5) + (-0.2)(-1.0) + 0.5(2.0) + 0.1 = 0.15 + 0.2 + 1.0 + 0.1 = 1.45$$

$$z_2^{[1]} = 0.1(0.5) + 0.4(-1.0) + (-0.3)(2.0) + (-0.1) = 0.05 - 0.4 - 0.6 - 0.1 = -1.05$$

ReLU:

$$a_1^{[1]} = \max(0, 1.45) = 1.45$$

$$a_2^{[1]} = \max(0, -1.05) = 0$$

4.2 Tầng output

$$Z^{[2]} = 0.7(1.45) + (-0.5)(0) + 0.2 = 1.015 + 0 + 0.2 = 1.215$$

$$\hat{y} = \sigma(1.215) = \frac{1}{1 + e^{-1.215}} \approx 0.771$$

4.3 Binary Cross-Entropy Loss

$$L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] = -[1 \cdot \log(0.771) + 0] = -\log(0.771) \approx 0.260$$

4.4 Backprop tại output

Với sigmoid + binary cross-entropy, $dZ^{[2]} = \hat{y} - y$:

$$dZ^{[2]} = 0.771 - 1 = -0.229$$

$$dW^{[2]} = dZ^{[2]} \cdot (A^{[1]})^T = -0.229 \begin{pmatrix} 1.45 & 0 \end{pmatrix} = \begin{pmatrix} -0.332 & 0 \end{pmatrix}$$

$$db^{[2]} = dZ^{[2]} = -0.229$$

Câu 5. Decision Tree — Gini Index

5.1 Gini Index ban đầu

10 mẫu: 6 Có, 4 Không.

$$G(S) = 1 - \left(\frac{6}{10}\right)^2 - \left(\frac{4}{10}\right)^2 = 1 - 0.36 - 0.16 = 0.48$$

5.2 Gini Gain — Thuộc tính Tuổi

- **Trẻ** (dòng 1,2,8,9): 1 Có, 3 Không $\rightarrow G = 1 - (1/4)^2 - (3/4)^2 = 1 - 0.0625 - 0.5625 = 0.375$
- **Trung niên** (dòng 3,7): 2 Có, 0 Không $\rightarrow G = 0$
- **Già** (dòng 4,5,6,10): 3 Có, 1 Không $\rightarrow G = 1 - (3/4)^2 - (1/4)^2 = 0.375$

$$G_{split} = \frac{4}{10}(0.375) + \frac{2}{10}(0) + \frac{4}{10}(0.375) = 0.15 + 0 + 0.15 = 0.30$$

$$\text{Gain}_{\text{Tuổi}} = 0.48 - 0.30 = 0.18$$

5.3 Gini Gain — Thuộc tính Sinh viên

- **Có** (dòng 5,6,7,9,10): 4 Có, 1 Không $\rightarrow G = 1 - (4/5)^2 - (1/5)^2 = 1 - 0.64 - 0.04 = 0.32$
- **Không** (dòng 1,2,3,4,8): 2 Có, 3 Không $\rightarrow G = 1 - (2/5)^2 - (3/5)^2 = 1 - 0.16 - 0.36 = 0.48$

$$G_{split} = \frac{5}{10}(0.32) + \frac{5}{10}(0.48) = 0.16 + 0.24 = 0.40$$

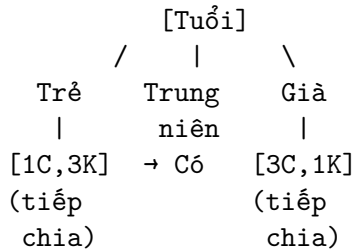
$$\text{Gain}_{\text{Sinh viên}} = 0.48 - 0.40 = 0.08$$

5.4 Kết luận

Thuộc tính	Gini Gain
Tuổi	0.18
Sinh viên	0.08

Chọn **Tuổi** vì Gini Gain lớn hơn.

5.5 Cây sau bước chia



Câu 6. K-means Clustering — Trace

6.1 Iteration 0 — Gán cụm

$$\mu_1 = (1, 2), \mu_2 = (9, 8)$$

STT	$d^2(\cdot, \mu_1)$	$d^2(\cdot, \mu_2)$	Cụm
1	$(1-1)^2 + (2-2)^2 = 0$	$(1-9)^2 + (2-8)^2 = 100$	1
2	$(2-1)^2 + (1-2)^2 = 2$	$(2-9)^2 + (1-8)^2 = 98$	1
3	$(2-1)^2 + (3-2)^2 = 2$	$(2-9)^2 + (3-8)^2 = 74$	1
4	$(8-1)^2 + (7-2)^2 = 74$	$(8-9)^2 + (7-8)^2 = 2$	2
5	$(9-1)^2 + (8-2)^2 = 100$	$(9-9)^2 + (8-8)^2 = 0$	2
6	$(8-1)^2 + (9-2)^2 = 98$	$(8-9)^2 + (9-8)^2 = 2$	2

$$C_1 = \{(1, 2), (2, 1), (2, 3)\}, C_2 = \{(8, 7), (9, 8), (8, 9)\}$$

Distortion:

$$J = \frac{1}{6}(0 + 2 + 2 + 2 + 0 + 2) = \frac{8}{6} \approx 1.333$$

6.2 Cập nhật tâm

$$\mu_1^{(1)} = \left(\frac{1+2+2}{3}, \frac{2+1+3}{3} \right) = \left(\frac{5}{3}, 2 \right) \approx (1.667, 2.000)$$

$$\mu_2^{(1)} = \left(\frac{8+9+8}{3}, \frac{7+8+9}{3} \right) = \left(\frac{25}{3}, 8 \right) \approx (8.333, 8.000)$$

6.3 Iteration 1 — Gán lại

Với tâm mới, tính khoảng cách. Vì hai nhóm cách nhau rất xa, nhãn không đổi.

$$C_1 = \{(1, 2), (2, 1), (2, 3)\}, C_2 = \{(8, 7), (9, 8), (8, 9)\} \text{ — không thay đổi.}$$

Tâm cập nhật lại cho cùng kết quả → **Hội tụ**

6.4 Giải thích

K-means luôn hội tụ vì: - Bước gán cụm giảm hoặc giữ nguyên J (mỗi điểm chọn tâm gần nhất). - Bước cập nhật tâm giảm hoặc giữ nguyên J (trung bình là nghiệm tối ưu cho tổng bình phương khoảng cách cố định thành viên). - $J \geq 0$ nên dãy giảm bị chặn dưới phải hội tụ.

Tuy nhiên, vì hàm mục tiêu không lồi, nghiệm cuối phụ thuộc vào khởi tạo và có thể là cực tiểu cục bộ. Giải pháp: chạy nhiều lần với K-means++ và chọn nghiệm có J nhỏ nhất.

Câu 7. Bias-Variance Decomposition

7.1 Phương trình

$$\text{Test MSE} = \text{Bias}^2(\hat{f}) + \text{Var}(\hat{f}) + \sigma^2$$

7.2 Chứng minh

Cho $y = f(\mathbf{x}) + \epsilon$ với $\mathbb{E}[\epsilon] = 0$, $\text{Var}(\epsilon) = \sigma^2$.

$$\begin{aligned}\mathbb{E}[(y - \hat{f})^2] &= \mathbb{E}[(f + \epsilon - \hat{f})^2] \\ &= \mathbb{E}[(f - \hat{f})^2 + 2\epsilon(f - \hat{f}) + \epsilon^2]\end{aligned}$$

Vì ϵ độc lập với $\hat{f}$ và $\mathbb{E}[\epsilon] = 0$:

$$= \mathbb{E}[(f - \hat{f})^2] + \sigma^2$$

Thêm bớt $\bar{f} = \mathbb{E}[\hat{f}]$:

$$\begin{aligned}\mathbb{E}[(f - \hat{f})^2] &= \mathbb{E}[(f - \bar{f}) + (\bar{f} - \hat{f})]^2 \\ &= (f - \bar{f})^2 + 2(f - \bar{f})\mathbb{E}[\bar{f} - \hat{f}] + \mathbb{E}[(\bar{f} - \hat{f})^2]\end{aligned}$$

Vì $\mathbb{E}[\bar{f} - \hat{f}] = 0$:

$$= (f - \bar{f})^2 + \mathbb{E}[(\hat{f} - \bar{f})^2]$$

$$= \text{Bias}^2 + \text{Variance}$$

Kết luận: $\mathbb{E}[(y - \hat{f})^2] = \text{Bias}^2 + \text{Var} + \sigma^2$ ■

7.3 Chọn λ

λ	Trung bình validation MSE
0.00	$(1.50 + 1.65 + 1.40 + 1.55)/4 = 1.525$
0.05	$(0.85 + 0.90 + 0.82 + 0.88)/4 = 0.8625$
0.50	$(0.60 + 0.65 + 0.58 + 0.62)/4 = \mathbf{0.6125}$
5.00	$(0.95 + 1.00 + 0.92 + 0.98)/4 = 0.9625$

Chọn $\lambda = 0.50$ vì validation MSE trung bình nhỏ nhất. $\lambda = 0$ overfit, $\lambda = 5$ underfit.

Câu 8. Tính phi tuyến trong Mạng Neural (Nâng cao)

Nếu dùng hàm kích hoạt tuyến tính $g(z) = z$, phép tính ở mỗi tầng chỉ là một biến đổi tuyến tính: Tầng 1: $\mathbf{A}^{[1]} = \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}$ Tầng 2: $\mathbf{A}^{[2]} = \mathbf{W}^{[2]}\mathbf{A}^{[1]} + \mathbf{b}^{[2]} = \mathbf{W}^{[2]}(\mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}) + \mathbf{b}^{[2]} = (\mathbf{W}^{[2]}\mathbf{W}^{[1]})\mathbf{x} + (\mathbf{W}^{[2]}\mathbf{b}^{[1]} + \mathbf{b}^{[2]})$

Đặt $\mathbf{W}' = \mathbf{W}^{[2]}\mathbf{W}^{[1]}$ và $\mathbf{b}' = \mathbf{W}^{[2]}\mathbf{b}^{[1]} + \mathbf{b}^{[2]}$, ta có $\mathbf{A}^{[2]} = \mathbf{W}'\mathbf{x} + \mathbf{b}'$. Đây lại là một phép biến đổi tuyến tính duy nhất.

Bất kể mạng có sâu bao nhiêu tầng, sự kết hợp của các phép biến đổi tuyến tính vẫn chỉ tạo ra một biến đổi tuyến tính. Mạng sẽ sụp đổ (collapse) thành một mô hình hồi quy tuyến tính/logistic thông thường, hoàn toàn mất đi khả năng học và mô phỏng các ranh giới phân loại phi tuyến phức tạp (như bài toán XOR).

HẾT ĐÁP ÁN

Exam Appendix 5: CS114 - Exam_02

Source: `exam/exam_02/CS114 - Exam_02.md`

ĐỀ THI MACHINE LEARNING — ĐỀ 02

- **Thời gian làm bài:** 90 phút
- **Hình thức:** Tự luận + tính toán
- **Ghi chú:** Sinh viên được sử dụng máy tính cầm tay. Trình bày rõ công thức, các bước biến đổi và kết luận.

Câu 1. MLE cho Linear Regression — Chứng minh OLS

Cho tập dữ liệu $\{(x_i, y_i)\}_{i=1}^n$ với giả định nhiễu Gaussian:

$$y_i = \beta^T \mathbf{x}_i + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

Hãy:

1. Viết hàm mật độ xác suất $P(y_i | \mathbf{x}_i; \beta)$ dưới dạng Gaussian.
2. Viết hàm Likelihood $L(\beta)$ cho toàn bộ tập dữ liệu (giả định i.i.d.).
3. Chứng minh rằng việc cực đại hóa Log-Likelihood tương đương với việc cực tiểu hóa tổng bình phương sai số (OLS).
4. Suy ra Normal Equations từ gradient của hàm loss ma trận $L = \frac{1}{2} \|\mathbf{y} - \mathbf{X}\beta\|_2^2$.

Câu 2. Gaussian Discriminant Analysis (GDA)

Cho bảng dữ liệu 1 chiều ($p = 1$) với nhãn nhị phân:

STT	x	y
1	1.0	0
2	1.5	0
3	2.0	0
4	2.5	0
5	5.0	1
6	5.5	1
7	6.0	1
8	6.5	1

Hãy:

1. Ước lượng prior $P(y = 0)$, $P(y = 1)$.
2. Ước lượng mean μ_0 , μ_1 .
3. Ước lượng variance chung σ^2 theo công thức GDA.
4. Phân loại điểm mới $x = 3.5$ bằng cách tính $P(y = 0 | x)$ và $P(y = 1 | x)$.
5. Tìm ranh giới quyết định (điểm x^* mà tại đó $P(y = 0 | x) = P(y = 1 | x)$).

Câu 3. Logistic Regression — Gradient Descent Một Bước

Cho mô hình Logistic Regression có $\beta_{init} = (0, 0, 0)$ (bias, β_1 , β_2). Cho 6 mẫu:

STT	x_1	x_2	y
1	1	0	1
2	0	1	0
3	2	1	1
4	1	2	0
5	3	1	1
6	0	3	0

Hãy:

1. Tính xác suất dự đoán ban đầu cho tất cả 6 mẫu (khi $\beta = \mathbf{0}$).
2. Tính gradient $\nabla J = \frac{1}{n} \mathbf{X}^T (\hat{\mathbf{p}} - \mathbf{y})$.
3. Cập nhật trọng số sau một bước gradient descent với $\alpha = 0.5$.

4. Giải thích ý nghĩa hướng cập nhật của từng hệ số.

Câu 4. Decision Tree — Entropy và Information Gain

Cho bảng dữ liệu phân loại rủi ro tín dụng:

STT	Trình độ	Nợ	Thu nhập	Rủi ro
1	Cao	Nhiều	Thấp	Cao
2	Cao	Ít	Trung bình	Cao
3	Cao	Ít	Cao	Thấp
4	Trung bình	Nhiều	Thấp	Cao
5	Trung bình	Ít	Cao	Thấp
6	Trung bình	Ít	Trung bình	Thấp
7	Thấp	Nhiều	Thấp	Cao
8	Thấp	Ít	Cao	Thấp
9	Thấp	Ít	Trung bình	Thấp
10	Thấp	Ít	Thấp	Thấp

Hãy:

1. Tính Entropy ban đầu $H(S)$.
2. Tính Information Gain cho thuộc tính **Nợ**.
3. Tính Information Gain cho thuộc tính **Thu nhập**.
4. Chọn thuộc tính tốt nhất để chia tại node gốc.

Câu 5. Backpropagation — Mạng 2-2-1 với Squared Error

Cho mạng neural network:

- 2 input: $x_1 = 2, x_2 = -1$
- 1 hidden layer gồm 2 neuron, dùng sigmoid
- 1 output neuron, dùng sigmoid
- Loss: $L = \frac{1}{2}(\hat{y} - y)^2$
- Nhân thật: $y = 0$

Trọng số:

Kết nối	Trọng số
$x_1 \rightarrow h_1$	0.5
$x_2 \rightarrow h_1$	-0.3
$x_1 \rightarrow h_2$	-0.2
$x_2 \rightarrow h_2$	0.4
$h_1 \rightarrow o$	0.6
$h_2 \rightarrow o$	-0.4

Bias: $b_{h1} = 0.2, b_{h2} = -0.1, b_o = 0.3$

Hãy thực hiện:

1. Forward propagation: tính $h_1, h_2, \hat{y}$.
2. Tính loss L .
3. Tính $\delta_o, \delta_{h1}, \delta_{h2}$ (backpropagation).
4. Tính gradient cho tất cả 6 trọng số.
5. Cập nhật trọng số với $\eta = 0.1$.

Biết rằng: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

Câu 6. K-means — Chọn K và Distortion

Cho 8 điểm dữ liệu 1D:

$$x = \{1, 2, 3, 10, 11, 12, 20, 21\}$$

Hãy:

1. Với $K = 2$, khởi tạo $\mu_1 = 2, \mu_2 = 11$. Thực hiện 2 vòng lặp K-means. Tính distortion sau mỗi vòng.
2. Tính distortion cuối cùng cho $K = 3$ nếu các cụm tối ưu là $\{1, 2, 3\}, \{10, 11, 12\}, \{20, 21\}$.
3. Dựa trên kết quả, vẽ đồ thị distortion theo K và chọn K hợp lý theo phương pháp elbow.

Câu 7. Thử thách: Hội tụ của K-means và Giới hạn của Đảm bảo Hội tụ

Dựa vào hàm mục tiêu K-means:

$$J(\mathbf{c}, \mu) = \sum_{i=1}^m \left\| \mathbf{x}^{(i)} - \mu_{c_i} \right\|_2^2$$

với $c_i \in \{1, \dots, K\}$ là nhãn cụm của điểm $\mathbf{x}^{(i)}$.

Hãy:

1. Chứng minh bước gán cụm không thể làm tăng J nếu các tâm cụm được giữ cố định.
2. Chứng minh bước cập nhật tâm cụm không thể làm tăng J nếu các nhãn cụm được giữ cố định.
3. Kết hợp hai ý trên để lập luận vì sao K-means hội tụ sau hữu hạn bước lặp.
4. Dùng bộ dữ liệu 1D $x = \{0, 2, 3, 10, 11, 12\}$ với $K = 2$:
 - Trường hợp A: khởi tạo $\mu_1 = 0, \mu_2 = 10$.
 - Trường hợp B: khởi tạo $\mu_1 = 2, \mu_2 = 3$. Thực hiện K-means đến khi hội tụ và tính SSE cuối cùng cho mỗi trường hợp.
5. Giải thích vì sao việc J giảm đơn điệu và hội tụ hữu hạn vẫn không đủ để chứng minh K-means luôn đạt cực tiểu toàn cục.
6. Nêu một chiến lược thực tế để giảm rủi ro bị kẹt ở nghiệm cục bộ.

HẾT

Exam Appendix 6: CS114 - Exam_02_Answer

Source: exam/exam_02/CS114 - Exam_02_Answer.md

ĐÁP ÁN ĐỀ THI MACHINE LEARNING — ĐỀ 02

Câu 1. MLE cho Linear Regression — Chứng minh OLS

1.1 Hàm mật độ

$$P(y_i | \mathbf{x}_i; \beta) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y_i - \beta^T \mathbf{x}_i)^2}{2\sigma^2}\right)$$

1.2 Hàm Likelihood

$$L(\beta) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y_i - \beta^T \mathbf{x}_i)^2}{2\sigma^2}\right)$$

1.3 Chứng minh MLE OLS

Log-likelihood:

$$\begin{aligned} \ell(\beta) &= \sum_{i=1}^n \left[\log \frac{1}{\sqrt{2\pi}\sigma} - \frac{(y_i - \beta^T \mathbf{x}_i)^2}{2\sigma^2} \right] \\ &= n \log \frac{1}{\sqrt{2\pi}\sigma} - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \beta^T \mathbf{x}_i)^2 \end{aligned}$$

Hạng đầu là hằng số theo β . Hệ số $\frac{1}{2\sigma^2} > 0$.

$$\max_{\beta} \ell \iff \min_{\beta} \sum_{i=1}^n (y_i - \beta^T \mathbf{x}_i)^2 \quad \blacksquare$$

1.4 Normal Equations

$$\begin{aligned} L &= \frac{1}{2}(\mathbf{y} - \mathbf{X}\beta)^T(\mathbf{y} - \mathbf{X}\beta) \\ &= \frac{1}{2}(\mathbf{y}^T\mathbf{y} - 2\beta^T\mathbf{X}^T\mathbf{y} + \beta^T\mathbf{X}^T\mathbf{X}\beta) \end{aligned}$$

Gradient:

$$\nabla_{\beta} L = -\mathbf{X}^T\mathbf{y} + \mathbf{X}^T\mathbf{X}\beta = \mathbf{0}$$

$$\implies \mathbf{X}^T\mathbf{X}\beta = \mathbf{X}^T\mathbf{y}$$

$$\implies \beta = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} \quad \blacksquare$$

Câu 2. Gaussian Discriminant Analysis

2.1 Prior

$$P(y = 0) = \frac{4}{8} = 0.5, \quad P(y = 1) = \frac{4}{8} = 0.5$$

2.2 Mean

$$\mu_0 = \frac{1.0 + 1.5 + 2.0 + 2.5}{4} = \frac{7.0}{4} = 1.75$$

$$\mu_1 = \frac{5.0 + 5.5 + 6.0 + 6.5}{4} = \frac{23.0}{4} = 5.75$$

2.3 Variance chung

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu_{y_i})^2$$

Lớp 0: $(1.0 - 1.75)^2 + (1.5 - 1.75)^2 + (2.0 - 1.75)^2 + (2.5 - 1.75)^2 = 0.5625 + 0.0625 + 0.0625 + 0.5625 = 1.25$

Lớp 1: $(5.0 - 5.75)^2 + (5.5 - 5.75)^2 + (6.0 - 5.75)^2 + (6.5 - 5.75)^2 = 0.5625 + 0.0625 + 0.0625 + 0.5625 = 1.25$

$$\sigma^2 = \frac{1.25 + 1.25}{8} = \frac{2.50}{8} = 0.3125$$

2.4 Phân loại $x = 3.5$

$$P(x = 3.5 \mid y = 0) = \frac{1}{\sqrt{2\pi(0.3125)}} \exp\left(-\frac{(3.5 - 1.75)^2}{2(0.3125)}\right) = \frac{1}{1.4005} \exp\left(-\frac{3.0625}{0.625}\right) = 0.7142 \cdot e^{-4.9} \approx 0.00532$$

$$P(x = 3.5 \mid y = 1) = \frac{1}{1.4005} \exp\left(-\frac{(3.5 - 5.75)^2}{0.625}\right) = 0.7142 \cdot e^{-8.1} \approx 0.000216$$

Nhân prior:

$$\text{score}_0 = 0.00532 \times 0.5 = 0.00266$$

$$\text{score}_1 = 0.000216 \times 0.5 = 0.000108$$

Posterior:

$$P(y = 0 \mid x = 3.5) = \frac{0.00266}{0.00266 + 0.000108} = \frac{0.00266}{0.00277} \approx 0.961$$

Kết luận: Phân loại $x = 3.5$ vào **lớp 0**.

2.5 Ranh giới quyết định

Vì prior bằng nhau, ranh giới là điểm cách đều hai mean:

$$x^* = \frac{\mu_0 + \mu_1}{2} = \frac{1.75 + 5.75}{2} = 3.75$$

(Khi prior bằng nhau và variance chung, ranh giới nằm giữa hai mean.)

Câu 3. Logistic Regression — Gradient Descent

3.1 Xác suất ban đầu

Khi $\beta = (0, 0, 0)$: $z_i = 0$ cho mọi mẫu, nên $\hat{p}_i = \sigma(0) = 0.5$ cho tất cả.

3.2 Gradient

Ma trận $\mathbf{X}$ (cột 1 cho bias):

$$\mathbf{X} = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \\ 1 & 3 & 1 \\ 1 & 0 & 3 \end{pmatrix}$$

Sai số: $\hat{p}_i - y_i$:

STT	$\hat{p}$	y	$\hat{p} - y$
1	0.5	1	-0.5
2	0.5	0	0.5
3	0.5	1	-0.5
4	0.5	0	0.5
5	0.5	1	-0.5
6	0.5	0	0.5

$$\nabla J = \frac{1}{6} \mathbf{X}^T (\hat{\mathbf{p}} - \mathbf{y})$$

Thành phần bias ($x_0 = 1$):

$$\frac{1}{6} \sum (\hat{p}_i - y_i) = \frac{1}{6} (-0.5 + 0.5 - 0.5 + 0.5 - 0.5 + 0.5) = 0$$

Thành phần β_1 :

$$\begin{aligned} \frac{1}{6} \sum (\hat{p}_i - y_i) x_{i1} &= \frac{1}{6} [(-0.5)(1) + (0.5)(0) + (-0.5)(2) + (0.5)(1) + (-0.5)(3) + (0.5)(0)] \\ &= \frac{1}{6} (-0.5 + 0 - 1.0 + 0.5 - 1.5 + 0) = \frac{-2.5}{6} = -0.4167 \end{aligned}$$

Thành phần β_2 :

$$\begin{aligned} \frac{1}{6} \sum (\hat{p}_i - y_i) x_{i2} &= \frac{1}{6} [(-0.5)(0) + (0.5)(1) + (-0.5)(1) + (0.5)(2) + (-0.5)(1) + (0.5)(3)] \\ &= \frac{1}{6} (0 + 0.5 - 0.5 + 1.0 - 0.5 + 1.5) = \frac{2.0}{6} = 0.3333 \end{aligned}$$

$$\nabla J = \begin{pmatrix} 0 \\ -0.4167 \\ 0.3333 \end{pmatrix}$$

3.3 Cập nhật

$$\beta_{new} = \beta - \alpha \nabla J = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} - 0.5 \begin{pmatrix} 0 \\ -0.4167 \\ 0.3333 \end{pmatrix} = \begin{pmatrix} 0 \\ 0.2083 \\ -0.1667 \end{pmatrix}$$

3.4 Ý nghĩa

- β_1 tăng (dương): x_1 lớn tương quan dương với $y = 1$. Đúng vì các mẫu dương có x_1 lớn (1, 2, 3).
- β_2 giảm (âm): x_2 lớn tương quan dương với $y = 0$. Đúng vì các mẫu $y = 0$ có x_2 lớn (1, 2, 3).
- Bias không đổi vì số mẫu lớp 0 và lớp 1 bằng nhau.

Câu 4. Decision Tree — Entropy

4.1 Entropy ban đầu

10 mẫu: 4 Cao, 6 Thấp.

$$\begin{aligned}H(S) &= -\frac{4}{10} \log_2 \frac{4}{10} - \frac{6}{10} \log_2 \frac{6}{10} \\&= -0.4 \log_2 0.4 - 0.6 \log_2 0.6 = -0.4(-1.3219) - 0.6(-0.7370) \\&= 0.5288 + 0.4422 = 0.9710\end{aligned}$$

4.2 Information Gain — Ng

- **Nhiều** (dòng 1,4,7): 3 Cao, 0 Thấp $\rightarrow H = 0$
- **Ít** (dòng 2,3,5,6,8,9,10): 1 Cao, 6 Thấp $\rightarrow H = -\frac{1}{7} \log_2 \frac{1}{7} - \frac{6}{7} \log_2 \frac{6}{7} = 0.5917$

$$G_{split} = \frac{3}{10}(0) + \frac{7}{10}(0.5917) = 0 + 0.4142 = 0.4142$$

$$\text{Gain}_{Ng} = 0.9710 - 0.4142 = 0.5568$$

4.3 Information Gain — Thu nhập

- **Thấp** (dòng 1,4,7,10): 3 Cao, 1 Thấp $\rightarrow H = -\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} = 0.8113$
- **Trung bình** (dòng 2,6,9): 1 Cao, 2 Thấp $\rightarrow H = -\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3} = 0.9183$
- **Cao** (dòng 3,5,8): 0 Cao, 3 Thấp $\rightarrow H = 0$

$$G_{split} = \frac{4}{10}(0.8113) + \frac{3}{10}(0.9183) + \frac{3}{10}(0)$$

$$= 0.3245 + 0.2755 + 0 = 0.6000$$

$$\text{Gain}_{Thu\ nh\ ap} = 0.9710 - 0.6000 = 0.3710$$

4.4 Kết luận

Thuộc tính	Information Gain
Nợ	0.5568
Thu nhập	0.3710

Chọn Nợ vì Information Gain lớn nhất. Nhánh “Nhiều” nợ $\rightarrow$ thuần nhất lớp Cao (entropy = 0).

Câu 5. Backpropagation — Mạng 2-2-1

5.1 Forward Propagation

$$z_{h1} = 0.5(2) + (-0.3)(-1) + 0.2 = 1.0 + 0.3 + 0.2 = 1.5$$

$$h_1 = \sigma(1.5) = \frac{1}{1+e^{-1.5}} \approx 0.8176$$

$$z_{h2} = (-0.2)(2) + 0.4(-1) + (-0.1) = -0.4 - 0.4 - 0.1 = -0.9$$

$$h_2 = \sigma(-0.9) = \frac{1}{1+e^{0.9}} \approx 0.2891$$

$$z_o = 0.6(0.8176) + (-0.4)(0.2891) + 0.3 = 0.4906 - 0.1156 + 0.3 = 0.6750$$

$$\hat{y} = \sigma(0.6750) \approx 0.6625$$

5.2 Loss

$$L = \frac{1}{2}(0.6625 - 0)^2 = \frac{1}{2}(0.4389) = 0.2195$$

5.3 Backpropagation

$$\delta_o = (\hat{y} - y) \cdot \sigma'(z_o) = (0.6625 - 0) \cdot \hat{y}(1 - \hat{y}) = 0.6625 \times 0.6625 \times 0.3375 = 0.1481$$

$$\delta_{h1} = \delta_o \cdot w_{h1 \rightarrow o} \cdot h_1(1 - h_1) = 0.1481 \times 0.6 \times 0.8176 \times 0.1824 = 0.01325$$

$$\delta_{h2} = \delta_o \cdot w_{h2 \rightarrow o} \cdot h_2(1 - h_2) = 0.1481 \times (-0.4) \times 0.2891 \times 0.7109 = -0.01217$$

5.4 Gradient

Trọng số	Gradient
$\partial L / \partial w_{h1 \rightarrow o} = \delta_o \cdot h_1$	$0.1481 \times 0.8176 = 0.1211$
$\partial L / \partial w_{h2 \rightarrow o} = \delta_o \cdot h_2$	$0.1481 \times 0.2891 = 0.04282$
$\partial L / \partial w_{x1 \rightarrow h1} = \delta_{h1} \cdot x_1$	$0.01325 \times 2 = 0.02650$
$\partial L / \partial w_{x2 \rightarrow h1} = \delta_{h1} \cdot x_2$	$0.01325 \times (-1) = -0.01325$
$\partial L / \partial w_{x1 \rightarrow h2} = \delta_{h2} \cdot x_1$	$-0.01217 \times 2 = -0.02434$
$\partial L / \partial w_{x2 \rightarrow h2} = \delta_{h2} \cdot x_2$	$-0.01217 \times (-1) = 0.01217$

5.5 Cập nhật ($\eta = 0.1$)

Kết nối	w_{old}	Gradient	$w_{new} = w_{old} - 0.1 \times \text{grad}$
$x_1 \rightarrow h_1$	0.500	0.02650	0.4974
$x_2 \rightarrow h_1$	-0.300	-0.01325	-0.2987
$x_1 \rightarrow h_2$	-0.200	-0.02434	-0.1976
$x_2 \rightarrow h_2$	0.400	0.01217	0.3988
$h_1 \rightarrow o$	0.600	0.1211	0.5879
$h_2 \rightarrow o$	-0.400	0.04282	-0.4043

Câu 6. K-means

6.1 Iteration 0

$$\mu_1 = 2, \mu_2 = 11$$

x	$d(x, \mu_1)$	$d(x, \mu_2)$	Cụm
1	1	100	1
2	0	81	1
3	1	64	1
10	64	1	2
11	81	0	2
12	100	1	2
20	324	81	2
21	361	100	2

$$C_1 = \{1, 2, 3\}, C_2 = \{10, 11, 12, 20, 21\}$$

$$J_0 = \frac{1}{8}(1 + 0 + 1 + 1 + 0 + 1 + 81 + 100) = \frac{185}{8} = 23.125$$

Cập nhật:

$$\mu_1^{(1)} = \frac{1 + 2 + 3}{3} = 2$$

$$\mu_2^{(1)} = \frac{10 + 11 + 12 + 20 + 21}{5} = \frac{74}{5} = 14.8$$

Iteration 1

Với $\mu_1 = 2$, $\mu_2 = 14.8$:

x	$d^2(x, 2)$	$d^2(x, 14.8)$	Cộng
1	1	190.44	1
2	0	163.84	1
3	1	139.24	1
10	64	23.04	2
11	81	14.44	2
12	100	7.84	2
20	324	27.04	2
21	361	38.44	2

Không đổi: $C_1 = \{1, 2, 3\}$, $C_2 = \{10, 11, 12, 20, 21\}$

$$J_1 = \frac{1}{8}(1 + 0 + 1 + 23.04 + 14.44 + 7.84 + 27.04 + 38.44) = \frac{112.8}{8} = 14.1$$

$$\mu_1^{(2)} = 2, \quad \mu_2^{(2)} = 14.8 \quad (\text{hội tụ})$$

6.2 Với $K = 3$

$C_1 = \{1, 2, 3\}$: $\mu_1 = 2$, distortion = $(1 + 0 + 1)/3 = 0.667$

$C_2 = \{10, 11, 12\}$: $\mu_2 = 11$, distortion = $(1 + 0 + 1)/3 = 0.667$

$C_3 = \{20, 21\}$: $\mu_3 = 20.5$, distortion = $(0.25 + 0.25)/2 = 0.25$

$$J_{K=3} = \frac{1}{8}(1 + 0 + 1 + 1 + 0 + 1 + 0.25 + 0.25) = \frac{4.5}{8} = 0.5625$$

6.3 Elbow

K	Distortion
1	rất lớn
2	14.1
3	0.5625

Giảm rất mạnh từ $K = 2$ sang $K = 3$ (từ 14.1 xuống 0.56). **Chọn** $K = 3$ vì dữ liệu có 3 nhóm rõ ràng ($\{1,2,3\}$, $\{10,11,12\}$, $\{20,21\}$).

Câu 7. Thử thách: Hội tụ của K-means và Giới hạn của Đảm bảo Hội tụ

7.1 Bước gán cụm

Khi các tâm $\mu_1, \dots, \mu_K$ cố định, mỗi điểm $\mathbf{x}^{(i)}$ đóng góp:

$$\left\| \mathbf{x}^{(i)} - \mu_{c_i} \right\|_2^2.$$

Chọn cụm gần nhất nghĩa là:

$$c_i^{new} = \arg \min_k \left\| \mathbf{x}^{(i)} - \mu_k \right\|_2^2.$$

Do đó với từng điểm:

$$\left\| \mathbf{x}^{(i)} - \mu_{c_i^{new}} \right\|_2^2 \leq \left\| \mathbf{x}^{(i)} - \mu_{c_i^{old}} \right\|_2^2.$$

Cộng trên mọi điểm suy ra J không tăng.

7.2 Bước cập nhật tâm

Khi nhân cụm cố định, bài toán của cụm C_k là:

$$\min_{\mu_k} \sum_{\mathbf{x}^{(i)} \in C_k} \|\mathbf{x}^{(i)} - \mu_k\|_2^2.$$

Lấy gradient:

$$\nabla_{\mu_k} J_k = -2 \sum_{\mathbf{x}^{(i)} \in C_k} \mathbf{x}^{(i)} + 2|C_k|\mu_k.$$

Đặt bằng 0:

$$\mu_k = \frac{1}{|C_k|} \sum_{\mathbf{x}^{(i)} \in C_k} \mathbf{x}^{(i)}.$$

Vì mean là nghiệm tối ưu của bài toán bình phương nhỏ nhất trong cụm, bước cập nhật tâm cũng không làm tăng J .

7.3 Hội tụ hữu hạn

Mỗi vòng lặp gồm hai bước đều làm J giảm hoặc giữ nguyên. Đồng thời $J \geq 0$. Hơn nữa, với m điểm và K cụm, số cách gán cụm tối đa là K^m , hữu hạn. Vì thuật toán không thể tạo vô hạn trạng thái gán cụm khác nhau với J giảm mãi, K-means phải dừng sau hữu hạn bước khi nhân cụm không đổi.

7.4 Ví dụ hai khởi tạo

Dữ liệu: $x = \{0, 2, 3, 10, 11, 12\}$, $K = 2$.

Trường hợp A: $\mu_1 = 0$, $\mu_2 = 10$.

Gán cụm:

$$C_1 = \{0, 2, 3\}, \quad C_2 = \{10, 11, 12\}.$$

Cập nhật:

$$\mu_1 = \frac{0 + 2 + 3}{3} = \frac{5}{3}, \quad \mu_2 = \frac{10 + 11 + 12}{3} = 11.$$

Gán lại không đổi, nên hội tụ. SSE:

$$J_A = (0 - \frac{5}{3})^2 + (2 - \frac{5}{3})^2 + (3 - \frac{5}{3})^2 + (10 - 11)^2 + (11 - 11)^2 + (12 - 11)^2.$$

$$J_A = \frac{25}{9} + \frac{1}{9} + \frac{16}{9} + 1 + 0 + 1 = \frac{42}{9} + 2 = \frac{20}{3} \approx 6.667.$$

Trường hợp B: $\mu_1 = 2, \mu_2 = 3$.

Lần gán đầu:

$$C_1 = \{0, 2\}, \quad C_2 = \{3, 10, 11, 12\}.$$

Cập nhật:

$$\mu_1 = 1, \quad \mu_2 = \frac{3 + 10 + 11 + 12}{4} = 9.$$

Gán lại:

$$C_1 = \{0, 2, 3\}, \quad C_2 = \{10, 11, 12\}.$$

Cập nhật tiếp:

$$\mu_1 = \frac{5}{3}, \quad \mu_2 = 11.$$

SSE cuối cùng cũng là:

$$J_B = \frac{20}{3} \approx 6.667.$$

7.5 Vì sao vẫn không đảm bảo global minimum?

Trong ví dụ này hai khởi tạo đều đi về cùng nghiệm tốt. Tuy nhiên, đây chỉ là một ví dụ thuận lợi, không phải một chứng minh global optimum cho mọi dữ liệu.

Lý do cốt lõi là $J(\mathbf{c}, \mu)$ không lồi theo đồng thời cả nhãn cụm rời rạc $\mathbf{c}$ và tâm cụm μ . K-means chỉ tối ưu từng khối biến một cách tham lam:

- Tối ưu nhãn khi tâm cố định.
- Tối ưu tâm khi nhãn cố định.

Một trạng thái có thể ổn định theo hai bước này nhưng vẫn không phải cách chia tốt nhất trong toàn bộ không gian phân cụm. Do đó, hội tụ hữu hạn chỉ đảm bảo đạt một điểm cố định của thuật toán Lloyd, không đảm bảo cực tiểu toàn cục.

7.6 Cách giảm rủi ro nghiệm cục bộ

Trong thực tế thường dùng:

- Chạy K-means nhiều lần với nhiều khởi tạo khác nhau rồi chọn nghiệm có SSE nhỏ nhất.
- Dùng K-means++ để chọn tâm ban đầu xa nhau hơn và có xác suất khởi tạo tốt hơn.
- Kiểm tra elbow/silhouette để tránh chọn K quá sai so với cấu trúc dữ liệu.

HẾT ĐÁP ÁN

Exam Appendix 7: CS114 - Exam_03

Source: `exam/exam_03/CS114 - Exam_03.md`

ĐỀ THI MACHINE LEARNING — ĐỀ 03

- **Thời gian làm bài:** 90 phút
- **Hình thức:** Tự luận + tính toán
- **Ghi chú:** Sinh viên được sử dụng máy tính cầm tay. Trình bày rõ công thức, các bước biến đổi và kết luận.

Câu 1. Hồi quy Tuyến tính Đơn (OLS từ đầu)

Cho 6 điểm dữ liệu:

STT	x	y
1	1	3
2	2	5
3	3	6
4	4	9
5	5	11
6	6	13

Hãy:

1. Tính $\bar{x}$, $\bar{y}$.
2. Tính $S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y})$ và $S_{xx} = \sum (x_i - \bar{x})^2$.
3. Tính hệ số góc β_1 và hệ số chặn β_0 .
4. Dự đoán $\hat{y}$ khi $x = 8$.
5. Tính $RSS = \sum (y_i - \hat{y}_i)^2$ và kiểm tra $\sum e_i = 0$.

Câu 2. Chứng minh Hàm Sigmoid từ Exponential Family

Hãy:

1. Viết phân phối Gaussian $\mathcal{N}(\mu, \sigma^2)$ với $\sigma^2 = 1$ dưới dạng Exponential Family. Xác định η , $T(y)$, $a(\eta)$, $b(y)$.
2. Sử dụng tính chất $\mathbb{E}[y] = a'(\eta)$, kiểm tra rằng $\mathbb{E}[y] = \mu$ cho Gaussian ($\sigma^2 = 1$).
3. Sử dụng tính chất $\text{Var}(y) = a''(\eta)$, kiểm tra rằng $\text{Var}(y) = 1$ cho Gaussian ($\sigma^2 = 1$).
4. Giải thích vì sao Gaussian GLM cho ra Linear Regression (identity link) và Bernoulli GLM cho ra Logistic Regression (logit link).

Câu 3. Naive Bayes với Laplace Smoothing

Cho bảng dữ liệu phân loại bệnh nhân có bệnh tim hay không:

STT	Huyết áp	Cholesterol	Hút thuốc	Bệnh tim
1	Cao	Cao	Có	Có
2	Cao	Bình thường	Không	Không
3	Bình thường	Cao	Có	Có
4	Bình thường	Bình thường	Không	Không
5	Cao	Cao	Không	Có
6	Bình thường	Bình thường	Có	Không
7	Cao	Cao	Có	Có
8	Bình thường	Cao	Không	Không

Bệnh nhân mới: Huyết áp = Cao, Cholesterol = Cao, Hút thuốc = Không.

Hãy:

1. Tính prior probability.
2. Tính conditional probabilities cần thiết (có Laplace smoothing $\alpha = 1$).
3. Tính posterior cho hai lớp.
4. Kết luận bệnh nhân mới có bệnh tim hay không.

Câu 4. Regularization và Ridge Regression

Cho mô hình hồi quy tuyến tính với hàm loss Ridge:

$$L_{ridge}(\beta) = \frac{1}{2n} \sum_{i=1}^n (y_i - \beta^T \mathbf{x}_i)^2 + \lambda \|\beta\|_2^2$$

Hãy:

1. Viết gradient ∇L_{ridge} theo β .
2. Chứng minh nghiệm đóng Ridge Regression là: $\beta_{ridge} = (\mathbf{X}^T \mathbf{X} + 2n\lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$.
3. Cho $\beta = (3.0, -1.5, 0.8)$, train loss = 0.15, $\lambda = 0.2$. Tính Ridge objective.
4. Giải thích sự khác biệt giữa Ridge (L_2) và Lasso (L_1) về mặt ảnh hưởng đến hệ số.

Câu 5. Neural Network — Xác định Kích thước Ma trận

Cho mạng fully connected có kiến trúc: 4 input $\rightarrow$ 5 neuron ẩn (tầng 1) $\rightarrow$ 3 neuron ẩn (tầng 2) $\rightarrow$ 2 output. Mini-batch có $m = 10$ mẫu.

Hãy:

1. Liệt kê kích thước của $\mathbf{W}^{[1]}$, $\mathbf{b}^{[1]}$, $\mathbf{Z}^{[1]}$, $\mathbf{A}^{[1]}$.
2. Liệt kê kích thước của $\mathbf{W}^{[2]}$, $\mathbf{b}^{[2]}$, $\mathbf{Z}^{[2]}$, $\mathbf{A}^{[2]}$.
3. Liệt kê kích thước của $\mathbf{W}^{[3]}$, $\mathbf{b}^{[3]}$, $\mathbf{Z}^{[3]}$, $\mathbf{A}^{[3]}$.
4. Tổng số tham số (weights + biases) của mạng.
5. Với gradient $\mathbf{dW}^{[2]}$, xác nhận kích thước phải bằng kích thước $\mathbf{W}^{[2]}$.
6. Giải thích vì sao hidden layer hiện đại dùng ReLU thay vì Sigmoid.

Câu 6. K-means — Chứng minh Tâm Cụm Là Trung Bình

Cho cụm $C_k = \{x_1, x_2, \dots, x_m\}$. Hàm mục tiêu cụm:

$$f(\mu) = \sum_{i=1}^m \|x_i - \mu\|_2^2$$

Hãy:

1. Khai triển $f(\mu)$ dưới dạng $\sum \|x_i\|_2^2 - 2\mu^T \sum x_i + m\|\mu\|_2^2$.
 2. Tính gradient $\nabla_{\mu} f$ và đặt bằng 0.
 3. Chứng minh $\mu^* = \frac{1}{m} \sum_{i=1}^m x_i$ (trung bình cộng).
 4. Áp dụng: Cho cụm gồm 4 điểm $(1, 3)$, $(2, 5)$, $(4, 1)$, $(5, 3)$. Tính tâm cụm tối ưu và distortion.
-

Câu 7. Thử thách: Newton's Method cho Logistic Regression

Cho Logistic Regression nhị phân với m mẫu, ma trận thiết kế $\mathbf{X} \in \mathbb{R}^{m \times p}$, nhãn $\mathbf{y} \in \{0, 1\}^m$, xác suất dự đoán:

$$\mathbf{p} = \sigma(\mathbf{X}\beta)$$

và negative log-likelihood:

$$J(\beta) = - \sum_{i=1}^m [y_i \log p_i + (1 - y_i) \log(1 - p_i)].$$

Hãy:

1. Chứng minh $\nabla J(\beta) = \mathbf{X}^T(\mathbf{p} - \mathbf{y})$.
2. Chứng minh Hessian có dạng $\nabla^2 J(\beta) = \mathbf{X}^T \mathbf{R} \mathbf{X}$, trong đó $\mathbf{R}$ là ma trận đường chéo với $R_{ii} = p_i(1 - p_i)$.
3. Suy ra vì sao J là hàm lồi.
4. Viết công thức cập nhật Newton:

$$\beta_{new} = \beta - (\mathbf{X}^T \mathbf{R} \mathbf{X})^{-1} \mathbf{X}^T(\mathbf{p} - \mathbf{y}).$$

5. So sánh Newton's Method với Gradient Descent về tốc độ hội tụ và chi phí tính toán khi p rất lớn.

HẾT

Exam Appendix 8: CS114 - Exam_03_Answer

Source: `exam/exam_03/CS114 - Exam_03_Answer.md`

ĐÁP ÁN ĐỀ THI MACHINE LEARNING — ĐỀ 03

Câu 1. Hồi quy Tuyến tính Đơn

1.1

$$\bar{x} = \frac{1 + 2 + 3 + 4 + 5 + 6}{6} = \frac{21}{6} = 3.5$$

$$\bar{y} = \frac{3 + 5 + 6 + 9 + 11 + 13}{6} = \frac{47}{6} \approx 7.833$$

1.2

x_i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$
1	-2.5	-4.833	12.083	6.25
2	-1.5	-2.833	4.250	2.25
3	-0.5	-1.833	0.917	0.25
4	0.5	1.167	0.583	0.25
5	1.5	3.167	4.750	2.25
6	2.5	5.167	12.917	6.25

$$S_{xy} = 12.083 + 4.250 + 0.917 + 0.583 + 4.750 + 12.917 = 35.500$$

$$S_{xx} = 6.25 + 2.25 + 0.25 + 0.25 + 2.25 + 6.25 = 17.500$$

1.3

$$\beta_1 = \frac{S_{xy}}{S_{xx}} = \frac{35.5}{17.5} \approx 2.029$$

$$\beta_0 = \bar{y} - \beta_1 \bar{x} = 7.833 - 2.029(3.5) = 7.833 - 7.101 = 0.733$$

Mô hình: $\hat{y} = 0.733 + 2.029x$

1.4

$$\hat{y}(8) = 0.733 + 2.029(8) = 0.733 + 16.229 = 16.962$$

1.5

x	y	$\hat{y}$	$e = y - \hat{y}$	e^2
1	3	2.762	0.238	0.057
2	5	4.790	0.210	0.044
3	6	6.819	-0.819	0.671
4	9	8.848	0.152	0.023
5	11	10.876	0.124	0.015
6	13	12.905	0.095	0.009

$$\sum e_i = 0.238 + 0.210 - 0.819 + 0.152 + 0.124 + 0.095 \approx 0.0$$

$$RSS = 0.057 + 0.044 + 0.671 + 0.023 + 0.015 + 0.009 = 0.819$$

Câu 2. Exponential Family

2.1 Gaussian dạng EFD ($\sigma^2 = 1$)

$$P(y | \mu) = \frac{1}{\sqrt{2\pi}} e^{-y^2/2} \cdot \exp\left(y\mu - \frac{\mu^2}{2}\right)$$

- $\eta = \mu$
- $T(y) = y$
- $a(\eta) = \frac{\eta^2}{2} = \frac{\mu^2}{2}$
- $b(y) = \frac{1}{\sqrt{2\pi}} e^{-y^2/2}$

2.2 Kiểm tra $\mathbb{E}[y]$

$$a'(\eta) = \frac{d}{d\eta} \frac{\eta^2}{2} = \eta = \mu \quad \checkmark$$

2.3 Kiểm tra $\text{Var}(y)$

$$a''(\eta) = \frac{d^2}{d\eta^2} \frac{\eta^2}{2} = 1 = \sigma^2 \quad \checkmark$$

2.4 Giải thích

- **Gaussian GLM:** $\eta = \mu$, link function là identity ($g(\mu) = \mu$), inverse link $g^{-1}(\eta) = \eta$. Dự đoán $\hat{y} = \beta^T \mathbf{x} \rightarrow$ **Linear Regression**.
- **Bernoulli GLM:** $\eta = \log \frac{\phi}{1-\phi}$, link function là logit, inverse link $\phi = \sigma(\eta)$. Dự đoán $P(y=1) = \sigma(\beta^T \mathbf{x}) \rightarrow$ **Logistic Regression**.

Toàn bộ sự khác biệt giữa các GLM nằm ở inverse link function.

Câu 3. Naive Bayes

3.1 Prior

4 mẫu “Có bệnh tim” (dòng 1,3,5,7), 4 mẫu “Không” (dòng 2,4,6,8).

$$P(\text{Có}) = P(\text{Không}) = 0.5$$

3.2 Conditional Probabilities (Laplace $\alpha = 1$)

Lớp Có (4 mẫu):

- Huyết áp=Cao: dòng 1,5,7 $\rightarrow \text{count} = 3 \rightarrow P = \frac{3+1}{4+2} = \frac{4}{6} = 0.667$
- Cholesterol=Cao: dòng 1,3,5,7 $\rightarrow \text{count} = 4 \rightarrow P = \frac{4+1}{4+2} = \frac{5}{6} = 0.833$
- Hút thuốc=Không: dòng 5 $\rightarrow \text{count} = 1 \rightarrow P = \frac{1+1}{4+2} = \frac{2}{6} = 0.333$

Lớp Không (4 mẫu):

- Huyết áp=Cao: dòng 2 $\rightarrow \text{count} = 1 \rightarrow P = \frac{1+1}{4+2} = \frac{2}{6} = 0.333$
- Cholesterol=Cao: dòng 8 $\rightarrow \text{count} = 1 \rightarrow P = \frac{1+1}{4+2} = \frac{2}{6} = 0.333$
- Hút thuốc=Không: dòng 2,4,8 $\rightarrow \text{count} = 3 \rightarrow P = \frac{3+1}{4+2} = \frac{4}{6} = 0.667$

3.3 Posterior

Email mới: Huyết áp=Cao, Cholesterol=Cao, Hút thuốc=Không.

$$\text{score}_{\text{Có}} = P(\text{Có}) \times 0.667 \times 0.833 \times 0.333 = 0.5 \times 0.185 = 0.0926$$

$$\text{score}_{\text{Không}} = P(\text{Không}) \times 0.333 \times 0.333 \times 0.667 = 0.5 \times 0.0740 = 0.03700$$

$$P(\text{Có} \mid \mathbf{x}) = \frac{0.0926}{0.0926 + 0.03700} = \frac{0.0926}{0.1296} \approx 0.714$$

3.4 Kết luận

Vì $P(\text{Có} \mid \mathbf{x}) \approx 0.714 > 0.5$, bệnh nhân mới được phân loại là **Có bệnh tim**.

Câu 4. Ridge Regression

4.1 Gradient

$$\nabla L_{\text{ridge}} = -\frac{1}{n} \mathbf{X}^T (\mathbf{y} - \mathbf{X}\beta) + 2\lambda\beta$$

4.2 Nghiệm đóng

Đặt gradient bằng 0:

$$-\frac{1}{n} \mathbf{X}^T \mathbf{y} + \frac{1}{n} \mathbf{X}^T \mathbf{X} \beta + 2\lambda\beta = \mathbf{0}$$

$$\frac{1}{n} \mathbf{X}^T \mathbf{X} \beta + 2\lambda\beta = \frac{1}{n} \mathbf{X}^T \mathbf{y}$$

$$\left(\frac{1}{n} \mathbf{X}^T \mathbf{X} + 2\lambda \mathbf{I} \right) \beta = \frac{1}{n} \mathbf{X}^T \mathbf{y}$$

Nhân hai vế cho n :

$$(\mathbf{X}^T \mathbf{X} + 2n\lambda \mathbf{I})\beta = \mathbf{X}^T \mathbf{y}$$

$$\beta_{ridge} = (\mathbf{X}^T \mathbf{X} + 2n\lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y} \quad \blacksquare$$

4.3 Ridge objective

$$\|\beta\|_2^2 = 3.0^2 + (-1.5)^2 + 0.8^2 = 9 + 2.25 + 0.64 = 11.89$$

$$L_{ridge} = 0.15 + 0.2(11.89) = 0.15 + 2.378 = 2.528$$

4.4 Ridge vs Lasso

- **Ridge (L_2):** Co nhỏ tất cả hệ số một cách mượt và tỷ lệ. Không bao giờ đẩy hệ số về đúng 0. Phù hợp khi tất cả đặc trưng đều có ảnh hưởng.
- **Lasso (L_1):** Có thể đẩy một số hệ số về chính xác 0, thực hiện chọn đặc trưng tự động (feature selection). Nghiệm thưa (sparse). Phù hợp khi nhiều đặc trưng không liên quan.

Câu 5. Neural Network — Kích thước

5.1 Tầng 1 ($n_0 = 4 \rightarrow n_1 = 5$)

- $\mathbf{W}^{[1]} \in \mathbb{R}^{5 \times 4}$
- $\mathbf{b}^{[1]} \in \mathbb{R}^{5 \times 1}$
- $\mathbf{Z}^{[1]} \in \mathbb{R}^{5 \times 10}$
- $\mathbf{A}^{[1]} \in \mathbb{R}^{5 \times 10}$

5.2 Tầng 2 ($n_1 = 5 \rightarrow n_2 = 3$)

- $\mathbf{W}^{[2]} \in \mathbb{R}^{3 \times 5}$
- $\mathbf{b}^{[2]} \in \mathbb{R}^{3 \times 1}$
- $\mathbf{Z}^{[2]} \in \mathbb{R}^{3 \times 10}$
- $\mathbf{A}^{[2]} \in \mathbb{R}^{3 \times 10}$

5.3 Tầng 3 ($n_2 = 3 \rightarrow n_3 = 2$)

- $\mathbf{W}^{[3]} \in \mathbb{R}^{2 \times 3}$
- $\mathbf{b}^{[3]} \in \mathbb{R}^{2 \times 1}$
- $\mathbf{Z}^{[3]} \in \mathbb{R}^{2 \times 10}$
- $\mathbf{A}^{[3]} \in \mathbb{R}^{2 \times 10}$

5.4 Tổng tham số

$$\text{Weights} = 5 \times 4 + 3 \times 5 + 2 \times 3 = 20 + 15 + 6 = 41$$

$$\text{Biases} = 5 + 3 + 2 = 10$$

$$\text{Tổng} = 41 + 10 = 51$$

5.5

$$d\mathbf{W}^{[2]} = \frac{1}{m} d\mathbf{Z}^{[2]} (\mathbf{A}^{[1]})^T$$

Kích thước: $(3 \times 10)(10 \times 5) = 3 \times 5 =$ kích thước $\mathbf{W}^{[2]}$

5.6 ReLU vs Sigmoid

- **Sigmoid:** Đạo hàm $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq 0.25$. Qua nhiều tầng, gradient nhân chuỗi nhiều số $< 0.25 \rightarrow$ tiệm cận 0 $\rightarrow$ **vanishing gradient**.
 - **ReLU:** $g'(z) = 1$ khi $z > 0$, gradient không bị thu nhỏ $\rightarrow$ truyền ngược hiệu quả qua nhiều tầng.
-

Câu 6. K-means — Chứng minh Tâm Cụm

6.1 Khai triển

$$\begin{aligned} f(\mu) &= \sum_{i=1}^m \|x_i - \mu\|_2^2 = \sum_{i=1}^m (x_i - \mu)^T (x_i - \mu) \\ &= \sum_{i=1}^m (\|x_i\|_2^2 - 2\mu^T x_i + \|\mu\|_2^2) \\ &= \sum_{i=1}^m \|x_i\|_2^2 - 2\mu^T \sum_{i=1}^m x_i + m\|\mu\|_2^2 \quad \blacksquare \end{aligned}$$

6.2 Gradient

$$\nabla_{\mu} f = -2 \sum_{i=1}^m x_i + 2m\mu = \mathbf{0}$$

6.3 Nghiệm

$$2m\mu = 2 \sum_{i=1}^m x_i \implies \mu^* = \frac{1}{m} \sum_{i=1}^m x_i \quad \blacksquare$$

6.4 Áp dụng

$$\mu^* = \frac{1}{4}[(1, 3) + (2, 5) + (4, 1) + (5, 3)] = \frac{(12, 12)}{4} = (3, 3)$$

Distortion:

$$|(1, 3) - (3, 3)|^2 = 4 + 0 = 4$$

$$|(2, 5) - (3, 3)|^2 = 1 + 4 = 5$$

$$|(4, 1) - (3, 3)|^2 = 1 + 4 = 5$$

$$|(5, 3) - (3, 3)|^2 = 4 + 0 = 4$$

$$f(\mu) = (4 + 5 + 5 + 4) = 18$$

Câu 7. Thử thách: Newton's Method cho Logistic Regression

7.1 Gradient

Đặt:

$$z_i = \mathbf{x}_i^T \beta, \quad p_i = \sigma(z_i).$$

Với một mẫu:

$$J_i = -y_i \log p_i - (1 - y_i) \log(1 - p_i).$$

Ta có:

$$\frac{\partial J_i}{\partial p_i} = -\frac{y_i}{p_i} + \frac{1 - y_i}{1 - p_i}, \quad \frac{\partial p_i}{\partial z_i} = p_i(1 - p_i).$$

Nhân lại:

$$\frac{\partial J_i}{\partial z_i} = \left(-\frac{y_i}{p_i} + \frac{1 - y_i}{1 - p_i} \right) p_i(1 - p_i) = p_i - y_i.$$

Vì $z_i = \mathbf{x}_i^T \beta$:

$$\nabla_{\beta} J_i = (p_i - y_i) \mathbf{x}_i.$$

Cộng toàn bộ mẫu:

$$\nabla J(\beta) = \mathbf{X}^T (\mathbf{p} - \mathbf{y}).$$

7.2 Hessian

Gradient là:

$$\nabla J = \sum_{i=1}^m (p_i - y_i) \mathbf{x}_i.$$

Vì:

$$\frac{\partial p_i}{\partial \beta} = p_i(1 - p_i) \mathbf{x}_i,$$

nên:

$$\nabla^2 J = \sum_{i=1}^m p_i(1 - p_i) \mathbf{x}_i \mathbf{x}_i^T.$$

Dạng ma trận:

$$\nabla^2 J = \mathbf{X}^T \mathbf{R} \mathbf{X},$$

trong đó:

$$\mathbf{R} = \text{diag}(p_1(1 - p_1), \dots, p_m(1 - p_m)).$$

7.3 Tính lồi

Với mọi vector $\mathbf{v}$:

$$\mathbf{v}^T \nabla^2 J \mathbf{v} = \mathbf{v}^T \mathbf{X}^T \mathbf{R} \mathbf{X} \mathbf{v} = (\mathbf{X} \mathbf{v})^T \mathbf{R} (\mathbf{X} \mathbf{v}).$$

Vì $p_i(1 - p_i) \geq 0$, $\mathbf{R}$ là positive semidefinite. Do đó:

$$\mathbf{v}^T \nabla^2 J \mathbf{v} \geq 0.$$

Suy ra J là hàm lồi.

7.4 Cập nhật Newton

Newton's Method dùng:

$$\beta_{new} = \beta - (\nabla^2 J)^{-1} \nabla J.$$

Thay gradient và Hessian:

$$\beta_{new} = \beta - (\mathbf{X}^T \mathbf{R} \mathbf{X})^{-1} \mathbf{X}^T (\mathbf{p} - \mathbf{y}).$$

7.5 So sánh với Gradient Descent

Gradient Descent dùng:

$$\beta_{new} = \beta - \alpha \mathbf{X}^T (\mathbf{p} - \mathbf{y}).$$

Nó rẻ hơn mỗi bước nhưng phụ thuộc learning rate α và thường cần nhiều vòng lặp. Newton dùng thông tin độ cong nên thường hội tụ rất nhanh gần nghiệm tối ưu, nhưng phải tạo và giải hệ tuyến tính với Hessian $p \times p$. Chi phí có thể lên tới $\mathcal{O}(p^3)$ nếu nghịch đảo trực tiếp, nên khó dùng khi số đặc trưng p rất lớn.

HẾT ĐÁP ÁN

Exam Appendix 9: CS114 - Exam_04

Source: `exam/exam_04/CS114 - Exam_04.md`

ĐỀ THI MACHINE LEARNING — ĐỀ 04

- **Thời gian làm bài:** 90 phút
- **Hình thức:** Tự luận + tính toán
- **Ghi chú:** Sinh viên được sử dụng máy tính cầm tay. Trình bày rõ công thức, các bước biến đổi và kết luận.

Câu 1. Logistic Regression — Chứng minh Tính Lỗi

Cho hàm mất mát Binary Cross-Entropy của Logistic Regression:

$$J(\beta) = - \sum_{i=1}^n \left[y^{(i)} \log \sigma(\beta^T \mathbf{x}^{(i)}) + (1 - y^{(i)}) \log(1 - \sigma(\beta^T \mathbf{x}^{(i)})) \right]$$

Hãy:

1. Tính gradient $\nabla J(\beta)$ bằng chain rule (trình bày 3 thành phần: Loss $\rightarrow$ Sigmoid $\rightarrow$ Tuyến tính).
2. Tính ma trận Hessian $H = \nabla^2 J(\beta)$.
3. Chứng minh $H \succeq 0$ (bán xác định dương), từ đó suy ra J là hàm lồi.
4. Giải thích hệ quả thực tiễn: Logistic Regression không có cực tiểu cục bộ.

Câu 2. OLS bằng Ma trận — Bộ dữ liệu 2 biến

Cho dữ liệu với mô hình $\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2$:

STT	x_1	x_2	y
1	1	2	10
2	2	1	12
3	3	3	18
4	4	2	20
5	5	4	28

Hãy:

1. Xây dựng $\mathbf{X}$ (có cột 1) và $\mathbf{y}$.
2. Tính $\mathbf{X}^T \mathbf{X}$ và $\mathbf{X}^T \mathbf{y}$.
3. Giải normal equations (có thể dùng elimination hoặc máy tính).
4. Dự đoán $\hat{y}$ khi $x_1 = 3, x_2 = 5$.

Câu 3. GDA — Chứng minh Ranh giới Tuyến tính

Trong Gaussian Discriminant Analysis với $\mathbf{x} \mid y = 0 \sim \mathcal{N}(\mu_0, \Sigma)$ và $\mathbf{x} \mid y = 1 \sim \mathcal{N}(\mu_1, \Sigma)$ (ma trận hiệp phương sai chung).

Hãy:

1. Viết $P(y = 1 \mid \mathbf{x})$ theo Định lý Bayes.
2. Chứng minh rằng $P(y = 1 \mid \mathbf{x}) = \sigma(\theta^T \mathbf{x} + \theta_0)$ bằng cách:
 - Tính log-ratio $\log \frac{P(\mathbf{x} \mid y=1)}{P(\mathbf{x} \mid y=0)}$.
 - Chỉ ra số hạng bậc hai $\mathbf{x}^T \Sigma^{-1} \mathbf{x}$ bị triệt tiêu nhờ Σ chung.
 - Xác định θ và θ_0 .
3. Giải thích vì sao nếu $\Sigma_0 \neq \Sigma_1$, ranh giới quyết định sẽ là bậc hai (quadratic).

Câu 4. Decision Tree — Regression Tree với MSE

Cho bảng dữ liệu hồi quy:

STT	x	y
1	1	2.5
2	2	3.0
3	3	4.5
4	4	4.0
5	5	8.0
6	6	7.5
7	7	9.0
8	8	10.0

Hãy:

1. Tính MSE khi không chia (dùng toàn bộ dữ liệu làm 1 lá, dự đoán $= \bar{y}$).
2. Xét split $x \leq 4$. Tính $\bar{y}_{left}$, $\bar{y}_{right}$, SSE trái, SSE phải, tổng SSE.
3. Xét split $x \leq 3$. Tính tương tự.
4. Chọn split tốt hơn theo tiêu chí SSE nhỏ hơn.

Câu 5. Backpropagation qua Mạng 2-3-1

Cho mạng gồm:

- 2 input: $x_1 = 1, x_2 = -1$
- 1 hidden layer gồm 3 neuron, dùng sigmoid
- 1 output neuron, dùng sigmoid
- Loss: Binary cross-entropy $L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$
- Nhân thật: $y = 1$

Trọng số tầng ẩn:

$$\mathbf{W}^{[1]} = \begin{pmatrix} 0.2 & -0.3 \\ 0.4 & 0.1 \\ -0.1 & 0.5 \end{pmatrix}, \quad \mathbf{b}^{[1]} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

Trọng số tầng output:

$$\mathbf{W}^{[2]} = \begin{pmatrix} 0.5 & -0.3 & 0.4 \end{pmatrix}, \quad b^{[2]} = 0$$

Hãy thực hiện:

1. Forward pass: tính $\mathbf{Z}^{[1]}$, $\mathbf{A}^{[1]}$, $Z^{[2]}$, $\hat{y}$.
2. Tính loss.
3. Tính $dZ^{[2]}$ (biết sigmoid + BCE cho $dZ = \hat{y} - y$).
4. Tính $dW^{[2]}$ và $db^{[2]}$.

Câu 6. Gradient Descent với Regularization

Cho mô hình tuyến tính $\hat{y} = \beta_0 + \beta_1 x$ với Ridge regularization. Bắt đầu từ $\beta_0 = 0$, $\beta_1 = 0$.

Dữ liệu:

x	y
1	3
2	5
3	7
4	9

Hãy:

1. Tính gradient của loss OLS (không có regularization) tại $\beta = (0, 0)$.
2. Tính gradient của Ridge penalty $\lambda \|\beta\|_2^2$ tại $\beta = (0, 0)$ với $\lambda = 0.1$.
3. Tính gradient tổng và cập nhật β sau một bước với $\alpha = 0.1$.
4. So sánh β mới với trường hợp không regularization. Nhận xét.

Câu 7. Thử thách: Batch Normalization Forward và Backward

Cho một mini-batch gồm 4 giá trị tiền kích hoạt của một neuron:

$$z = (1, 3, 5, 7), \quad \epsilon = 0, \quad \gamma = 2, \quad \beta = -1.$$

Batch Normalization được định nghĩa bởi:

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^m z_i, \quad \sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_{i=1}^m (z_i - \mu_{\mathcal{B}})^2,$$

$$\hat{z}_i = \frac{z_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \quad \tilde{z}_i = \gamma \hat{z}_i + \beta.$$

Hãy:

1. Tính $\mu_{\mathcal{B}}$, $\sigma_{\mathcal{B}}^2$, toàn bộ $\hat{z}_i$ và $\tilde{z}_i$.
2. Giả sử gradient đi từ tầng sau là $\frac{\partial L}{\partial \tilde{z}} = (1, -1, 2, -2)$. Tính:

$$\frac{\partial L}{\partial \gamma} = \sum_{i=1}^m \frac{\partial L}{\partial \tilde{z}_i} \hat{z}_i, \quad \frac{\partial L}{\partial \beta} = \sum_{i=1}^m \frac{\partial L}{\partial \tilde{z}_i}.$$

3. Chứng minh công thức ngắn gọn sau cho gradient đi về z_i :

$$\frac{\partial L}{\partial z_i} = \frac{\gamma}{m\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \left[mg_i - \sum_{j=1}^m g_j - \hat{z}_i \sum_{j=1}^m g_j \hat{z}_j \right],$$

trong đó $g_i = \frac{\partial L}{\partial \tilde{z}_i}$. 4. Tính $\frac{\partial L}{\partial z_i}$ cho cả 4 phần tử. 5. Giải thích tại sao kết quả gradient theo z có tổng bằng 0 trong mini-batch này.

HẾT

Exam Appendix 10: CS114 - Exam_04_Answer

Source: `exam/exam_04/CS114 - Exam_04_Answer.md`

ĐÁP ÁN ĐỀ THI MACHINE LEARNING — ĐỀ 04

Câu 1. Logistic Regression — Chứng minh Tính Lỗi

1.1 Gradient

Đặt $z^{(i)} = \beta^T \mathbf{x}^{(i)}$, $\hat{y}^{(i)} = \sigma(z^{(i)})$.

Áp dụng chain rule cho từng mẫu:

$$\frac{\partial J^{(i)}}{\partial \beta} = \frac{\partial J^{(i)}}{\partial \hat{y}^{(i)}} \cdot \frac{\partial \hat{y}^{(i)}}{\partial z^{(i)}} \cdot \frac{\partial z^{(i)}}{\partial \beta}$$

Thành phần Loss:

$$\frac{\partial J^{(i)}}{\partial \hat{y}^{(i)}} = -\frac{y^{(i)}}{\hat{y}^{(i)}} + \frac{1 - y^{(i)}}{1 - \hat{y}^{(i)}} = \frac{\hat{y}^{(i)} - y^{(i)}}{\hat{y}^{(i)}(1 - \hat{y}^{(i)})}$$

Thành phần Sigmoid:

$$\frac{\partial \hat{y}^{(i)}}{\partial z^{(i)}} = \hat{y}^{(i)}(1 - \hat{y}^{(i)})$$

Thành phần Tuyến tính:

$$\frac{\partial z^{(i)}}{\partial \beta} = \mathbf{x}^{(i)}$$

Nhân lại, mẫu số triệt tiêu:

$$\frac{\partial J^{(i)}}{\partial \beta} = (\hat{y}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}$$

$$\nabla J = \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}$$

1.2 Hessian

$$H = \frac{\partial}{\partial \beta} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}$$

Vì $y^{(i)}$ là hằng số và $\frac{\partial \hat{y}^{(i)}}{\partial \beta} = \hat{y}^{(i)}(1 - \hat{y}^{(i)}) \mathbf{x}^{(i)}$:

$$H = \sum_{i=1}^n \hat{y}^{(i)}(1 - \hat{y}^{(i)}) \mathbf{x}^{(i)} (\mathbf{x}^{(i)})^T$$

1.3 Chứng minh $H \succeq 0$

Với vector $\mathbf{v}$ bất kỳ:

$$\begin{aligned} \mathbf{v}^T H \mathbf{v} &= \sum_{i=1}^n \hat{y}^{(i)}(1 - \hat{y}^{(i)}) \cdot \mathbf{v}^T \mathbf{x}^{(i)} (\mathbf{x}^{(i)})^T \mathbf{v} \\ &= \sum_{i=1}^n \hat{y}^{(i)}(1 - \hat{y}^{(i)}) \cdot (\mathbf{v}^T \mathbf{x}^{(i)})^2 \end{aligned}$$

- $\hat{y}^{(i)} \in (0, 1) \rightarrow \hat{y}^{(i)}(1 - \hat{y}^{(i)}) > 0$
- $(\mathbf{v}^T \mathbf{x}^{(i)})^2 \geq 0$

Do đó $\mathbf{v}^T H \mathbf{v} \geq 0$ với mọi $\mathbf{v} \rightarrow H \succeq 0 \rightarrow J$ lồi ■

1.4 Hệ quả

Vì J lồi, mọi cực tiểu cục bộ cũng là cực tiểu toàn cục. Gradient descent và Newton's method đều đảm bảo hội tụ đến nghiệm tối ưu duy nhất.

Câu 2. OLS 2 biến

2.1

$$\mathbf{X} = \begin{pmatrix} 1 & 1 & 2 \\ 1 & 2 & 1 \\ 1 & 3 & 3 \\ 1 & 4 & 2 \\ 1 & 5 & 4 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 10 \\ 12 \\ 18 \\ 20 \\ 28 \end{pmatrix}$$

2.2

$$\mathbf{X}^T \mathbf{X} = \begin{pmatrix} 5 & 15 & 12 \\ 15 & 55 & 40 \\ 12 & 40 & 34 \end{pmatrix}$$

Chi tiết: $-\sum 1 = 5$, $\sum x_1 = 15$, $\sum x_2 = 12$ - $\sum x_1^2 = 1 + 4 + 9 + 16 + 25 = 55$ - $\sum x_2^2 = 4 + 1 + 9 + 4 + 16 = 34$ - $\sum x_1 x_2 = 2 + 2 + 9 + 8 + 20 = 41$

Sửa: $\sum x_1 x_2 = 1(2) + 2(1) + 3(3) + 4(2) + 5(4) = 2 + 2 + 9 + 8 + 20 = 41$

$$\mathbf{X}^T \mathbf{X} = \begin{pmatrix} 5 & 15 & 12 \\ 15 & 55 & 41 \\ 12 & 41 & 34 \end{pmatrix}$$

$$\mathbf{X}^T \mathbf{y} = \begin{pmatrix} 88 \\ 318 \\ 242 \end{pmatrix}$$

Chi tiết: $\sum y = 88$, $\sum x_1 y = 10 + 24 + 54 + 80 + 140 = 308$

Sửa: $\sum x_1 y = 1(10) + 2(12) + 3(18) + 4(20) + 5(28) = 10 + 24 + 54 + 80 + 140 = 308$

$$\mathbf{X}^T \mathbf{y} = \begin{pmatrix} 88 \\ 308 \\ 242 \end{pmatrix}$$

$\sum x_2 y = 2(10) + 1(12) + 3(18) + 2(20) + 4(28) = 20 + 12 + 54 + 40 + 112 = 238$

$$\mathbf{X}^T \mathbf{y} = \begin{pmatrix} 88 \\ 308 \\ 238 \end{pmatrix}$$

2.3 Giải normal equations

Giải hệ (bằng máy tính hoặc elimination):

$$\begin{pmatrix} 5 & 15 & 12 \\ 15 & 55 & 41 \\ 12 & 41 & 34 \end{pmatrix} \beta = \begin{pmatrix} 88 \\ 308 \\ 238 \end{pmatrix}$$

Giải ra: $\beta_0 \approx 2.073$, $\beta_1 \approx 3.219$, $\beta_2 \approx 1.461$

Mô hình: $\hat{y} = 2.073 + 3.219x_1 + 1.461x_2$

2.4

$$\hat{y}(3, 5) = 2.073 + 3.219(3) + 1.461(5) = 2.073 + 9.657 + 7.305 = 19.035$$

Câu 3. GDA — Ranh giới Tuyến tính

3.1

$$\begin{aligned} P(y = 1 \mid \mathbf{x}) &= \frac{P(\mathbf{x} \mid y = 1)P(y = 1)}{P(\mathbf{x} \mid y = 1)P(y = 1) + P(\mathbf{x} \mid y = 0)P(y = 0)} \\ &= \frac{1}{1 + \frac{P(\mathbf{x} \mid y = 0)P(y = 0)}{P(\mathbf{x} \mid y = 1)P(y = 1)}} \end{aligned}$$

3.2

$$\log \frac{P(\mathbf{x} \mid y = 1)}{P(\mathbf{x} \mid y = 0)} = -\frac{1}{2}(\mathbf{x} - \mu_1)^T \Sigma^{-1}(\mathbf{x} - \mu_1) + \frac{1}{2}(\mathbf{x} - \mu_0)^T \Sigma^{-1}(\mathbf{x} - \mu_0)$$

Khai triển:

$$= -\frac{1}{2}\mathbf{x}^T \Sigma^{-1} \mathbf{x} + \mu_1^T \Sigma^{-1} \mathbf{x} - \frac{1}{2}\mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2}\mathbf{x}^T \Sigma^{-1} \mathbf{x} - \mu_0^T \Sigma^{-1} \mathbf{x} + \frac{1}{2}\mu_0^T \Sigma^{-1} \mu_0$$

Số hạng $\mathbf{x}^T \Sigma^{-1} \mathbf{x}$ triệt tiêu nhờ Σ chung:

$$= (\mu_1 - \mu_0)^T \Sigma^{-1} \mathbf{x} - \frac{1}{2}\mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2}\mu_0^T \Sigma^{-1} \mu_0$$

Đặt $z = \theta^T \mathbf{x} + \theta_0$ với:

$$\theta = \Sigma^{-1}(\mu_1 - \mu_0)$$

$$\theta_0 = -\frac{1}{2}\mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2}\mu_0^T \Sigma^{-1} \mu_0 + \log \frac{\phi}{1 - \phi}$$

$$\therefore P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-z}} = \sigma(\theta^T \mathbf{x} + \theta_0) \quad \blacksquare$$

3.3

Nếu $\Sigma_0 \neq \Sigma_1$, các số hạng $\mathbf{x}^T \Sigma_1^{-1} \mathbf{x}$ và $\mathbf{x}^T \Sigma_0^{-1} \mathbf{x}$ không triệt tiêu nhau. Log-ratio chứa số hạng bậc hai theo $\mathbf{x}$, nên ranh giới quyết định là một mặt bậc hai (conic section: ellipse, parabola, hoặc hyperbola). Mô hình này gọi là **Quadratic Discriminant Analysis (QDA)**.

Câu 4. Regression Tree

4.1 Không chia

$$\bar{y} = \frac{2.5 + 3.0 + 4.5 + 4.0 + 8.0 + 7.5 + 9.0 + 10.0}{8} = \frac{48.5}{8} = 6.0625$$

$$\text{SSE} = (2.5 - 6.0625)^2 + (3 - 6.0625)^2 + \dots + (10 - 6.0625)^2$$

$$= 12.691 + 9.378 + 2.441 + 4.253 + 3.753 + 2.066 + 8.628 + 15.503 = 58.713$$

Tính chi tiết:

$$= (-3.5625)^2 + (-3.0625)^2 + (-1.5625)^2 + (-2.0625)^2 + (1.9375)^2 + (1.4375)^2 + (2.9375)^2 + (3.9375)^2$$

$$= 12.691 + 9.379 + 2.441 + 4.254 + 3.754 + 2.066 + 8.629 + 15.504 = 58.718$$

$$\text{MSE} = \frac{58.718}{8} = 7.340$$

4.2 Split $x \leq 4$

Trái ($x \leq 4$): $\{2.5, 3.0, 4.5, 4.0\}$

$$\bar{y}_L = \frac{2.5 + 3.0 + 4.5 + 4.0}{4} = 3.5$$

$$\text{SSE}_L = (2.5 - 3.5)^2 + (3 - 3.5)^2 + (4.5 - 3.5)^2 + (4 - 3.5)^2 = 1 + 0.25 + 1 + 0.25 = 2.5$$

Phải ($x > 4$): $\{8.0, 7.5, 9.0, 10.0\}$

$$\bar{y}_R = \frac{8 + 7.5 + 9 + 10}{4} = 8.625$$

$$\text{SSE}_R = (8 - 8.625)^2 + (7.5 - 8.625)^2 + (9 - 8.625)^2 + (10 - 8.625)^2 = 0.391 + 1.266 + 0.141 + 1.891 = 3.688$$

$$\text{Tổng SSE} = 2.5 + 3.688 = 6.188$$

4.3 Split $x \leq 3$

Trái: $\{2.5, 3.0, 4.5\}$, $\bar{y}_L = \frac{10}{3} = 3.333$

$$\text{SSE}_L = (2.5 - 3.333)^2 + (3 - 3.333)^2 + (4.5 - 3.333)^2 = 0.694 + 0.111 + 1.361 = 2.167$$

Phải: $\{4.0, 8.0, 7.5, 9.0, 10.0\}$, $\bar{y}_R = \frac{38.5}{5} = 7.7$

$$\text{SSE}_R = (4 - 7.7)^2 + (8 - 7.7)^2 + (7.5 - 7.7)^2 + (9 - 7.7)^2 + (10 - 7.7)^2$$

$$= 13.69 + 0.09 + 0.04 + 1.69 + 5.29 = 20.8$$

$$\text{Tổng SSE} = 2.167 + 20.8 = 22.967$$

4.4 Kết luận

Split	Tổng SSE
$x \leq 4$	6.188
$x \leq 3$	22.967

Chọn split $x \leq 4$ vì SSE nhỏ hơn rất nhiều.

Câu 5. Backpropagation 2-3-1

5.1 Forward Pass

$$\mathbf{Z}^{[1]} = \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]} = \begin{pmatrix} 0.2(1) + (-0.3)(-1) \\ 0.4(1) + 0.1(-1) \\ -0.1(1) + 0.5(-1) \end{pmatrix} = \begin{pmatrix} 0.5 \\ 0.3 \\ -0.6 \end{pmatrix}$$

$$\mathbf{A}^{[1]} = \sigma(\mathbf{Z}^{[1]}) = \begin{pmatrix} \sigma(0.5) \\ \sigma(0.3) \\ \sigma(-0.6) \end{pmatrix} \approx \begin{pmatrix} 0.6225 \\ 0.5744 \\ 0.3543 \end{pmatrix}$$

$$Z^{[2]} = 0.5(0.6225) + (-0.3)(0.5744) + 0.4(0.3543) + 0 = 0.3113 - 0.1723 + 0.1417 = 0.2807$$

$$\hat{y} = \sigma(0.2807) \approx 0.5697$$

5.2 Loss

$$L = -[1 \cdot \log(0.5697) + 0] = -\log(0.5697) \approx 0.5628$$

5.3

$$dZ^{[2]} = \hat{y} - y = 0.5697 - 1 = -0.4303$$

5.4

$$\begin{aligned}dW^{[2]} &= dZ^{[2]} \cdot (\mathbf{A}^{[1]})^T = -0.4303 \begin{pmatrix} 0.6225 & 0.5744 & 0.3543 \end{pmatrix} \\&= \begin{pmatrix} -0.2679 & -0.2473 & -0.1525 \end{pmatrix} \\db^{[2]} &= dZ^{[2]} = -0.4303\end{aligned}$$

Câu 6. Gradient Descent với Ridge

6.1 Gradient OLS tại $\beta = (0, 0)$

Mọi dự đoán = 0, residual = y_i . Gradient dạng “cộng”:

$$\begin{aligned}\frac{\partial L}{\partial \beta_0} &= -\frac{1}{4} \sum r_i = -\frac{1}{4}(3 + 5 + 7 + 9) = -\frac{24}{4} = -6 \\ \frac{\partial L}{\partial \beta_1} &= -\frac{1}{4} \sum r_i x_i = -\frac{1}{4}(3 + 10 + 21 + 36) = -\frac{70}{4} = -17.5\end{aligned}$$

6.2 Gradient Ridge penalty

$$\nabla(\lambda \|\beta\|_2^2) = 2\lambda\beta = 2(0.1) \begin{pmatrix} 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

(Tại $\beta = 0$, penalty gradient = 0.)

6.3 Cập nhật

Gradient tổng = $(-6, -17.5) + (0, 0) = (-6, -17.5)$

$$\beta_{new} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} - 0.1 \begin{pmatrix} -6 \\ -17.5 \end{pmatrix} = \begin{pmatrix} 0.6 \\ 1.75 \end{pmatrix}$$

6.4 Nhận xét

Tại $\beta = (0, 0)$, Ridge penalty không ảnh hưởng vì gradient penalty = 0 ở gốc tọa độ. Sự khác biệt xuất hiện ở các bước sau: khi β xa 0, penalty đẩy hệ số về phía 0, co nhỏ chúng so với OLS thuần.

Câu 7. Thử thách: Batch Normalization Forward và Backward

7.1 Forward pass

Với $z = (1, 3, 5, 7)$:

$$\mu_{\mathcal{B}} = \frac{1 + 3 + 5 + 7}{4} = 4.$$

$$\sigma_{\mathcal{B}}^2 = \frac{(1-4)^2 + (3-4)^2 + (5-4)^2 + (7-4)^2}{4} = \frac{9 + 1 + 1 + 9}{4} = 5.$$

Vì $\epsilon = 0$:

$$\sqrt{\sigma_{\mathcal{B}}^2} = \sqrt{5}.$$

Do đó:

$$\hat{z} = \left(\frac{-3}{\sqrt{5}}, \frac{-1}{\sqrt{5}}, \frac{1}{\sqrt{5}}, \frac{3}{\sqrt{5}} \right) \approx (-1.342, -0.447, 0.447, 1.342).$$

Với $\gamma = 2, \beta = -1$:

$$\tilde{z} = 2\hat{z} - 1 \approx (-3.683, -1.894, -0.106, 1.683).$$

7.2 Gradient theo γ và β

Cho:

$$g = \frac{\partial L}{\partial \tilde{z}} = (1, -1, 2, -2).$$

Ta có:

$$\frac{\partial L}{\partial \beta} = \sum_i g_i = 1 - 1 + 2 - 2 = 0.$$

$$\frac{\partial L}{\partial \gamma} = \sum_i g_i \hat{z}_i = 1 \left(\frac{-3}{\sqrt{5}} \right) (-1) \left(\frac{-1}{\sqrt{5}} \right) 2 \left(\frac{1}{\sqrt{5}} \right) (-2) \left(\frac{3}{\sqrt{5}} \right).$$

$$\frac{\partial L}{\partial \gamma} = \frac{-3 + 1 + 2 - 6}{\sqrt{5}} = \frac{-6}{\sqrt{5}} \approx -2.683.$$

7.3 Công thức backward

BatchNorm có ba bước: trừ mean, chia độ lệch chuẩn, rồi scale-shift. Khi gom các đạo hàm trung gian, gradient về từng z_i có thể viết:

$$\frac{\partial L}{\partial z_i} = \frac{\gamma}{m\sqrt{\sigma_{\mathcal{B}}^2} + \epsilon} \left[mg_i - \sum_{j=1}^m g_j - \hat{z}_i \sum_{j=1}^m g_j \hat{z}_j \right].$$

Công thức này có ba thành phần trực giác:

- mg_i : gradient trực tiếp của phần tử i .
- $-\sum_j g_j$: hiệu chỉnh vì mean phụ thuộc vào tất cả phần tử.
- $-\hat{z}_i \sum_j g_j \hat{z}_j$: hiệu chỉnh vì variance cũng phụ thuộc vào tất cả phần tử.

7.4 Tính gradient về z

Ta đã có:

$$\sum_j g_j = 0, \quad \sum_j g_j \hat{z}_j = \frac{-6}{\sqrt{5}}.$$

Vì $m = 4$, $\gamma = 2$, $\sqrt{\sigma_{\mathcal{B}}^2} = \sqrt{5}$:

$$\frac{\gamma}{m\sqrt{\sigma_{\mathcal{B}}^2}} = \frac{2}{4\sqrt{5}} = \frac{1}{2\sqrt{5}}.$$

Do $\sum_j g_j = 0$:

$$\frac{\partial L}{\partial z_i} = \frac{1}{2\sqrt{5}} \left[4g_i - \hat{z}_i \left(\frac{-6}{\sqrt{5}} \right) \right].$$

Tính từng phần:

$$i = 1 : \quad \hat{z}_1 = \frac{-3}{\sqrt{5}}, \quad 4g_1 - \hat{z}_1 \frac{-6}{\sqrt{5}} = 4 - \frac{18}{5} = 0.4.$$

$$\frac{\partial L}{\partial z_1} = \frac{0.4}{2\sqrt{5}} \approx 0.0894.$$

$$i = 2 : \quad \hat{z}_2 = \frac{-1}{\sqrt{5}}, \quad 4g_2 - \hat{z}_2 \frac{-6}{\sqrt{5}} = -4 - \frac{6}{5} = -5.2.$$

$$\frac{\partial L}{\partial z_2} = \frac{-5.2}{2\sqrt{5}} \approx -1.1628.$$

$$i = 3 : \quad \hat{z}_3 = \frac{1}{\sqrt{5}}, \quad 4g_3 - \hat{z}_3 \frac{-6}{\sqrt{5}} = 8 + \frac{6}{5} = 9.2.$$

$$\frac{\partial L}{\partial z_3} = \frac{9.2}{2\sqrt{5}} \approx 2.0572.$$

$$i = 4 : \quad \hat{z}_4 = \frac{3}{\sqrt{5}}, \quad 4g_4 - \hat{z}_4 \frac{-6}{\sqrt{5}} = -8 + \frac{18}{5} = -4.4.$$

$$\frac{\partial L}{\partial z_4} = \frac{-4.4}{2\sqrt{5}} \approx -0.9839.$$

7.5 Vì sao tổng gradient bằng 0?

Tổng gradient:

$$0.0894 - 1.1628 + 2.0572 - 0.9839 \approx 0.$$

Điều này xảy ra vì BatchNorm trừ mean của mini-batch. Nếu cộng cùng một hằng số vào mọi z_i , các giá trị chuẩn hóa $\hat{z}_i$ không đổi. Do đó loss không nhạy với dịch chuyển đồng đều của toàn batch, nên gradient theo hướng $(1, 1, 1, 1)$ phải bằng 0.

HẾT ĐÁP ÁN

Exam Appendix 11: CS114 - Exam_05

Source: `exam/exam_05/CS114 - Exam_05.md`

ĐỀ THI MACHINE LEARNING — ĐỀ 05

- **Thời gian làm bài:** 90 phút
- **Hình thức:** Tự luận + tính toán
- **Ghi chú:** Sinh viên được sử dụng máy tính cầm tay. Trình bày rõ công thức, các bước biến đổi và kết luận.

Câu 1. Cross-Entropy Loss và Logistic Regression Hoàn chỉnh

Cho 6 mẫu dữ liệu huấn luyện và mô hình Logistic Regression ban đầu với $\beta = (-0.5, 1.0, -0.8)$ (bias, β_1 , β_2):

STT	x_1	x_2	y
1	2	1	1
2	0	2	0
3	1	0	1
4	3	2	1
5	-1	1	0
6	1	3	0

Hãy:

1. Tính z_i , $\hat{p}_i = \sigma(z_i)$, lớp dự đoán, cross-entropy loss cho từng mẫu.
2. Tính cross-entropy loss trung bình và accuracy.
3. Tính gradient $\nabla J = \frac{1}{n} \mathbf{X}^T (\hat{\mathbf{p}} - \mathbf{y})$.
4. Cập nhật β sau một bước gradient descent với $\alpha = 0.5$.
5. Nhận xét β mới so với cũ.

Câu 2. Naive Bayes — So sánh Có và Không có Laplace Smoothing

Cho bảng phân loại tin nhắn rác:

STT	Từ “giảm giá”	Từ “miễn phí”	Từ “hộp”	Từ “dự án”	Nhãn
1	Có	Có	Không	Không	Rác
2	Có	Không	Không	Không	Rác
3	Không	Có	Không	Không	Rác
4	Không	Không	Có	Có	Bình thường
5	Không	Không	Có	Không	Bình thường
6	Không	Không	Không	Có	Bình thường

Tin nhắn mới: “giảm giá” = Không, “miễn phí” = Có, “hộp” = Không, “dự án” = Không.

Hãy:

1. **Không dùng** Laplace smoothing: tính $P(\text{Rác} \mid \mathbf{x})$. Nêu vấn đề gặp phải.
2. **Dùng** Laplace smoothing ($\alpha = 1$): tính lại $P(\text{Rác} \mid \mathbf{x})$ và $P(\text{Bình thường} \mid \mathbf{x})$.
3. Kết luận và giải thích tại sao Laplace smoothing quan trọng.

Câu 3. Decision Tree — So sánh Entropy và Gini

Cho bảng dữ liệu nhị phân đơn giản:

STT	A	B	Nhãn
1	0	0	-
2	0	1	-
3	0	0	-
4	1	0	+
5	1	1	+
6	1	1	+
7	1	0	+
8	0	1	+

Hãy:

1. Tính Information Gain của thuộc tính A theo **Entropy**.
2. Tính Information Gain của thuộc tính A theo **Gini**.
3. Tính Information Gain của thuộc tính B theo **Entropy**.
4. So sánh và kết luận: A hay B tốt hơn? Hai metric có cho cùng kết luận không?

Câu 4. Neural Network — Dropout và Batch Normalization

Hãy:

1. Cho tầng ẩn có 4 neuron với activation $\mathbf{a} = (0.8, 0.3, 0.6, 0.9)$. Nếu áp dụng Dropout với $p = 0.5$ và mask $\mathbf{M} = (1, 0, 1, 0)$, tính output sau Dropout (inverted dropout).
2. Cho $\mathbf{z} = (2.0, 4.0, 6.0, 8.0)$ trước khi đưa vào activation. Tính Batch Normalization:
 - Tính $\mu_{\mathcal{B}}$ và $\sigma_{\mathcal{B}}^2$.
 - Tính $\hat{z}_i$ (chuẩn hóa) với $\epsilon = 10^{-8}$.
 - Nếu $\gamma = 2$, $\beta = 1$, tính $\tilde{z}_i$.
3. Giải thích vì sao Dropout giống ensemble và vì sao Batch Normalization tăng tốc hội tụ.

Câu 5. Dropout trong Neural Network

Trình bày kỹ thuật Inverted Dropout trong huấn luyện mạng Neural sâu. Tại sao Dropout thường được giải thích là có tác dụng tương tự như việc tạo ra một tập hợp (ensemble) gồm rất nhiều mạng con (sub-networks)?

Câu 6. Chứng minh MLE của Exponential Family là Lỗi

Cho phân phối thuộc Exponential Family:

$$P(y | \eta) = b(y) \exp(\eta T(y) - a(\eta))$$

Hãy:

1. Viết hàm Log-Likelihood $\ell(\eta)$ cho n mẫu i.i.d.
 2. Tính gradient $\nabla_{\eta} \ell$.
 3. Tính Hessian $\nabla_{\eta}^2 \ell$.
 4. Chứng minh Hessian $\preceq 0$ (tức log-likelihood lõm, nên negative log-likelihood lồi).
 5. Nêu ý nghĩa thực tiễn: MLE cho Exponential Family luôn có nghiệm tối ưu toàn cục duy nhất.
-

Câu 7. Thử thách: Vanishing Gradient bằng Quy tắc Chuỗi

Xét một mạng neural sâu dạng vô hướng để đơn giản hóa:

$$a^{[0]} = x, \quad z^{[\ell]} = w_{\ell} a^{[\ell-1]}, \quad a^{[\ell]} = \sigma(z^{[\ell]}), \quad \ell = 1, \dots, L,$$

với loss $L_{oss} = \frac{1}{2}(a^{[L]} - y)^2$ và sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$.

Hãy:

1. Chứng minh $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq \frac{1}{4}$ với mọi z .
2. Dùng quy tắc chuỗi để viết $\left| \frac{\partial L_{oss}}{\partial w_1} \right|$ dưới dạng tích các đạo hàm tầng sau.
3. Giả sử $|w_{\ell}| \leq 1$ với mọi ℓ , chứng minh gradient về tầng đầu bị chặn bởi một hằng số nhân với $\left(\frac{1}{4}\right)^{L-1}$.
4. Với $L = 8$ và $|a^{[L]} - y| \leq 1$, ước lượng cận trên xấp xỉ của phần suy giảm gradient do các sigmoid tầng sau.
5. Giải thích vì sao ReLU, Batch Normalization và residual connection giúp giảm vấn đề này theo các cơ chế khác nhau.

HẾT

Exam Appendix 12: CS114 - Exam_05_Answer

Source: `exam/exam_05/CS114 - Exam_05_Answer.md`

ĐÁP ÁN ĐỀ THI MACHINE LEARNING — ĐỀ 05

Câu 1. Logistic Regression Hoàn chỉnh

$$\beta = (-0.5, 1.0, -0.8), z = -0.5 + 1.0x_1 - 0.8x_2$$

1.1 Bảng tính

STT	z	$\hat{p}$	Lớp	y	Đúng?	Loss
1	$-0.5 + 2 - 0.8 = 0.7$	0.668	1	1		$-\log(0.668) = 0.404$
2	$-0.5 + 0 - 1.6 = -2.1$	0.109	0	0		$-\log(0.891) = 0.115$
3	$-0.5 + 1 - 0 = 0.5$	0.622	1	1		$-\log(0.622) = 0.475$
4	$-0.5 + 3 - 1.6 = 0.9$	0.711	1	1		$-\log(0.711) = 0.342$
5	$-0.5 - 1 - 0.8 = -2.3$	0.091	0	0		$-\log(0.909) = 0.095$
6	$-0.5 + 1 - 2.4 = -1.9$	0.130	0	0		$-\log(0.870) = 0.139$

1.2

$$\bar{J} = \frac{0.404 + 0.115 + 0.475 + 0.342 + 0.095 + 0.139}{6} = \frac{1.570}{6} = 0.262$$

$$\text{Accuracy} = \frac{6}{6} = 1.0 \quad (100\%)$$

STT	$\hat{p} - y$	x_0	x_1	x_2
-----	---------------	-------	-------	-------

1.3 Gradient

Sai số $\hat{p}_i - y_i$:

STT	$\hat{p} - y$	x_0	x_1	x_2
1	-0.332	1	2	1
2	0.109	1	0	2
3	-0.378	1	1	0
4	-0.289	1	3	2
5	0.091	1	-1	1
6	0.130	1	1	3

$$\nabla_{\beta_0} = \frac{1}{6}(-0.332 + 0.109 - 0.378 - 0.289 + 0.091 + 0.130) = \frac{-0.669}{6} = -0.1115$$

$$\nabla_{\beta_1} = \frac{1}{6}[(-0.332)(2) + (0.109)(0) + (-0.378)(1) + (-0.289)(3) + (0.091)(-1) + (0.130)(1)]$$

$$= \frac{1}{6}(-0.664 + 0 - 0.378 - 0.867 - 0.091 + 0.130) = \frac{-1.870}{6} = -0.3117$$

$$\nabla_{\beta_2} = \frac{1}{6}[(-0.332)(1) + (0.109)(2) + (-0.378)(0) + (-0.289)(2) + (0.091)(1) + (0.130)(3)]$$

$$= \frac{1}{6}(-0.332 + 0.218 + 0 - 0.578 + 0.091 + 0.390) = \frac{-0.211}{6} = -0.0352$$

1.4 Cập nhật

$$\beta_{new} = \begin{pmatrix} -0.5 \\ 1.0 - 0.8 \end{pmatrix} - 0.5(-0.1115 - 0.3117 - 0.0352) = \begin{pmatrix} -0.4443 \\ 1.1559 - 0.7824 \end{pmatrix}$$

1.5 Nhận xét

- Bias tăng: mô hình đẩy xác suất lớp 1 lên một chút.
 - β_1 tăng: x_1 lớn tương quan mạnh hơn với $y = 1$.
 - β_2 tăng nhẹ (bớt âm): điều chỉnh nhẹ mối quan hệ âm với x_2 .
 - Accuracy đã 100% nên các thay đổi nhỏ, mô hình đang tinh chỉnh để tăng confidence.
-

Câu 2. Naive Bayes — Laplace Smoothing

2.1 Không dùng Laplace

Prior: $P(\text{Rác}) = 3/6 = 0.5$, $P(\text{BT}) = 3/6 = 0.5$

Lớp Rác (3 mẫu: dòng 1,2,3): - $P(\text{giảm giá=Không} \mid \text{Rác}) = 1/3$ - $P(\text{miễn phí=Có} \mid \text{Rác}) = 2/3$ - $P(\text{hộp=Không} \mid \text{Rác}) = 3/3 = 1$ - $P(\text{dự án=Không} \mid \text{Rác}) = 3/3 = 1$

$$\text{score}_{\text{Rác}} = 0.5 \times \frac{1}{3} \times \frac{2}{3} \times 1 \times 1 = 0.1111$$

Lớp BT (3 mẫu: dòng 4,5,6): - $P(\text{giảm giá=Không} \mid \text{BT}) = 3/3 = 1$ - $P(\text{miễn phí=Có} \mid \text{BT}) = 0/3 = 0 \leftarrow \text{Vấn đề!}$

$$\text{score}_{\text{BT}} = 0.5 \times 1 \times 0 \times \dots = 0$$

Vấn đề: Xác suất bằng 0 triệt tiêu toàn bộ posterior. Mô hình luôn kết luận Rác dù “miễn phí” chỉ đơn giản chưa xuất hiện trong lớp BT ở tập huấn luyện.

2.2 Với Laplace ($\alpha = 1$)

Công thức: $P(w \mid c) = \frac{\text{count}(w,c)+1}{N_c+2}$

Lớp Rác ($N_c = 3$): - $P(\text{giảm giá=Không} \mid \text{Rác}) = \frac{1+1}{3+2} = 2/5 = 0.4$ - $P(\text{miễn phí=Có} \mid \text{Rác}) = \frac{2+1}{3+2} = 3/5 = 0.6$ - $P(\text{hộp=Không} \mid \text{Rác}) = \frac{3+1}{3+2} = 4/5 = 0.8$ - $P(\text{dự án=Không} \mid \text{Rác}) = \frac{3+1}{3+2} = 4/5 = 0.8$

$$\text{score}_{\text{Rác}} = 0.5 \times 0.4 \times 0.6 \times 0.8 \times 0.8 = 0.5 \times 0.1536 = 0.07680$$

Lớp BT ($N_c = 3$): - $P(\text{giảm giá=Không} \mid \text{BT}) = \frac{3+1}{3+2} = 4/5 = 0.8$ - $P(\text{miễn phí=Có} \mid \text{BT}) = \frac{0+1}{3+2} = 1/5 = 0.2$ - $P(\text{hộp=Không} \mid \text{BT}) = \frac{1+1}{3+2} = 2/5 = 0.4$ - $P(\text{dự án=Không} \mid \text{BT}) = \frac{1+1}{3+2} = 2/5 = 0.4$

$$\text{score}_{\text{BT}} = 0.5 \times 0.8 \times 0.2 \times 0.4 \times 0.4 = 0.5 \times 0.02560 = 0.01280$$

Posterior:

$$P(\text{Rác} \mid \mathbf{x}) = \frac{0.07680}{0.07680 + 0.01280} = \frac{0.07680}{0.08960} = 0.857$$

2.3 Kết luận

Phân loại: **Rác** (xác suất 85.7%).

Laplace smoothing cần thiết vì nó ngăn xác suất bằng 0 triệt tiêu toàn bộ tích xác suất. Nó “cho phép” các sự kiện chưa quan sát có một xác suất nhỏ hợp lý.

Câu 3. Decision Tree — Entropy vs Gini

Dữ liệu: 8 mẫu, 5 (+), 3 (-)

3.1 IG(A) theo Entropy

Entropy cha: $H = -\frac{5}{8} \log_2 \frac{5}{8} - \frac{3}{8} \log_2 \frac{3}{8}$

$$= -0.625(-0.678) - 0.375(-1.415) = 0.424 + 0.531 = 0.954$$

A=0 (dòng 1,2,3,8): 1(+), 3(-) $\rightarrow H = -\frac{1}{4} \log_2 \frac{1}{4} - \frac{3}{4} \log_2 \frac{3}{4} = 0.811$

A=1 (dòng 4,5,6,7): 4(+), 0(-) $\rightarrow H = 0$

$$IG_E(A) = 0.954 - \frac{4}{8}(0.811) - \frac{4}{8}(0) = 0.954 - 0.406 = 0.549$$

3.2 IG(A) theo Gini

Gini cha: $G = 1 - (5/8)^2 - (3/8)^2 = 1 - 0.391 - 0.141 = 0.469$

A=0: $G = 1 - (1/4)^2 - (3/4)^2 = 1 - 0.0625 - 0.5625 = 0.375$

A=1: $G = 0$

$$IG_G(A) = 0.469 - \frac{4}{8}(0.375) - \frac{4}{8}(0) = 0.469 - 0.188 = 0.281$$

3.3 IG(B) theo Entropy

B=0 (dòng 1,3,4,7): 2(+), 2(-) $\rightarrow H = 1.0$

B=1 (dòng 2,5,6,8): 3(+), 1(-) $\rightarrow H = -\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} = 0.811$

$$IG_E(B) = 0.954 - \frac{4}{8}(1.0) - \frac{4}{8}(0.811) = 0.954 - 0.500 - 0.406 = 0.049$$

3.4 So sánh

Thuộc tính	IG (Entropy)	IG (Gini)
A	0.549	0.281
B	0.049	—

A tốt hơn B theo cả hai metric. Entropy và Gini thường cho cùng kết luận về thứ tự, chỉ khác giá trị tuyệt đối.

Câu 4. Dropout và Batch Normalization

4.1 Inverted Dropout

Dropout mask: $\mathbf{M} = (1, 0, 1, 0)$, $p = 0.5$

$$\mathbf{a}_{dropout} = \frac{\mathbf{a} \odot \mathbf{M}}{1 - p} = \frac{(0.8, 0, 0.6, 0)}{0.5} = (1.6, 0, 1.2, 0)$$

4.2 Batch Normalization

$\mathbf{z} = (2, 4, 6, 8)$

$$\mu_{\mathcal{B}} = \frac{2 + 4 + 6 + 8}{4} = 5$$

$$\sigma_{\mathcal{B}}^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = \frac{9 + 1 + 1 + 9}{4} = 5$$

$$\hat{z}_i = \frac{z_i - 5}{\sqrt{5 + 10^{-8}}} \approx \frac{z_i - 5}{2.236}$$

z_i	$\hat{z}_i$	$\tilde{z}_i = 2\hat{z}_i + 1$
2	-1.342	-1.683
4	-0.447	0.106
6	0.447	1.894
8	1.342	3.683

4.3 Giải thích

Dropout giống ensemble: Mỗi bước huấn luyện, dropout tạo ra một mạng con (sub-network) khác nhau bằng cách tắt ngẫu nhiên các neuron. Khi test, sử dụng toàn bộ mạng tương đương với lấy trung bình dự đoán của exponentially nhiều sub-network $\rightarrow$ giống bagging/ensemble.

Batch Normalization tăng tốc: Chuẩn hóa input của mỗi tầng về $\mu = 0, \sigma = 1$ giảm hiện tượng internal covariate shift (phân phối input thay đổi liên tục khi train). Điều này cho phép dùng learning rate lớn hơn mà không gây dao động, tăng tốc hội tụ.

Câu 5. Dropout trong Neural Network

Kỹ thuật Inverted Dropout ngẫu nhiên tắt đi (đặt giá trị bằng 0) một tỷ lệ p các nơ-ron ở các tầng ẩn trong mỗi bước lặp khi huấn luyện. Cụ thể, vector kích hoạt $\mathbf{a}$ được nhân phần tử (element-wise) với một ma trận mặt nạ $\mathbf{M}$ (chứa 0 và 1) rồi chia cho $(1 - p)$ để giữ nguyên cường độ tín hiệu (kỳ vọng toán học): $\mathbf{a}_{drop} = \frac{\mathbf{a} \odot \mathbf{M}}{1-p}$.

Ý nghĩa Ensemble: Ở mỗi vòng lặp, bằng cách tắt ngẫu nhiên các nơ-ron khác nhau, mô hình thực chất đang huấn luyện một mạng con (sub-network) khác nhau được trích xuất từ mạng toàn cục ban đầu. Khi bước vào giai đoạn dự đoán (inference/test mode) và không dùng Dropout, toàn bộ nơ-ron được giữ lại và tham gia tính toán. Hiệu ứng này tương đương về mặt toán học với việc lấy trung bình dự đoán (model averaging / bagging) của hàng tỷ mạng con đã được huấn luyện, giúp mô hình đạt được sự vững chãi (robust) và giảm thiểu Overfitting hiệu quả.

Câu 6. MLE Exponential Family

6.1 Log-Likelihood

$$\ell(\eta) = \sum_{i=1}^n \log P(y^{(i)} | \eta) = \sum_{i=1}^n [\log b(y^{(i)}) + \eta T(y^{(i)}) - a(\eta)]$$

6.2 Gradient

$$\nabla_{\eta} \ell = \sum_{i=1}^n [T(y^{(i)}) - a'(\eta)] = \sum_{i=1}^n [T(y^{(i)}) - \mathbb{E}[T(y) | \eta]]$$

6.3 Hessian

$$\nabla_{\eta}^2 \ell = - \sum_{i=1}^n a''(\eta) = -n \cdot a''(\eta)$$

6.4 Chứng minh

Ta biết $a''(\eta) = \text{Var}(T(y) | \eta)$ (tính chất chuẩn của Exponential Family).

Vì phương sai luôn ≥ 0 :

$$a''(\eta) = \text{Var}(T(y)) \geq 0$$

$$\Rightarrow \nabla_{\eta}^2 \ell = -n \cdot a''(\eta) \leq 0$$

Tức Hessian $\preceq 0 \rightarrow$ log-likelihood là hàm **lõm** $\rightarrow$ negative log-likelihood là hàm **lồi** ■

6.5 Ý nghĩa

Vì negative log-likelihood lồi, mọi bài toán MLE cho phân phối thuộc Exponential Family (Gaussian, Bernoulli, Poisson, ...) đều: - Có tối đa **một** cực tiểu toàn cục (không có cực tiểu cục bộ). - Gradient descent/Newton luôn hội tụ đến nghiệm tối ưu. - Đây là lý do nền tảng mà Linear Regression (Gaussian) và Logistic Regression (Bernoulli) đều có hàm loss lồi.

Câu 7. Thử thách: Vanishing Gradient bằng Quy tắc Chuỗi

7.1 Chứng minh $\sigma'(z) \leq \frac{1}{4}$

Với:

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

ta có:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

Đặt $s = \sigma(z)$. Vì sigmoid luôn nằm trong $(0, 1)$:

$$s(1 - s) = s - s^2.$$

Đây là parabola quay xuống, đạt cực đại khi:

$$\frac{d}{ds}(s - s^2) = 1 - 2s = 0 \Rightarrow s = \frac{1}{2}.$$

Giá trị cực đại:

$$\frac{1}{2} \left(1 - \frac{1}{2}\right) = \frac{1}{4}.$$

Vậy:

$$0 < \sigma'(z) \leq \frac{1}{4}.$$

7.2 Gradient tầng đầu bằng tích đạo hàm

Với:

$$L_{oss} = \frac{1}{2}(a^{[L]} - y)^2,$$

ta có:

$$\frac{\partial L_{oss}}{\partial a^{[L]}} = a^{[L]} - y.$$

Gradient theo w_1 :

$$\frac{\partial L_{oss}}{\partial w_1} = \frac{\partial L_{oss}}{\partial a^{[L]}} \frac{\partial a^{[L]}}{\partial a^{[L-1]}} \cdots \frac{\partial a^{[2]}}{\partial a^{[1]}} \frac{\partial a^{[1]}}{\partial w_1}.$$

Với tầng $\ell \geq 2$:

$$\frac{\partial a^{[\ell]}}{\partial a^{[\ell-1]}} = \sigma'(z^{[\ell]})w_\ell.$$

Với tầng đầu:

$$\frac{\partial a^{[1]}}{\partial w_1} = \sigma'(z^{[1]})a^{[0]}.$$

Do đó:

$$\frac{\partial L_{oss}}{\partial w_1} = (a^{[L]} - y) \left[\prod_{\ell=2}^L \sigma'(z^{[\ell]})w_\ell \right] \sigma'(z^{[1]})a^{[0]}.$$

7.3 Cận trên cho gradient

Lấy trị tuyệt đối:

$$\left| \frac{\partial L_{oss}}{\partial w_1} \right| \leq |a^{[L]} - y| \left[\prod_{\ell=2}^L |\sigma'(z^{[\ell]})| |w_\ell| \right] |\sigma'(z^{[1]})| |a^{[0]}|.$$

Vì $|\sigma'(z)| \leq \frac{1}{4}$ và $|w_\ell| \leq 1$:

$$\prod_{\ell=2}^L |\sigma'(z^{[\ell]})| |w_\ell| \leq \left(\frac{1}{4} \right)^{L-1}.$$

Thêm $|\sigma'(z^{[1]})| \leq \frac{1}{4}$:

$$\left| \frac{\partial L_{oss}}{\partial w_1} \right| \leq |a^{[L]} - y| |a^{[0]}| \left(\frac{1}{4} \right)^L.$$

Nếu tách riêng phần suy giảm do các tầng sau, tín hiệu truyền từ output về tầng 1 bị nhân bởi ít nhất:

$$\left(\frac{1}{4} \right)^{L-1}.$$

7.4 Ước lượng với $L = 8$

Phần suy giảm do các sigmoid tầng sau:

$$\left(\frac{1}{4} \right)^7 = \frac{1}{16384} \approx 0.000061.$$

Tức là trước khi gradient chạm tới tầng đầu, chỉ riêng chuỗi đạo hàm sigmoid đã có thể làm tín hiệu nhỏ đi hơn 16 nghìn lần. Nếu nhiều z rơi vào vùng bão hòa của sigmoid, $\sigma'(z)$ còn nhỏ hơn $\frac{1}{4}$ rất nhiều, nên gradient có thể gần như biến mất.

7.5 Vì sao các kỹ thuật hiện đại giúp giảm vấn đề?

- **ReLU:** $g'(z) = 1$ khi $z > 0$, nên gradient không luôn bị nhân với một số tối đa 0.25 như sigmoid. Điều này giúp tín hiệu truyền ngược qua nhiều tầng tốt hơn.
- **Batch Normalization:** Giữ phân phối z ổn định hơn, giảm khả năng sigmoid/tanh rơi sâu vào vùng bão hòa nơi đạo hàm gần 0.
- **Residual connection:** Tạo đường truyền tắt dạng $a^{[\ell+1]} = F(a^{[\ell]}) + a^{[\ell]}$. Khi backprop, gradient có một nhánh cộng trực tiếp qua identity path, giúp tín hiệu không phải đi qua toàn bộ chuỗi đạo hàm phi tuyến.

HẾT ĐÁP ÁN

Cheers

Cheers, and good luck on the exam.